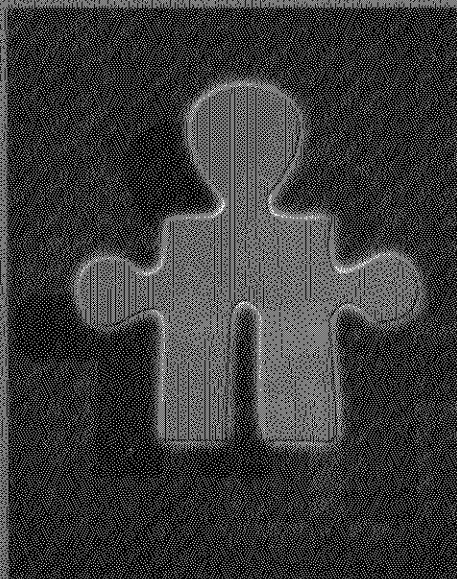


软件工程经典系列



个体软件过程

Introduction to the Personal Software Process™



PSP™

〔美〕Watts S. Humphrey 著

吴超英 车向东 译

周伯生 审校

ADDISON
WESLEY

SEI SERIES IN SOFTWARE ENGINEERING



人民邮电出版社
www.pptph.com.cn

图书在版编目(CIP)数据

个体软件过程/(美)汉弗莱(Humphrey,W.S.)著;吴超英,车向东译.

—北京:人民邮电出版社,2001.10

(软件工程经典系列)

ISBN 7-115-09650-3

I.个... II.①汉...②吴...③车... III.软件工程—教材 IV.TP311.5

中国版本图书馆CIP数据核字(2001)第062639号

软件工程经典系列

个体软件过程

◆ 著 [美] Watts S. Humphrey

译 吴超英 车向东

审 校 周伯生

责任编辑 俞 彬

◆ 人民邮电出版社出版发行 北京市崇文区夕照寺街14号

邮编 100061 电子函件 315@pptph.com.cn

网址 <http://www.pptph.com.cn>

读者热线 010-67129212 010-67129211(传真)

北京汉魂图文设计有限公司制作

北京顺义向阳胶印厂印刷

新华书店总店北京发行所经销

◆ 开本:787×1092 1/16

印张:15.5

字数:365千字

2001年10月第1版

印数:1-6000册

2001年10月北京第1次印刷

著作权合同登记 图字:01-2001-1376号

ISBN 7-115-09650-3/TP·2465

定价:30.00元

本书如有印装质量问题,请与本社联系 电话:(010)67129223

软件工程经典系列

个体软件过程

[美] Watts S. Humphrey 著

吴超英 车向东 译

周伯生 审校

人民邮电出版社

《个体软件过程》(简称“PSPI”),是 Watts S.Humphrey 于 1997 年特为美国大学一年级学生编写的教科书。书中描述了很多资深的软件工程师解决软件工程问题的方法,特别是有关软件项目计划和软件质量控制方面的先进方法,并提供了很多练习来帮助读者掌握这些方法。

美国 Embry-Riddle 航空大学计算机科学系以本书初稿为教材,为该系一年级学生讲授了这门课程,经验证明,这对造就学生成为合格的软件专业人员很有帮助。每章之后还附有习题,以帮助读者复习与掌握该章的主要内容。由于本书是在两个学期的计算机科学或软件工程导论课程中使用,所以本书的内容分成两部分,在第一学期中讲授时间管理(前 10 章),在第二学期中讲授质量问题(后 10 章)。

本书实用性与可读性较强,可作为高等学校计算机软件工程课程的教材,也可作为工程技术人员自学个体软件过程的教材,是进行软件过程改善和能力成熟度模型 SW-CMM 评估的重要参考资料。本书同样适用于软件开发项目经理、程序员和一般编程爱好者在开发软件时参考。

版权声明

Introduction to the Personal Software Process, First Edition
by Watts Humphrey

Copyright © 1997 by Addison Wesley Longman, Inc. All
rights reserved.

Published by arrangement with the original publisher,
Addison Wesley Longman, a Pearson Education Company.

人民邮电出版社经 Pearson 教育集团的 Addison
Wesley Longman 公司授权出版。版权所有，侵权必究。

本书是 Watts S. Humphrey 特地为大学一年级学生编写的教科书。该书描述了很多资深的软件工程师解决软件工程问题的方法，并提供了很多练习来帮助学生掌握这些方法。根据美国 Carnegie-Mellon（卡内基梅隆）大学软件工程研究所 Humphrey 等人的实践数据，在应用了 PSP 之后，软件产品的总的缺陷减少了 58.0%，在测试阶段发现的缺陷减少了 71.9%，生产效率提高了 20.8%。在美国，Embry-Riddle 航空大学计算机科学系首先以本书初稿为教材，为该系一年级学生讲授了这门课程，取得了很好的效果，接着在几十所大学中开设了这门课程。经验证明，这对造就学生成为合格的软件专业人员很有帮助。

为了在企业中推广 PSP，需要高层经理的大力支持，需要有合格的教员和合适的教材，而且还需要所有有关人员的积极参与。然而，遵循个体软件过程 PSP 开发软件并不是一件轻而易举的事，需要人们改变自己的工作方式。但是，人们通常很难改变自己的工作习惯，他们虽然愿意做些细微的改变，但往往离不开原有的习惯。只有在用他们自己的事实证明新的方法确实有效之后，才愿意真正进行改变。因此，推广个体软件过程需要以认真的实践为基础，并努力总结亲身经历的经验和教训。

个体软件过程可以作为两学期的软件工程课程的一部分，也可作为一门独立的课程。但这是一门实践性很强的学科，需要通过编写程序才能真正掌握 PSP 的精髓。为了精简讲授内容，减少重复实践，可以将 PSP 课程的程序设计练习与其它课程的作业结合起来。在最近与 Humphrey 的一次通信中，他告诉我们如果在讲授个体软件过程 PSP 之前，先讲一些统计过程控制原理，对理解 PSP 的方法和理念非常有利。

诚然，PSP 方法的成功与否，在很大程度上取决于教师激发学生学习和实践这些概念的能力。这不仅需要教师本人对 PSP 持积极态度，而且需要努力实践，认真积累数据，用自己的经验向学员展示 PSP 方法的威力。可以认为，教员是否按 PSP 原理进行认真的实践，是讲授好 PSP 课程的重要前提。

在 Embry-Riddle 航空大学计算机科学系的教员和学员分别为

本书撰写的“给教师的话”和“给学生的话”以及 Humphrey 本人在本书正文和他为本书撰写的前言中，都特别强调采集数据的重要性，并特别指出千万不要对数据进行粗制滥造，否则所收集的数据就没有什么用处。因此，我们诚恳地期望学员在学习 PSP 课程时，按正确的方法去做，努力理解 PSP 的整体框架和所蕴涵的概念，努力采集真实的数据，在自己的工作中坚持贯彻 PSP 的原则。因此，开展 PSP 培训，不仅需要学员的认真参与，而且需要在后续课程中进一步提高、巩固和扩展 PSP 方法。可以认为，这是掌握 PSP 方法的基本保证。

自从 1994 年以来，Humphrey 一直大力倡导这种方法，他在美国很多著名公司推行 PSP 方法，获得了很好的结果。而且还从 1996 年开始，大力倡导群组软件过程（也称小组软件开发过程）TSP 方法。软件能力成熟度模型 SW-CMM、个体软件过程 PSP 和群组软件过程 TSP 三者相互配合，各有侧重，形成了一个不可分割的整体，犹如一张具有三条腿的凳子，缺一不可。在国际上，印度的软件产业界颇为重视 SW-CMM、PSP 和 TSP 的培训和推广，因而在软件产品质量、软件项目的进度和费用等方面都得到了大幅度的改善。

此外，我们注意到在美国 Carnegie-Mellon 大学软件工程研究所对外开设的课程中，对个体软件过程 PSP 给予了特别的重视。其中 *Introduction to the Personal Software Process* 是一门历时两天的课程，讨论了 PSP 的关键概念以及相关的术语和理念，并为软件工程师参加群组软件过程作好准备。其中为软件工程师开设的 PSP 课程有两门，第一门讨论软件项目规划，第二门讨论软件产品质量，每门课都是五天。在第一门课中，除了讨论个体软件过程引论，还讨论规模测量、规模估计、如何基于历史数据来估计规模和资源以及如何进行过程测量等问题。第二门课主要讨论缺陷管理、设计过程、设计验证、如何将 PSP 应用于大型项目以及如何进行过程开发等问题。这两门课程都是以 Humphrey 所著的另一本书 *The Discipline for Software Engineering* 为教本。

北京航空航天大学软件工程研究所从 1997 年开始在硕士研究生中开设《个体软件过程引论》课程，最近两年改在本科生的软件工程课程中讲授 PSP 的内容，今年还对一些公司的软件工程师进行了 PSP 培训，这些都取得了一定的效果。但是由于实践时间不长，数据采集不全，需要进一步坚持改善与总结提高。

从 2000 年下半年以来，在我国软件企业界掀起了能力成熟度模型 CMM 评估的热潮。需要着重指出的是，在软件能力成熟度模型 SW-CMM 的 18 个关键过程域中，有 12 个与个体软件过程 PSP 紧密相关，有 16 个与群组软件过程 TSP 紧密相关。因此，如果能熟悉个体软件过程 PSP 和群组软件过程 TSP，不仅有助于工程师改善工作效率，而且也非常有利于组织的过程改善。可以认为，在大学中讲授 PSP 和 TSP 课程，对培养合格的软件专业人员和加速我国软件产业的发展将起积极的推动作用。

我们把 *Introduction to the Personal Software Process* 一书的中文译本奉献给读者，希望能对我国软件产业的发展起到推动作用。但由于我们水平的限制，缺点和错误在所难免，敬请读者对本书翻译中的缺点和错误提出批评、指正，并希望对授课方式和练习内容提出积极建议。

吴超英 车向东 周伯生

2001 年 4 月 22 日于北京

我们在 Embry-Riddle 航空大学计算机科学系,以本书初稿为教材,为该系一年级学生讲授过程原理课程。本书包含个体软件过程(PSP)的元素和活动的主要子集,一年级学生可以像传统的程序设计课程那样来接受这些内容。本书还为引导学生进行规范化的个体实践提供动力和一种结构。我们已经领略了以本书作为教材进行教学的好处,它对造就我们的学生成为合格的软件专业人员很有帮助。

多年来,我们一直在尝试如何对学生提供实际的软件工程经验。我们曾经在低年级课程中引入软件工程理论和实践,在高级课程中增设小组项目,并取得了一定的成功。但不幸的是,我们发现学生在做这些项目时,他们并不懂得如何进行时间管理、进度安排和质量管理。正如软件工业化生产已经证实的,工程小组高效开发高质量软件产品的能力,在很大程度上取决于个体工程师的能力。对学生在时间管理和质量管理上的问题我们并不感到惊奇,因为我们并未给学生讲授如何计划和如何管理方面的课程。因此,我们尝试着在大学低年级课程中开始介绍过程的概念。

我们感到,大学一年级新生最容易接受时间管理的实践并从中获益。所以我们在计算机科学系的第一门课程(CS1)中介绍本书第1章~第10章的内容。虽然所有选修CS1课程的学生都有一定的程序设计经验,但他们离正式定义的软件开发过程的要求还相距甚远。因此,在他们真正深刻理解软件工程师的角色和实践之前,需要了解现代软件开发工作的各种问题。

在完成了CS1课程和大学第一学期其它课程的学习之后,学生已经能用更加规范化的方法来开发程序,我们采用本书第11章~第20章的内容,在CS2课程中介绍了PSP过程。这时,学生已经可以为他们的每一个程序设计项目制订计划。他们遵循所定义的PSP实践,使用自己的历史数据来估计项目规模、工作量和质量(缺陷分布情况),并在项目总结报告中收集和记录每个项目的实际数据。

在进行了一年的实践之后，我们发现给计算机科学系一年级新生介绍过程活动的方法相当有效。这里说有效是指学生学会了如何使用本书所描述的过程。他们最终确实明白了记录工作量、程序规模和质量数据的价值，并且能利用这些数据计划项目和分析他们个人工作的有效性。他们在工作中收集数据，就为估计提供了定量基础。他们定期进行结构化复查，并学会在工作中遵循已定义的开发阶段(例如：计划、设计、编码、编译和后置处理)进行工作。我们也感到，如果把介绍 PSP 的课程推迟到另一个学期(或另一年)，只会使不正确、无规范的程序设计实践在学生心目中更加根深蒂固，从而对改变将更加抵触。

PSP 已经帮助学生认识到规范化方法对开发软件的重要性，也为将来进一步介绍高级个体过程和群组过程奠定了更严谨的基础。学生的大部分数据是准确的，但必须仔细分析并抛弃有怀疑的数据。我们感到遗憾的是，学生的计划工作能力并无提高，很多学生仍然把作业拖到指定的交付日期才交。这是一个新程序员常见的问题。

我们毫不惊奇地发现，PSP 方法的成功与否，在很大程度上取决于教师激发学生学习和实践这些概念的能力。我们使用本书的概念和论点鼓励学生对过程方法持积极的态度；对课堂数据提供定期的反馈和分析，以激发学生进一步考虑个人数据的兴趣；并邀请工业界专业人员在课堂上讨论他们的过程经验。这些对学生学好这门课程都很有帮助。

在开始讲授这些新课时存在一些困难。例如，我们开始没有将 PSP 与 CS1 和 CS2 课程的其它内容紧密地结合起来。因此，学生很难把时间管理活动与他们的程序设计工作相结合。此外，我们也没有对课堂的整体数据提供足够的反馈。

PSP 的一个有趣且有益的副产物是给教师提供了大量的数据。在 CS1 课程中，我们获得了学生花费在 CS1 课程活动上的时间的周报告。在 CS2 课程中，我们获得了有关每个程序设计项目规模、工作量和缺陷数据的 PSP 总结报告。这些数据经常引起课堂上对所讲授的方法以及它们如何影响程序员的生产率和程序质量进行热烈的讨论。这些数据为这些问题的详细讨论和分析提供了定量的基础。

在我们的计划中，将在大学一年级继续安排讲授 PSP 课程。我们也要求修完 CS1 和 CS2 课程的学生在数据结构和算法课程中继续使用 PSP 方法。我们相信，这将为他们在三、四年级遇到复杂的群组项目时作好准备。我们还计划在他们的后续课程中指导学生进一步提高和扩展 PSP 方法。

我们已经看到，本书对学生学习专业人员的软件规范很有帮助。我们希望其他使用本书的学生和教师也能得到同样的收益。

Thomas B. Hilburn, Aboalfazl Salimi, Massood Towhidnejad
Embry-Riddle 航空大学

当我们完成了一年级的 PSP 课程之后, Embry-Riddle 航空大学的一些教师问我们是否愿意联名为这本书写一篇前言。我们同意了。但由于我们不能肯定如何才能写好一篇前言, 他们建议我们仅仅回答一些问题。下面是教师向我们提出的问题以及我们对这些问题的答案:

一、你们在 PSP 课程中做了哪些类型的工作?

我们记录了在程序设计作业和项目上花费的所有时间。为了跟踪时间数据, 需要做大量的文书工作。我们还记录了程序的规模和缺陷, 并用所收集的数据估计今后项目的时间、规模和缺陷数目。

二、本书的内容合适吗? 这个教材与一年级其它课程的教材是怎样相配合的?

它很好地满足了课程的要求。而且有了一个估计之后, 可以帮助你正在进行的工作产生自信。

刚开始时, PSP 课程似乎对其它课程有干扰。但在课程结束时, 我们认识到, 这些活动对完成其它课程也确有帮助。在学习这门课程的整个过程中, 我们经常问自己: “为什么我要做这些工作?” 但是后来认识到, 对自己要完成的程序工作有了估计, 实际上对自己很有帮助。

非常重要的一点是, 千万不要对数据(时间)进行粗制滥造, 否则所收集的数据就没有什么用处。

三、你学到了什么?

除了上面已经提到的那些之外, 还学习如何才能更有效地利用时间以及在上机之前要做那些准备工作。在上机之前应该完成许多纸面工作。

通过示例和讨论, 还将了解自己和他人所犯的错误。由于在上机前做了许多纸面工作, 因此在进行程序设计时井井有条。我们还认为, PSP 不仅可以用于软件开发; 只要对表格稍作修改, 也可

用于其它各种活动。

四、对将来要使用 **PSP** 的学生有什么建议？

按正确的方法去做，对数据不要粗制滥造，要遵循指南。要努力理解 **PSP** 的整体框架和所蕴涵的概念。不要对文书工作产生抵触，它最终是有回报的。

Ben Bishop, Andrew Henderson, Michael Patrick
Embry-Riddle 航空大学

如果你正在学习成为一个软件工程师，这本书就是为你写的。书中描述了很多资深的工程师解决软件工程问题的方法，并提供了很多练习来帮助你掌握这些方法。每章描述了一个不同的主题，当你做完作业规定的练习之后，你将掌握这个技能。完成每个练习中的例题，对检查自己的学习情况很有帮助。

一、为什么我要写本书？

开发软件产品不仅是将程序设计指令汇集在一起，并使其能在一台计算机上运行，而且要求在商定的费用和进度下满足用户的需求。为此，软件工程师需要始终如一地在计划的费用下、按进度开发出高质量的程序。本书向你展示如何进行这些工作，介绍个体软件过程(PSP)，PSP是用规范化的个人实践从事高级软件工程的指南。

PSP将向你展示如何制订计划并跟踪你的工作，并指出怎样始终如一地生产高质量的软件产品。使用PSP也将向你提供工作有效性的数据并识别出自己的优势和弱点。PSP犹如运动场上的秒表和测距器，要对参赛做出明智的判定，需要通过测量来了解自己的擅长以及那些地方需要改善。就像一个田径队，软件工程也有许多专业，软件工程师需要各种技能和才干。为了事业的成功，需要了解和改善自己的技能和才智，并在工作中充分利用自己独特的才能。PSP对帮助你做到这一点很有帮助。

二、使用 PSP

通过使用PSP，你将对专业软件工程师经过多年的试验和失败所开发（研究、探索）出来的各种技能和方法进行实践。在前人经验的基础上前进，将会学得更快，而且可以避免重复前人所犯的错误。作为一个专业人员，关键是要了解别人已经做过什么，并注意吸取他们的经验和教训。

1. 学生将如何受益

虽然现在通常在研究生的软件工程课程中介绍PSP，但其基本

原理也可以在大学本科新生中讲授和实践。本书旨在学生学习其它课程的同时，循序渐进地了解 PSP 方法。当阅读每一章之后，要完成课后的练习。这些内容将向你展示如何管理时间，如何制订计划并跟踪你的工作，以及怎样始终如一地生产高质量的程序。

要掌握有效的技能和习惯需要花费时间，因此在每一个软件作业中都应该用 PSP 方法进行实践。如果你这样做了，在从事软件工程工作需要这些技能之前，你就已经学习、实践和完善了这些技能。

2. 已经工作的工程师将如何利用本书

已经工作的软件工程师也可从本书学习 PSP 的基础。我建议你从头到尾做一遍书中的练习，并将它们作为改善你的日常工作方法的指南。实践每个练习直到熟悉这种方法，然后再阅读下一章，以增添新的方法。然后采取同样的方法，在进行下一步学习之前，既要实践新方法，也要实践以前学过的方法。关键是一定要花时间来掌握某一个方法，然后才学习下一个方法。

只要坚持并遵循一定的规范，自学本书不应该感到困难。但若在一个班级中或几个同事组织起来学习，可以互相交流经验并分享见解，这样做效果会更好。无论采取哪一种方式，大约每周要花费 1~2 个小时学习 PSP 课本，记录和分析有关数据，并把 PSP 方法应用于你的工作。虽然学习 PSP 所需要花费的时间与你现在的习惯和实践有关，但若一旦完成了本书的学习，对你今后专业生涯的持续开发就奠定了坚实的基础。然而应该注意，学习 PSP 的关键是要对你的工作数据进行考察和思考，看看这些数据对你的个人性能说明些什么。

三、对教员的一些建议

本书是作为传统上两学期的计算机科学或软件工程课程的一部分而设计的，只要求大学一年级新生的程度。本书一步一步地介绍 PSP，学生可以在他们的日常课程中使用。前 10 章的练习都是一般性的，既可以面向程序设计工作，也可以面向其它工作。后 10 章的练习是为 6~8 小时或更多一点时间的小型程序设计练习而设计的。

虽然有些学生在进入大学后才第一次学习程序设计，但现在有很多学生在高中就学习程序设计的基础知识。因此将本书设计成既可在第一门程序设计课程中使用，也可在更高级的程序设计课程中使用。无论学生是否已经学过程序设计，他们都应该很容易理解本书的内容，并会立刻发现本书对他们非常有用。

本书是 PSP 引论，而不是 PSP 的全部内容。例如，本书没有包含准确估计或数据分析所需要的统计技术；也没有包含适用于大项目的 PSP 方法，即没有包含将 PSP 用于编写大型程序时所需要的过程定义和过程改善技术。因此，全面介绍 PSP 技术应在学生教育计划的后期进行¹。

当你指导学生学习本书并完成作业时，在第一学期，学生将学习跟踪和监控工作、管理时间以及制订计划的方法。在第二学期，学生将学习有关程序质量、进行复查以及利用各种

¹ 在我的另一本教科书《软件工程规范》(A Discipline for Software Engineering, Reading MA: Addison-Wesley, 1995)中，对 PSP 和 PSP 课程有更为详细的描述。在该书的辅助材料中，有教员指南以及包含课堂投影胶片和作业材料的光盘。

质量测量和质量管理的办法。他们也将学习有关缺陷、引起缺陷的原因以及工程师对他们生产的产品质量应该承诺的个人责任。在第二学期课程结束时，学生们可以掌握 PSP 的基本内容。为了在这个基础上继续深造，并帮助他们积累使用这种方法的经验，在后续课程中应该要求学生继续使用 PSP。

1. 教学方案

由于本书是在两个学期的计算机科学或软件工程导论课程中使用，所以把本书的内容分成两部分，在第一学期中讲授时间管理(前 10 章)，在第二学期中讲授质量问题。在两个学期中讲授本书共需要 6 个课内学时。由于学生在他们现在进行的课程中使用 PSP 方法，讲授本书并没有明显加重学生的课业负担。学生在学习这种方法上所花费的时间，都将在其它工作中提高效率而得到补偿。

当讲授这本书时，应在每章的课后安排作业。经验证明，最好在第一季度的前几周讲完前 10 章的内容。然后在该学期的后几周对所介绍的方法进行实践。第二学期应该遵循同样的策略，在前几周进行 PSP 的课堂讲授，在后几周对所介绍的方法进行实践。

特别重要的是，应把本书所讲授的内容作为整个计算机科学或软件工程导论课程的有机组成部分。要对学生讲清，本书所介绍的各种方法是软件工程方法的精髓，学生必须学习和实践。当你安排每个作业时，还要向学生说明，他们的分数不仅依赖于他们工作的质量，也依赖于他们运用 PSP 方法的水平。他们必须完成每个 PSP 练习，并且在介绍该 PSP 方法后继续使用它。课程方案、建议讲授的内容以及作业所需要的工具等，在本书最后一页的教师指南和支持材料中介绍。

2. 教员的准备

在讲授这门课程的过程中，你将会发现自己使用这种方法也有好处。例如，你可以使用计划和时间管理的方法来准备课堂讲座或批改作业。在你亲身使用 PSP 方法之后，将会更加赞同需要个体规范。有了这种背景，对你向学生解释并指导他们使用 PSP 方法都有好处。当学生发现你已经使用过 PSP，他们也就更有可能使用它。

四、致 谢

在撰写本书时，我要特别感谢 Embry-Riddle 航空大学计算机科学的全体教职员工。他们鼓励我编写本书并友好地评阅了我的手稿。由于他们在给几个新生班级教课，并且自己也在使用 PSP 方法，因此他们给我提出了很多有益的建议。在此我要特别感谢 Tom Hilburn、Iraj Hirmanpour、Aboalfazl Salimi、Davie Srachet 和 Massood Towhidnejad 教授对我的支持和鼓励。我也要感谢他们的三位学生 Ben Bishop、Guillermo Jose Hernandez 和 Richard Rickert 为我提供了他们的课程数据和课堂经验。此外，我还要感谢 Ben Bishop、Andrew Henderson 和 Michael Patrick 在本书的“给学生的话”中非常诚恳地描述了他们使用本书的经验。

在 SEI 和其他单位的一些朋友和同事也友好地评阅了我的手稿，并提出了许多有益的建议和评注。我很感谢 Steve Burke、Howie Dow、John Eikenberry、Andy Huber、Julia Mullaney、Glenn Rosander、Marie Silverthorn 和 Bob Stoddard。我的秘书 Marlene MacDonald 在阅读和评

注部分手稿以及在分发手稿进行评阅的过程中，也给了我很大的帮助。我还要感谢 Peter Gordon 和 Helen Goldstein 以及 Addison-Wesley 的专业人员在本书的出版过程中的有益支持。

最后，我为有一个很好的妻子感到骄傲。在我编写一本又一本书的过程中，我的妻子 Barbara 给我极大的帮助。就像我在出版其它书时一样，在本书交付出版之前，总是由她阅读本书的最后一版手稿。虽然我得到这么多的帮助和支持，但我深信，无论是 Barbara 或给我帮助的很多同事，都不可能发现我所有的疏忽和错误。所有留在书中的错误完全由我自己负责。

Watts S. Humphrey
Sarasota, Florida

第 1 章 软件工程师的任务	1
1.1 什么是软件工程	1
1.2 为什么工程质量很重要	1
1.3 个体软件过程	2
1.4 高质量工作的规范	2
1.5 高质量工作的重要性	3
1.6 如何提高工作质量	3
1.7 过程改进的步骤	4
1.8 本书的策略	5
1.9 总结	5
1.10 练习 1	5
第 2 章 时间管理	7
2.1 时间管理的逻辑原理	7
2.2 了解时间的使用情况	8
2.3 工程记事本	9
2.4 工程记事本的设计	9
2.5 工程记事本示例	10
2.6 总结	12
2.7 练习 2	13
第 3 章 时间跟踪	15
3.1 为什么要跟踪时间	15
3.2 记录时间数据	15
3.3 跟踪时间	16
3.4 使用标准的时间记录日志	17
3.5 处理中断	19
3.6 跟踪已完成的任務	19
3.7 在工程记事本中登记时间日志	21
3.8 时间记录的提示	21

3.9 总结	22
3.10 练习 3	22
第 4 章 阶段计划与产品计划	23
4.1 阶段计划和产品计划	23
4.2 周活动总结表	24
4.3 总结每周的时间分配	26
4.4 计算阶段时间和工作效率	28
4.5 使用周活动总结表	32
4.6 总结	32
4.7 练习 4	33
第 5 章 产品计划	35
5.1 产品计划的必要性	35
5.2 产品计划的用途	35
5.3 什么是产品计划	36
5.4 产品计划	36
5.5 制订小型任务的计划	37
5.6 术语定义	37
5.7 作业编号日志	37
5.8 关于使用作业编号日志的几点建议	42
5.9 使用产品的时间和效率数据	42
5.10 总结	43
5.11 练习 5	43
第 6 章 产品规模	45
6.1 产品计划过程	45
6.2 规模测量	45
6.3 使用规模测量的注意事项	46
6.4 程序规模	47
6.5 其它的规模测量方法	49
6.6 程序规模估计	49
6.7 较大规模的估计	50
6.8 在作业编号日志中使用规模测量的方法	55
6.9 总结	56
6.10 练习 6	56
第 7 章 管理好时间	57
7.1 时间管理的要素	57

7.2	活动分类	57
7.3	收集活动的时间数据	58
7.4	时间分配的评价	58
7.5	作出时间安排	58
7.6	找出更多的时间	60
7.7	制订基本规则	61
7.8	设定时间分配的优先级	64
7.9	管理好时间安排	66
7.10	关于管理可变动的时间的几点建议	67
7.11	时间管理的目标	67
7.12	总结	68
7.13	练习 7	68
第 8 章	契约的管理	69
8.1	什么是契约	69
8.2	认真制订契约	70
8.3	契约的一个示例	71
8.4	工业中的示例	72
8.5	处理没有完成的契约	73
8.6	管理契约的重要性	73
8.7	不对契约进行管理的后果	74
8.8	管理契约的方法	75
8.9	总结	75
8.10	练习 8	76
第 9 章	进度管理	77
9.1	进度管理的必要性	77
9.2	Gantt 图	78
9.3	制订项目进度表	79
9.4	检查点	79
9.5	跟踪项目计划	81
9.6	跟踪积分	83
9.7	总结	86
9.8	练习 9	86
第 10 章	项目计划	89
10.1	项目计划的必要性	89
10.2	项目计划总结表	89
10.3	项目总结	92

10.4	程序规模	93
10.5	开发阶段的时间	95
10.6	估计的准确性	97
10.7	总结	97
10.8	练习 10	97
第 11 章	软件开发过程	99
11.1	为什么使用过程	99
11.2	一些定义	99
11.3	过程脚本	100
11.4	检查点和阶段	102
11.5	更新的项目计划总结表	102
11.6	一个计划的示例	105
11.7	累计时间值计算的示例	109
11.8	总结	111
11.9	练习 11	112
第 12 章	缺陷	113
12.1	什么是软件质量	113
12.2	缺陷和质量	113
12.3	什么是缺陷	114
12.4	缺陷与 Bug	115
12.5	缺陷类型	116
12.6	了解缺陷	117
12.7	缺陷记录日志	117
12.8	统计缺陷个数	120
12.9	使用缺陷记录日志	121
12.10	更新的 PSP 过程	122
12.11	总结	127
12.12	练习 12	128
第 13 章	缺陷查找技术	129
13.1	个人对产品质量的承诺	129
13.2	发现缺陷的步骤	129
13.3	发现和修复缺陷的方法	130
13.4	代码复查	131
13.5	为什么要尽早发现缺陷	131
13.6	发现和修复缺陷的费用	132
13.7	利用代码复查发现缺陷	133

13.8	编译前的复查	135
13.9	编译与测试缺陷的数据	135
13.10	更新后的 PSP 项目计划总结表	136
13.11	其它种类的代码复查	141
13.12	总结	142
13.13	练习 13	142
第 14 章	代码复查检查表	145
14.1	检查表的用途	145
14.2	代码复查检查表的示例	146
14.3	使用代码复查检查表	149
14.4	建立个人检查表	149
14.5	改进检查表	153
14.6	编码标准	156
14.7	总结	158
14.8	练习 14	159
第 15 章	缺陷预测	161
15.1	缺陷率	161
15.2	缺陷数据的使用	162
15.3	缺陷密度	163
15.4	缺陷率的预测	163
15.5	缺陷估计	164
15.6	更新的项目计划总结表和示例	165
15.7	登入实际的数据	172
15.8	总结	173
15.9	练习 15	173
第 16 章	缺陷排除的经济效益	175
16.1	高质量工作的必要性	175
16.2	缺陷排除问题	176
16.3	缺陷排除时间	176
16.4	缺陷引入和排除的经验	176
16.5	节省缺陷排除时间	178
16.6	在 PSP 项目计划总结表中每小时缺陷数的计算	179
16.7	缺陷排除效益的计算	183
16.8	提高缺陷排除率	184
16.9	减少缺陷引入率	185
16.10	总结	186

16.11 练习 16	186
第 17 章 设计缺陷	187
17.1 设计缺陷的本质	187
17.2 识别设计缺陷	188
17.3 什么是设计	189
17.4 设计过程	189
17.5 设计缺陷的起因	190
17.6 设计缺陷的影响	191
17.7 设计表达	191
17.8 总结	195
17.9 练习 17	195
第 18 章 产品质量	197
18.1 质量第一	197
18.2 测试	197
18.3 过滤器概念	198
18.4 仔细工作的好处	199
18.5 缺陷排除效益的计算	201
18.6 最终的缺陷排除效益的估计	202
18.7 100%过程效益的好处	203
18.8 缺陷排除效益的经验	203
18.9 原型方法	205
18.10 总结	205
18.11 练习 18	206
第 19 章 过程质量	207
19.1 过程测量	207
19.2 缺陷排除中的矛盾	207
19.3 缺陷排除策略	208
19.4 质量的成本	209
19.5 质量成本的计算	209
19.6 质检/过失比	213
19.7 改进复查的效率	216
19.8 质量成本的精确计算	217
19.9 总结	218
19.10 练习 19	218

第 20 章 个人对质量的承诺	221
20.1 质量的重要性	221
20.2 低质软件的危险正在增长	221
20.3 制订个人质量承诺	222
20.4 个人的目标	223
20.5 成就的回报	223

第 1 章 软件工程师的任务

本章除了描述软件工程的任务及其中的某些关键活动之外，还概述了有助于学习并改善有关软件工程方面的技能的策略，并概括性地给出了在学习本书过程中所要完成的主要任务。

1.1 什么是软件工程

一个软件工程师的任务就是要在预定的时间和进度下交付高质量的软件产品。因此，要进行有效的软件工程的工作，要从以下三个方面进行：开发出高质量的软件产品；在预期的费用内进行工作；在预定的进度下完成任务。经过多年来痛苦的经历，许多软件工程师已经懂得，要想使所做的工作富有成效，需要进行下列工作：

1. 制订工作计划；
2. 按照此计划进行工作；
3. 尽最大努力生产出高质量的产品；

本书的主要目标就是要说明如何做。

1.2 为什么工程质量很重要

从历史上看，很少有软件开发组织能够确实满足费用及进度方面的要求。如此不乐观的记录不仅损坏了软件工程的名声，而且导致严重的业务问题。有很多这方面的示例，如业务失败、合同纠纷、法律诉讼及客户投诉等。FAA(联邦航空管理局)几十亿美元的空中交通管理项目由于软件问题其费用竟比预算超出一倍多，而且多次延误了项目的进度。一个相当大的软件公司(Ashton Tate)由于其低劣的软件质量而破产。软件中的缺陷甚至已经伤害了人的生命[Leveson]。

现在计算机软件对于许多行业都至关重要。它使大部分现代化的工厂得以正常地运转，它每天要处理上万亿美金的国际汇兑，它是现代社会中人们所用的所有产品与服务必不可少

的因素。随着软件的业务意义与日俱增，软件工程组织的有效性也越来越重要。由此可以认为软件工程师最重要的个人资产，就是始终以高质量的产品满足契约要求的能力。

1.3 个体软件过程

个体软件过程(Personal Software Process, PSPSM)¹是为帮助软件工程师完成出色的工作而设计的。它告诉工程师如何把这些先进的工程方法应用到日常的任务中去。它提供详细的估计和计划的方法，告诉他们如何按照计划去跟踪任务的执行情况，向他们详细解释如何利用已定义的过程来指导他们的工作。

在研究生的课程中讲述个体软件过程(PSP)的全部内容需要 15 节课的时间，在这期间，学生通过 10 道程序设计题和 5 道分析题来实践 PSP 的方法。这些内容将帮助他们了解 PSP 的方法是如何工作的。很多大学都开设了 PSP 的课程，而且 PSP 正在推广到许多工业企业。从数以千计的 PSP 课程中得到的数据表明，PSP 的方法对于提高工程师的计划能力和产品质量都很有效。

PSP 在工业化软件生产中也很有效。例如，一个由三位工程师组成的小组，在学习 PSP 之前，他们开发一个软件系统中 3 个组件所用的时间是预计的 5 倍。当经过 PSP 培训之后，他们开发同一个产品中另外 6 个组件所用的时间比计划的时间少了 10.4%。从用户所发现的产品缺陷来看，他们在经过培训后开发的组件的质量是培训前的 5 倍。

本书的每一章都将介绍 PSP 的一种方法，它应该应用到今后的工作中去。在使用 PSP 方法的过程中，将知道它是如何有效，并且提供了实践的机会。

PSP 的方法需要付出很大的努力，但是这是学习它们的唯一方法。听课和阅读课本固然重要，但最重要的学习工具是在完成每一道 PSP 练习时收集你自己的实践数据。保存这些数据非常重要，你可以看到 PSP 是如何改善你的工作性能。

1.4 高质量工作的规范

规范是指为掌握或提高某种技能的活动或训练。通常有人把规范看成繁琐的限制，但实际上是一个学习与自我提高的框架。PSP 的规范提供了软件工程师所需要的改进个人技能的结构化框架和方法。问题不在于是否需要某些个人的技能，而在于掌握这些技能需要多长时间以及是否能经常使用它们。PSP 的规范将会加速你的学习过程。

其它许多领域的专业人员都要通过他们各自的正规教育来学习和实践专业所要求的技能与方法。化学家要学习使用实验室的仪器进行精确的分析；在进行外科手术之前，实习生要在有经验的医生指导下来观察与实践，没有人会让他们在实际工作中学习有关卫生、感染

The Personal Software Process 和 PSPSM 是 Carnegie Mellon 大学的服务商标。

或手术室规程的知识；在进行首次商业飞行之前，飞行员要经过严格的训练；音乐家在初次演出之前也要经过多年的练习；在其它领域，专业人员也都要在表现出他具有基本的能力之后才会被允许进行甚至最简单的操作。然而，在软件行业，没有经过 PSP 培训的工程师不得不在工作中掌握这种技能。这不仅要付出很大的代价和时间，而且具有越来越大的风险。

1.5 高质量工作的重要性

作为实际工作中的软件工程师，将来很可能开发较大规模的产品或系统中的某一部分，无论开发的部分在整个产品中是多么小或是多么不重要，潜伏在其中的任何缺陷都可能破坏整个系统。一个软件系统的质量不仅是由它每一部分的质量决定的，而且在支持程序中任何细小的错误也都可能带来破坏性的后果。

现代的计算系统每秒可以执行几百万条指令。那么，即使十亿次才出现一次不常见的缺陷，在一天中就可能出现几次。软件中的异常情况随时都可能发生；看起来不可能的情况的出现周期也仅仅是稍稍长一点而已。大系统中微小部分的缺陷都可能引起不可预料的有时甚至是很严重的问题。如果由于细小的错误而在产品中留下缺陷，就可能给用户带来极大的麻烦甚至造成设备的损坏。

为了生产高质量的软件系统，每一个工程师都必须学会做高质量的工作。如果能不断编写出高质量的程序，开发人员及其产品就会获得雇主和客户的高度评价。

1.6 如何提高工作质量

当我在美国海军服役时，必须学会用机枪射击。开始训练时是用猎枪打泥鸽子，我的成绩很糟糕，并且经过练习还是没有提高。教官对我进行了一段观察后，建议我改用左手射击。作为一个习惯用右手的人，刚开始时我觉得不习惯，但是练了几次之后，我的成绩几乎总是接近优秀。

这个示例说明了几个问题。首先，需要使用测量来诊断一个问题。通过了解我击中了几只鸽子和脱靶的情况，教官和我就容易看出必须对我作些调整。然后，我们必须客观地分析测量的数据。通过观察我的射击，教官就可分析我射击的过程——上膛、就位、跟踪目标、瞄准，最后击发。教官的目的就是要发现我哪些步骤存在问题。他很快找到问题所在，于是就建议我改用左手射击。

最后，也是最重要的，就是自身的变化。过程改进是非常困难的，因为人们不愿意尝试新事物。他们传统的习惯看起来很自然，以至于不相信改变会有什么帮助。我总是使用右手，从来也没想过用左手射击会是什么样子。但是自从采纳了教官的建议，我的成绩就提高了。

定义测量方法并不是件容易的事情，但它几乎总是可能的。本书定义了软件过程的一些测量方法。一旦规定了测量方法，就必须搜集和分析数据。如果需要作些改进，接下来就要

分析工作过程，看看什么地方需要作些改变。最后，要想真正得到改进，必须切实作出改变。

如果不修改我的射击过程，我的成绩几年后都不会有什么变化，也不会成为一个优秀射手。仅仅进行测量并不会产生什么提高，仅仅靠努力也不会有什么提高。在很大程度上工作方式决定了所能得到的结果。如果还是按照老办法做事，得到的结果还会是老样子。

1.7 过程改进的步骤

改进工作方式所需要的步骤与我学习射击泥鸽子的步骤一样。它们并不复杂，如图 1.1 所示：

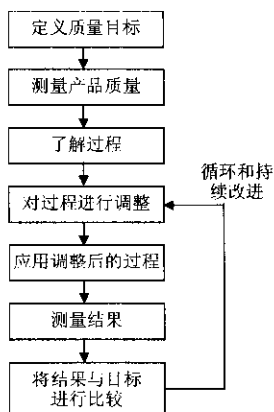


图 1.1 工作改进过程

● 定义质量目标。很显然，我的目标就是尽我所能射中靶子。百发百中是我最终的目标。

● 测量产品质量。教官和我都看到了我的糟糕成绩，觉得需要作些改进。

● 了解过程。教官观察我的操作，看看我应该作哪些改变。

● 对过程进行调整。教官建议我改用左手射击。

● 应用调整后的过程。我用左手进行了几轮射击。

● 测量结果。我们数了数我击中的和脱靶的鸽子数。

● 将结果与目标作比较。从统计数据可以看出，我的成绩大大地提高了。

● 循环和持续改进。因为学习射击泥鸽子的过程是一个简单的过程，所以就不需要这一步了。

如果是一个较复杂的过程，通常需要进行几个循环的过程改进。像软件开发这样复杂的过程，应该不断进行过程改进的循环。

1.8 本书的策略

本书分章介绍一些关键的软件工程方法。这些方法都很重要，但是讲述这些方法时应该按照某种顺序进行。这就像教人游泳一样，要把如何换气、如何蹬腿、如何飘浮以及如何用手臂划水等当作单个活动分别告诉别人，但实际游泳时要把这些动作连贯起来才行。通常情况下，要先教别人换气，而其它步骤的顺序则没有多大区别。但是不管怎样，这几个步骤都要告诉别人。在任何情况下，只有把这几步都学会了，才能学会游泳。有了这本书作指导，我们就比较幸运。书中的绝大部分方法都可以独立地使用，每种方法都有它们各自的作用；然而它们确实有相互促进的作用。因此，在把这些方法全都使用了之后，才能得到一个规范化的软件工程过程所带来的全部好处。就像学游泳就必须先学会在水中换气一样，规范化的软件工程必须从搜集数据开始。

本书的策略是介绍图 1.1 中所列的过程改进的步骤。由于顺序不是很关键，我就把内容分为两部分。本书的前 10 章讲述计划和时间管理的问题——即计划过程。后 10 章讲述产品质量的问题——即缺陷管理过程。首先讲述计划过程非常重要，因为这为讲述缺陷管理过程打下了坚实的基础。

1.9 总 结

软件工程师应该计划要做的工作，然后按照这个计划来工作。这样就能够在规定的时间范围内开发出高质量的产品。个体软件过程（PSP）就是为使软件工程师更好地工作而设计的一个框架。它指出如何估计和计划工作，如何按照这些计划来跟踪自己的性能，以及如何提高程序的质量。

提高质量的方法需要花费一定的时间去学习和实践，但这对你的工程生涯是有益的。为了不断提高工作的质量，必须树立目标、对产品质量进行测量、了解工作的过程、改变并重用这个过程、测量和分析结果，最后要反复地对过程进行持续改进。

本书的策略是指出如何调度和计划要做的工作，如何管理所生产的程序的质量。在了解工作过程中，第一步要明确所要完成的任务，估计每个任务所要花费的时间，并对所花费的时间进行测量。

1.10 练 习 1

在了解工作过程中，第一步是要明确所要完成的任务。例如，作为一个学生，要去听课、编写程序、读教科书，还要做各种各样的家庭作业。有些时候还需要为考试作准备。作为家

庭作业的一部分，你也要编写程序。描述这些任务的一种方法，列一个像表 1.1 那样的表格。这里，X 学生每周计划花费大约 1200 分钟学习，另外每学期还计划花费 300 分钟来准备考试。那么，每周的学习时间就是 20 个小时，此外每学期还要用 5 个小时来准备考试。

练习 1：要求如表 1.1 那样，把学习这门课的主要活动列出来。这项工作之后，估计这些任务可能发生的频率，估计在每项任务上要花费的时间。对于每周的任务，估计每周要花费的时间。对于每月或每学期的任务，按月或学期估计要花费的时间。这个练习并不要求对所花费的时间进行测量，作个估计就可以了。在交作业之前，作个备份。还有，别忘了在表格上写姓名和日期。

表 1.1

课程任务的示例

工程师	学生 X	日期	9/9/96
任务	频度	时间(min)	
听课	周一、周三、周五	每周 180	
阅读课本	每周一次	每周 180	
家庭作业	每周一次	每周 420	
编写程序	每周一次	每周 420	
准备考试	每学期一次	每学期 300	
复习笔记	在做家庭作业和学习时	已包括在其它的时间中	

参 考 文 献

[Leveson] Leveson, Nancy G. Safeware, System Safety and Computers. Reading, MA: Addison-Wesley, 1995.

第2章 时间管理

本章将阐述如何对时间进行管理及时间管理的重要性。同时还描述如何使用工程记事本以及为什么要使用它。为了完成本章的课后练习，需要准备一个在工作中使用的工程记事本。

2.1 时间管理的逻辑原理

时间管理的逻辑原理是：

人们很可能像上星期那样安排这星期的时间。人们通常利用上星期分配时间的方式为以后几个星期作参考。但是，也有很多例外的情况。例如，在考试的那一周，你就可能不会去听同样的课，你很可能花费较多的时间去复习，而用较少的时间去做家庭作业。

为了制订切实可行的计划，必须对所用的时间进行跟踪。人们大概很清楚上周的时间是怎么利用的，但是当看到实际的数据时，很可能会感到吃惊。人们往往记住一些事情，而同时忘掉另外一些事情。例如，做作业花费的时间可能会比估计的少得多；而花费在吃饭或和朋友消遣的时间会比预期的多得多。当我们乐意做某件事时，时间好像过得特别快，用的时间也似乎特别少。相反，那些进展缓慢、令人头疼或难以应付的活动占用的时间似乎要比实际花费的时间多得多。因此，为了搞清楚时间都花费在什么地方，必须保留一份准确的记录。

为了检查时间估计和计划的准确性，必须把它们写成文档并在今后与实际情况进行比较。虽然在学校里这并不是一个很严重的问题，但对于实际工作中的工程师来说，它却是至关重要的。作计划是一种技能，很少有人能作得很准确。但是，可以学习和实践这些已知的作计划的方法。学习制订好的计划，第一步就是要先作计划，然后把该计划写下来，以便今后与实际的数据相比较。

为了制订出更准确的计划，需要知道以前的计划中存在哪些错误，哪些地方可以进行改进。当按照计划进行工作时，记录下所花费的时间。如果笔记较详细的话，这些数据会很很有用处。例如，当学习某门功课时，分别记录下花费在听课、看书、写程序和复习考试上的时间。同样，当编写较大的程序时，你会发现记录下进行这件工作的各个阶段（设计、编码、

编译和测试)所用的时间也很有用处。当然,对于一些时间周期很短的工作,并不需要详细到这个程度,但是对于那些要花费几个小时甚至更长时间的项目,这个方法就非常有用。

当计划已文档化且记录了实际所花费的时间,就能很容易地把实际的结果与原来的计划相比较,然后就能发现计划中存在哪些错误以及如何改进作计划的过程。制订准确计划的关键就是要坚持制订计划,并把每个计划与实际的结果相比较,然后就会知道如何才能制订出更好的计划。

为了管理好时间,首先制订时间分配计划,然后按照计划去做。知道如何去作一个较好的计划是比较容易的,能真正按照计划去实施才是最困难的。世界上未能实施的计划的示例数不胜数,譬如节食或戒烟。

开始的时候,按照计划进行工作可能比较困难。你可能会有很多借口,最常见的就是这份计划作得不好。但只有按照计划去做,你才可能知道它的优劣。按照预先制订的计划进行工作会有两点好处:第一,了解计划中还存在哪些错误,有助于更好地计划下一个项目。第二,按照计划好的方式完成工作。这看起来好像不重要,但是事实上软件工程中的许多错误都是由于考虑不周、粗心大意或是不注意细节而造成的。在很多情况下,已经知道了合适的方法并且也作了详细的说明,但就是没有得到实施。因此,学会制订可行的计划是重要的,但是学会实际执行这些计划才是最关键的。

按照计划进行工作的另一个更加微妙的好处就是它实际上在改变你的工作方式。有了计划,就不用浪费时间去考虑下一步要干什么,它会帮助你把精力集中在所做的事情上,很少分心,从而提高了工作效率。

2.2 了解时间的使用情况

进行时间管理的实践,第一步要了解现在是如何利用时间的,为此需要遵循下面几个步骤:

将主要活动分类。当开始分配时间时,你会发现大部分时间都用在相对很少的几个活动上。这是很正常的。要想完成某项工作,必须把精力集中在几件最重要的事情上。如果把时间分给太多种类的活动,很难分析时间的利用情况并得出结论。对一门课程来说,一般把活动分成三到五类来跟踪它所花费的时间就足够了。如果以后需要更详细的数据,可以再把其中比较一般的类进行细分。

记录每项主要活动所花费的时间。坚持记录时间需要很强的自我约束能力。要想进行精确的记录,必须记录下每件主要工作开始和结束的时间。开始的时候可能经常忘记这么做,但是经过一段实践之后,就会形成习惯。第3章将详细讨论记录时间的问题。

用标准的方法记录时间。必需使用标准的时间日志。因为需要采集的时间数据的数量增长得很快,如果不认真记录和储存这些数据,它们很可能丢失或变得混乱。数据丢失或数据混乱将不利于查找或对它进行解释。如果不打算对这些数据进行适当的整理、归纳,就根本不要去收集数据。第3章描述了可用于搜集 PSP 时间数据的标准的时间记录日志。

将时间数据保存在合适的地方。把时间记录日志放好。当学习这门课程时，需要保存好时间记录日志，因此要把它放在方便使用的地方，这正是工程记事本的一个主要用途。

2.3 工程记事本

在本课程中，将使用工程记事本来记录时间花费情况，也可以用它记录其它的事情，例如记录作业，跟踪所承诺的工作，作课堂笔记以及记录设计思想或是进行计算。

对于一个软件专业人员，工程记事本也有许多用途，譬如记录时间日志、运算结果以及设计方案等，甚至可以作为你所遵循正确的工程实施方案的凭证。当公司涉及有关产品责任的法律纠纷时，这些记录就成了非常重要的证据。受害者起诉公司，想说明责任在于供应商的疏忽。对于公司来说，最有利的辩护证据就是工程师遵循了正确的工程实践。从这方面来说，使用工程记事本是一个好的习惯。

使用工程记事本的另一个好处就是可以保护雇主的知识资产。例如，记录下可取得专利的想法。有一次在一个设计会议上，我的同事和我提出了一个我们认为可以申请专利的想法，我们立刻把它记录在我的工程记事本上，并且每一页都签了名。专利律师告诉我们这对于确定发明的日期很有帮助。后来，我们的确在美国取得了这项发明的专利，也因此受到了公司的现金奖励。

这些看起来好像和学生没有什么关系，但是你需要学习这些方法，养成这样的习惯，这是将来作为软件工程师必不可少的。这就是为什么现在就要准备工程记事本并养成使用它的习惯。

2.4 工程记事本的设计

对记事本进行特殊的设计不很重要，但通常工业界用的是由螺旋线装订的活页夹。如果把每一页都编号，螺旋式的设计既可以使各页保持有序，又可以给工作提供有法律依据的记录。当然，不方便的就是必须按时间顺序记录，并且不能随便插入或取出记录页。

表 2.1 推荐了一种工程记事本封面的设计。在封面的顶端，要给记事本编号。几年之后，你将积累很多工程记事本，给它们编号以便将它们按时间顺序保存。然后，把姓名、电话号码或 E-mail 地址写在记事本上。随后，记下你在记事本中第一条记录的日期，当记事本记满时，将最后一条记录的日期登在终止日这一栏中。

在记事本里面，给每一页编号。留出前两页作为目录。在目录中，列出那些特殊的、你认为将来需要查找的条目，譬如说，课程作业。这样将为在整本书中查找提供方便。那些对以后不会再有参考价值的页，就不必把它的内容列出来了。

表 2.1

工程记事本封面的示例

编号: <u>1</u>
工程记事本 公司或大学的名称
姓名 <u> </u> 电话/E-mail <u> </u>
开始日期 <u>9/9/96</u> 终止日期 <u> </u>

2.5 工程记事本示例

表 2.2 给出了工程记事本的目录页的样版。对于那些在今后要参考的材料，把它的页码列在记事本目录页的左边，随后在主题部分写上简单的描述。例如，学生在第 3 页记录了两周的全部的 CS1 作业，这目录中还指出了在 11 页继续记录了其它的 CS1 作业。表 2.3 是记事本第 3 页的一部分内容的示例。

表 2.2

工程记事本目录的示例

目 录			1
页 码	主 题	日 期	
3	CS1 的作业	9/9—16	
4	CS1 的笔记，课程讲座 1、2、3、4	9/9—13	
8	设计笔记，第一个程序	9/11	
9	电话，等等	9/11	
10	CS1 的笔记（续），课程讲座 4	9/13—	

表 2.3

工程记事本页的示例

[illegible]

2.6 总 结

为了对时间进行有效的管理,必须对时间进行计划并按照计划来工作。为了制订切实可行的计划,首先要跟踪时间的使用情况。然后,把计划写成文档,并把它们与实际的情况进行比较。为了提高计划的准确性,就要找出以前计划中的不足之处,并确定怎样才能使计划制订的更好。

计划和管理时间的第一步是要弄清楚现在是如何利用时间的。为了做到这一点，需要把活动分成几个主要的类。而后，用一种标准的方法把每项活动所花费的时间记录下来，为了方便记录和查阅数据，采用了工程记事本。在这门课程中，也将使用这种记事本来记录作业和记笔记，同时也可作为设计思路的笔记以及各种计算的笔记。

2.7 练习 2

为这门课程将所要做的工作制作一个工程记事本。它的封面应当包含 2.4 节所描述的条目，它的目录应当至少包含一条此课程的作业或是有关当前周的工作安排。下次上课时，把该记事本交给老师。以后，老师可能会偶尔要你交回并检查你的工程记事本。注意，这是一个选做作业，如果老师没有要求的话，可以不做。

第3章 时间跟踪

本章描述了时间使用情况进行跟踪和记录的规程和表格，还给出了几种时间记录的示例。在课后作业中，要建立并使用时间记录日志。

3.1 为什么要跟踪时间

正如第1章描述的那样，要想改进工作质量，首先要搞清楚你当前要做的事情。这意味着必须了解要完成哪些任务、如何完成以及将要得到怎样的结果。这个过程的第一步，就是要明确定义这些任务并估计出完成每一项任务所要花费的时间。为了做到这一点，必须对时间进行实际的测量。本章将描述如何测量时间，并介绍一种有助于测量时间的表格。

我以前写书的经验证明这样做对工作会有帮助。在我计划写一本名为 *A Discipline for software Engineering* 的书时，我查阅了自己以前写书的数据，发现每周用在写书上的时间平均是 26 小时。由于我的工作日程安排得很满，只能利用晚上和周末的时间来写书。于是我计划每周用 20 小时来写这本新书。按照这样的速度，可以在明年一月完成书的初稿。

当这项工作进行了一个月以后，我发现事情比我预想的要困难得多。通过查阅记录的数据，发现每章我用的时间都比预计的要多。而且我每周实际花费的时间是 30 多个小时，而不是 20 小时。虽然工作量要比预计的要多，但是这些时间日志上的数据使我能够预计到会比原计划提前四个月完成初稿。事实上我也确实是在那个时间完成的。

即使你不打算写很多的书，也会发现控制项目所用的时间要比提高生产效率容易得多。当人们谈论他们努力工作时，实际上是指他们的工作时间更长了。除非你知道自己实际上用了多少时间，否则就不可能管理好使用时间的方式。

3.2 记录时间数据

当记录时间数据时，要记住，目的是要获得实际工作情况的数据，只要数据准确、完整，收集数据的格式和规程并不重要，我自己使用的方法正是本章里描述的，我也通过课堂把它

教给了许多学生。我还把这种方法介绍给许多工程技术人员，他们在应用了这个方法之后都获得了相当的成功。在他们克服了对使用这些表格和规程的原有的抵触情绪之后，大部分工程师发现这些方法既简捷又方便。你们很可能也会有这样的体会。

在学习这本书的过程中，要使用本章所描述的记录时间的方法。你可能想知道：“为什么应该使用这种表格？为什么不自己设计表格？”回答是这样的：

- 在课堂上，每一个学生都使用一种不同的表格或方法是不实际的。
- 如果以前没有收集数据的经验，一般很难设计出一种实用的表格和规程。
- 在完成这门课程之后，你就有了知识和经验来修改这些表格和规程，使其适合你的工作。

换句话说，先使用这里讲述的方法，然后通过课程使用了这些方法之后，就可以对它们进行更改或设计出全新的东西。不过，无论使用什么样的表格，要想管理好自己的时间，就必须不断地对它们进行跟踪。

3.3 跟踪时间

通常当人们谈论所做的事情时，常常以小时作为测量单位。事实表明以小时为单位没有多大用处。原因就是几乎不会在完整的一个小时内一直做某件事情。例如，图 3.1 是学生 7 在最近的课程中编写一些小程序时所用时间的总结。这些练习平均每一个要用 4~6 个小时，要求学生跟踪每次开始工作与结束工作的时间。从图中可以看出，学生 7 的连续工作时段约有 90% 都少于 1 小时。

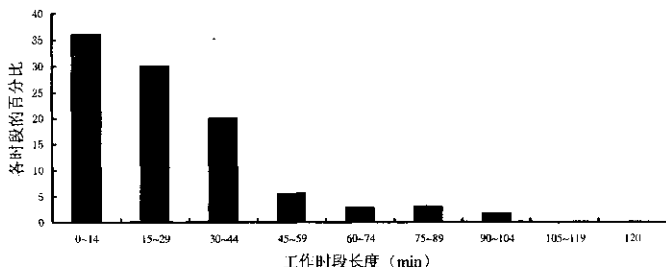


图 3.1 学生 7 的连续工作时段

工程师在完成任务中不间断工作的时间一般都少于 1 小时。因此，以小时为单位对工作时间进行测量不能提供用以计划和管理工作所需要的详细数据。另一方面，用分钟要比用带小数的小时跟踪时间数据容易得多。不妨设想一下，用带小数的小时记录的时间数据会是什么样子。那些记录中可能会充满像 0.38 或 1.27 小时这样的数字。虽然在每周或每月的总结中以小时为单位可能会更有用，但是这些小数部分既难以计算又难以解释。例如，23 分钟比 0.38

小时更容易理解和记录。一旦决定进行时间跟踪，用分钟作为测量单位将比用小时更容易。

3.4 使用标准的时间记录日志

表 3.1 给出了用于记录时间的表格。在表格的顶端是表头信息，这里的空格是用来记录你的姓名、填写表头信息时的日期、指导教师的姓名和课程的名称或编号。表格中的列用来记录时间数据。按照下面的格式在表格的每一行登入每个活动使用时间的情况：

- 日期。进行某个活动的日期，譬如听课或编程序的日期。
 - 开始时间。开始这个活动的时间。
 - 结束时间。结束这个活动的时间。
 - 中断时间。由于中断而占用的时间（见下一节）。
 - 净时间。以分钟计算这项活动从开始到结束用的时间，不包含中断所占用的时间。
 - 活动。任务的描述性名称。
 - 备注。有关所做事情的更加完整的注释、中断的类型以及任何对于以后分析时间数据有用的内容（参见下一节中有关中断的内容）。
- C（完成）：当完成一项任务后，例如阅读了书中的某一章或是编写了一个程序，在这一栏中作个标记（参见3.6节）。
- U（单元）：完成一项任务所用的工作单元（参见3.6节）。

表 3.1

时间记录日志

学生 _____ 日期 _____

教员 _____ 课程 _____

[illegible]

学生	学生 Y	日期	9/9/96
教员	先生 Z	课程	CS1

[illegible]

按照这种方法记录时间，意味着必须对时间进行跟踪。如果没有手表的话，最好去买一块。

3.5 处理中断

在跟踪时间时，一个常见的问题是中断。我们被电话、和别人聊天、偶尔的烦恼以及必要的休息打断的次数多得令人吃惊。使用时间记录日志处理中断的方法就是把这部分时间登入中断时间那一栏。如表 3.2 中给出的那样，学生 Y 在 10 日 11:06 开始做作业，到 12:19 做完。在这期间，她的活动被中断过两次，一次用了 6 分钟，另一次用了 5 分钟。她花费在做作业上的时间就是 73 分钟再减去中断用的 11 分钟，即总共 62 分钟。学生 Y 不仅把这些时间登入了时间记录日志，还在备注那一栏中登入了中断的简要说明。

跟踪中断的一个有用的技巧就是使用秒表。在被中断时，启动秒表；当准备重新工作时，终止计时。在再次开始工作之前，把中断时间登入日志中并将秒表复位。这比记录每次中断的开始时间要方便，也比猜测的数据更准确。一位爱好象棋的工程师告诉我，他发现下象棋用的时钟最适合跟踪中断的时间。当然，用什么技术并不重要，只要它合适就行。

由于中断的时间不是有效的工作时间，所以必须记录中断所占用的时间。如果中断时间的长短基本不变，则处理它就不很麻烦。然而，中断时间的变化幅度很大，如果不对它进行测量，实质上就是在时间记录中加入了—个随机数，也就很难使用这些时间数据来计划和管理你的时间。

时间日志中的数据也能帮助你了解工作被打断的频度。中断不仅浪费时间，还会打断你的思路，导致效率降低和错误的产生。因此了解被打断的频度有助于提高工作的质量和效率。一些工程师告诉我，通过对时间进行跟踪，他们最大的收获之一就是学会控制中断的次数和中断持续的时间。

3.6 跟踪已完成的任务

为了对时间的使用情况进行跟踪，还需要对产生的结果进行跟踪。例如，对于听课或开会，一份简单的时间记录就足够了。但是，在开发程序、读书或是写报告时，就必须知道有多少工作已经完成了，这样才能计算出工作效率。譬如，阅读书籍的一个章节或编写一个程序要多长时间。知道了这些，就能为将来的工作制订出更好的计划。

时间记录日志右侧的 C 和 U 栏是 Completed（已完成）和 Unit（单元）的缩写。这两栏有助于快速地确定不同任务所花费的时间和已经完成的任务。从本书第 4 章开始将使用这些信息。

这里，一个单元就是一个工作单元。例如，读了书的一个章节，就是完成一个工作单元。编写完一个程序是另一个工作单元。第 6 章将介绍更详细的工作单元的测量方法，如书以页

作为单元，源程序以行作为单元。那么，11 页书就是 11 个工作单元。虽然可以用任何有用的单元来测量工作，但是通常像页或代码行这样的小单元要比章节或程序这样的大单元更有用。不过，现在仍使用大单元，在第 6 章再详细讨论单元的测量问题。

为了登录 C 栏，在完成一项任务时，在这一栏作上标记。例如，在表 3.2 中，学生 Y 在 9 月 9 日从 6:25 到 7:45 在读书。在这段时间里，她读完了两章，就在那一行的 C 栏中作了标记。为了标出她完成的工作单元，在 U 栏登入了 2。以后，当她总结一周工作时，就能很快扫视这几列，找出已完成的活动的项。她也可以知道完成了多少单元的工作以及这些工作花费了多长时间。例如，在表 3.2 中，有三行有关于课文的记录，但是只有两行在 C 栏中作了标记。在这两行中，学生 Y 一共花费了 $80+28=108$ 分钟阅读了一共 $2+1=3$ 章课文。使用这些数据，学生 Y 可以看到，她阅读每章课文平均用 36 分钟。

为了进行准确的时间记录，每次完成具有可测量结果的任务时都要填写 C 栏和 U 栏，这样做非常重要。如果忘记了，虽然通常也还能找到相关信息，但是在完成任务之后马上填写要容易得多。时间记录日志的说明在表 3.3 中又重复了一遍，仅供参考；在第一次使用日志之前回顾一下这些说明，以后再核对一下以确保能够正确地记录时间。

表 3.3 时间记录日志使用说明

目的	用来记录时间在本课程中的使用情况；将时间记录日志放在工程记事本的后面
一般性说明	记录花费在本课程中的所有时间；以分钟为单位进行记录；尽可能做准确
表头	如下输入： 你的名字和填表的日期 教师的名字和课程名字或课程号 确保把名字写在提交的时间记录日志和家庭作业上
日期	登入记录的时间
示例	9/14/96
开始时间	登入开始一项任务的时间
示例	9:15
结束时间	登入结束上述任务的时间
示例	11:59
中断时间	登入任何类型的中断所用的时间和中断的原因。如果有多个中断，登入中断的总时间
示例	5+3+22，休息，接电话，聊天
净时间	登入花费在上述任务上的总时间，不包括中断时间
示例	从 9:15 到 11:59，再减去 30 分钟，总计 134 分钟
活动	登入任务的名字或其标志的名字，或所做工作的活动
示例	复习
备注	登入相关的注释信息，以便将来可以了解关于该活动的任何意外的情况

续表

示例	准备考试
C (完成)	当一项任务已经完成时, 在这一栏中作个标记
示例	在 9/9 的 7:45, 读完了课本的一章或几章, 就在这里作个标记
U (单元)	登入所完成的工作的单元数
示例	9/9 从 6:25 到 7:45, 读完了课本的两章, 因此在这里登入 2
注意事项	记录用在本课程中的所有时间; 如果忘记记录时间, 马上作自己最好的估计; 如果忘记随身携带时间记录日志, 用其它方式记录时间并迅速把它们拷贝到时间日志中

3.7 在工程记事本中登记时间日志

当你在这门课上花费时间时, 必须随手有一份时间日志来保留完整的时间记录。工程记事本可以方便地保存时间记录日志。因为我经常同时忙于不同的项目, 就把所用的时间记录在记事本的后面。由于我随时都有新开始的项目, 也有即将结束的项目, 所以我发现用工程记事本记录时间比用多个时间表要方便。

我认为最方便的方法就是从记事本的最后一页开始填写时间日志, 每次向前翻一页。当记满了一页, 就向前翻一页。我这样一直记录事情和时间, 直到从前面开始记的内容遇到了从后开始记的时间日志, 这时就换一本新的记事本。

但是, 你可能希望为每个项目保存不同的时间日志。当一次只做一个项目时, 使用单独的几页保存时间记录会更方便。而且, 在某些课程中, 指导教师可能希望用单独的时间日志来记录时间, 以便于作业的上交和批改。当然, 只要记录了时间, 选择何种特殊的方法并没有多大的差别。

3.8 时间记录的提示

跟踪时间在概念上很简单, 但是, 有一些窍门能帮你提高记录时间数据的一致性和准确性。

- **随时准备好工程记事本。**这看起来很容易, 但是刚开始时你偶尔会忘记这么做。如果确实忘记了, 把时间信息记录在一张纸上, 并尽快把它登入时间日志中。此外, 如果你用工程记事本跟踪其它的重要信息, 如课堂笔记、作业、约会, 就更应该记住随身带上它。

- **当偶尔忘了记录开始时间、结束时间或中断的持续时间时, 凭记忆尽早作出估计。**这不如当时就记录时间准确, 但是, 这是能够作出的最好的补救措施。它一般都比较接近实际的时间。

- **用秒表来跟踪中断的时间。**这么做看起来过于精确了，但是这比每次记录中断的开始和结束时间要容易。

- **及时总结记录的时间数据。**使用周活动记录来总结每周在学习此课程上所花费的时间。我们将在第 4 章讨论怎样和为什么做总结。

另一种记录时间数据的方法就是把数据记录在计算机上。我试过这种方法，发现它要花费更多的时间，而且没有在纸上记录方便。计算机系统应该是一种理想的方案，但是需要适当的应用程序来支持。在这种支持工具开发出来之前，你的选择只有自己来开发这种支持程序，或者使用手工记录的方式，要么就不对时间进行跟踪。

3.9 总 结

本章描述了如何跟踪时间。使用时间记录日志以分钟为单位对时间进行跟踪，并把时间记录日志写在记事本的后面。在执行课程中的任务时，随时准备好记事本。如果忘了随身带记事本，可以先记下任务所用的时间，并尽快把这些数据拷贝到记事本中。建议使用秒表跟踪中断。

3.10 练 习 3

使用时间日志跟踪你在这门课程的不同活动上花费的时间。作为这个练习的一部分，检查你为练习 1 建立的活动表并对它进行必要的调整。下周交一份你的时间日志。从现在开始到这门课结束，你每周都要交一份时间日志。

第 4 章 阶段计划与产品计划

本章介绍阶段计划和产品计划以及它们与个人工作的联系。另外，还要学习使用周活动总结表来统计迄今为止你在这门课程时间记录日志中的数据。

4.1 阶段计划和产品计划

这里，我们介绍两种计划。一种计划是基于时间段的，时间段即日历上的一段时间，如一天、一周、一个月或一年，这种计划称作阶段计划。阶段计划是关于在这段时期内对时间的安排。第二种计划是基于活动的，例如开发一个程序或写一份报告，这种计划称为产品计划。产品可以有形的，如程序或报告；也可以是无形的，如从课本中学到的知识或是在办公室提供的服务。产品计划是关于制作产品活动期间的时间安排。

以阅读本书这个活动为例来说明阶段计划和产品计划的区别。为了计划这项工作，首先应估计出整个任务应花费多少时间。例如，你可能希望用 20 小时阅读全书 20 章的内容。对于这个任务来说，产品计划就是以 20 小时读完全书为目标，阶段计划就是每周安排 1 小时读书这种方式。

阶段计划和产品计划之间的关系

在日常生活中，我们既用到阶段计划也用到产品计划。例如，在业务领域，这两种计划之间的关系可以简单地用图 4.1 来表示。图中左边的部分说明基于产品的任务，右边的部分是基于阶段的任务。

公司管理部向工程和制造部提供资金用来开发和制造产品。工程部在开发出产品后，把它们交给生产部进行生产。通过市场部，制造部把这些产品交付客户，客户则向公司付款。另外，工程和制造部要把产品计划报告给财务和经营部，这些部门则根据产品计划制订季度和年度的收入和支出的阶段计划。财务和经营部把阶段计划上报给公司管理部。他们还要为工程和制造部提供估价和预测信息，以便这些部门知道每月该生产多少产品以及应该向客户收取多少费用。公司管理部根据阶段计划决定付给投资者多少股息和利息，并且决定公司需要多少新的投资。当公司管理部知道公司以后将获得多少利润时，管理部门就能够决定给工程部多少资金用于以后的产品开发和制造，这样就完成了这个循环。

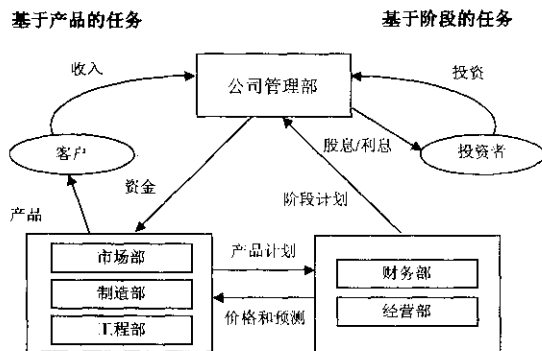


图 4.1 阶段计划与产品计划的关系

阶段计划和产品计划的重要性

产品计划在业务运作中很重要，因为市场部要根据产品计划确定什么时候新产品可以卖给用户，财务部要根据产品计划计算开发和制造所需要的费用，决定产品的价格，并向公司管理部提供准确的阶段计划。然后管理部根据工程和制造部的需要分配资金。如果产品计划作得不准确，那么阶段计划也不会准确。这样，一方面管理部、市场部、投资者甚至客户都将收不到准确的信息；另一方面，当工程和制造部需要必要的资金时，资金却不能到位。没有足够的资金，工程和制造部就不得不削减计划；一旦削减计划，员工就可能被解雇。

这样的计划循环听起来可能太理论化了，也许需要几年的时间，但是这种威胁确实是存在的。在任何公司中，工作职位稳定的关键是该组织财务系统的健全；而一个公司的财务系统健全的关键就是要有准确的阶段计划和产品计划。因此，工程师懂得怎样制订好的阶段计划和产品计划是非常重要的。

虽然阶段计划和产品计划之间存在着必然的联系，但两者是两个不同的计划。工作进度、个人收入以及其它日常活动都是在一个时间段中进行的。也就是说，睡觉、吃饭、工作以及休息都是发生在一天或一周中特定的时间段内。支出和收入由每周、每月或每年的进度决定。因此，人们生活在一个有阶段活动的、有规则和有约束的世界里。然而，人们工作的主要目的就是要向他人提供有价值的产品和服务。因此，这些货物或服务费用、进度和质量都非常重要。由于你的工作是生产产品，你的生活又处于这些周期之中，所以，阶段计划和产品计划对你都很重要。如果不能做好其中的任何一个，另一个也就不会做好。本章的重点是阶段计划，下一章讨论产品计划。

4.2 周活动总结表

为了制订阶段计划，必须清楚时间的使用情况。首先要做的就是使用第 3 章中介绍的时

间记录日志跟踪时间。在收集了一两周的时间数据之后，你就能渐渐明白自己是如何支配时间的。由于时间日志中的数据过于详细，不适合直接用于制订计划，因而需要用一种更有用的表格来总结这些数据。表 4.1 所示的周活动总结表给出了组织时间数据的示例，这对制订阶段计划是比较方便的。

表 4.1

周活动总结表

姓名:

日期:

1	任务 日期								日总计
2	日								
3	一								
4	二								
5	三								
6	四								
7	五								
8	六								
9	周总计								

阶段时间和效率

周数 (上一次周活动总结表的周数+1):

不包括上一周在内的累计时间

12	总计								
13	平均								
14	最大								
15	最小								

以前各周的累计时间

17	总计								
18	平均								
19	最大								
20	最小								

周活动总结表的 1~9 行记录这学期上一周每天各主要活动所花费的时间。从 11~15 行分别给出这学期上一周以前的各周在各项任务上花费的平均时间、最长时间和最短时间。17~20 行给出到目前为止这学期内 (包括上一周) 在各类工作中花费的总时间、平均时间、最长时间和最短时间。

当制订下一周的计划时，可参考最近的周活动总结表。根据以前各个任务花费的时间，就能判断出下一周将在这些任务上花费多少时间。制订这种计划最简单的方法就是假设将要使用的时间与过去平均使用的时间相同。一种精确的方法是首先考虑下周将要做的工作内容，然后根据以前的最长和最短时间来估计出一个合适的时间。这样通常会得到一个更加准确的计划。下面的几节将讲述如何完成周活动总结表。

4.3 总结每周的时间分配

表 4.2 是一个部分完成的周活动总结表的示例。表中的数据是根据表 4.3 的时间记录日志中的数据计算得来的。具体的步骤如下：

1. 登入姓名和日期。
 2. 在表格的第 1 行中顺序列出工作的类别。在表 4.2 给出的示例中，工作类别包括听课、编写程序、准备考试以及阅读课本。
 3. 将第 1 周星期日的日期登入日期的那一列（第 2 行）。
 4. 计算时间记录日志中每一天各类任务所用时间的总和，把结果登入相应的列中。例如，在表 4.2 中，第一类任务是听课。在表 4.3 的时间记录日志中，学生 Y 在 9 月 9 日上午从 9:00~9:50 花费了 50 分钟时间听课，就在表 4.2 中 9 月 9 日对应的第 3 行听课的那一列登入 50。你可以看到时间日志中那天有两条编程的记录：分别用 38 和 58 分钟，合计 96 分钟，就知道这天编程所花费的时间的总量，然后就在表 4.2 的第 3 行编程的那一列中登入 96。
 5. 在完成每一天所有的记录之后，计算其总和，并把结果登入当天的总计一栏。例如，表 4.2 第 3 行的总时间是 $50+96+80=226$ 分钟。
 6. 对这一周的每一天都重复上述步骤（4~8 行）。
 7. 计算各类任务每周花费的总时间，并登入第 9 行。例如，在编程那一列，将得到 $96+62+69+114=341$ 分钟。
 8. 下一步，统计周总计行（第 9 行）中的数字以得到一周的全部活动所用的总时间。在这个示例中，这个总数是 $150+341+134+146=771$ 分钟。
 9. 最后，把每天的总时间加起来与周总计行的结果相比较。在这个示例中， $226+62+147+114+88+134=771$ 分钟，与周总计行的结果一致，所以没有错误。如果这两个数不一样，就要重新合计各天的总时间和各类任务的总时间来找出错误。
 10. 按照 4.2 节所解释的那样完成当前这周的时间总结。
- 减少计算错误的一个好方法就是把每列的数字加两遍：从上到下加一遍，再从下到上加一遍，以确保得到相同的数字。另外一种减少计算错误的方法就是使用本书的支持材料所提供的 EXCEL 电子表格¹。

¹ 参阅本书最后一页，可以得到本书的支持材料的描述和使用说明。

表 4.2

周活动总结表

姓名: 学生 Y

日期: 9/16/96

1	任务 日期	听课	编写 程序	准备 考试	阅读 课本					日总计
2	日 9/8									
3	一	50	96		80					226
4	二		62							62
5	三	50	69		28					147
6	四		114							114
7	五	50			38					88
8	六			134						134
9	周总计	150	341	134	146					771

10 阶段时间和效率

周数 (上一次周活动总结表的周数+1): 1

11 不包括上一周在内的累计时间

12	总计								
13	平均								
14	最大								
15	最小								

16 以前各周的累计时间

17	总计	150	341	134	146				771
18	平均	150	341	134	146				771
19	最大	150	341	134	146				771
20	最小	150	341	134	146				771

第一周的时间记录日志的示例

日期 9/9/96

课程 CS1

[illegible]

4.4 计算阶段时间和工作效率

- 28 -

怎么得来的，下面我们一起来完成表 4.4 所示的周活动总结表，其中用到的数据是表 4.2 给出的学生 Y 第一周的活动总结。利用这些数据，按照下列步骤来计算表 4.4 中的阶段时间和效率。

表 4.4

周活动总结表

姓名：学生 Y

日期：9/23/96

1	任务 日期	听课	编写 程序	准备 考试	阅读 课本					日总计
2	日 9/15									
3	一	50	93		80					223
4	二		95							95
5	三	50			71					121
6	四		77							77
7	五	50	74		40					164
8	六				33					33
9	周总计	150	339		224					713

10 阶段时间和效率

周数（上一次周活动总结表的周数+1）：2

11 不包括上一周在内的累计时间

12	总计	150	341	134	146					771
13	平均	150	341	134	146					771
14	最大	150	341	134	146					771
15	最小	150	341	134	146					771

16 以前各周的累计时间

17	总计	300	680	134	370					1484
18	平均	150	340	67	185					742
19	最大	150	341	134	224					771
20	最小	150	339	134	146					713

1. 登入已经过去的周数。查看前一周活动总结的第 10 行，了解已经过了几周。由于表 4.2 第 10 行的空格中登的是 1，所以表 4.4 中周数的那一栏应为 $1+1=2$ 。

2. 把前一周表格（表 4.2）中 17~20 行的所有记录复制到目前这周表格（表 4.4）的 12~15 行。

3. 合计到目前为止各任务所花费的时间。在表 4.4 第 17 行的每一列中登入第 12 行和第 9 行的时间之和。对于第一列听课这一项来说, 就是 $150+150=300$ 。对于第 2 列编程就是 $339+341=680$ 。在其它各列登入相应的记录。现在第 17 行各列表明了这学期前两周各项任务所花费的时间。

4. 计算这学期每周各项任务所花费的平均时间。用周数分别去除第 17 行中的数字, 将得到的结果登入第 18 行。第 1 条平均记录是 $300/2=150$, 第 2 条平均记录是 $680/2=340$ 。对于余下的各列进行同样的运算。

5. 为了找出一周内某项任务所用的最多时间, 需要计算这个值。为此, 要比较第 14 行与第 9 行中各列相同项的大小, 把最大的数字登入第 19 行。对于读书这一列, 上一周第 14 行中最长时间是 146, 而这一周第 9 行中的值是 224。由于 224 更大一些, 所以就把它登入第 19 行。

6. 为了了解每周花费在各项任务上的最少时间, 需要计算最小值。通过比较第 15 行和第 9 行相同各列的数字, 就能计算出要登在第 20 行的最短时间, 并把这个较小的数登入第 20 行。对于读书这列, 第 15 行中以前的最短记录是 146, 第 9 行中上一周的值是 224。146 更小一些, 所以在第 20 行中登入 146。现在第 20 行就给出了以前各周每项任务所花费的最短时间。注意, 在计算最短时间时, 应当不考虑 0 值。

7. 注意, 除了第一周以外, 以前各周花费在各项任务上的时间的最大值或最小值, 通常与不包括上一周的时间累计花费在各项任务的最大值或最小值并不相等。

准备考试这一列给出一个示例, 它说明在计算最少时间时为什么要忽略 0 值。由于这周没有在这项活动上花费时间, 所以平均时间是 67 分钟。也就是说学生 Y 在一周用了 134 分钟, 而在另一周用了 0 分钟, 平均为 67 分钟。最大值正如以前计算的那样, 但是最小值 0 不是一个有用的数字。因为已经知道在一学期中大部分的周不会花费时间准备考试。把 0 作为最小值没有增加任何新的信息。然而有趣的是, 当确实要复习准备考试时, 就可以知道要花费的最短时间是 134 分钟。由于做这项任务时想知道的只是完成这项任务所要用的最短时间, 所以只有用第 9 行中较小的非 0 值代替第 15 行中的值才有意义。

表 4.5 和 4.6 给出了周活动总结表的使用指南。

表 4.5

周活动总结表使用指南

(1) 周时间总结表

目的	本表用来跟踪和分析本课程的时间使用情况。这些数据应该从时间记录日志中总结而得
一般性说明	在每周末从时间记录日志中总结数据。如果任务分类不合适, 请变更。如果没有足够的列用于记录所有的任务, 请增加一个表格
表头	登入: 名字和今天的日期
任务	登入本课程使用了时间的各项主要任务的名字
示例	听课, 编写程序, 准备考试, 阅读课本, 等

续表

日期	登入星期日的日期
示例	9/8
日总计	从时间记录日志中得到该周每一天花费在每个任务上的总时间
示例	周一, 9/9, 在时间记录日志中, 编写程序用了 38 和 58 分钟。因此, 在第三行(周一, 9/9)的编写程序一栏登入其和(即 96 分钟)
周总计(第9行)	在第 9 行登入整个星期每个任务的时间总数
示例	在周总计行的编写程序一栏登入 $96+62+69+114=341$ min
总计	在最右边的一行, 登入每天花费在各项任务上的时间之和。如果使用了多个表格, 把所有的此行的数据汇总在第 1 页
示例	周一, 9/9 总计为 $50+96+80=226$ min
第一次检查	计算本周所有花费在各项活动上的总时间, 即将最右列中每天活动总时间相加
示例	本周总计为 $226+62+147+114+88+134=771$ min
最后的检查	将第 9 行各项任务的总时间相加, 得到 $150+341+134+146=771$, 此数等于最右列的总计数(见上面的示例), 所以这个总计是正确的

表 4.6

周活动总结表使用指南

(2) 阶段时间段和效率总结

目的	本表用来跟踪和分析本课程的时间使用情况。这些数据应该从时间记录日志中总结而得
一般性说明	此表为上一个周活动总结表使用指南(1)的续表。表头及其它信息见上表
周数(第10行)	本表总结的数据。所包含的总周数。参阅上一次周活动总结表的周数(见该表第 10 行), 然后加 1, 就得到本次周活动总结表的周数
累计时间 (17~20行)	从上一次周活动总结表中的 17~20 行拷贝来每行的数据。包括每一列及其总计列的总计值、平均值、最大值、最小值
当前时间值 (第17行)	分别把第 9 行和第 12 行的数据加在一起, 登入第 17 行的每列
示例	在表 4.4 中, 第 17 行编写程序的计算为: $339+341=680$
当前时间值 (第18行)	将本表第 17 行中各列之总计值除以第 10 行中给出的周数
示例	在表 4.4 中, 第 17 行编写程序的平均值为: $680/2=340$
当前时间值 (第19行)	此行的数据是第 14 行和第 9 行比较后得出的, 把两行中较大的值登入第 19 行。分别计算第 19 行每一列的最大值
示例	在表 4.4 中, 编写程序最大值为 341, 而本周为 339, 所以这个最小值为 341
另一个示例	在表 4.4 中, 阅读课本最大值为 146, 而本周为 224, 所以这个最大值为 224

续表

当前时间值 (第 20 行)	此行的数据是第 15 行和第 9 行比较后得出的, 把两行中较小的非零值登入第 20 行。分别计算第 20 行每一列的最小值
示例 1	在表 4.4 中, 编写程序的最小值为 341, 而本周为 339, 所以这个最小值为 339
示例 2	在表 4.4 中, 阅读课本最小值为 146, 而本周为 224, 所以这个最小值为 146
示例 3	在表 4.4 中, 复习考试的最小值为 134, 而本周为 0, 所以这个最小值为 134

4.5 使用周活动总结表

通过每周用这种方法完成表格, 可获得每周花费在各个活动上的平均时间、最长时间和最短时间的记录。虽然这种表格看起来有些复杂, 但是经过一点练习就显得很清楚。然后就会发现这是一种组织和保存大量时间数据的好方法。如果使用电子表格计算那些数据, 则将更加简单。

在周活动总结表中的数据可以帮助你了解时间都用在哪些地方。例如, 有了这些数据, 就能判断出一个人的任务所需要的时间可能接近总结表中的最长时间, 而一个简单的任务所需要的时间可能接近最短时间。这样, 就可以使用这些数据对以后的几周进行计划。随后的几章将进一步讨论计划的问题。

开始你会从周活动总结表中获得大量的信息。但是很快每周你能得到的新信息就相对少了。当达到这一步时, 每个月准备一次周活动总结表来检查当前的时间分配就足够了。另外, 在第 5 章之后开始收集项目数据时, 就会知道你在项目上花费的时间, 并且当需要时可以很容易地在周活动总结表中总结这些数据。

经过一段时间的实践和不断的改进, 现在我使用月活动总结表来跟踪我在两个主要活动中花费的时间: 写书和做其它项目。由于我只能在业余时间写书, 上班时间做项目, 了解每周或每月我有多少可以利用的业余时间和做项目的时间非常重要。这会帮助我制订一些准确的计划来满足我承担的任务。为了这个目的, 按月进行跟踪就足够了。不管怎样, 每月对时间进行总结确实对我很有帮助。

4.6 总 结

本章指明阶段计划和产品计划的关系, 同时还介绍阶段计划。进行阶段计划的第一步就是要了解自己的时间数据。从时间记录日志中基本的时间数据开始, 每周完成一份周活动总结表。周活动总结记录了本学期各个活动所用的时间以及每周各个活动所用的平均时间、最长时间和最短时间。最终你可不用每周都作总结, 只需在时间分配有变化时再进行总结, 最后会发现每月总结一次就足够了。

4.7 练习 4

分别完成并提交前两周的周活动总结表，使用时间日志中的数据来填写这两周的周活动总结表。同时，提交一份你以前没有提交的那些时间日志表。

第5章 产品计划

本章描述如何利用时间日志中的数据制订产品计划。本章还介绍一种新的表格，即作业编号日志，以帮助你跟踪产品的历史数据，并且用一个示例告诉你如何完成这种表格。在本章的练习中，将利用本课程开设以来获得的产品数据完成作业编号日志。

5.1 产品计划的必要性

几年前我在 IBM 工作的时候，公司让我主管一个大的软件开发部门，它包括很多项目。那时这个部门的大部分项目都严重延误工期，公司的高层经理非常着急。我的任务就是整顿这种混乱的局面。首先，我检查主要的项目。令我吃惊的是，这些项目都没有任何书面的计划。我立即坚持要求工程师订出所有的项目计划。由于他们以前从来没有写过正式的计划，所以花费了几个月才做完这项工作。我们甚至还专门开设了项目计划课程。然而，在他们制订了计划之后，才能建立更符合实际工作的进度安排。

制订计划这一行动产生了显著的效果。这个开发组以前从来没有按时发布过产品，但是从制订新的计划开始后的两年半时间内，他们从未延误过一个订单。从此以后，他们对所有的工作都进行计划。作计划是软件工程师工作的一个重要部分，要想成为一名有才干的工程师，就必须知道如何制订计划。关键是实践，为了获得更多的实践机会，从现在开始就制订计划，并在以后的项目中继续这样做下去。

5.2 产品计划的用途

在这本书中，我建议对所有的项目或主要的任务都制订产品计划，如编写程序、阅读课本或是准备一份报告。产品计划帮助你判断某项工作将要花费多少时间以及什么时候能够完成。计划还可以帮助你跟踪工作的进展状况。

当工程师在开发组中工作时，就需要计划个人的工作。计划是按期完成承诺的任务的可

靠基础，而且计划也可以在工程师合作开发产品过程中协调他们的工作。个人的产品计划能使每个人互相约定各自的独立任务的期限，以便完全满足所承诺的各项任务。

在一些业务活动中，使用产品计划也是基于同样的原因：即计划和管理所做的工作。一份好的计划一般都包括项目费用的估计，这对于项目开发合同来说是必不可少的，因为客户需要提前知道产品的价格。进行产品开发也需要进行估计。项目费用是产品价格中很主要的部分，必须用尽可能低的价格使产品在市场中具有竞争力。

工程师也利用产品计划来了解项目的状态。有了足够详细而准确的计划，就能根据计划判断项目进行到了什么阶段：是否已经延期并需要帮助；还是必须把进度推后；也许提前了进度并能拿出时间来帮助同事工作，或者可以提前发布产品。在工程师制订了计划之后，他们能够更好地分配时间并且避免在交付产品前最后一分钟出现危机。这样，出错的可能性也减小了，并且通常能够开发出更好的产品。

正因为计划是如此重要，因此需要知道如何制订准确的计划，也需要知道如何将这些计划与实际结果相比较，从而学会制订更好的计划。

5.3 什么是产品计划

制订产品计划的第一步就是要有一份待开发产品的清晰定义。虽然这是显而易见的事情，然而令人吃惊的是，人们常常在为定义要做的事之前，便陷入产品开发的具体技术。只有在知道要做什么之后，才应该开始思考如何去做，此时就是开始制订计划的时候。

一份合适的产品计划包括以下三个方面：

- 待开发产品的规模及其重要的特性。
- 完成工作所需时间的估计。
- 项目进度计划。

产品可以是一个可执行的程序、一个程序设计方案或是一个测试计划。产品计划要说明所要开发的产品，它包含产品规模、工作时间和进度三方面的估计。更复杂的产品需要更加详细的计划以及许多其它方面的信息，如责任分工、人员配置计划、产品或过程的规格说明、与其它项目组的依赖关系，或者特殊的测试或质量保证条款等。但在这本书中，我们只讨论计划的三个基本方面：规模估计、项目时间和项目进度。

5.4 产品计划

制订产品计划是可以通过实践加以提高的一种技能。从现在开始练习制订计划，培养作计划的技能。这样，当需要对一个重要的项目进行计划时，你就知道如何去做。从这一章开始，应该为每一项主要的任务制订计划。然后，从第十章开始，为开发程序制订全面的计划。

在这本书中，从制订小任务的计划开始，这是学习如何制订计划的一种好的方法。总之，

如果不能作好小任务的计划,怎么可能作好大型项目的计划呢?在实际工作中,工程师完成了很多小任务,这些小任务是每一个大任务中的一部分。如果对每一个小任务都进行计划,就能获得更多机会来培养你的计划技能。对于较大的作业而言,你可以把很多小的计划合成一个较大的整体作业计划。事实证明,这是为较大任务制订准确计划的最有效的方法。

5.5 制订小型任务的计划

根据工作任务的规模和复杂程度制订一份合适的计划很重要。例如,为一个只用一两个小时的任务制订一份详细的计划是没有意义的。相反,对于一项要花费几周时间完成的任务来说,就需要制订一份具体的计划。

最基本的产品计划只包括对任务或作业所需时间的估计。一旦能够准确地估计出完成某项作业所需要的时间,通常其它计划问题解决起来就相当容易了。

通过收集以前不同任务所用时间的数据,就能够预测将来类似的任务大概所需要的时间。这种方法几乎适用于任何与产品相关的任务,无论这些任务需要几小时还是几天。关键的数据是以前做类似工作所用的时间。例如,如果计划阅读一章教材,了解阅读前几章所用的时间很有用。有了平均、最长和最短阅读时间的数据,就能更好地预测阅读新的一章可能花费的时间。

5.6 术语定义

在讲述下面的内容之前,需要定义以下一些术语。

- 一个产品是你为合作者、雇主或客户生产的物品。
- 一个项目通常生产一种产品。
- 一个任务是已定义的一部分工作。
- 一个过程定义了完成项目的方法。第11章将更全面地描述过程。
- 计划描述完成一个特定项目的方法,包括如何完成、何时能完成以及费用是多少?也可以为个人的任务制订计划。
- 一个作业是你所做的事情,一个项目或一项任务。

5.7 作业编号日志

表5.1所示的作业编号日志是为了记录估计的和实际的时间数据而设计的。应该注意,这个日志是一份产品计划文档,因为它记录的是有关产品的数据。相反的,时间记录日志和周活动总结表包含的是基于周的阶段数据,所以它们是阶段计划文档。

表 5.1

作业编号日志

学生 _____			日期 _____									
作业号	日 期	过 程	估计数据		实际数据			累计数据				
			时间	单元	时间	单元	平均值	时间	单元	平均值	最大值	最小值
		描述:										
		描述:										
		描述:										
		描述:										
		描述:										

表 5.2

作业编号日志及其示例

学生 学生 Y _____			日期 9/9/96 _____									
作业号	日 期	过 程	估计数据		实际数据			累计数据				
			时间	单元	时间	单元	平均值	时间	单元	平均值	最大值	最小值
1	9/9	编写程序	100	1	158	1	158	158	1	158	158	158
		描述: 编写程序 1										
2	9/9	阅读课本	50	2	80	2	40	80	2	40	40	40
		描述: 阅读课本的第 1 章和第 2 章										
3	9/11	编写程序	158	1	69	1	69	227	2	113.5	158	69
		描述: 编写程序 2										
4	9/11	阅读课本	40	1	28	1	28	108	3	36	40	28
		描述: 阅读课本的第 3 章										
5	9/12	编写程序	114	1	114	1	114	341	3	113.7	158	69
		描述: 编写程序 3										

续表

作业号	日期	过程	估计数据		实际数据			累计数据				
			时间	单元	时间	单元	平均值	时间	单元	平均值	最大值	最小值
6	9/13	阅读课本	60	1	118	1	118	226	4	56.5	118	28
	描述: 阅读课本的第4章											
7	9/16	编写程序	114	1	93	1	93	434	4	108.5	158	69
	描述: 编写程序4											
8	9/17	编写程序	109	1	95	1	95	529	5	105.8	158	69
	描述: 编写程序5											
9	9/18	阅读课本	57	1	71	1	71	297	5	59.4	118	28
	描述: 阅读课本的第5章											
10	9/19	编写程序	106	1	151	1	151	680	6	113.3	158	69
	描述: 编写程序6											
11	9/20	阅读课本	59	1	40	1	40	337	6	56.2	118	28
	描述: 阅读课本的第6章											
12	9/21	阅读课本	56	1	33	1	33	370	7	52.9	118	28
	描述: 阅读课本的第7章											

表 5.3

时间记录日志的示例

学生 学生 Y 日期 9/9/96
 教员 先生 Z 课程 CS1

日期	开始时间	结束时间	中断时间	净时间	作业号	备 注	C	U
9/9	9:00	9:50		50	听课	讲座		
	12:40	1:18		38	1	练习 1		
	2:45	3:53	10	58	1	练习 1		
	6:25	7:45		80	2	阅读课本第1章和第2章	X	2
9/10	11:06	12:19	6+5	62	1	练习 1, 休息, 聊天	X	1
9/11	9:00	9:50		50	听课	讲座		
	1:15	2:35	3+8	69	3	练习 2, 休息, 电话	X	1
	4:18	5:11	25	28	4	阅读课本第3章, 聊天	X	1
9/12	6:42	9:04	10+6+12	114	5	练习 3	X	1
9/13	9:00	9:50		50	听课	讲座		
	12:38	1:16		38	6	阅读课本第4章		

续表

日期	开始时间	结束时间	中断时间	净时间	作业号	备 注	C	U
9/14	9:15	11:59	5+3+22	134	复习	准备考试, 休息, 电话		
9/16	9:00	9:50		50	听课	讲座		
	2:10	4:06	4+19	93	7	练习 4, 休息, 电话	X	1
	7:18	8:49	11	80	6	阅读课本第 4 章, 聊天	X	1
9/17	9:26	11:27	4+22	95	8	练习 5, 休息, 电话	X	1
9/18	9:00	9:50		500	听课	讲座		
	4:21	5:43	11	71	9	阅读课本第 5 章, 休息	X	1
9/19	6:51	9:21	51+16+6	77	10	练习 6		
9/20	9:00	9:50		50	听课	讲座		
	12:33	1:18	5	40	11	阅读课本第 6 章, 休息	X	1
	1:24	2:38		74	10	练习 6	X	1
9/21	11:18	11:51		33	12	阅读课本第 7 章		

表 5.2 是一个已经完成的作业编号日志的示例。这个示例使用的数据来自表 5.3 中学生 Y 的时间记录日志。表 5.4 给出了作业编号日志的填写说明。应注意, 学生 Y 在日志中跟踪了两个一般的活动: 听课和复习考试。她还跟踪了两个类似项目的活动: 编写程序和阅读课本。按照如下的填写说明完成表 5.2 中的各个条目:

1. 作业号 在计划一个活动时, 为它分配一个作业号, 从 1 开始顺序进行。
2. 日期 登入开始做这项作业的时间。
3. 过程 登入任务的类型。比如说阅读课本, 编写程序或准备报告。
4. 估计时间 估计这项任务所要用的时间, 并把它登入估计时间这一栏。进行估计时, 查看以前类似项目的有关数据并使用这些数据估计。注意在估计程序 6 (作业 10) 时, 学生 Y 使用了程序 1~5 的平均时间。她通过查看前一个程序 (作业 8) 的累计平均值这一栏, 发现编写每个程序的平均时间是 105.8 分钟。
5. 估计单元 对于只有一个单元的作业, 在估计单元这栏中登入 1。下一章将介绍更精确的规模测量方法。
6. 实际时间 在某项作业完成时, 登入所用的实际时间。对于作业 10 来说, 这个时间是 151 分钟。
7. 实际单元 在作业完成之后, 记录实际的作业单元数。这里, 每项作业都是 1。下一章, 作业单元将使用产品规模测量单位。
8. 实际平均值 实际平均值是由实际时间除以实际单元得到的。程序 6 (作业 10) 的实际效率是 $151/1=151$ 分钟/每个程序。

9. 累计时间 在某项作业完成时, 计算到这时为止完成同类作业所用的累计时间。例如, 程序 6 所用的实际时间是 151 分钟, 从程序 1 到程序 5 所用的累计时间是 529 分钟(见作业 8), 那么到程序 6 为止, 编程所用的累计时间是 $151+529=680$ 分钟。
10. 累计单元 把到目前为止完成同类任务的累计单元数登入这一栏。又如程序 6, 把到它前一个同类任务(也是作业 8)为止的累计单元个数加上这项作业的实际单元: 即 $5+1=6$ 。
11. 累计平均值 累计平均值就是用累计单元数去除累计时间总数, 如 $680/6=113.3$ 分钟/每个程序, 这是做这类作业所用的平均时间。
12. 最大值 为了找出同类任务中每单元的最长时间, 将最近这项任务的实际时间与前面同类作业中的最长时间相比, 将较大的数值登入该栏。例如, 对于程序 6 来说, 到程序 5(作业 8)为止每单元的最长时间是 158 分钟, 它比程序 6 的每单元 151 分钟要大, 所以在程序 6 的最大值那一栏登入 158。
13. 最小值 最小值是同类任务所用的每单元的最短时间。要找出这个数值, 需要将目前作业的实际时间与前面同类作业中的最短时间相比, 将较小的数值登入该栏。例如, 对于程序 6 的实际时间为每单元 151min, 比到程序 5(作业 8)为止每单元的最小值时间 69min 要大, 所以到程序 6 为止的最小时间仍然是 69min。

有了这些数据, 就能很容易地查看任何给定类型任务的平均值、最大值和最小值。作业编号日志的主要优点是它为记录和访问大量的历史项目数据提供了一种简洁的方法。从中可以感到, 这是进行准确估计的关键; 而准确的估计又是作好计划的关键。

表 5.4 作业编号日志使用说明

目的	跟踪每个项目作业的数量。记录每个项目的关键信息。项目可以是你希望进行跟踪的任何活动, 例如开发一个程序或写一份报告
一般性说明	当开始一个项目时, 在这个表格中登入新作业的编号。从 1 开始顺序编号
表头	登入名字 登入开始使用此作业编号日志真的日期
作业号#	登入所选择的作业号
日期	登入开始这项作业的日期
过程	登入任务的类型。例如, 对于撰写技术论文, 就登论文; 对于开发程序, 就登编写程序
估计时间	登入进行这项作业估计花费的总时间(min)。使用累计平均值、最大值、最小值作为参考。如果这些值看起来不合理, 按照自己的判断进行估计
估计单元	登入估计使用的作业单元数。如果完成一项任务, 就登入 1
实际时间	登入完成这项作业实际所用的总时间

续表

实际单元	登入完成这项作业的实际单元数。如果完成一项任务，就登入 1
实际平均值	实际时间/实际单元
累计时间	找到最近开发的上一个同类型的作业，把它的累计时间值加上当前作业的实际时间值，就得到当前作业的累计时间值
累计单元	找到最近开发的上一个同类型的作业，把它的累计单元值加上当前作业的实际单元值，就得到当前作业的累计单元值
累计平均值	用累计时间除以累计单元就得到累计平均值。累计平均值代表到目前为止完成该类作业的平均值
最大值	登入已经完成的同类作业的最大时间
最小值	登入已经完成的同类作业的最小时间
描 述	登入要完成的作业描述。描述应足够清楚，以便易于说明当前作业的内容。在第一次记录作业类型的数据时，应写明测量单位

5.8 关于使用作业编号日志的几点建议

下面的实践应能帮助你有效地使用作业编号日志：

在第一次做某种给定类型的作业时，学生 Y 没有相关的历史数据来指导她作出估计。那么她只好猜测。第一次的时候猜测是可以的，只要已经开始收集数据，以后就不必再继续猜测了。

通常，在为一项新的作业估计时间时，会使用最近的同类作业的累计的平均效率的数据。学生 Y 一般就是这样做的。对于作业 6，学生 Y 并没有使用每章 36 分钟的平均读书时间。我推测，她认为阅读第 4 章所用的时间要比读前几章的平均时间要长。结果她是对的，但是她所用的时间比预计的时间更长。记住，作业编号日志中的数据是用来帮助制订计划的。在实际估计时应该参考这些数据。但是，如果认为采用再长些或再短些的时间估计更准确，就使用自己的判断。

为了迅速找出给定编号的某项作业的全部记录，在时间记录日志的活动那一列中加上作业号是有用的，如表 5.3 所示。实际上，最方便的办法是按活动类型组织记录，这样只需要通过引用活动编号就可以了。从下章开始，我们将采用这种方法。

在学会使用作业编号日志后，你很可能发现使用电子表格进行计算更方便。本书在提供的支持材料中给出了一个有关电子表格的示例。在本书的最后一页给出获得这些支持材料的指南。

5.9 使用产品的时间和效率数据

假设有表 5.2 中的数据，希望对下一周的时间进行计划。例如，可能计划编写两个程

序,读四章书。也可能还知道这两个程序中有一个比以前做过的任何一个程序都复杂,而另一个为中等难度。

为了估计编写这两个程序所要用的时间,查看一下表 5.2 中作业 10 的数据。这里记录着编写前六个程序的平均时间是 113.3 分钟,也就是将近 2 小时。最长时间是 158 分钟,也就是大约两个半小时。那么那个中等难度的程序大约可能会用 2 小时,这个比较复杂的程序至少要用 158 分钟甚至更长的时间。为了保险起见,假定这个复杂的程序要用 3.5 个小时即 210 分钟。那么下一周编程的时间预计一共需要 330 分钟。

对于这几章书来说,估计其中两章用的时间和平均时间差不多,而另两章要用稍多点时间。从作业 12 的数据中可以看出,阅读每章所用的平均时间是 52.9 分钟。因此可以假定阅读中等难度的两章时每章各用一个小时;对于较为复杂的两章,可能所用的时间和最长时间(118 分钟)差不多,大约也就是各用 2 小时。

现在,你预计下周一共要用 5.5 小时编写程序,6 个小时阅读课本。有了这些信息,就能更好计划你的工作。

你很快会发现,小任务所用的时间很接近估计值,这是正常的。然而,好的估计方法的关键就是要作出均衡估计。均衡估计是指在—组估计中,高估和低估的数目基本是相同的。在表 5.2 的示例的 11 个估计中,5 个估计低了,6 个估计高了,作业 5 正好合适。低估的作业编号是 1, 2, 6, 9 和 10; 高估的编号是 3, 4, 7, 8, 11 和 12。这样学生 Y 就做到了均衡估计。当她要估计一项新的作业所要用的时间时,通常要使用以前的平均数据。只要能不断地这么去做并且所做的作业基本相似,就能很快作出均衡估计。

均衡估计的优点是作业所用的时间平均起来将和估计的一样。也就是说,一项作业所用的时间比预计的多了,这部分时间将由其它作业节省的时间来补偿。当估计大作业时,把它的许多小作业估计收集在一起,那么所作的估计就很可能十分接近整体计划。

5.10 总 结

产品计划帮助总结和管理要生产的产品的时间数据,这些产品可以是编写的程序、写的报告、或者是阅读课本以及准备考试。作业编号日志是跟踪生产这些产品所用时间的一种工具。作业编号日志中用到的时间数据是来自于时间记录日志。

完成了作业编号日志后,它就提供了迄今为止完成每个项目需要的平均时间、最长时间和最短时间的数据。可以使用这些信息计划下周的工作或是设计以后几章所要求的更复杂的计划。

5.11 练 习 5

完成并提交一份到目前为止在这门课程中你所完成工作的作业编号日志。如果有估计过

的数据，把它们登入日志，否则就空着。使用时间记录日志完成这些表格。继续将每一个项目的记录登入作业编号日志。在开始每一项作业之前，先估计要用的时间和作业单元。在完成它时，记录实际的数据和到目前为止的累计数据。

第6章 产品规模

到目前为止，我们已经考虑过像程序和书的章这样较大的作业单元。但是为了制订产品计划，还需要使用更加精确的测量方法。本章将讲述如何测量和估计产品的规模。本章的练习是要求对前几章练习中所做的作业进行测量并且计算作业的生产效率。

6.1 产品计划过程

产品计划不是一个准确的过程。应该尽自己所能制订最好的计划，但同时也要看到，这些计划可能会不准确。作计划是一种技能，可以逐步地培养和提高。培养的途径就是为所做的每一项作业制订计划。然后，对于每一项已经完成的作业，将它的计划与实际的结果相比较。这样可以更清楚了解计划中的差错并且学会制订更好的计划。

在制订产品计划时，把计划要做的作业与以前做过的作业相比较。从类似作业的数据开始，把新的任务与以前的这些任务相比较。从你曾计划过的作业中找出相同规模的产品，基于这些作业所花费的时间估计新的作业要用的时间。

在这门课程里，收集作业的数据的主要原因是要了解不同作业所用的时间。例如，如果判断一项新的作业与以前某两项作业非常类似，而那两项作业各用了3小时和4.2小时，那么就会得出这样的结论：即这项新的作业用的时间与那两项作业的平均时间大约相同，也就是约3.6小时。这样得来的粗略估计会比没有历史数据的估计准确。

6.2 规模测量

由于各种任务的规模和复杂程度的差异很大，所以掌握一种比较它们规模的方法是很有帮助的。想一想阅读许多章节的书这项作业。有了阅读前五章所用时间的数据，就可以估计出阅读第6章要用的时间。一种方法就是使用阅读前几章的平均时间。这种方法当然比没有什么参考要好，但它没有区分章节的长短。假设，阅读较短的章所用的时间一般要比阅读较长

的章所用的时间少，那么，应该考虑用每页多少分钟而不是用每章多少分钟来测量阅读每章所用的时间。

表 6.1

学生 Y 章阅读时间表

学生	学生 Y	日期	9/30/96
教员	先生 Z	课程	CS1
章	阅读时间(min)	页数	分钟/页
1&2	80	20	4.00
3	28	12	2.33
4	118	16	7.38
5	71	17	4.18
6	40	12	3.33
总计	337	77	
平均	56.17	12.83	4.38

现在看一下表 6.1 中学生 Y 阅读课本各章所用时间的数据，其范围为 28~118 分钟，最长时间是最短时间的 4 倍多。当以分钟/页为单位来进行测量时，这个范围是从第 3 章的每页 2.33 分钟到第 4 章的每页 7.38 分钟。这个范围仍然很大，大概是 3:1。虽然可能还会有很大的变化，但是如果根据章的规模和以前的平均阅读速率（以分钟/页为单位）来进行估计，就有可能作出比较准确的估计。为了计算平均阅读速率，把所有的阅读时间加起来，除以总页数，即：

$$\begin{aligned}\text{平均速率} &= (80+28+118+71+40) / (20+12+16+17+12) = 337/77 \\ &= 4.38 \text{ min/页}\end{aligned}$$

6.3 使用规模测量的注意事项

虽然这种规模测量方法看起来很简单，但是其中确实存在一些复杂的因素。首先，有些文档读起来要比其它的文档困难得多。这意味着应该考虑工作所涉及的类型，而不只是它的规模。

另外，你大概遇到过这样的问题。例如，假设要准备考试，你读以前学过的章的时间比最初读它时要少。同样，即使是重新阅读自己写的文档，随着阅读目的不同，所用的时间也会变化很大。对刚起草的第一版文档，你读每页可能会用 15 或 20 分钟。但是，如果是快速浏览完整的文档，每页可能只用 1 分钟甚至更少的时间。

对程序开发计划，也存在同样的问题。不同类型的作业，譬如说重用以前开发的程序、

修改已经存在的程序或是开发新的程序，它们的生产效率是很不同的。为了处理这些问题，应该为不同类型的作业按其规模和时间分别记录它们的数据。

6.4 程序规模

当估计编写某个程序所用的时间时，根据以前编写类似程序所用的时间进行估计。如表 6.2 所示，学生 Y 编写程序所用的时间从最短的 69 分钟到最长的 158 分钟。虽然在这个范围中最长的时间只是最短时间的两倍多，但是当学生 Y 编写更大的程序时，这个范围还将加大。因此，基于程序的规模来估计时间是个好主意。

表 6.2 学生 Y 的程序开发时间

学生	学生 Y	日期	9/30/96
教员	先生 Z	课程	CS1
程序	开发时间(min)	代码行	分钟/代码行
1	158	20	7.90
2	69	11	6.27
3	114	14	8.14
4	93	10	9.30
5	95	14	6.79
6	151	18	8.39
总计	680	87	
平均	110.0	14.5	7.82

测量程序规模的方法是统计源程序有多少文本行。也就是说，如果打印的程序清单有 16 行，这个程序就有 16 个代码行 (Lines of Code, LOC)。在统计 LOC 时，约定不统计空行和注释行。如果一行既有注释又有程序语句就算一个代码行 (LOC)。虽然可以选择任何一种前后一致的标准，但本书约定：统计代码行时不统计空行和注释行。这样，下面 Ada 程序的片段就有 5LOC。

例 1:

```
-- comment describing the program's function
If (X_Average >= 100) then
    Size := X_Average;
else
    Size := X_Average/2;
end if;
```

类似地，如果不写注释并且使用更紧凑的格式重写这部分程序，它就只有 3LOC。

例 2:

```
If (X_Average >= 100) then
    Size := X_Average;
else Size := X_Average/2; end if;
```

虽然它们是同样的程序并且它们的开发时间可能也相同，但是用这个统计方法，它们的规模是不一样的。只要在编程时使用一致的格式，这些统计上的变化不会很大。但是为了确保规模统计的一致，建议采用标准的编程格式。虽然你应该遵循指导教师所要求的格式，我个人更愿意使用例 1 中那种展开的格式。

代码行的测量方法通常适用于大部分程序设计语言。例如，用 C++ 编写的相同功能的代码如下：

例 3:

```
// comment describing the program's function (描述程序的功能)
if (X_Average >= 100)
    Size = X_Average ;
else
    Size = X_Average / 2 ;
```

当使用这种书写格式时，这段 C++ 的代码有 4LOC。下面稍微大一点的 C++ 的代码段有 12 LOC：

例 4:

```
while ( n>0 )
{
    push (n);
    cout << "Enter a positive integer. \n" ;
    cout << "Enter 0 to stop. \n" ;
    cin >> n ;
}

// read out the stack (从堆栈读出)
while ( stack_pointer != NULL )
{
    cout.width (8) ;
    cout << pop () ;
}
```

使用这种代码行统计方法，表 6.2 给出了学生 Y 编写的 6 个程序的规模。编写一行代码用的时间从程序 2 的 6.27 分钟到程序 4 的 9.3 分钟不等。随着经验的增加，编写一行程序所用的时间将会有很大的变化，应该以分钟/代码行这个速率进行跟踪，并且以最近 5~10 个程序所获得的数据为依据进行估计。

6.5 其它的规模测量方法

尽管在本书中没有使用其它的规模测量方法，但存在很多规模测量方法。例如，工业化的软件开发通常包含文档，它们是以页为单位进行测量的。即使对程序来说，代码行测量方法也不能覆盖所有的情况。不适合用代码行统计的产品类型的示例有菜单、文件、报告或屏幕数量。除非能设计出一种合适的测量方法，否则就应该使用第5章中的单元和平均值来进行统计。

另外，如果是使用程序开发辅助工具来生成各种类型的屏幕、窗口或其它标准程序元素，统计的方法就要稍微复杂一些。这里重要的一点是，只统计自己编写的代码行，而不统计由程序开发辅助工具生成的代码行。

无论使用什么测量方法，主要目的是为了估计开发的工作量。基于这一点，需要使用与产品的工作量相关的规模测量方法。也就是说，对于花费时间较多的产品，应该有成比例增大的规模测量方法。¹

6.6 程序规模估计

虽然看起来现在已经可以估计编写一个程序的时间，但是实际上并不那么简单。对于阅读几章课本，只要统计出要读的页数并用过去阅读每页所用的平均时间，就能够计算出可能的阅读时间。甚至可以按阅读的难度把各章分级，对于复杂的或包含不太熟悉的材料的章，每页分配较多的时间去读。通过浏览新的一章，就能够估计出它的相对难度并根据以往的经验选取阅读每页的时间。

但是对于编程来说，在把它开发出来以前，是没有程序可供统计的。为了估计编程的时间，首先必须估计程序可能会有多少代码行，然后估计编写每行代码需要多少分钟。这样就能计算出总共需要的时间。

有很多种方法可以在开发程序之前估计出程序的规模，但是所有的规模测量方法都涉及大量的判断。首先要查看所开发程序的需求，然后把新程序按程序的规模等级与从前编写的程序排列在一起。最后，判断新程序的规模可能落于从前编写的程序的哪个规模范围，进而估计出可能的代码行。

表 6.3 给出了这个规程的一个示例。这是学生 Y 以前所编写的程序清单，按规模从小到大的次序排列。这个清单给出了以代码行表示的程序规模、以分钟表示的开发时间、以分钟/代码行为单位的效率以及程序功能的简单描述。通过查看表中的数据，并且考虑已知

¹ 关于规模测量与规模估计的完整讨论，参阅 Watts S. Humphrey 的另一本书 *A Discipline for Software Engineering* (软件工程规范)，Reading, MA: Addison-Wasley, 1995.

的有关要开发的程序的信息，就能够判断出新程序处于表中哪个规模等级，这将帮助你估计新程序的规模范围。根据过去每行代码用时间的历史数据，就能估计出开发新程序所要用的时间。

表 6.3

学生 Y 的程序规模排序清单

学生	学生 Y	日期	9/30/96	
教员	先生 Z	课程	CSI	
程序	时间	代码行	分钟/代码行	功能
4	93	10	9.30	简单的 While-loop
2	69	11	6.27	小的 Case 语句
3	114	14	8.14	大的 Case 语句
5	95	14	6.79	中等的 repeat-until
6	151	18	8.39	小的链表
1	158	20	7.90	小的计算

例如，假设学生 Y 计划编写一个含有中度复杂的 while 循环的新的程序。通过查看表 6.3 中的数据，她估计这个程序比程序 5（14 行代码）要大，比程序 1（20 行代码）要小。这样，她可能会取这两个的平均数，即 $(14+20)/2=17\text{LOC}$ 。

6.7 较大规模的估计

虽然表 6.3 所示的方法适合于以前编写过的那些类型的小程序，但是并不适合于较大的或是新的程序类型。原因就是即使是再小的程序也是由函数和过程构成的。对于较大的程序，很难把某个新的程序与以前开发的程序联系起来。

解决这个问题的一种方法就是使用如表 6.4 所示的一种表格。这里列出了不同的程序或程序函数和过程。有了大量的与程序有关的数据，甚至可以像学生 Y 在表 6.5 中那样把一份清单分成几类(循环 loop、选择 case、数据结构 data 和计算 calculation)。其它有用的类的示例有文本、控制、逻辑、显示和打印。目的就是建立一个历史记录，把以前编写的各个元素以及它们各自所包含的代码行的有关数据收集在一起。当考虑一个新程序的功能时，就可以估计每一个功能的规模，然后把所有这些功能的估计加在一起就得到了对整个程序的估计。

表 6.4

程序规模估计表

学生 _____ 日期 _____
 教员 _____ 课程 _____

程序	代码行	以前的功能	估计的功能	最小	平均	最大

注释: _____

表 6.5

学生 Y 的程序规模估计

学生 学生 Y 日期 10/7/96
 教员 先生 Z 课程 CS1

程序	代码行	以前的功能	估计的功能	最小	平均	最大
Loops						
4	10	简单的 while-loop 循环				
5	14	中等的 repeat-until 循环	Repeat until	7	11	14
Case						
2	11	小的 case 语句	Case	5	8	11
3	14	大的 case 语句				
Data						
6	18	小的链表				
Calc.						
1	20	小的计算	Calculation	10	15	20
估计				22	34	45

注释：此程序有一个简单的选择语句、一个循环和一个计算。假定该程序的最大值规模值将是这些典型功能的规模值之和，即 $11+14+20 = 45\text{LOC}$ 。假定这些功能在一个程序中构造比单独构造的效率要高，取 $\text{LOC}=22$ 作为该程序的最小规模值。最后取这两个端点的中点 $\text{LOC}=34$ 作为该程序的平均规模值。

对于小程序（如表 6.5 列出每一个程序）来说，随着程序规模的增大，它们可能会包含很多函数和过程。把每一个函数和过程都分别列出来，就能快速建立起历史估计数据的基础。有了更多的数据之后，可以把每一类程序用不同的表格保存（如表 6.4 所示）。可以用一份表格保存计算，一份保存文本处理过程，一份保存控制等。在每一份表格中，把程序的各个功能按规模大小排列，并且列出包括这些功能的程序的名称和数量。

表 6.5 给出了如何使用这种方法进行规模估计的示例。这里，学生 Y 已经收集了各类程序的一些数据。但是由于这些程序都很小，她仅列出这些程序的总规模。在估计新程序可能包含的代码行数时，她就判断新程序可能需要的各种类型的功能有多少行代码。

学生 Y 作估计时，首先查看新程序的需求并且建立如何构造此程序的一般策略。她希望使用一个 repeat-until 循环和一个简单的 case 语句，还希望作一个相当简单的计算。但是在作这些估计时，她不能完全肯定各个程序的规模，所以对每一个功能都估计出一个最小规模、一个最大规模和一个平均规模。虽然作估计时通常会用平均规模的数据，但考虑最大和最小规模有助于避免将规模估计的过大或过小。

虽然这并不是一个十分可靠的规模估计方法，但是它比猜测要好得多。随着编写的程序越来越多，你也可以把这些程序加入这个程序清单。这样，逐渐增加的历史数据将帮助你作出更好的估计。

没有任何方法可以保证估计的结果一定准确。规模估计是一种技能。作出好的规模估计的关键是要有大量的历史数据，要进行多次规模估计，并且要定期的将实际结果与估计值作比较。

表 6.6

时间记录日志的示例

学生	学生 Y					日期	9/9/96		
教员	Z 先生					课程	CSI		
日 期	开始时间	结束时间	中断时间	净时间	作业号	备 注	C	U	
9/9	9:00	9:50		50	听课	讲座			
	12:40	1:18		38	1	练习 1			
	2:45	3:53	10	58	1	练习 1			
	6:25	7:45		80	2	阅读课本第 1 章和第 2 章	X	20	
9/10	11:06	12:19	6+5	62	1	练习 1, 休息, 聊天	X	20	
9/11	9:00	9:50		50	听课	讲座			
	1:15	2:35	3+8	69	3	练习 2, 休息, 电话	X	11	
	4:18	5:11	25	28	4	阅读课本第 3 章, 聊天	X	12	

续表

日期	开始时间	结束时间	中断时间	净时间	作业号	备 注	C	U
9/12	6:42	9:04	10+6+12	114	5	练习 3	X	14
9/13	9:00	9:50		50	听课	讲座		
	12:38	1:16		38	6	阅读课本第 4 章		
9/14	9:15	11:59	5+3+22	134	复习	准备考试, 休息, 电话		
9/16	9:00	9:50		50	听课	讲座		
	2:10	4:06	4+19	93	7	练习 4, 休息, 电话	X	10
	7:18	8:49	11	80	6	阅读课本第 4 章, 聊天	X	16
9/17	9:26	11:27	4+22	95	8	练习 5, 休息, 电话	X	14
9/18	9:00	9:50		50	听课	讲座		
	4:21	5:43	11	71	9	阅读课本第 5 章, 休息	X	17
9/19	6:51	9:21	51+16+6	77	10	练习 6		
9/20	9:00	9:50		50	听课	讲座		
	12:33	1:18	5	40	11	阅读课本第 6 章, 休息	X	12
	1:24	2:38		74	10	练习 6	X	18
9/21	11:18	11:51		33	12	阅读课本第 7 章		

表 6.7

作业编号日志及其示例

学生 学生 Y

日期 9/9/96

作业号	日 期	过 程	估计数据		实际数据			累计数据				
			时间	单元	时间	单元	平均值	时间	单元	平均值	最大	最小
1	9/9	编写程序	100		158	20	7.90	158	20	7.90	7.90	7.90
	描述: 编写程序 1(分钟/LOC)											
2	9/9	阅读课本	50	20	80	20	4.00	80	20	4.00	4.00	4.00
	描述: 阅读课本的第 1 章和第 2 章(分钟/页)											
3	9/11	编写程序	158		69	11	6.27	227	31	7.32	7.90	6.27
	描述: 编写程序 2											
4	9/11	阅读课本	40	12	28	12	2.33	108	32	3.38	4.00	2.33
	描述: 阅读课本的第 3 章											
5	9/12	编写程序	114		114	14	8.14	341	45	7.58	8.14	6.27
	描述: 编写程序 3											

续表

作业号	日期	过程	估计数据		实际数据			累计数据				
			时间	单元	时间	单元	平均值	时间	单元	平均值	最大	最小
6	9/13	阅读课本	60	16	118	16	7.38	226	48	4.71	7.38	2.33
		描述: 阅读课本的第4章										
7	9/16	编写程序	114		93	10	9.30	434	55	7.89	9.30	6.27
		描述: 编写程序4										
8	9/17	编写程序	109		95	14	6.79	529	69	7.67	9.30	6.27
		描述: 写程序5										
9	9/18	阅读课本	57	17	71	17	4.18	297	65	4.57	7.38	2.33
		描述: 阅读课本的第5章										
10	9/19	编写程序	106		151	18	8.39	680	87	7.82	9.30	6.27
		描述: 写程序6										
11	9/20	阅读课本	59	12	40	12	3.33	337	77	4.38	7.38	2.33
		描述: 阅读课本的第6章										
12	9/21	阅读课本	56									
		描述: 阅读课本的第7章										

表 6.8

作业编号日志使用说明

目的	本表用于跟踪每个项目的作业号, 也记录每个项目的关键信息。项目可以是任何你希望进行跟踪的活动, 例如开发一个程序、读一本书或写一篇文章
一般性说明	当开始一个项目时, 在这个表格中登入新作业的编号。从1开始顺序编号
表头	登入名字 登入开始使用此作业编号日志页的日期
作业号#	登入所选择的作业号
日期	登入开始这项作业的日期
过程	登入任务的类型。例如, 对于撰写技术论文, 就登论文; 对于程序开发, 就登编写程序
估计时间	登入做这项作业估计花费的总时间(min)。使用累计效率、最大值、最小值作为参考。如果这些值看起来不合理, 按照自己的判断进行估计
估计单元	登入完成作业估计的单元数。例如, 对一个程序开发, 估计预期完成的代码行的数量
实际时间	登入完成这项作业实际所用的总时间
实际单元	登入实际完成作业单元数。例如, 对一个程序来说, 计算所完成的程序的代码行数

续表

实际平均值	实际时间/实际单元
累计时间	找到最近开发的上一个同类型的作业，把它的累计时间值加上当前作业的实际时间，就得到当前作业的累计时间值
累计单元	找到最近开发的上一个同类型的作业，把它的累计单元值加上当前作业的实际单元值，就得到当前作业的累计单元值
累计平均值	用累计时间除以累计单元就得到累计平均值。累计平均值代表到目前为止完成该类作业的平均时间
最大值	登入已经完成的同类作业的最大时间
最小值	登入已经完成的同类作业的最小时间
描述	登入要完成的作业描述。应足够清楚，以便于说明当前作业的内容。在第一次记录作业类型的数据时，应写明测量单位

6.8 在作业编号日志中使用规模测量的方法

当使用规模测量而不是单元统计时，应该如表 6.6 和表 6.7 那样，把规模数据保存在时间记录日志和作业编号日志中。这些示例使用了表 6.1 和表 6.2 中的程序规模数据。这里不仅仅要标记读完一、两章或写完一、两个程序，而且要把它们的规模数据登入单元这一列。如表 6.6 所示，学生 Y 在 9 月 10 日完成了一个 20 行的程序，所以她就 U（单元）这一列登入 20。同样，在 9 月 11 日，她读了第 3 章，这章有 12 页书，用了 28 分钟，就把 12 登入表 6.6 中单元这一列，并把 28 登入时间这一列。

由于已经把规模数据登入了作业编号日志，所以再把这些数据登入时间日志可能显得多余。但是，依照我的经验，如果通常每周填写一次或两次作业编号日志，那时就必须查找有关作业规模的数据。然而在做这项作业的同时，将规模数据登入时间日志则通常不会花费多少时间。表 6.8 给出了作业编号日志使用说明。

在表 6.7 中，学生 Y 使用这些规模数据计算产品的生产效率。她按照表 6.8 中作业编号日志的说明来完成这项任务。例如，她没有对作业 1 进行规模估计，所以她就估计的作业单元这一栏空着。实际的作业时间还是 158 分钟，但是实际的作业单元这一栏填的是 20 行代码。这意味着累计的作业单元也是 20 行代码，平均效率是 $159/20=7.90$ 分钟/代码行（LOC）。由于这是第一个程序，所以最大和最小值也都是 7.90。

对于第二个程序，即作业 3，学生 Y 也没有对它作规模估计，所以估计的作业单元这一栏仍然是空白。编写程序 2 所用的实际时间是 69 分钟，程序 2 的实际作业单元是 11 行代码。现在给出的累计作业时间和前面算法一样是 227 分钟，而累计作业单元是 $20+11=31$ 行代码。学生获得这个数字是通过查看同类任务中最近前一项（作业 1）的累计作业单元，再把这项作业的实际作业单元 11 行代码加上以前累计的 20 行代码。现在知道累计代码行是 31 和累计时间是 227 分钟，累计平均值就是 $227/31=7.32\text{min}/\text{代码行}$ 。再看作业 1 的最大值是 7.90，

它比作业 3 的 6.27 分钟/代码行的值要大,所以最大值仍然是 7.90。作业 1 的最小值也是 7.90,但是作业 3 的实际值是 6.27,它比 7.90 要小,所以到目前为止的最小值就被当前值 6.27 所代替。

随后的每个程序都遵循同样的步骤,直到把到目前为止所编写的所有程序的有关数据全部登入作业编号日志中。以后,在编程之前可以先估计每个程序有多少行代码,并把这个数字登入表中。虽然这费点事,但是只要有过去的相关数据,就能够相对容易地遵循 6.6 节描述的过程进行估计。对于阅读课本来说,除了要统计所读课本的页数估计作业单元以外,这个过程是一样的。

通过使用这些数据,就能知道编写程序和阅读课本需要多少时间。对于其它作过规模和时间测量的任务也是一样的。

6.9 总 结

本章介绍了产品规模测量,并且说明如何在产品计划过程中使用规模测量。

介绍规模测量方法是为了帮助估计产品规模。本章讲述了如何从代码行来确定程序的规模。工业化的软件项目常常需要其它的规模测量,如菜单、文件、屏幕和报告。

产品计划的第一步是要估计产品的规模。为了作出准确的估计,需要用到以前的规模数据。把以前的规模数据按照功能分类是有帮助的。这样,就可以估计出新程序中各类代码会有多少行。随着所积累的数据越来越多,作出的估计就会越来越准确。作业编号日志为记录大量的历史的规模和效率数据提供了一种简便的方法。

6.10 练 习 6

复习到日前为止所完成的作业,以代码行和页为单位统计它们的规模。按照表 6.1 的格式呈交有关对于阅读本章练习的数据,按照表 6.3 的格式呈交有关编程的数据。此外,呈交一份更新过的以“分钟/代码行”和以“分钟/页”为单位的周活动总结表和作业编号日志。如果还没有完成所有的程序,至少要交一份最近的三个程序作业的有关数据。并且,交一份以前没有交过的时间记录日志、作业编号日志以及周活动总结表。从现在起,呈交所有日志和总结时,在适当的地方都要带有以“min/代码行”和以“min/页”为单位的数据。

第 7 章 管理好时间

本章描述时间安排表，并介绍如何使用这种表。本章还解释如何作时间安排，并给出一些满足这种安排的建议。在本章的练习中，要求制订一个时间安排表。

7.1 时间管理的要素

迄今为止，这门课程已经测量了几周内使用时间的方​​式。现在，已经有了一定数量的数据，并且对不同任务所要花费多少时间有了较好的了解。获得这些知识并不困难，而且，现在你可以看到这些数据对于制订今后的计划有怎样的帮助。本章将使用这些数据按照如下的步骤来管理时间：

1. 决定如何使用时间。
2. 制订时间安排表。
3. 对照制订的安排表跟踪使用时间的方​​式。
4. 决定应该改变什么以使自己的行动达到所作安排的要求。

随后的几节将讨论这些主题。

7.2 活动分类

复查时间的分类情况，了解这些类别是否包含了所有主要的活动。如果没有，补充缺少的类别或修订已有的类别。例如，你可能发现在这门课程中用了大量的时间分析和使用自己收集的数据，因此，你可能希望把数据分析这类活动加入周活动总结。

尽管现在对于每周任务所用的时间有了很好的了解，但是对于那种一个月、一学期或是一学年才偶尔出现一次的任务，目前还没有多少有关的数据，譬如准备考试或作学期规划。当然，除非以前做过这些项目，否则无法收集到相关的数据，但是尽可能现在标识出这些项目的类别，以便在计划中为这类项目列上一些条款。

7.3 收集活动的时间数据

周活动总结表给出了每周用在各个活动上的平均时间、最大时间和最小时间。一个好办法就是现在检查一下这些活动的分类，看看是否有些类别包含的范围过大了，而另一些又是否分得过细。不均衡的分类可能会导致某类活动会占用 50% 或者更多的时间，而其它类别的活动仅占用 5% 或者更少的时间。

为了对时间进行管理，必须把重点放在那些占用大部分时间的少数几项活动。这就需要详细了解所做工作的内容。如果 25% 的时间用在标有“其它”这类活动上，就应该把它再分成几个定义更加明确的活动。这些“其它”类型活动如果没有更精确的数据，就很难控制在这些活动上所用的时间。

7.4 时间分配的评价

既然可以知道时间都用在了什么地方，就可问一问自己是否正在按照所希望的方式使用时间。明确哪些活动是最重要的，并且考虑是否给这些活动分配了足够的时间。是否有一些活动用了很长的时间以至于没有时间去做另外一些更重要的活动？是否分配了足够的时间来阅读课本？你是否有一个职业以及你个人的承诺是什么？你是否及时开始工作以完成分派给你的任务？或者经常在最后期限才紧张工作？这里没有如何使用时间的一般指南。你必须协调好学校工作、各项任务、娱乐和社会生活，但这纯属个人的决定。其中有些是个人问题，需要作出各种复杂的权衡，特别是当你有一个职业同时又承担家庭责任时情况更是如此。

7.5 作出时间安排

时间安排表是如何使用时间的计划。根据以前如何使用时间的数据，就能够分配以后各类活动所需要的时间。表 7.1 给出了学生 Y 初步制订的时间安排表。

表 7.1 周时间安排表示例

学生	学生 Y	日期	9/23/96
教员	先生 Z	课程	CS1
活动类别	计划时间 (分钟)	实际时间 (分钟)	
听课	150		
编写程序	360		

续表

活动类别	计划时间 (分钟)	实际时间 (分钟)
阅读课本	180	
复习准备考试	120	
其它	30	
总计	840	

表 7.2

学生 Y 的周活动总结表

姓名: 学生 Y

日期: 9/23/96

1	任务 星期	听课	编写 程序	准备 考试	阅读 课本				其它	日总计
2	日(9/15)									
3	一	50	93		80					223
4	二		95							95
5	三	50			71					121
6	四		77							77
7	五	50	74		40					164
8	六				33					33
9	周总计	150	339		224					713

10 周期时间和效率

周数 (上一周活动总结表的周数+1): 2

11 不包括上一周在内的累计时间

12	总计	150	341	134	146					771
13	平均	150	341	134	146					771
14	最大	150	341	134	146					771
15	最小	150	341	134	146					771

16 以前各周的累计时间

17	总计	300	680	134	370					1484
18	平均	150	340	67	185					742
19	最大	150	341	134	224					771
20	最小	150	339	134	146					713

表 7.2 是表 4.4 的拷贝，给出学生 Y 的周活动总结表。看一下第 18 行，将所作的时间安排与她实际使用的时间相比较。可以得出以下几点结论：

- 迄今为止，学生 Y 在这门课程上所用的总时间平均为每周 742 分钟，即每周 12.37 小时。在新的安排中，她计划每周用 840 分钟（即 14.0 小时），比原来的平均时间多出了约 1.5 小时。这是一个相当大的变化，她每周可能很难找出这些额外的时间。

- 学生 Y 安排给听课与她以前使用的时间一样，这是合理的。
- 她计划每周增加大约 20 分钟来编写程序，这或许也是合理的。
- 她计划每周用与以前平均时间大约相同的时间来阅读课本，这看起来是合理的。
- 主要时间增加在复习考试和其它类型的活动上。这两类活动每周所用的时间加起来共计 150 分钟（即 2.5 小时）。如此大的变化可能是不切合实际的。对学生 Y 来说，尽管可以把规划的总时间增加到迄今为止她在这门课程中所用的最大时间 771 分钟左右，但是期望做太多的工作是不切合实际的。

管理好时间的关键是逐步对使用时间的方式进行反复平衡。即使希望以后在某些任务上多用一些时间，但是除非能够在另外一些任务中少用一些时间，否则，这常常只能是一个愿望而已。重要的是要确保现在是否能够最有效地利用这些时间。人们经常把很多时间浪费在决定下一步打算做什么上。建立个人计划和时间安排将使你知道下一步该做什么。令人感到惊讶的是，这样做很快就能提高工作效率。

你可能需要更多的时间去做所有你想做的事情。但是，要实际一些。当面临危机时，通常能够找到一些额外的时间，但是没有人希望总是处于危急之中。所以应该为健康、家庭、朋友、业余爱好、睡觉和饮食合理地分配一定数量的时间。当然你可以在这些方面作出适当的调整，但只能逐步地进行。建议你为这一学年中要进行的工作建立优先级。当陷入时间危机时，可以削减掉一项或几项工作，但是不可能总这样做。如果必须在某些活动上多用一些时间，就必须从另外的活动中抽出这些时间。复查工作的优先等级，并且决定哪些工作是最重要的，然后制订相应的时间安排计划。

7.6 找出更多的时间

在复查了时间安排表之后，你可能需要增加时间的总量。该怎么去做呢？有这样几种选择。

如果进度不是很紧张的话，可以很容易找出一些额外的时间。遗憾的是，很少有人会有空闲的时间。

更多的情形是会发现自已承诺了太多的事情。在这种情况下，查看一下所有的委托书，然后，可能需要对用在所有的课程、主要的工作或者闲暇活动上的时间进行跟踪。

问题是一天只有 24 小时，你无法增加这一时间。如果需要在某些活动上多用一些时间，就必须从另一些活动中抽出一些时间。因此，只要知道各类活动要用多少时间，就可以对时间安排作出实际的调整。

7.7 制订基本规则

既然已经决定了想如何使用时间，就必须切实地按照预定的方式去做。这个简单的规则将会带来意想不到的收获。原因是时间安排表只是对想要做的事情的一般陈述。实际的时间管理需要对每一分钟该做什么作出决定。因此，为了切实按照预定的安排去做，就必须有非常具体的计划。

大概已经注意到了，管理听课所用的时间很容易，原因是听课是有规律、按照一定的时间间隔发生的。当人们有了遵循的具体规程时，他们通常能够按它去做。我们在做许多事情时都是按照别人制订的规则去做的。为了对时间进行有效的管理，也需要有规则可循。但是，必须自己制订这些规则。实际上，时间管理的安排就是为管理自己的时间而制订的规则。

为了给每天的活动提供有用的指导，需要作出每天的时间安排表。每日和每周所作的时间安排表可能不同，所以需要表 7.3 那样的格式。

表 7.3

每周活动预算

姓名 _____ 日期 _____

第 1 周的安排

任务 星期									日总计
日									
一									
二									
三									
四									
五									
六									
周总计									

第 2 周的安排

任务 星期									日总计
日									
一									

续表

任务 星期									日总计
二									
三									
四									
五									
六									
周总计									

第 3 周的安排

任务 星期									日总计
日									
一									
二									
三									
四									
五									
六									
周总计									

现在可能明白为什么表 7.1 给出的安排不是非常有用。例如，每周都花费时间准备考试是没有意义的。既不希望到考试的前一天才作准备，也不希望过早就开始复习，以防止到考试时忘掉复习的内容。

此外，表 7.1 中“其它”这一类活动是对未作计划的活动的考虑。尽管为预料不到的事情做一些考虑是个好主意，但是并没有方法为偶然发生的事情制订详细的计划。处理预料不到的事情的一种方法，就是在过去平均时间的基础上为各个活动多分配一点时间。加进来的这些富裕时间使你能够处理计划中的错误和一些预想不到的事情。

对于学生 Y 来说，一种更加切合实际的时间安排表如表 7.4 所示。

表 7.4

每周活动安排的示例

姓名 学生 Y

日期 9/23/96

第1周的安排 常规周安排

任务 星期	听课	编写 程序	准备 考试	阅读 课本	复习			其它	日总计
日									
一	50			40					90
二		120		40					160
三	50			40					90
四		120		40					160
五	50			40					90
六		120							120
周总计	150	360		200					710

第2周的安排 考试前一周的安排

任务 星期	听课	编写 程序	准备 考试	阅读 课本	复习			其它	日总计
日									
一	50		40						90
二		120	40						160
三	50		40						90
四		120	40						160
五	50		120						170
六			150						150
周总计	150	240	430						820

第3周的安排 考试周的安排

任务 星期	听课	编写 程序	准备 考试	阅读 课本	复习			其它	日总计
日									
一			300						300
二			450						450
三	170		150						320
四	50								50
五									
六									
周总计	220		900						1120

7.8 设定时间分配的优先级

管理好时间的一个基本步骤是建立优先次序。有些时间是固定不变的，如什么时候去听课或去做兼职工作，可以把这些时间称为固定时间。进行其它活动的时间就是可变动的时间，只要有时间就可以去做这些活动。但是不确定的活动又有两类：需要的和自行斟酌的。需要的活动包括做家庭作业、阅读课本或准备考试等。虽然它们是需要的活动，但它们的时间是可变动的，因为无论何时找出时间都可做这些事情，并且每周在这些活动上所用的时间是不同的。自行斟酌的活动就是要做的所有其它的那些事情：吃饭、睡觉、社交、参加或观看比赛以及其它的娱乐活动。

当作出全面的时间安排时，知道以前在各类活动上用了多少时间是有帮助的。固定活动的安排没有什么问题。最常见的问题就是分配可变动的的时间。例如，到呈交作业时，才发现没有足够的时间完成它了。这时，唯一的方法就是从自行斟酌的活动中抽出那些额外的时间。这就要十分了解固定的、必需的和自行斟酌的时间，以决定可以在什么地方作出调整。

一种能够更加有效地观察个人时间分配状况的工具就是表 7.5 给出的全面的时间总结。为了完成这份总结，至少要粗略地对各类活动所用的时间进行跟踪。尽管可以使用时间记录日志，但是最初的记录并不需要详细到那种程度。开始时，在一天结束后把各类活动所用的时间记录下来就足够了。如果能够肯定每天的时间加起来是 24 小时并且每周的总时间是 168 小时或 10080 分钟的话，总的时间可能就相当准确了。如果用这种方法不能得到足够详细的记录，那么就需要使用时间记录日志了。

表 7.5

每周时间总结表

学生	学生 Y	日期	9/23/96				
教员	先生 Z	课程	CS1				
活动	计算机	物理	数学	英语	吃饭/休息	其它	总计
固定的							
听课	150	150	100	100			500
需要的							
做作业	360	240	240	360			1200
阅读课本	240	240	180	60			720
自由的							
吃饭					1260		1260
睡觉					3150		3150
运动						600	600
娱乐						360	360
休息						2290	2290
总计	750	630	520	520	4410	3250	10080

有了表 7.5 给出的全面总结,就有了用来决定从哪儿抽取需要的补充时间的基本的数据。但是,还是需要实际一些。例如,减少睡觉和吃饭的时间有害于身体健康并且会降低工作效率,省去几顿饭或是通宵工作可能是克服短期危机的一种方法,但是要想从这些地方抽取大量的时间,一般来说不是个好办法。

还有另外一个因素对时间安排是否合理产生影响,即是否真愿意按照这时间安排来执行。一份没有社交、休息或体育锻炼的时间安排表会令人厌烦的。当需要减少用在休息和娱乐上的时间时,留有一定的闲暇时间是必不可少的。当所有的时间全部用到工作上时,人很容易感到疲劳和精神压抑。就像没有薯条、巧克力和冰淇淋会使人食欲不振一样,没有休息时间会导致人们将管理好时间的想法全部推翻。作时间安排以及跟踪时间是重要的,但是时间安排一定要是自己实际愿意接受的。

随着继续对时间进行跟踪,将实际使用时间的情况与时间安排表进行比较。如果总是能够按照安排管理时间,就不必作出太多的改变。但是更多的情况是需要对最初的计划进行调整,不要认为改变安排和改变基本规则有什么不好,每周的活动都在不断地变化,因此对时间安排表就需要做必要的修改。但是要记住,要把新的安排写下来。

7.9 管理好时间安排

为了制订管理好时间的基本规则，使用表 7.6 所示的方法，把每天所用的各类时间都列出来。如果打算一天里在某类活动上用两个时间段，那么或者使用两列来记录，或者把表格中的空格做得大一些。表格的格式并不重要，重要的是要建立起一套清晰明确的基本规则。

表 7.6 周时间进度和安排的示例

学生 学生 Y 日期 9/23/96

第 1 周的安排 常规周的安排

任务 星期	听课	编写 程序	准备 考试	阅读 课本	复习		其它	日总计
日								
	9:00- 9:50			10:20- 11:00				90
一		8:30- 10:30		10:20- 11:00				160
二	9:00- 9:50			10:20- 11:00				90
三		8:30- 10:30		10:20- 11:00				160
四	9:00- 9:50			10:20- 11:00				90
五		8:30- 10:30		10:20- 11:00				160
六								
周总计	150	360		240				750

按照时间安排表工作的能力很大程度上取决于个人的自觉性，但是它还取决于要做的工作的数量和它们的优先次序。预料不到的事件是生活中很自然并且是很正常的一部分，特别是在软件工程中。危机常常会破坏人们的计划，因此不得不作出调整。例如，为了在截止日期之前完成任务，人们可能不得不工作到深夜或是推迟已计划好的家庭活动、娱乐或者社交活动。

在第一次使用时间安排表时，可能感到它不是很有用。这是正常的，不要因为第一次它没有起作用就放弃对时间作安排的过程。而是要考虑所发生的事情，看看是否存在一些不可

能再发生的反常事件、或者存在虽由正常的事件引起然而意外地花费了很多时间？如果真的是紧急的情况，就不必对时间安排作大的改动，而在下一周再试着用它，然后再复查一下结果。但是如果一些经常发生的事情扰乱了安排，应考虑对安排进行调整，为以后可能发生的类似事件提前做好准备。

7.10 关于管理可变动的时间的几点建议

在为管理可变动时间建立基本规则时，应考虑以下几个问题：

- **目前最优先考虑的是什么项目？**首先努力完成最重要的任务。推迟困难的或是没有吸引力的任务是很自然的，但是这样做并不使人感到轻松。如果发现自己将重要的工作推迟了，停下来并且考虑一下你正在做些什么。这时，将会发现当把重要的任务推迟时，你不自觉的会为这些任务担心。实际上决定立刻处理这些事情常常是更有效的，并且这也将给人们带来一种完成任务的成就感。此外，记住一旦开始了一项令人生畏的任务，就很少会感到像你想象的那么困难。

- **是否有一些事情必须在特定的时间去做？**准备复习考试、准备一次实验或是与系里的指导老师谈工作就是这样的示例。在时间安排表中为这些活动分配特定的时间。

- **是否一有时间就想去做某些活动？**在工程记事本中，列出需要尽快做的事情，例如编写分配的程序作业或是阅读课本的下一章。当完成这些任务之后就把它标注出来。

为了管理好每天的时间，必须随时都带着时间安排表。一种管理这些安排的好方法就是把它们放在工程记事本里，把它们夹在前面或是后面封皮的内侧，或者把它们钉在记事本的某一页上。每当对安排进行更新时，记录下最近一份的日期并将旧的那份也保存下来。在作出改动之前复查一下以前的版本，这有助于防止改动过多。例如，为了弥补对某项任务的过高估计，在下次很可能就对这项任务低估一些。复查以前的安排就会发现这种趋势，这有助于选取一个更加合适的中间值。软件工程中一条必须遵循的规则就是：不要丢掉过去的计划、数据或程序。通常会惊奇的发现，以后还会需要它们。

7.11 时间管理的目标

按照这种方法对时间管理了几周之后，可以考虑通过将时间类别合并成很少的几类来简化获取时间数据的方法。这时的目标是要得到时间的整体分布情况而不是去了解细节。例如，在我对自己的时间跟踪了几年之后，就把我的工作时间分为两类：公司项目所用的时间和个人项目所用的时间。由于我对每一类项目分别进行规划，对时间段再进行细分并不会增添多少有用的信息。对于我来说，最重要的问题就是我希望了解每周或者每月在这两类项目上各用了多少时间。

记住，收集时间数据是为了帮助自己管理时间。如果收集的数据被证明是没有用的，就

需要重新考虑自己收集时间数据的方法。但是，只有在已经实践了安排时间之后再这样做。即使作了时间安排表，如果由于一些原因对时间安排变化很大，那么也应该收集更多的数据，直到自己明白当前是如何使用时间为止。

7.12 总 结

为了更好地管理好时间：要分析自己使用时间的历史数据；制订时间安排表；并且按照这个安排对时间进行跟踪。要制订时间安排表：首先决定你自己打算如何使用时间；然后作一个进度计划，它能够反映出你的选择，并且能够显示出每天所用的时间；对于不同的星期可能需要不同的时间安排表。

时间安排表的基本规则是有帮助的：即要确定固定的和可变化的时间安排。把可变化的时间分成需要完成的任务和自行斟酌的任务。分析一下目前时间划分的种类。请记住：总时间是固定的；如果想在一些活动上多用一些时间，就必须在另一些活动上少用一些时间。

最后，按照时间安排表跟踪实施的性能，要继续收集时间数据。根据经验复查时间安排表，再根据经验和经验修改安排，要逐步地作出改变。在改变时间安排表时，要保存以前的版本。

7.13 练 习 7

使用所有花费时间的数据，为这学期的后几周建立常规周时间安排表，也为可预计的特殊几周确定时间安排表。对于安排与平均时间有很大不同的情况，简要地解释你作出这种选择的理由。使用如表 7.4 和表 7.6 那样的安排表格，把这些安排和作业一起交来。注意，时间安排表是选做的作业，除非你的指导教师有要求，否则可以不完成。

递交你还没有交的时间记录日志和周活动总结表。

第 8 章 契约的管理

本章我们把精力集中在契约上，讨论契约是什么，契约为什么重要以及如何管理契约。在练习中，评阅并且列出自己目前的契约。

8.1 什么是契约

作出承诺是一种思想状态。无论是出于什么原因，你承担了一项工作，就会觉得应该做这件事情。但是，契约不仅仅是你要做的那些事情；它还说明有人希望你来做这些事情。实际上，这是契约的关键问题：你向谁承诺了契约？从法律或者合同的角度来说，你对别人负有义务：你的教授，你的经理，你的雇主。但是，更重要的是你给自己制订了更深层次的契约。

许多软件开发的进度和计划中存在的主要问题就是：管理人员把这些进度和计划看成是类似于合同的契约，而软件工程师则并不把它看成是个人的契约。我们将会看到这种差异主要在于如何制订契约。在这一章里，将学习如何制订合同式的契约并使它们成为个人的承诺。

对于带有合同性质的契约，在它生效之前，两个或是更多的人必须在预期的行动上达成一致。例如，A 先生和 B 小姐都同意 A 先生为 B 小姐提供一些产品或者完成一些任务。你向你的老师承诺完成这门课程的家庭作业就是契约的一个示例。你同意为客户编写程序是另一个示例。

当 A 先生制订一个契约时，他同意在规定的时间内为 B 小姐完成指定的任务并且收取一定的报酬。这个示例又指出了契约的另外两个要素：除了规定的任务之外，双方还在完成任务的时间、A 先生作为回报而得到的收入或其它的报酬上达成了一致。你同意在一个星期之内完成并上交家庭作业、指导教师同意给你的作业一个成绩就是一个示例。另外一个示例就是客户有义务向你支付为开发和安装某个软件的费用。

个人契约的一个关键特征就是双方都是自愿的。例如，假设你的客户发现他需要早一些得到程序，并且让你比原计划提前两周完成任务。他从未问过你是否能够做到这一点，并且你也没有同意。你只是被告知了新的最后期限。即使你经过努力能够按照新的期限完成任务，但你很可能觉得自己并没有义务这样做。

为了达成真正的契约，必须透彻地考虑另外一种可供选择的方案，并决定是否能够完成这项任务以及是否愿意去做。仅仅被别人告知必须做某件事情并未使你作出个人的承诺。实际上，当被命令去做某些事情时，人们常常会觉得受到了威胁并会感到气愤。他们会对作出这项指示的人感到不满，甚至会想进行报复。当然一种报复的方式就是拒绝去做被命令去做的事情。虽然对正常的商业要求做出这种反应可能会显得很幼稚，但是很多人无意中都会做出这种反应。

真正达成一致是个人契约的最重要的特征。双方必须在要做什么、何时完成任务以及提供什么回报这几方面达成一致。

一个真正的契约既是人的承诺也有合同的效应，并且它需要双方或多方在如下问题上达成明确和自愿的一致：

- 要做什么（事情）。
- 判断工作是否完成的准则。
- 由谁来完成任务。
- 何时完成任务。
- 作为回报所支付的报酬或其它考虑。
- 由谁来提供这些报酬或其它考虑。

8.2 认真制订契约

除了已经描述的特征之外，契约要认真制订并要进行适当的管理。可以遵照如下的步骤来确保契约是认真制订并且是管理得很好。

在同意契约之前要对所做的工作进行分析。契约双方必须有诚意缔结契约。你自己确实想要做这项工作，而对方也愿意提供适当的报酬作为回报。但是，问题是双方在多大程度上确认能够达到契约的要求。例如，你是否足够细致的查看了要做的工作并判定能够完成这项工作？同样，对方是否有能力给你支付报酬？软件契约往往只是基于双方的愿望。即使是双方真正想要履行各自的职责，但仅凭良好的愿望并不能为可靠的契约提供一个合理的基础。

用一个计划支持契约。对于任何规模的工作，认真制订契约的方法是先为所做的工作制订一个计划。做计划要花费一定的工夫，但不一定要花费很长的时间。事实上，如果有制订正式计划的经验，通常很快就可以完成这些计划。

把协议文档化。虽然这看起来很容易，但实际上并不是这样。一般的错误概念认为，诚实的人们只需要简单的几句话再握手就行了。然而语言常常会引起误解。即使两个人达成口头上的一致，他们也经常很难就书面的协议语句达成一致。这说明他们原来的协议只是表面的并且不是真实的。第二个问题就是当出现问题时双方该怎么办。这实际上是绝大多数合同要解决的主要问题。当所有的事情都按照计划进展时，就不需要合同了；只有在存在问题时，才需要一份合同。

如果不能达到契约的要求，及时告诉对方并努力把对对方的影响减小到最小的程度。当

你学会管理契约时，几乎总是能达到契约的要求。不幸的是，即使有了最好的计划，所做的工作偶尔也会比预想的要复杂，或是出现一些预想不到的事情。

8.3 契约的一个示例

学生 Y 同意为大学的招生办公室每周做 10 个小时的兼职工作。她的案例说明制订一个合适的契约所包含的步骤。

1. 学生 Y 到招生办公室去找主管人员，后者向学生 Y 介绍了期望她去做的工作。这是契约过程的需求阶段，在这个阶段学生 Y 精确地了解了她要去做的工作。对于办公室的工作来说，这个阶段的产品就是一份预期任务的清单，它可能以工作的描述形式给出。

2. 学生 Y 考虑了这些任务并得出结论：她愿意做这项工作，并且有能力去做这项工作。这回答了第一个问题：能否胜任这项工作？下一个问题是：能否在规定的时间内和需要的资源下完成任务？为了回答这个问题，学生 Y 是这样做的：

3. 她查看了个人的时间契约，并且得出结论：她每周能抽出 10 小时（也就是 600 分钟）来做这项工作。把表 7.5 与表 8.1，进行比较，你可看到她每周通过将娱乐时间减少 600 分钟来得到这部分时间。

4. 她审阅了表 8.2 所示的自己的契约，并得出结论：她每周从周一到周五的下午 4:00~6:00 有時間去做这份工作。

5. 既然她认为自己能够做这份工作，她就和办公室的主管商讨报酬和开始工作的日期。

6. 所有的问题都解决之后，学生 Y 和这位主管在所有讨论过的问题上达成一致意见，并且主管同意给学生 Y 一封概括了关键问题的邀请信。

这是一份负责任的契约。学生 Y 用一定的时间去了解这份工作以及她做这份工作的能力。她对自己处理这份工作的能力感到满意并能够抽出一定的时间。她也同这位主管就报酬和其它工作条件等问题达成了一致。

表 8.1 学生 Y 的每周时间安排总结

学生	学生 Y				日期	9/30/96	
教员	先生 Z				班级	CS1	
活动	计算机	物理	数学	英语	吃饭/休息	其它	总计
固定的							
上课	150	150	100	100			500
工作						600	600
需要的							
做作业	360	240	240	360			1200
读课本	240	240	180	60			720

续表

活动	计算机	物理	数学	英语	吃饭/休息	其它	总计
自由处理的							
吃饭					1260		1260
睡觉					3150		3150
运动						600	600
娱乐						360	360
放松时间						1690	1690
总计	750	630	520	520	4410	3250	10080

表 8.2

学生 Y 每周固定的契约

姓 名 学生 Y 日期 9/30/96

第 1 周的预算 固定的契约

任务 星期	计算机	物理	数学	英语	工作			日总计
日								
一	9:00- 9:50	10:00-10:50			4:00- 6:00			220
二			9:00- 9:50	10:00- 10:50	4:00- 6:00			220
三	9:00- 9:50	10:00-10:50			4:00- 6:00			220
四			9:00- 9:50	10:00- 10:50	4:00- 6:00			220
五	9:00- 9:50	10:00-10:50			4:00- 6:00			220
六								
周总计	150	150	150	150	150			1100

8.4 工业中的示例

一家重要的汽车公司的一位工程师接到了一项紧急的软件开发任务。以前，管理部门只

是简单地给他一定的时间来完成任务。但是由于这位工程师和他的小组刚刚经过了 PSP 的培训，他要求用一定的时间去查看这项任务，第二天早晨他带了一份计划返回公司。

这位工程师估算了这项工作的规模，并用他的 PSP 数据确定这项工作可能要用的时间。他返回到经理那儿并向他解释这个计划并说明为什么他认为这个计划是可行的。经理同意了他作的计划，而工程师也按照他承诺的进度完成了这项工作。

以前由管理部门指定的进度很少能按时完成。虽然其中绝大部分的进度都是不切合实际的，然而更主要的原因是工程师缺少个人的承诺。在与经理共同制订契约的过程中，通过自己积极的参与，工程师能够建立合理的进度。在这个过程中，他也亲自作出了承诺。

8.5 处理没有完成的契约

当你能熟练地制订计划之后，通常不会延误最后的期限。但是没有人能保证从不超过规定的限期。因为软件工程师所做的许多事情都是以从未做过的并且是具有创造性的，所以也就存在很大的风险。如果考虑所有可能的风险，你的估算就会高得不切合实际，因此在进度或费用上出现偶尔的失误是不可避免的。

如果已经确定无法完成某个契约，要尽快通知对方，并共同解决已经发生的问题。例如，双方商量可以延长工期或是缩小工作的范围。对于软件开发来说，通常的策略是在原进度或接近原进度的基础上交付一个具有较少功能的版本，然后再对产品进行一次或多次添加功能。通过适当地定义追加功能的顺序和时间，往往能够将客户所受的干扰减少到最小的程度。

处理偶尔出现的延误或是无法完成的契约的方法是很重要的。契约的问题总是令人不悦。你可能由于制订了糟糕的计划而变得心情不好，而那些你对其作出承诺的人会觉得受到了误导或欺骗。此时也可能会导致财务上的问题，有时还需要找出承担责任的人。这种不悦会更耽误时间。希望情况会有所改变而推迟通知对方通常会使事情更糟。最好在发现问题后，尽快面对不愉快的现实。

但是有一点很重要：如果还没有非常努力地去满足契约，就不能轻易放弃。问自己这样一些问题：你是否和有关专家讨论过去找出更好的方法？增加资源能否加快工作的进度？是否还有更巧妙的设计方案？彻底地考察各种可能的情况，然后，如果确实没有其他的方法，不要耽误时间，马上处理这些问题。

没有完成契约通常会带来不便和不快。这就是为什么人们在面对这种问题时经常要推迟到最后一刻。就像鸵鸟一样，他们在推迟这种不快，并希望问题会自己解决。不幸的是，把头埋在沙堆里只会增加破坏性，耽误客户选择可能有的其他方案，加大了不满意的程度。

8.6 管理契约的重要性

管理契约的主要原因就是使你不会忽略或忘记任何一件事情。工程师在工作时通常会有

许多的契约。他们要参与评审、写报告、开会、对程序进行修订以及提交更新过的程序模块。他们还可能必须撰写设计文档、答复客户的要求、会见经销商或是参与各种会议。工程师同时处理许多契约也是很正常的。所以学会如何管理契约是很重要的，这样你就不会遗漏或者忘记其中的任何一个契约。

对契约进行管理的另一个原因就是，当要做的工作超出了可以利用的时间时，它会对你有所帮助。如果你对工作做了计划，通常不会把时间用尽；但即使制订了严密的契约，偶尔也会出现问题。在这种情况下，及时确定出现问题的契约并迅速把它通知给其他当事人。

8.7 不对契约进行管理的后果

在学会管理契约之前，通常会遇到下面的一些不愉快的事情。

完成工作所要用的时间超过了可用的时间。经常会有比你能够完成的工作更多的事情需要你去完成。如果不保存一份契约的清单，可能会承担一些不应该承担的新的契约。例如，你可能会参加一次聚会，而这时你还有明天就要交的家庭作业要做。直到很晚你才想起了作业，然后不得不熬夜做作业。更糟糕的是，你可能根本就想不到还有作业要做。

无法达到契约的要求。软件开发工作通常要比预想的复杂。当不按照一定顺序制订契约时，可能会认为某项工作比实际情况要简单。这样一来，从开始这项工作的那一刻起，你就作了过多的承诺。

错误的优先次序。当作了过多的承诺时，人们经常会按照先后顺序而不是按照重要程度来安排工作的优先次序。当有比你能够处理的数量更多的事情时，你很自然就会处理下一件必须要做的事情。不幸的是，立刻处理这些最近发生的紧急情况通常是一种错误的策略。当作了过多的承诺时，需要重新安排契约以做合适的工作。通过推迟或减少一些需要立即处理的任务，才可能完成后而出现的更重要的工作。

低质量的工作。在进度的压力之下，通常软件工程师想走一些捷径。这时更容易发生粗心大意或愚蠢的错误，也是最需要关注产品质量的时候。当时间很短时，工程师应该特别注意避免错误。不幸的是，经验表明，在这种情况下，工程师和他们的经理最不可能花费时间复查、审查所做的工作或进行彻底的测试。

失去信任。如果你经常不能完成契约，人们就会注意这个问题。他们知道当你某事做出承诺以后，通常不会遵守自己的诺言。这样的名声是很难改变的，并且这会影响到你的业绩、职业等级、工作待遇甚至是工作保障。

失去别人对你的观点应有的尊重。当人们不相信你所说的话时，他们不可能再征求你的意见，并且他们更可能会坚持让你按照不合理的进度工作。

软件工程师能够拥有的最重要的个人资产就是完成承诺的声誉。为了使人们相信你说的话，你要说出你的计划，然后按照你所说的去做。这本书的一个重要目的就是提供一些工具以帮助你制订能够完成的、切合实际的契约。

8.8 管理契约的方法

管理契约是从列出已有契约的清单开始。注意每一个契约的完成日期和它可能要用的时间。表 8.3 给出了学生 Y 契约的清单。

表 8.3 学生 Y 的契约清单

日期	契约	另一方	时间	签订日期	回报
每周					
一、三、五	上课	指导教师	1.5		成绩
一、三、五	上缴计算机作业	指导教师	6.0		成绩
二、四	阅读课本	指导教师	4.0		成绩
一到五	兼职, 下午 4:00-6:00	管理处	10.0	9/1	工资
其它					
11/28	学期论文	指导教师	24	9/11	成绩

为了在软件工程中管理契约, 重要的是要记住软件业务生涯中的以下几个事实:

- 如果进度落后了, 除非做出一些变化, 否则就会继续延误下去。
- 只一味努力是没有用的。记住, 你已经很努力了。
- 如果不能精确地知道项目的进展状况和还有多少工作要做, 你几乎已经遇到麻烦了。
- 当必须要有很好的运气才能完成契约时, 往往得不到这种运气。
- 当估算错误时, 几乎总是低估了。
- 几乎所有的变更都将包含更多的工作。

努力工作以完成契约是很重要的, 但是, 如果多用几个小时仍然不能完成工作, 就需要正视这个问题并认真地解决它。

8.9 总 结

本章定义了契约, 解释了为什么要履行契约规定的义务以及如何管理契约。当双方或者多方的当事人, 在任务是什么、谁来完成、什么时候完成任务以及应提供什么样的报酬这几方面达成一致时, 他们就制订了关于某项任务的契约。对于制订恰当的契约, 应有满足契约的合理的保证条件, 并要制订一个计划来支持其实现。此外, 计划和契约都要存档。

应该管理契约以避免过多的承诺。如果作了过多的承诺, 你将不能完成某些契约并且会得到不可靠的名声。总是完不成契约会毁掉你的职业生涯, 会影响你的业绩以及得到和保持

一份职业的能力。

管理契约的步骤是：(1) 制订一份当前契约的清单，(2) 包括要做什么和什么时候完成 (3) 估计每个契约的工作量有多少。

软件工程师需要面对的现实情况是：如果不作出变化，就会继续落后；只是更加努力是没有用的；要准确地知道你的工作进行到了什么程度以及还有多少工作要做；不能指望运气帮你度过难关；当估计出错时，往往是低估了；几乎所有的变更都需要做更多的工作。

8.10 练习 8

列出一份契约的清单。包括关于契约内容的简要总结、由谁来完成契约、何时完成以及你希望得到什么样的回报。使用表 8.3 所示的格式，将一份这样的契约和你的作业一起上交。在你的工程记事本中保存一份契约的副本。注意，契约清单为选做的作业，除非你的指导教师要求了，否则你可以不完成这项作业。

上交你还没有交的时间记录日志、工作编号日志和周工作总结表。

第9章 进度管理

本章阐述如何制订和使用进度表来跟踪作业的进展情况。将使你学会如何对照进度表使用检查点来跟踪作业的进展情况。在本章的练习中，要识别和描述用来跟踪某个编程项目进展情况的几个检查点。

9.1 进度管理的必要性

制订作业进度的目的是为了完成委托。当在同一时间内要完成多个委托时，就需要有一份进度表。对于小型的项目或任务，通常可以在下一项目开始前就能完成它。学生总是把家庭作业拖到临近交付日期才开始做作业，每次做一个作业，在完成一个作业后再开始下一个作业。当有足够的时间来完成所有的任务时，且当任务是短期的或非常简单可按固定的顺序就能完成时，这种策略是可行的。但是，当任务变得庞大复杂、或作业量增加、或进度受到限制时，就需要更加复杂的时间管理方案。这时你就必须在各任务之间交替作业，在某任务上工作一会儿，再转到另一项任务上去。

例如，假设你要在三周内完成一个程序，在四周内完成一篇学术论文。假定你估计编写程序要用大约5个小时，但编程时要用到一个特殊的系统，而你每天只能使用该系统1个小时。还假定完成这篇学术论文需要到图书馆查阅一些资料，并在一周之内上交一份提纲。你估计完成论文大约要用10个小时。由于计划每周在这些作业上用5个小时，所以用三周左右的时间就能完成这两项任务。

按照先完成一项任务再完成另一项的策略，由于要在一周内上交学术论文的提纲，所以先做论文。正如估计的那样，你查找了资料、列出了提纲并且在两周之内完成了报告。然后，开始编程。工作了一段时间之后，发现编写这个程序要用大约10个小时而不是预计的5个小时。不幸的是，即使全力以赴，也没有足够的上机时间来按进度完成这个程序。

但是，假设第一周在这两项任务上都做一些初始的工作，就会发现编程要用的时间会比估计的多，这样就有三周的时间来找出需要的额外上机时间。

随着要从事的项目越来越大，精心地安排时间就会越来越重要。一个典型的软件工程师可能一边给某个开发项目测试一些模块，同时还要与一位技术人员一起编写产品文档。如果前面开发的程序出现问题，就必须召开技术和项目评审会。工程师必须用某种方式，把这些

任务安排到每天的进度中去，并且不能遗漏或忽视它们中的任何一个。他们也必须经常与许多人协调工作。要使这些工作无一例外地运转起来，就需要使用一份个人进度表。

9.2 Gantt 图

进度是按时间顺序计划活动的一个列表，其格式通常如图 9.1 所示。我们称之为 Gantt 图，它有以下几个关键的成分。

- 横跨图顶部排列的是日历表。根据所需要的详细程度，每列的时间单位可以是日、周、月或年。如果需要的话，也可以是小时。
- 最左边的一列包含了每项任务的标识号 (ID)。
- 左边第二列是要做的任务的名称。通常按照预期要做的顺序排列。
- 在图表当中，任务条指示各项任务计划的开始和结束时间。
- 在表的左下方是项目名称、进度表的作者和制订此进度表的原始日期。
- 图中的小椭圆是不同的检查点（它们的 ID 分别是 2，5，7，9 和 14）。检查点将在本章的后面讨论。

编号	名字	十一月				十二月					一月		
		11/5	11/12	11/19	11/26	12/3	12/10	12/17	12/24	12/31	1/7	1/14	1/21
1	需求分析												
2	结束												
3	估计和计划												
4	建议												
5	接受												
6	设计												
7	完成												
8	编码												
9	完成												
10	测试												
11	安装计划												
12	安装												
13	试运行												
14	正式运行												

项目: 项目 ABC

作者: 工程师 X

日期: 11/12/95

图 9.1 Gantt 图的示例

从图中可以看出，Gantt 图是展现项目中各个任务进展状况的一种有用的工具。这种图表对于协调多种活动特别有用。后面的解释将指出，Gantt 图也提供了一种按照计划和进度跟踪项目进展情况的有效途径。

9.3 制订项目进度表

对于任何规模的项目，制订进度表的第一步，是把作业分析到足够详细来确定它由哪几项任务组成。然后，估计这些细化了的任务的规模并决定每项任务可能的作业量。最后，在 Gantt 图中列出每一项任务并用任务条说明各项任务开始和结束的时间。

如果是为有若干人参与的作业制订进度表，还需要几个附加的步骤：

1. 确保每个人都知道要完成的任务。
2. 获取每项任务承诺的完成时间。
3. 确定各任务之间的相互依赖关系。每个人开始处理工作之前他们必须得到什么样的输入，以及他们从谁那里得到这些输入。
4. 把每一个相互依赖的关系都文档化。
5. 与所有涉及到的人一起复查计划的进度表和各任务之间的依赖关系，以确保没有冲突、分歧和误解。
6. 复查进度表，以确保它包含了完成整个作业所需要的全部任务。

虽然还需要更多的步骤来为软件开发项目制订一个完整的进度表，但是这些基本的原则已经能够帮助你制订个人的工作计划，以及为那些包含你的同学或同事的小型群组项目制订计划。

9.4 检查点

除了非常小的项目之外，对任何项目进行计划时，都要把工作分成几部分，分别对它们估计和规划。然后，每一个部分都可以作为一个计划项。这样，当每一部分完成之后，你就达到了一个定义的进展阶段。像这样可以测量的进度点称为检查点或里程碑。检查点是项目规划和项目管理中重要部分。

检查点是一个项目中能够客观地标识的点，例如，完成某些特殊的项目活动或某一重要的行动。如果一份计划包含了若干个检查点，而且每个检查点都标明了计划完成的日期，那么就很容易看出当前是达到了进度的要求还是落后了。

检查点必须清晰明确、没有歧义才是有用的。也就是说，检查点必须是某一特别的动作，要么它已经完成，要么还没有完成。一些明确的检查点的示例如下：

- 完成并提交了一份学术报告。
- 制订了编写某个程序的计划，并且用标准的格式写成了文档。

- 指导教师和你复查了开发计划，并按照建议作出了一些修改。
- 完成了某个程序的设计，并用特定的格式把它写成了文档。
- 实现、编译并修改了某个程序，因此该程序在编译时没有错误。

有很多明确的检查点的示例适合作计划时使用。关键是每个检查点的完成是可以客观地验证的。

不满足可验证性准则的一般陈述不能用作检查点。

不适当的检查点的示例如下：

- 为编写程序制订了一个计划。
- 设计了一个程序。
- 完成了 90% 的编码工作。

虽然上面的有些陈述作些改进能够达到检查点的要求（即清晰而明确），但从写的情况来看，这些陈述太一般。譬如说“为编写程序制订了一个计划”这个陈述，怎么能知道这份计划是否包含了所需要的信息呢？虽然可以通过查看计划来找出这些信息，但是检查点本身并没有告诉你想要知道的信息。除非使用某些明确定义的计划框架，否则这个计划是否已经完成就不能肯定了。对于“设计了一个程序”这个陈述，你是否知道完整的设计由那些内容构成？从粗略的流程图到详细的设计几乎都能满足这种描述。但是，如果该陈述补充了使用特殊格式描述的设计方案，就能够成为合适的检查点。

Fred Brooks 把“完成了 90% 的代码”这样的陈述描述为在许多软件开发项目中含糊和引起误导的状态报告的典型[Brooks]。如果有人告诉你编码几乎就要完成了，所以项目进展良好，这肯定是一个危险的信号。但是如果这个人告诉你他或她已经完成、评审、编译以及改正了 9 个新模块中的 7 个模块的代码，并且开始处理第 8 个模块的代码了，你就能够放心了，因为那个人知道自己在说些什么。

关于建立检查点的几点建议

在制订计划时，为任何包括几个小时以上工作的项目建立检查点。尽可能在每周中都找出几个检查点。例如，如果一项作业要用两周时间，至少要建立两个检查点，最好是 4 个或 5 个。应该根据工作量的大小来选取相应数量的检查点。对于需要 3~5 个小时的任务，设置超过一个的检查点就太多了。

我发现为大约每 5 个小时的作业建立一个检查点是最有帮助的。过多的检查点会占用太多的时间去跟踪一些无关紧要的活动。对于为期少于 5 小时的任务，就把这项任务作为一个完整的单元来管理。在这种情况下，项目管理就变成时间管理。只要在项目上投入了需要的时间，就能按照进度完成作业。当然，应当为每项任务适当留一些缓冲的时间，以防该项任务所用的时间比预期的要多。

对于为期几周的任务，即使估计这项作业每周只用大约半小时的时间，每周至少也要设置一个检查点。这个检查点可用来提醒你不要忘记已计划的工作。没有这样的提醒，这些小的任务元素就容易被忽略。当然，如果不能为这些只有半小时的任务定义清楚的检查点，那么最好的办法是管理所投入的时间量。除非你已经真正完成了一个明确定义的检查点，否则只能粗略地估计工作的进展情况。

较大的项目经常要用几周或是几个月并且涉及若干个软件工程师。由于项目中的任务是

相互联系的，并且一个项目也依赖于其他的项目，在其他的工程师完成他们的任务之前，你可能无法完成自己的任务。这需要项目中的每个工程师都设置几个中间检查点，以便每个人都能知道其他人的工作状态。如果有人遇到了麻烦，其他人就可以帮助他或者重新安排他们的工作，以避免延误整个项目的进度。

9.5 跟踪项目计划

项目跟踪是项目管理的重要部分，也是软件工程师在实际作业中应掌握的一项关键技能。虽然进行项目跟踪对小任务也会有帮助，但是现在提出这个主题的主要原因，是让你接触到今后作计划和跟踪时所要用到的技术。在这本书中学到的技能，可以直接应用到今后作为软件工程师所要从事的较大规模的作业中去。

跟踪项目计划可以使你知道项目是否正在按照进度进行，进度超前了还是落后了。明确的检查点和详细的计划能够准确地显示出项目的哪些部分出了问题以及为了赶上进度哪些地方需要帮助。这些问题会随着项目规模的增大而变得越来越严重，也会随着无法完成委托的波及效应而使后果变得更严重。而对于出资的客户来说，给出明确的项目完成状态报告是基本的要求。

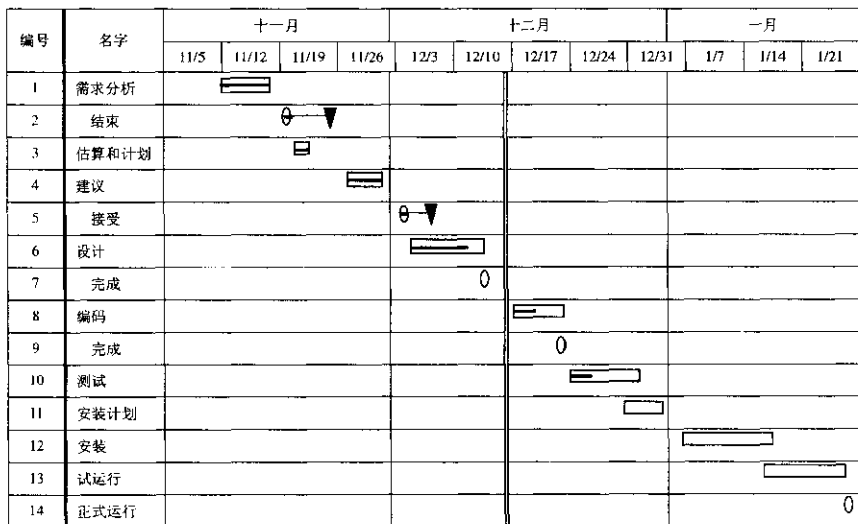
跟踪计划的另外一个原因，是为了能够在项目出现问题时及时地采取行动。通过对项目进行有效的跟踪，能够及早地发现问题，寻找解决问题的最佳办法，这常常是可以挽回损失的。实际上，一个良好的跟踪系统能够帮助你在问题严重到足以威胁项目的成败之前提前预防。

一个示例

Gantt 图不仅可以用来描述项目进度，还可以用来根据项目的进度表公布项目的进展情况。例如图 9.2，工程师 X 于 12 月 13 日公布了他的项目的进展状况。请注意以下几个方面：

- 项目进展状况更新的日期，是由穿过进度表的一个垂直的双线指示的，在这里是 12 月 13 日。
- 已完成的活动是由一条水平直线贯穿于完成的任务块指示的。例如，图中需求 (ID 1) 这项任务就已经完成了。
- 部分完成的活动，就在表示该活动的方块中画一条只能覆盖部分方块的水平直线。例如，设计作业 (ID 6) 应该在 12 月 12 日完成，但是如图所示它仅完成 80%。当然，在这项活动完成之前，80% 是个估计数字。
- 项目进度中提前进行的活动，由一条超过评价日期（由垂直的双线指示）的直线表示。例如，工程师 X 提前开始了一些编码和测试作业 (IDs 8 和 10)。
- 完成的检查点由完成的实际日期那里画一个实心的箭头来表示 (IDs 2 和 5)。

从这份更新过的 Gantt 图中，可以看到工程师 X 在设计阶段稍稍落后于进度，而在编码和测试阶段又比进度略有提前。但是，由于还没有到达“设计完成”的里程碑，所以可以认为这项目的进度落后了。



项目: 项目 ABC

作者: 工程师 X

日期: 11/12/95

更新日期: 12/13/95

图 9.2 Gantt 图的状态

关于跟踪项目进度的几点建议

在进行项目跟踪的过程中, 主要的问题是人们容易自欺欺人。他们可以通过跟踪模糊的检查点或频繁地改动进度计划进度表获得一种不真实的乐观情绪。下面的一些步骤可以帮助你避免这些事情的发生:

- 确保所有检查点的定义是明确的, 并已将他们写下来。
- 在制订一份全新的计划之前, 不要改动进度。
- 当按照计划公布项目进展状态时, 不要改动计划。
- 在给出新估计的项目完成日期时, 保留原来估计的日期, 然后用虚线注明新的日期。
- 保存原始的进度表和所有更新的版本。

关键是要记住, 根据没有改动过的进度表来公布项目的进展状态, 因为你正是根据这个计划来测量工作的。如果改动了原始的计划, 就没有了测量的依据。这就是为什么当评审项目进展状态时, 核对最初制订进度的日期是一个很好的主意。如果离评审的日期太近了, 更新进度表就不能提供有用的测量项目进展状况的信息。所以要求使用原始的进度表。

9.6 跟踪积分

对项目进行跟踪的一个问题常常是难于了解自己的进展情况。例如，假设跟踪我写这本书的手稿的进展状况。表 9.1 给出了前 10 章的原始计划。正如你所看到的，作业是从做计划和列提纲开始的，这项作业估计要用 681 分钟。在此之后，要写前言和各个章节。这个任务包括给每章打草稿、编辑草稿，然后再改写。表 9.1 给出了每一章估计要用的时间。这里，各项任务估计要用的时间都记在计划（分钟）这一列，各项任务估计的完成日期列在周数（计划）这一列。第一周从 1995 年 4 月 17 日星期一开始。这份计划参考了我以前写两本书时收集的数据。

表 9.1 手稿的开发计划

章 节		计划 /min	计划的积分		周数		实际 积分
			单元	累计	计划	实际	
做计划		681	3.243	3.243	1	1	3.243
前言	草稿	287	1.367	4.610	2	1	1.367
	编辑	800	3.810	8.420	2	2	3.810
	改写	760	3.619	12.039	3		
第 1 章	草稿	287	1.367	13.406	3	2	1.367
	编辑	800	3.810	17.216	4	2	3.810
	改写	760	3.619	20.835	4		
第 2 章	草稿	287	1.367	22.202	4	2	1.367
	编辑	800	3.810	26.012	5	3	3.810
	改写	760	3.619	29.631	6		
第 3 章	草稿	287	1.367	30.998	7	3	1.367
	编辑	800	3.810	34.808	7	3	3.810
	改写	760	3.619	38.427	8		
第 4 章	草稿	287	1.367	39.794	10	3	1.367
	编辑	800	3.810	43.604	10	4	3.810
	改写	760	3.619	47.223	10		
第 5 章	草稿	287	1.367	48.590	11	4	1.367
	编辑	800	3.810	52.400	11	4	3.810
	改写	760	3.619	56.019	12		

续表

章 节		计划 /min	计划的积分		周数		实际 积分
			单元	累计	计划	实际	
第 6 章	草稿	287	1.367	57.386	12	4	1.367
	编辑	800	3.810	61.196	14		
	改写	760	3.619	64.815	14		
第 7 章	草稿	287	1.367	66.182	15	4	1.367
	编辑	800	3.810	69.992	16		
	改写	760	3.619	73.611	17		
第 8 章	草稿	287	1.367	74.978	17		
	编辑	800	3.810	78.788	18		
	改写	760	3.619	82.407	18		
第 9 章	草稿	287	1.367	83.774	19		
	编辑	800	3.810	87.584	19		
	改写	760	3.619	91.203	20		
第 10 章	草稿	287	1.367	92.570	20		
	编辑	800	3.810	96.380	20		
	改写	760	3.619	100.000	21		
总计		20998	100.000				

在完成了这份计划之后，我在 Embry-Riddle 航空大学遇见了计算机系的教师，他们希望能从 9 月开始在课程中使用我这本书的手稿。这就是说我要在 8 月 10 日完成前 10 章的手稿。从表 9.1 中可以清楚的看到，我要在 21 周以后才能完成前 10 章，即 9 月 4 日，要比希望的晚 3 周多。

我没有重新制订计划，而是决定先对工作跟踪几周，来看看我是否能够按照缩短后的进度期限完成任务。我还决定不按照顺序完成每一章，而是要先写草稿和编辑所有草稿，最后在改写之前让我的助手复查草稿。这种方法的优点是使我能够知道我工作中的缺点，最终可能得到更好的产品。这样做的缺点是我很晚才完成各章的修订本。但如果这样，我怎么才能知道是否正在按照计划的进度进行工作呢？

这个问题的答案就是使用所谓的积分。基本上，我检查了计划并根据每项任务计划用的时间长短给每项任务分配一个积分。通过计算各任务所用时间占计划的项目总时间（20998 分钟）的百分比，来给出各任务的积分。这样，做计划用了 681 分钟，它的积分就是 $100 \times 681 / 20998 = 3.243\%$ 。为了满足新的进度要求，我要在 8 月 10 日就是 17 周之后而不是 21 周之后挣得 100% 的积分。

在完成了前四周的工作之后，我发现自己能够按时完成前 10 章的任务。这是根据表 9.2

和图 9.3 得到这个结论的。注意到前四周我得到的实际积分是 37.037，即平均每周 9.259。在这样的速度下，我估计可以得到表 9.2 中预测积分那一列的数值。这里，看起来在 11 周就可以达到 100%，即比计划要求的 17 周大大提前了。

这说明，通过使用积分，可以用与原计划不同的顺序工作，并且还可以按照计划跟踪工作进展状况。虽然也可以重新制订计划，但是那将耗费很大的工作量并会延误项目的进度。由于加上了复查这项任务，完成项目的周期比原计划要长，我实际是在第 15 周时完成了手稿，及时赶上了秋季开学的日期。

虽然本书并没有要求你使用积分的方法作进一步的跟踪，但是你会发现在今后管理较大项目的进度时，这是一个很有用的技术。在 PSP 课本[Humphrey]的第 6 章中更加透彻地讨论了积分的有关问题。

表 9.2 写书的进度表

周编号#	计划积分	实际积分	预测积分
1	3.243	4.610	
2	8.420	14.963	
3	13.406	25.317	
4	22.202	37.037	
5	26.012		46.296
6	29.631		55.556
7	34.808		64.815
8	38.427		74.074
9	38.427		83.333
10	47.223		92.592
11	52.400		101.852
12	57.386		
13	57.386		
14	64.815		
15	66.182		
16	69.992		
17	74.978		
18	82.407		
19	87.585		
20	96.380		
21	100.000		

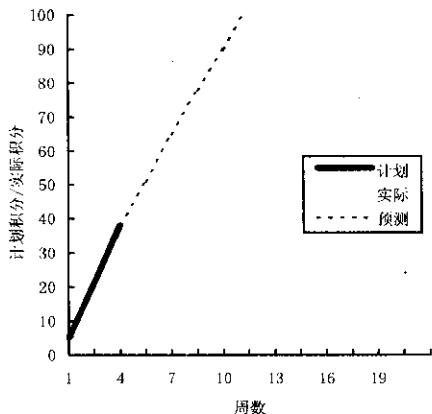


图 9.3 书写手稿的计划时间和实际时间

9.7 总 结

本章包括了进度表和检查点的内容，解释了为什么需要进度表以及如何制作 Gantt 图 and 如何使用 Gantt 图来管理项目进度。检查点是项目计划和项目管理的一个重要部分：一个检查点标识某个项目的特定事件的完成：它必须是明确的，而且已建立完成的准则。每 5~10 个小时工作至少应该建立一个检查点，为项目每周至少应该建立 1~2 个检查点。检查点可以帮助你查看项目进展的情况，并且当项目的进展落后于计划时，它能帮助你采取纠正行动。积分可以帮助你准确地跟踪项目的状态，准确地估算什么时候可以完成项目。

9.8 练 习 9

在编写中等规模的程序时，定义 3 个或者更多的检查点。上交一份简要的报告描述和说明每个检查点，并且要说明其他人如何判断你是否已经完成了检查点。注意，检查点练习是选做的作业，除非你的指导教师要求，否则就可以不做这个作业。

上交你还没有交的时间日志，作业编号日志和周作业总结表。

参 考 文 献

[Brooks] Brooks, Frederick P. *The Mythical Man-Month, Essays on Software*

Engineering, Anniversary Edition. Reading, MA: Addison-Wesley Publishing Company, 1995.

[Humphrey] Humphrey, W. S. A *Discipline for Software Engineering*. Reading, MA: Addison-Wesley, 1995.

第 10 章 项目计划

本章扩展对产品计划的讨论使其包括对时间的估计。还将介绍项目计划总结表，它用来记录估计的和实际的程序规模和开发时间。本章的练习，是为编写下一个程序制订一份项目计划。

10.1 项目计划的必要性

项目计划定义要完成的工作和如何做这项工作。它对每项主要任务作出定义，对所需要的时间和资源进行估计，还为管理部门的评审和控制提供框架。项目计划也是一个强大的学习工具。当把这个计划严格地写成文档时，它就是一个与实际性能比较的基准。这样的比较，可以使作计划的人看到估计中的错误，从而改善作计划的准确性[Humphrey 89]。

因此，项目计划是一个项目的基本部分。项目越大，制订计划就越重要。在这门课程中制订项目计划就是为了学习怎样制订计划。在有了更多的经验之后，制订准确计划的能力就会得到提高，并且在需要时就能够迅速制订出一份计划。就目前来说，你所制订的项目计划，仅仅包含产品规模的估计和完成工作所需时间的估计。在完成这个作业后，也将记录下实际的规模和时间数据。

一旦作了时间的估计，把这个数据记录下来，以便在完成作业时可以把估计的时间与实际花费的时间相比较。分析比较的结果有助于为今后的任务做出更好的估计。例如，如果你的估计一般都偏低，就要把它们调整得高一些，或者相反地，如果你的估计都偏高，你就要把它们调整得低一些。因为很容易调整过度，所以作这些调整时要谨慎。

10.2 项目计划总结表

表 10.1 给出了用于记录计划的和实际的项目数据的表格。在随后的几章里还会在这个相当简单的表格中补充一些条目。为了避免每章都介绍新的表格，我们真实地使用表 10.2 给出的项目计划表格，表格中用阴影标出的部分将在随后几章介绍。现在先忽略这些阴影部分。

完成这份表格时，应该在项目开始之前填写计划部分，在项目结束之后填写实际部分。从现在开始，每章的课后练习都要求把一份填好的项目计划总结表和每个完成的程序一起交上来。保存这些表格的副本，以便在将来的练习中可以使用有关你作业的数据。在以下的几节将介绍如何完成表 10.2 中所示的项目总结表

表 10.1

项目计划总结表

学生	_____	日期	_____
程序	_____	程序号#	_____
教师	_____	程序设计语言	_____

总结	计划	实际
Minutes/LOC	_____	_____
LOC/Hour	_____	_____
程序规模 (LOC)	计划	实际
新开发与更改的	_____	_____
最大值	_____	_____
最小值	_____	_____
开发时间 (min)	计划	实际
总计	_____	_____
最大值	_____	_____
最小值	_____	_____

表 10.2

PSP 项目计划总结表

学生	_____	日期	_____
程序	_____	程序号#	_____
教师	_____	程序设计语言	_____

总结	计划	实际	累计
Minutes/LOC	_____	_____	_____
LOC/Hour	_____	_____	_____
Defects/KLOC	_____	_____	_____
过程效益	_____	_____	_____
A/FR	_____	_____	_____
程序规模 (LOC)			
新开发与更改的			
最大值			

最小值					
开发时间/min	计划	实际	估计	累计	百分比
计划					
设计					
编码					
代码复查					
编译					
测试					
后置处理					
总计					
最大值					
最小值					

引入的缺陷	计划	实际	累计	累计	Def/Hour
				百分比	
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					

排除的缺陷	计划	实际	累计	累计	Def/Hour
				百分比	
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					

10.3 项目总结

在表中总结部分存放的是用来制订计划的效率数据。它还提供了在完成作业之后记录实际效率数据的地方。在总结部分，有一行是 Minutes/LOC（编写一行代码花费的分钟数）。在计划这一列，登入制订计划时使用的 Minutes /LOC 效率数据。表 10.3 中所给的示例说明，学生 X 使用了以前的程序或作业编号日志中的历史数据 7.82 Minutes /LOC。当然，如果没有以前的数据，就不得不猜测了。

除非你有特别好的理由使用另外的方法进行估计，否则就使用历史平均效率，从作业编号日志中最近程序的平均时间中得出。如果某个新的程序看起来特别难并且可能比平均时间花费更长，就可能需要使用不同的估计。或者相反，如果新程序与最近编写的某个程序非常类似，因此它所需要的工作量比较少。通过查看最近的程序记录中有关作业时间的平均值、最大值和最小值，就可以为新程序选取一个适当的值。

总结部分的第二项是 LOC/Hour（每小时编写的代码行数）。同样，把开始作业之前的估计值和完成作业之后得到的实际值填入表格。LOC/Hour 是用 60 除以 Minutes/LOC 得到的。在表 10.3 的示例里，学生 X 计划的 Minutes/LOC 的值是 7.82，所以计划的 LOC/Hour 就是 $60/7.82=7.67$ 。以 LOC/Hour 为单位的效率值常被工程师用于分析开发的生产率。

表 10.3 PSP 项目计划总结表的示例

学生	学生 X	日期	10/7/96
程序		程序号#	8
教师	先生 Z	程设计语言	Ada
总结	计划	实际	累计
Minutes /LOC	7.82	7.21	
LOC/Hour	7.67	8.32	
Defects/KLOC			
过程效益			
A/FR			
程序规模 (LOC)			
新开发与更改的	26	19	
最大值	36		
最小值	18		

续表

开发时间/min

计划

实际

累计

累计

百分比

计划

设计

编码

代码复审

编译

测试

后期处理

总计

203

133

最大值

282

最小值

141

引入的缺陷

计划

实际

累计

累计

Def/ Hour

计划

设计

编码

代码复审

编译

测试

总计

缺陷的缺陷

计划

实际

累计

累计

Def/Hour

计划

设计

编码

代码复审

编译

测试

总计

10.4 程序规模

项目计划总结表格中程序规模(LOC)部分包含了程序规模的估计数据和实际数据以及可能的规模范围。使用第6章描述的方法,估计程序的最终规模,并把这个数据填入新开发

的与更改的这一行中的计划这一列。在完成程序 8 的项目计划总结的过程中, 学生 X 作了规模估计, 如表 10.4 所示。

表 10.4 学生 X 的程序规模估计

学生 学生 X 日期 10/7/96
教师 先生 Z 课程 CS1

程序	代码行	以前的功能	估计的功能	最小	平均	最大
Case						
2	11	简单的 case 语句				
4	18	中等的 case 语句	大 case 语句	12	17	24
Loops						
1	5	简单的 do 循环	小 loop 语句	3	4	5
6	11	中等的 repeat-until 循环				
Calc.						
3	23	大的计算				
Text						
5	7	小的字符串操作	简单串操作	3	5	7
7	24	中等的字符串操作				
估计				18	26	36

注释: 此程序有一个比较大的 case 语句, 用来进行字符串的判断。因此, 这个 case 语句的大小应该和程序 4 的规模差不多, 可能要更大一些。另外, 还需要一个简单的循环来处理每个输入文本; 需要一个简单的文本分析器来格式化最终的输出文本。假定这个循环和分析器的规模的最大值与以前程序的规模相同, 而 case 语句要稍微大一些; 另外还假定程序规模的最小值是最大值的一半。

新开发的与更改的这部分只记录实际编写的代码的总行数。最初编写的每一个程序一般都是新开发的。但是编写过几个程序之后, 就可以使用程序库来执行标准函数, 或者可以通过对以前程序的添加或修改来得到一个新的程序。为了节省开发时间, 你也许会复制以前开发的程序的某些部分。例如, 可以重用以前自己开发的程序中的某个函数。这样, 不需要再把这个函数重写一遍, 只要把这部分代码复制过来, 然后在新程序中重用就行了。

当估计新程序的规模时, 不计算以前开发的代码行。原因就是我希望跟踪的是你自己实际编写的代码行数。这就是编写这部分代码要用的大部分时间, 并用以计算 Minutes/LOC 为单位的生产效率。项目计划总结中的这部分称为新开发的与更改的程序行数, 只计算新编写的代码和修改以前程序的代码。这样, 要估计开发一个新的程序所用的时间, 就用计划的开

发每行代码用的时间乘以计划的新编写的与修改的代码行数。

例如，如果你从以前的程序中复制了 25 行代码并且新编写 30 行代码，那么新编写的代码与修改的代码一共就是 30 行。另一方面，如果你从以前的程序中复制了 25 行代码并修改了其中的 7 行代码，那么新编写的与修改的代码行数就是 37：（即 7 行修改的代码和 30 行新编写的代码）。

只计算新编写的代码和修改的代码的原因就是，人们使用程序库或以前开发的代码的数量变化非常大。而且包含了这些代码所得到的以 Minutes/LOC 为单位的编码时间，一般都要大大少于编写新代码或修改已经存在的代码所需要的时间。这样，如果忽略这些重用的和复制的代码，你的估计会更准确。尽管统计修改的和重用的代码这项工作很复杂，但现在在你的规模估计中只包含新编写的和修改的代码，而没包含重用的代码。应该忽略从任何不同的系统库中或以前开发的程序中得到的代码，但是如果用在重用以前开发的程序上的时间太大似乎不能忽略时，请参考 PSP 课本[Humphrey95]中第 4 章和第 5 章中有关这个问题更完整的讨论。

最大规模和最小规模

程序规模 (LOC) 部分中的最大规模和最小规模的数量是用第 6 章中描述的方法估计的。表 10.4 的示例中，学生 X 使用程序 1 和 5 规模的总和作为两个函数的最大规模的估计值，并且为另一个函数选取了比程序 4 大一些的规模的估计值。为了计算最小值，假设合并的新程序的规模是最大值的一半。然后在这两个数字间的中点处，选取一个最可能的值。这样选择 36 作为规模估计的最大值，选择 18 作为最小值，选择 26 作为程序规模最可能的估计值。

在估计程序开发时间时，这些最大和最小规模的数字有助于判断可能的时间范围。例如，如果你非常担心能否在委托的日期内完成一个新的程序，考虑一下最大规模，把它作为所编写的程序有多大规模的指示。如果你估计新程序可能的规模是 26 LOC 并且最大规模是 36 LOC 的话，你可能希望在进度中留一些余量，以防编写出的程序接近估计的最大值时时间不够。

最小规模也是用第 6 章中描述的方法获得的。计算这个数字的主要原因是为了鼓励你去考虑所计划的程序的可能的规模范围。在一般情况下，把平均规模和最大规模的数字作为计划和委托书的根据。

注意，最大规模和最小规模完全是根据规模估计的范围得到的，它们不是用统计的方法得来的，不能靠它们来提供统计。除非已经有了充足的数据，否则使用最大规模和最小规模估计只能作为一般的计划指导。当有了更多的数据时，考虑使用 PSP 课本第 5 章和附录 A [Humphrey 95]中讲述的置信区间的计算方法。

10.5 开发阶段的时间

表 10.3 项目计划总结表的第三部分是开发阶段时间，主要用在后面几章中处理软件开发各阶段的时间数据。在本章中，仅使用程序开发的总时间。

为了估计开发一个新程序的总时间，首先估计程序有多少行代码，用计划的 Minutes/LOC

乘以估计的规模。这样就可以给出开发某个程序估计的总时间。表 10.3 中的示例也给出了这些数字。计划的总时间用编写一行代码计划所用的分钟数 Minutes/LOC 乘以计划的新编与修改程序的行数： $7.82 \times 26 = 203.32$ 或是 203 分钟。

此外还要计算最小和最大时间并要把它们登入计划那一列。得到这些值要用最小和最大的代码行数分别乘以 Minutes/LOC 。最小值就是 $7.82 \times 18 = 140.76$ ，最大值就是 $7.82 \times 36 = 281.52$ 。

注意，也可以通过这样的方法获得最大规模中的最大时间和最小规模中的最小时间，即用估计的代码行的最大值和最小值，分别乘以作业编号日志中记录的一行代码所用时间的最大值和最小值。由于这两个值给出的范围太大，无法提供有帮助的指导，所以我们在项目计划总结表中不使用它们。但是它们提供了可能预计到的开发时间的最坏和最好情况的概念。

在实际开发程序之前算出这些计划数据，并把它们登入项目计划总结表，在完成项目之后把实际的数据也登入表。当你工作时，对所用的时间进行跟踪，并且当作业完成时把总时间填入总计这一行的实际值这一列。此外在已经完成的程序中统计新编写的和修改的代码行，并计算编写一行代码实际所用的 Minutes/LOC 和 LOC/Hour 。为了查看完成项目计划总结表的方法，参考表 10.5 中的说明。

表 10.5

PSP 项目计划总结表使用说明

目 的	以一种方便而且易于存取的方式记录估计的和实际的项目数据
表头	如下输入： ● 你的名字和今天的日期 ● 程序名字和程序号 ● 教师的名字 ● 你将使用的编程语言
总结 Minutes/LOC	在开发之前： ● 登入估计的开发每行代码所需要的分钟数； 在开发之后： ● 用全部的开发时间除以实际的代码行数，得到每行代码花费的分钟数的值； ● 例如，如果项目总共花费了 266 分钟，代码行数是 34，那么，$\text{Minutes/LOC} = 266/34 = 7.82$
LOC/Hour	在开发之前： ● 用 60 除以计划的“Minutes/LOC”，就得到计划的“LOC/Hour”的值。 在开发之后： ● 用 60 除以实际的“Minutes/LOC”，就得到实际的“LOC/Hour”的值； ● 对于上面的示例，实际的 $\text{Minutes/LOC} = 7.82$，实际的 $\text{LOC/Hour} = 60/7.82 = 7.67$
程序规模 LOC	在开发之前： ● 在计划下面登入估计的新开发与更改的代码行数的总值、最大值和最小值。 在开发之后： ● 统计并登入实际的新开发与更改的代码行数

续表

目 的	以一种方便而且易于存取的方式记录估计的和实际的项目数据
开发阶段时间 计划	用新开发的与更改的代码行数，乘以开发效率“Minutes/LOC”，得到项目的总开发时间。 例如，用新开发的与更改的代码行数 26 乘以 7.82 Minutes/LOC，即总的开发时间 $=26 \times 7.82=203 \text{ min}$ 使用代码行的最大值乘以估计的开发效率“Minutes/LOC”得到开发时间的最大值；例如，用最大规模 36 LOC 乘以 7.82 Minutes/LOC，即最大时间 $=36 \times 7.82=282 \text{ min}$ 使用代码行的最小值乘以 Minutes/LOC 得到开发时间的最小值；例如，用最小规模 18 LOC 乘以 7.82 Minutes/LOC，即最小时间 $=18 \times 7.82=141 \text{ min}$
实际	在任务完成之后，根据时间记录日志中的数据，填入各个阶段的实际时间 (min)

10.6 估计的准确性

开始，估计出来的规模和时间可能不是很准确，这是正常的。首要的目标是要学会做出无偏估计。也就是说进行每 10 次估计，预计希望有 5 次高估和 5 次低估。尽管你希望它们都非常准确，但是这样高的准确性是不可能的。这些估计上的偏差只能逐渐地减少，但是你应该能够迅速地学会做出无偏估计——也就是说高估的次数要与低估的次数相同。有了这些经验，就能逐渐减少估计错误的平均值。

把估计文档化，进行研究并且从中学到一些东西。这将有助于你做出更好的估计。有些工程师就比其他工程师所作的估计要好。当测量估计的准确度时，你就会知道低估和高估的偏差有多大。这将有助于去判断制订自己不能满足的委托书的风险有多大。

10.7 总 结

本章讲述了如何制订项目计划。描述了制订产品计划的过程，定义了必需的数据和表格，并给出了一个已完成的项目计划的示例。对项目计划文档化，这样可以将程序的实际开发时间和规模与估计的数据进行比较，从而学会制订更好的计划。

为了制订好的计划，需要将当前的作业与以前的作业相比较，使用相似任务的历史数据，利用表格来记录所做的计划。为了获得制订计划的数据，需要测量已完成的产品的规模，估计下一项作业的规模，估计这项作业可能的开发效率，计算新的作业可能需要的时间，以及确定新的作业的规模 and 所需要的开发时间的范围。

10.8 练 习 10

为下一个编写的程序制订一个计划。制订计划时要以历史数据为基础，并且使用表 10.2

所示的记录计划数据的格式。从现在开始，为每一个编程的练习都制订一个计划并将计划交上来。在计划中，在程序开发之前记录下原始的估计数据，然后当你完成作业时，登入实际的规模和时间。此外上交一份使用表 10.4 来记录的程序规模的估计数据。

提交你还没有交的记录日志、作业编号日志和周活动总结表。从现在开始到学期末，上交在这门课程中已完成的你所开发的每一个程序的这些表格。

参 考 文 献

[Humphrey 89] Humphrey, W.S. Managing the Software Process . Reading, MA:Addison-Wesley, 1989.

[Humphrey 95] Humphrey, W.S. A Discipline for Software Engineering. Reading, MA:Addison-Wesley, 1995.

第 11 章 软件开发过程

遵循一个系统化的过程将帮助你更有效地计划和管理工作。本章说明如何利用过程来开发软件。在课后练习中，你将利用本章所描述的过程制订一个计划和编写一个程序。

11.1 为什么使用过程

在帮助许多软件组织改善他们的性能时，我发现重要的问题之一就是如何确定该组织当前的软件开发状况。例如，一个开发小组甚至不清楚有多少项目延期或超支。没有这些数据记录，开发小组无法知道其开发状况是变得越来越好还是越来越糟。为了解决这些问题，他们开始使用本章所讲的方法去收集有关数据。

一个过程是为完成某一作业而定义的一系列步骤。一个作业的每个步骤或阶段都有明确的入口准则，只有满足这些条件，才可以开始进入这个阶段。同样它们也有出口准则，只有满足这些条件，才可以结束这个阶段。而过程的步骤定义了要完成的任务以及如何完成这些任务。因为软件工程师的过程质量在很大程度上决定了他们的工作质量和效率，所以过程的设计和管理在软件工程中是非常重要的。本书所定义的个体过程是为了帮助你成为一个更有能力的软件工程师。

本书采用的过程称为个体软件过程（PSP）。它是一个用以帮助软件工程师测量和改进工作方式的框架。PSP 有两个目标，一个是帮助软件工程师开发程序，另一个是告诉你如何利用过程来改善工作方式。以后，当开发较大型的、更复杂的程序时，可以扩展 PSP 来完成这些工作。虽然本书没有讲述如何定义和扩展过程，但这些方法已经广为人知，并且可以在另一篇篇幅较多的 PSP 书籍中找到这些内容[Humphrey 95]。

11.2 一些定义

在讨论过程时，我们采用第 5 章定义的术语，同时也补充了一些术语：

- 一个产品是指为合作者、雇主或客户所生产的物品。
- 一个项目通常生产一种产品。
- 一个任务是已定义的一部分工作。
- 一个过程定义完成项目的方法。
- 过程有许多阶段或步骤，如计划、开发和测试。
- 一个过程阶段可以由许多任务或活动组成，像集成测试、产品测试和系统测试。
- 注意，一个过程可以包括一个或多个阶段，一个阶段可以包括一个或多个任务或活动。

动。

● 计划描述一个特定的项目的完成方式，包括如何做、什么时候做以及花多少费用。你也可个人的任务作计划。

- 一个作业是你为一个项目或一个任务所做的事情。

一个有完整描述的过程称为*已定义的过程*。已定义的过程一般由一些脚本、表格、模板和标准组成。一个过程脚本是一组书面的、过程的用户或代理在使用过程时应该遵守的步骤。各种表格（如一些日志和总结表）是用来记录和保存项目数据的。在本书后面的章节中，你将使用 PSP。图 11.1 列出了 PSP 的元素。注意，尽管图中列出了时间日志和缺陷日志，但是要等到第 12 章才开始收集缺陷数据。

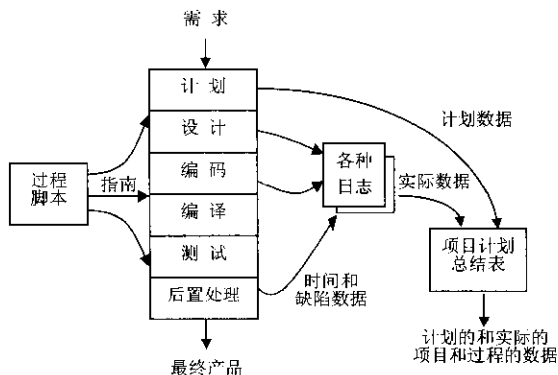


图 11.1 PSP 过程流程

11.3 过程脚本

最初的 PSP 脚本如表 11.1 所示，这是 PSP 脚本的几个版本中的第一个。在以后的章节中，将不断向这一脚本加入新的步骤。下面将逐一描述 PSP 过程的各个阶段。表 11.1 给出了各个阶段的总结性的说明。

计划阶段。首先获得项目的需求。然后完成项目计划总结表中不带阴影的计划部分。最后把作计划所花费的时间记录在时间记录日志上。

设计阶段。设计这个程序。设计不必过细，但在开始写程序代码前，必须考虑整个程序的逻辑结构。用流程图、伪代码或指导老师指定的其它格式记录设计的内容。在设计阶段结束时，把设计时间记录在时间记录日志中。设计过程和各種设计表示方法将在第 17 章进一步讨论。

编码阶段。利用所选择的程序设计语言来实现该设计。使用一致的编码格式并且按照指导老师所要求的编码标准进行编码。在编码阶段结束时，在时间记录日志上记录编码所花费的时间。

编译阶段。编译程序并改正所有发现的缺陷。继续编译和修复缺陷直到能正确编译而没有任何出错信息为止。所有花费在这一阶段的时间记为编译时间，其中包括改正代码或修改设计的时间。在编译阶段结束时，在时间记录日志上记录编译时间。

测试阶段。进行充分的测试以保证程序满足所有的需求，并且能够不带任何错误通过一组全面的测试集。花费在这一阶段的所有时间记为测试时间，其中包括改正代码、修改设计和重新编译的时间。在测试阶段结束时，在时间记录日志中记录测试时间。

后置处理阶段。按照本章后面所描述的方法，完成项目计划总结表中所有有关实际数据的那些栏目。由于必须在完成后置处理阶段之前填写后置处理的时间，所以应该完成尽可能多的工作，并留出几分钟的时间来做最后的计算，然后登入一个估算出来的后置处理阶段的时间。使用这个估计的后置处理时间来计算整个的开发时间和其它有关数据。

表 11.1 PSP 过程脚本

目的	指导用户进行小程序的开发
入口准则	问题描述； PSP 项目计划总结表； 以前开发的程序的实际规模和时间数据； 时间记录日志
1 计划	获取对程序功能的描述； 估计整个程序的代码行数及其最大值和最小值； 确定开发效率(Minutes/LOC)； 计算总开发时间及其最大值和最小值； 将计划数据填入项目计划总结表； 将计划阶段所花费的时间登入时间记录日志
2 设计	设计程序； 按照指定的格式记录设计文档； 将设计阶段所花费的时间登入时间记录日志
3 编码	实现设计； 使用标准的格式来书写程序代码； 将编码阶段所花费的时间登入时间记录日志

续表

目的	指导用户进行小型程序的开发
4 编译	编译程序; 修复所有发现的缺陷; 将编译阶段所花费的时间登入时间记录日志
5 测试	测试程序; 修复所有发现的缺陷; 将测试阶段所花费的时间登入时间记录日志
6 后置处理	把实际的规模和时间数据登入项目计划总结表; 将后置处理阶段所用的时间登入时间记录日志
出口准则	经过详尽测试的程序; 较好的设计文档; 完整的程序清单已经填好的项目计划总结表; 已经填好的时间日志

11.4 检查点和阶段

检查点已经在第 9 章介绍,它用来帮助制订和跟踪项目进度。通过定义明确的、可清楚地识别的项目检查点,可以制订更好的计划。称它们是更好的计划是因为在做这项工作时,检查点为测量项目状态提供了精确的参考点。

软件开发过程进一步把检查点的思想从几个点扩展到所有的过程阶段。对于一个已定义的过程,每个阶段产生一个特定的结果,阶段完成时刻即为一个可测量的检查点。通过使用一个已定义的过程,将有许多检查点来帮助你制订计划和跟踪你的工作。

11.5 更新的项目计划总结表

项目计划总结表是 PSP 过程中的一种表格,见表 11.2。它的使用说明见表 11.3。和前面一样,项目计划总结表中某些部分是带阴影的。这些部分现在可以忽略,因为它们在今后的章节中才用到。为了标识过程从一个版本到另一个版本的变化,新增加部分用**斜体**表示。通过比较表 10.2 与表 11.2 可以看出,现在开始使用一些以前带阴影的部分。例如,在

表的开发阶段时间, 计划、设计、编码、编译、测试和后置处理这些行都可以用了, 另有累计和累计百分比这两列也可以用了。为了标识表中这些新增加的行和列, 它们的标题用**斜体**表示显示。

在新的项目计划总结表中的项目阶段时间部分, 过程的每个阶段分别占一行。每一行中有每个阶段的计划时间和实际时间。在计划阶段, 在计划列中登入所有的计划数据。在后置处理阶段, 在实际这一列登入所有的实际数据。当用时间记录日志来记录时间时, 在注释一栏中填入当前阶段的名称。然后, 在后置处理阶段, 在实际时间这一列登入每个阶段花费的实际时间。

开始一个项目前, 按照第 10 章所叙述的方法在项目计划总结表的计划部分登入数据。唯一不同的是现在需要估计每个阶段所花费的时间。具体方法是参照从前项目各阶段所花费的时间比例, 把总的开发时间分配到每个阶段中去。如果是第一次, 则没有可以参照的实际数据, 就不得不进行猜测。但是, 对于此后的项目, 就可以根据前一个项目的数据来估计新项目的各个阶段的数据。这就是在项目计划总结表中设置累计百分比这一列的原因。

项目计划总结表中的累计和累计百分比提供了一种简单的方法来计算开发时间在各个阶段分布的百分比。累计列包括所有已开发的程序在各个阶段所花费的时间之和。累计百分比这列表示累计列在各个阶段时间分布的百分比。在 11.7 节的示例中, 说明如何计算表中累计和累计百分比这两列的值。

表 11.2

PSP 项目计划总结表

学生				日期	
程序				程序号#	
教师				程序设计语言	
总 结	计 划	实 际	累 计		
Minutes/LOC					
LOC/Hour					
Defects/KLOC					
缺陷数/千行					
程序规模 (LOC)					
新开发与更改的					
最大规模					
最小规模					
开发阶段时间/min	计 划	实 际	累 计	累计百分比	
计划					
设计					
编码					

续表

代码复查					
编译					
测试					
后置处理					
总计					
最大时间					
最小时间					
引入的缺陷	计划	实际	累计	累计百分比	Def./ Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					
排除的缺陷	计划	实际	累计	累计百分比	Def./ Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					

表 11.3

PSP 项目计划总结表使用说明

目 的	本表以一种方便且易于存取的方式记录估算的和实际的项目数据
表头	输入如下信息： ● 你的名字和今天的日期； ● 程序名字和程序号； ● 教师的名字； ● 你将使用的编写这个程序的语言。

续表

目 的	本表以一种方便且易于存取的方式记录估算的和实际的项目数据
总结 Minutes/LOC	在开发之前： ● 登入计划的开发每行代码的分钟数。使用在作业编号日志中最近开发项目的或者在最近开发项目总结表中的累计效率值作为参考。 在开发之后： ● 用全部的开发时间除以实际的代码行数，得到实际的开发效率的值； ● 例如，如果项目总共花了 196 分钟，代码行数是 29，那么 Minutes/LOC 的值就是 $196/29 = 6.76$。
LOC/Hour	在开发之前： ● 用 60 除以计划的“Minutes/LOC”，就得到计划的“LOC/Hour”的值。 在开发之后： ● 用 60 除以实际的“Minutes/LOC”，就得到实际的“LOC/Hour”的值； ● 对于上面的示例，Minutes/LOC = 6.76，那么，LOC/Hour = $60/6.76 = 8.88$。
程序规模(LOC)	在开发之前： ● 在计划部分登入估算的新开发的与更改的代码总行数，并登入最大值，最小值。 在开发之后： ● 统计并登入实际新开发的与更改的代码行数； ● 在累计一栏中，登入实际新开发的与更改的代码行数与上一个程序的新开发的与更改的代码行数的累计值之和。
开发阶段时间 计划	用估算的新开发的与更改的代码行数，乘以估算的开发效率“Minutes/LOC”，得到项目的总开发时间； 使用估算的代码行的最大值乘以估算的开发效率得到开发时间的最大值； 使用估算的代码行的最小值乘以估算的开发效率得到开发时间的最小值； 从最近的上一个程序的项目计划总结表中，找到各个阶段所花费的时间的累计百分比，乘以总开发时间，就得到各个阶段的开发时间。
实际	在作业完成之后，根据时间记录日志中的数据，登入各个阶段实际花费的时间（用分钟表示）
累计时间	对每个阶段，计算并登入实际开发时间与最近的前一个程序的累计时间之和
累计时间百分比(%)	对于每个阶段，用 100 乘以该阶段累计时间，再除以总的累计时间，此值则为累计时间百分比

11.6 一个计划的示例

学生 X 完成程序 9 时填写的项目计划总结表的计划部分如表 11.4 所示。他使用的数据来

自程序 8 的项目计划总结表, 见表 11.5。表 11.4 中各条目的填写方法如下:

表 11.4

PSP 项目计划总结表

学生	学生 X	日期	10/21/96
程序		程序号#	9
教师	先生 Z	程序设计语言	Ada

总 结	计 划	实 际	累 计	
Minutes/LOC	7.21			
LOC/Hour	8.32			
Defects/KLOC				
过程效益				
A/FR				
程序规模 (LOC)				
新开发与更改的	23			
最大规模	31			
最小规模	15			
开发阶段时间/min	计 划	实 际	累 计	累计百分比
计划	5			
设计	0			
编码	74			
代码复查				
编译	25			
测试	52			
后置处理	10			
总计	166			
最大时间	224			
最小时间	108			
引入的缺陷	计 划	实 际	累 计	累计百分比
计划				
设计				
编码				
代码复查				
编译				

测试					总计
总计					
存储和缺陷	计划	实际	累计	累计百分比	Def/Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					

表 11.5

PSP 项目计划总结表

学生	学生 X		日期	10/7/96	
程序			程序号#	8	
教师	先生 Z		程序设计语言	Ada	
总 结	计 划	实 际	累 计		
Minutes/LOC	7.82	7.21	7.21		
LOC/Hour	7.67	8.32	8.32		
Defects/LOC					
过程审查					
MTTR					
程序规模 (LOC)					
新开发与更改的	26	19	19		
最大规模	36				
最小规模	18				
开发阶段时间/min	计 划	实 际	累 计	累计百分比	
计划	10	4	4	2.9	
设计	19	0	0	0	
编码	118	61	61	44.6	
代码复查					
编译	12	21	21	15.3	
测试	29	43	43	31.4	

续表

后置处理	15	8	8		5.8
总计	203	137	137		100.0
最大时间	282				
最小时间	141				
引入的缺陷	计划	实际	累计	累计百分比	Def./Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					
排除的缺陷	计划	实际	累计	累计百分比	Def./Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					

● *Minutes/LOC*。在制订程序 9 计划的时，先找到以前程序实际的 *Minutes/LOC* 数据。从表 11.5 中的程序 8 中，取该项的值为 7.21 *Minutes / LOC*。

如果没有更好的方法，在计划新项目时，就先使用从前的实际的值。以后积累一定的数据后，在计划新项目时，就可以不再使用前一个项目的实际值，而使用以前开发的所有项目的平均值。PSP 中的累计百分比将在第 12 章详细讨论，目前主要涉及以下数据。

● *LOC/Hour*。学生 X 计算的 *LOC/Hour* 为 $60/7.21=8.32$ 。

● *程序规模*。学生 X 用第 6 章的方法估计新开发的和更改的代码行数、以及其最大值和最小值。在表 11.4 所示示例中，这些值分别为 23、31 和 15。

● *开发阶段时间的总计*。利用估计的规模为 23 行，每行代码所花费的时间 7.21 *Minutes / LOC*，可以计算出大概的开发时间为 $7.21 \times 23=166\text{min}$ 。

● *开发时间最大值*。最长时间估计为每行代码所花费的时间 (*Minutes / LOC*) 乘以程序规模的最大值： $7.21 \times 31=224\text{min}$ 。

● **开发时间最小值。**最小时间估计为每行代码所花费的时间 (Minutes / LOC) 乘以程序规模的最小值: $7.21 \times 15 = 108\text{min}$ 。

估计总的开发时间为 166min, 再利用表 11.5 中的累计百分比的数据就可以估计出开发程序 9 各个阶段所需时间。学生 X 进行了如下的计算:

- **计划** 估计的计划阶段所用时间为 $2.9 \times 166 / 100 = 4.81$, 或大约 5min。
- **设计** 估计的设计阶段所用时间为 $0 \times 166 / 100 = 0$ 。
- **编码** 估计的编码阶段所用时间为 $44.6 \times 166 / 100 = 74.04$, 大约是 74min。
- **编译** 估计的编译阶段所用时间为 $15.3 \times 166 / 100 = 25.40$, 大约是 25min。
- **测试** 估计的测试阶段所用时间为 $31.4 \times 166 / 100 = 52.12$, 大约是 52min。
- **后置处理** 估计的后置处理阶段所用时间为 $5.8 \times 166 / 100 = 9.63$, 大约是 10min。

值得注意的是, 当第一次用 PSP 开发程序时, 没有以前的项目数据可参考。由于学生 X 在程序 8 的设计时没有历史数据, 只好靠猜测。因此, 通过比较表 11.5 的计划的和实际的数据可以看出: 实际的时间分布与猜测有很大差别。对于程序 9, 学生 X 的计划数据与实际数据比较接近。

11.7 累计时间值计算的示例

在程序 9 的后置处理阶段, 学生 X 统计出新开发的和更改的代码行数为 29, 然后把这个数据记在项目计划总结表中, 如表 11.6 所示。接下来, 把时间记录日志中各阶段的实际时间数据记录在项目计划总结表中。从表中可以看出, 这一次估计的程序规模和时间基本接近实际的结果。学生 X 的实际开发程序时间比计划的要长一点。但是实际的每时代码行数与历史计划数据非常接近。另外, 时间分布与计划也相差不多。

表 11.6 PSP 项目计划总结表

学生	学生 X	日期	10/21/96
程序		程序号#	9
教师	先生 Z	程序设计语言	Ada
总结	计划	实际	估计
Mins/LOC	7.21	6.76	
LOC/Hour	8.32	8.88	
Defects/KLOC			
过程度量			
AFR			
程序规模(LOC)			
新开发与更改的	23	29	29

续表

最大规模	31				
最小规模	15				
开发时间 /min	计划	实际	累计	累计百分比	
计划	5	11	15	4.5	
设计	0	12	12	3.6	
编码	74	85	146	43.9	
代码复查					
编译	25	28	49	14.7	
测试	52	49	92	27.6	
后置处理	10	11	19	5.7	
总计	166	196	333	100.0	
最大时间	224				
最小时间	108				
引入的缺陷	计划	实际	累计	累计百分比	Def./Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					
排除的缺陷	计划	实际	累计	累计百分比	Def./Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					

为了完成表 11.6 的累计时间值的计算, 学生 X 将此表与表 11.5 中此列的值加起来。表 11.6 中累计时间百分比的值为每个阶段的累计时间值除以总计的累计时间值 333 再乘 100。这些值的计算过程如下:

- 总的新开发的与更改的代码行数 累计 LOC=19+29=48.
- 计划阶段 累计计划时间=4+11=15.
累计计划时间百分比=100×15/333=4.5.
- 设计阶段 累计设计时间=0+12=12.
累计设计时间百分比=100×12/333=3.6.
- 编码阶段 累计编码时间=61+85=146.
累计编码时间百分比=100×146/333=43.9.
- 编译阶段 累计编译时间=21+28=49.
累计编译时间百分比=100×49/333=14.7.
- 测试阶段 累计测试时间=43+49=92.
累计测试时间百分比=100×92/333=27.6.
- 后置处理阶段 累计后置处理时间=8+11=19.
累计后置处理时间百分比=100×19/333=5.7.
- 总开发时间 总开发时间累计=137+196=333.

使用程序 9 的数据, 学生 X 可以估计出下一个项目中花费在每个阶段的时间。不过为了更加实用, 应该使用几个项目的平均时间。累计和累计百分比列就是这样计算的。对于表 11.5 程序 8 的累计列数据使用了学生 X 开发程序的实际数值。当开发程序 9 时, 他把程序 8 和程序 9 的实际数据加起来得到新的累计数据。这样, 新的累计百分比数据也就是程序 8 和程序 9 的平均数。同理, 对于程序 10 的项目计划总结表, 可以使用程序 8、程序 9 和程序 10 的平均数, 依此类推。

项目不同, 总的开发时间也是不一样的。但是, 时间在各个阶段的分布却比较稳定。当然, 这取决于过程的质量。例如, 当在编译和测试中花费大量时间时, 由于大量不可预测的时间花在修复程序缺陷上, 计划的阶段时间就不够准确。然而, 对几个程序的时间取平均值时, 花在编译和测试阶段的时间的平均值就变化不大。这就是说, 只要不改变过程, 这些值的变化就不大。最初应用 PSP 时, 可能有三分之一到一半的时间花费在编译和测试阶段来查找和修复程序的缺陷。当使用后面章节所讲述的 PSP 方法, 你就能减少出现在编译和测试阶段的程序缺陷, 从而减少编译和测试时间。这将节省开发时间, 改善过程的可预测性, 提高计划的准确度, 并编写出更好的程序。

11.8 总 结

本章定义了过程, 描述了基本的 PSP 过程, 并且说明已定义过程如何帮助你制订更好的计划。扩展了项目计划总结表, 使它包括项目阶段时间, 计算每个阶段的累计时间和所花时间的百分比。在制订项目计划时, 使用前面程序累计百分比的经验值, 可以估计出花费在各个阶段的时间。在使用 PSP 时, 每个项目都用到一个项目计划总结表。在项目开始时登入计划部分的值; 当项目完成时, 在实际列登入实际数据并计算累计和累计百分比的值。

11.9 练习 11

利用表 11.2 所示的项目计划总结表为下一个要设计的程序制订一个计划。先用第 6 章所学内容估计程序规模。对于第一个 PSP 程序，猜测各个阶段的时间分布。今后利用项目计划总结表中的累计百分比那一列的数值来估计各个阶段的时间。开始作业前记录估计的数据，完成作业后记录实际的规模数据。

另外，如果以前没有交过时间记录日志、作业编号日志和每周活动总结表的学生，请交一份这些日志和总结表。

参考文献

[Humphrey 95] Humphrey, W.S. *A Discipline for Software Engineering*. Reading, MA: Addison - Wesley, 1995

第 12 章 缺 陷

本章描述的主题是软件缺陷（为什么称为缺陷而不叫 bug，见 12.4 节）。在用户使用软件产品时，其中的缺陷可能导致严重问题，而且发现和修复这些缺陷的代价是很大的。由于缺陷是由开发人员的错误引起的，所以软件工程师需要了解他们引入的缺陷，并且学会如何对它们进行管理。缺陷管理的第一步是收集引入到程序中的缺陷的有关数据。有了这些数据，就可以设计出更好的方法来发现和修复这些缺陷。本章的练习是要收集和报告程序中的缺陷数据。

到目前为止，本书只讨论了费用和进度的管理方法。这只是全部内容的一半。从本章开始，我们将讨论交付高质量软件产品的必要性。首先必须给质量下一个定义。

12.1 什么是软件质量

软件质量影响到开发费用、交付日期和用户满意度。因为软件质量如此重要，所以有必要先来讨论一下质量一词的含义。软件产品质量的定义应该对于产品的用户很有意义。因此，一个能给它的用户提供最重要的能力的产品是一个高质量的产品。用户的需要一般在需求文档中陈述。这些需求很重要，所以扩展需求、澄清需求和细化需求本身就是一个很大的主题。因此，本书不再对需求作进一步的讨论。不过一定要记住，直到已经有了明确的需求，才可开发出高质量的软件。在开始时可能没有清晰而明确的需求，但在结束之前，必须对需求有充分的了解。

软件质量是一个很大的主题，本书只讨论其中的一部分。书中提供一些必要的技能和实践来帮助理解自己引入的缺陷。有了这些知识，就能有效地发现和修复自己程序中的大部分缺陷，通过有关缺陷的数据可以使你在将来的开发中避免这些缺陷。最终，一旦能有效地管理缺陷，就可以把更多的注意力放在程序的可用性及其价值的质量因素。

12.2 缺陷和质量

在第 1 章中我们曾经讲到，软件工程师的任务就是按照预定的费用和进度交付高质量的

产品。软件产品既必需满足用户的功能要求，又必须稳定可靠地完成用户的作业。这里，能完成作业是关键点。尽管对使用者来说最重要的是软件的功能，但这些功能只有在软件能够正常运行时才能使用。为了能让软件运行，就必须排除软件中的缺陷。软件质量体现在许多方面，但首先要面对的而且必须解决的方面是软件缺陷。这并不是说只需要注意软件缺陷，或者缺陷是软件质量的最重要方面，而是说只有在排除了绝大部分程序缺陷后，才有可能考虑软件质量的其它目标。即使软件能够执行，如果缺陷太多，不管其它方面的质量如何，大型软件系统就不会正常运行，也没有人会使用它们。

缺陷之所以重要是因为人们会犯很多错误。事实上，即使是有经验的程序员，一般开发7~10行源程序就会引入一个缺陷。一般在编译和测试阶段能够发现和纠正大部分的缺陷，但还有许多缺陷仍然留在最终产品中。显然，首要的任务是要弄清引入的缺陷，并尽可能防止它们的引入。为此，就需要精通所用的程序设计语言，深入理解开发支持环境，掌握要开发的应用系统的类型。为了减少引入的缺陷，这些步骤及其它一些步骤都是必要的。

12.3 什么是缺陷

缺陷是指程序中存在的错误，例如语法错误、拼写错误、标点符号错误或者是一个不正确的程序语句。缺陷可能出现在程序中和设计中，甚至在需求、规格说明或其它的文档中。缺陷可能是冗余的语句、不正确的程序语句或是被忽略的程序部分。事实上，缺陷是任何影响到程序完整而有效地满足用户要求的东西。因此，一个缺陷是客观的事物，是可以标识、描述和统计的。

简单的编码错误可能产生导致破坏性后果的或者引起难以发现的缺陷。相反，许多复杂的设计缺陷通常很容易被发现。因此，设计错误的复杂性和所导致的缺陷的影响没有直接关系。一些微小的编码错误却可能引起严重的系统问题。事实上，绝大多数软件缺陷都源于程序员的疏忽大意。尽管设计问题一直很重要，但在第一次编写程序时，设计缺陷通常总是比由疏忽大意或生疏引起的缺陷要少。因此，为了提高程序质量，软件工程师学会管理所有引入到程序中的缺陷是很重要的。

把查找和标识缺陷与确定缺陷起因这两个问题区分开是很重要的。单纯地统计和记录软件产品中的缺陷并不能说明起因或追究责任。然而，缺陷确实是有起因的。它可能是由于拼错了一个参数名、忽略了一个标点或错误地调用了过程。这些错误都会引起缺陷。事实上，所有的缺陷都源于人的错误，而且软件工程师所犯的许多错误会引入程序缺陷。

错误是人做的不正确的事情，无论是谁或无论是在什么时候做的，缺陷都是程序中的故障元素。因此，是由于人们所犯的错误而导致程序有缺陷。当程序员犯了错误并因此产生缺陷时，我们称之为引入缺陷。这就意味着为了减少引入程序中的缺陷，就必须改变所做的事情。然而，为了排除产品中的缺陷，通常必须找到它们。因此，缺陷排除的过程比缺陷预防的过程要简单。缺陷预防是一个重要的也是很大的话题，需要对软件开发的全过程有深入的研究[Humphrey 89]。本书余下章节集中讨论缺陷排除问题。

除非工程师发现并修复已经引入的缺陷,否则这些缺陷将留在最终产品中。问题是发现和修复软件缺陷需要大量的时间和费用。为了减少缺陷,就必须研究已经引入的缺陷,确定引起这些缺陷的原因,并学会在将来如何避免重复同样的错误。由于有缺陷的产品测试费用高、难以修复、甚至严重影响使用,所以学会将产品中残留的缺陷减少到最小程度是非常重要的。本书将告诉你怎样做到这一点。

减少缺陷对于每个软件工程师都非常重要,这不仅是因为缺陷不仅影响用户的使用,而且一般的软件组织要用一半以上的精力来查找和修复缺陷。由于测试时间难以预计,产品缺陷常常是产生超支和延期的主要原因。

12.4 缺陷与 Bug

有人把软件缺陷称为 Bug,这是不正确的。当称为 Bug 时,令人想到的是那些讨厌的小虫子,应该把它们拍死或者对它们不予理睬。这会使一些重要的问题被视为琐碎小事,会养成一种错误的态度。当一个软件工程师说程序中只剩下几个 Bug 时,你可能感到一阵轻松。可是设想一下,如果我们称它们为定时炸弹而不是 Bug,这时如果程序员告诉你,程序已经进行了全部测试,程序中仅留有几个定时炸弹,你还会同样有轻松的感觉吗?只是用了不同的术语,你的态度就会完全改变。

缺陷并不像无足轻重的 Bug,而更像是定时炸弹。尽管不是所有的缺陷都会产生爆炸性的后果,但有些确实会这样。当程序广泛应用并且以设计者未曾预想到的方式使用时,看起来很小的错误都可能产生不可预测的后果。一些广泛应用的软件系统为满足新的需求而改进时,潜在的问题就会暴露出来,这些看起来很小的缺陷可能真的会产生破坏性的后果。

在这一点上,那些编过一些程序的读者可能会不赞同,觉得有些夸大其词。在某种意义上讲也许是这样。绝大部分细小缺陷没有引起严重后果。然而不幸的是,很小一部分看起来无关紧要的缺陷却能引起严重问题。有这样一个示例,一个简单的初始化错误引起缓冲区溢出,从而导致铁路控制系统数据丢失。在停机之后,系统无法迅速重新启动。这样,几公里铁路上的火车只好停运几个小时,以等待重新载入必要的数据。

程序中有一部分缺陷可能引起难以预料的后果。如果事先知道有哪些缺陷,我们只要修复它们,以后就不再为此担心了。但不幸的是,没有办法做到这一点。任何漏掉的缺陷都潜在地有引起严重后果的危险。尽管许多程序并不是用于那些会导致失败的应用系统,但是愈来愈多的程序将会用于这种场合。那么,虽然现在缺陷对你来说还不是一个重要问题,但很快就会成为一个重要问题。因此,现在学会管理缺陷是很重要的,这样可以为你将来生产高质量的程序作好准备。

编写程序的软件工程师本人最适合查找和修复程序中的缺陷。因此软件工程师应该承担所生产的软件的质量的责任,这一点很重要。学会写出无缺陷程序是一个巨大的挑战。这不是任何人都能很快地或轻易地做到的。它需要数据、有效的技术和技能。通过使用本书所描述的方法,可以开发和锻炼你的能力,以便能生产出高质量的程序。总之,如果你不朝着设

计无缺陷程序的方向努力，你永远也不可能做到这一点。

12.5 缺陷类型

在分析缺陷时，将缺陷进行分类是有帮助的。本书将缺陷分成 10 个类型。通过对缺陷分类，可以迅速找出那一类缺陷的问题最大，然后集中精力预防和排除这一类缺陷。而这正是缺陷管理的关键。把精力集中到最容易引起问题的几类缺陷上。一旦这几类缺陷得到控制，再进一步找到新的容易引起问题的几类缺陷上。照此办法继续下去。

本书中所用的缺陷类型如表 12.1 所示，这是 Chillarege 和他的同事在 IBM 研究院的工作成果[Chillarge]。他们在广泛研究 IBM 各种产品的缺陷的基础上，确定缺陷的主要类型。实践证明，这种缺陷分类方法也对 PSP 有益[Humphrey 95]。

在表 12.1 中，缺陷的分类是把几种具有普遍性的问题进行分类。例如，类型 20，语法缺陷，是指所有那些不符合程序设计语言规格说明的语句，例如，丢掉分号、不正确的 if 语句结构、拼写错误或错误的说明。也就是说，类型 20 的缺陷包括所有那些导致语法错误的缺陷，不管这些缺陷是如何引起或如何发现的。另一种缺陷类型 80 是指函数缺陷。比如说在 do_while 循环条件中的逻辑错误。

表 12.1 中的缺陷类型等级是按照引起缺陷的问题的复杂度来排列的。例如类型 10、20、30 缺陷一般是由于简单的疏忽和错误引起的。然而类型 80、90、100 缺陷一般是涉及到复杂的设计和系统问题。

表 12.1 缺陷类型标准

类型编号	类型名称	描述
10	文档	注释，消息
20	语法	拼写，标点符号，打字，指令格式
30	联编打包	变更管理，库，版本控制
40	赋值	说明，重名，作用域，限制
50	接口	过程调用和引用，输入/输出，用户格式
60	检查	出错信息，不合适的检查
70	数据	结构，内容
80	函数	逻辑，指针，循环，递归，计算，函数缺陷
90	系统	配置，计时，内存
100	环境	设计，编译，测试，其它支持系统问题

不要急于把 PSP 的 10 种类型的每一类细分出若干子类，直到你已经收集到大量程序的缺陷数据。那时，才能够看出哪里需要更详细以及补充什么样的信息才是最有用的。例如，

你可以把类型 20 语法缺陷细分成类型 21, 分号错误; 类型 22, 其它标点符号错误; 类型 23, 布尔表达式问题; 类型 24, 错误的指令格式, 等等。至少要收集到 100 个以上的缺陷数据, 才能考虑把一个缺陷类型细分成若干个子类型。即使如此, 你可能也会发现, 最初的缺陷类型分类已经足够了。

12.6 了解缺陷

缺陷管理的第一步是了解缺陷。为此, 必须首先收集缺陷数据, 然后才能了解这些缺陷, 并且找出如何预防它们。同时也能领会到如何更好地发现、修复甚至预防仍在引入的缺陷。请按照以下步骤收集程序中关于缺陷的数据:

- 为程序中发现的每一个缺陷做一个记录。
- 对于每个缺陷要记录足够详细的信息, 以便以后能更好地了解这个缺陷。
- 分析这些数据以找出哪些缺陷类型引起大部分的问题。
- 设计出发现和修复这些缺陷的方法。

发现自己引入程序中的缺陷只是事情的一个方面, 另一方面你总有一天要了解别人在你的程序中发现的缺陷。由于这些缺陷已经逃过你所有的缺陷预防和检测手段, 所以它们对于了解和确定你的个体软件过程的弱点是非常重要的。这些缺陷称为漏网者, 因为它们能够逃过你所有的缺陷排除手段。当改善个体过程时, 这些漏网者将成为个体过程改善的主要数据来源。

12.7 缺陷记录日志

设计缺陷记录日志是为帮助收集缺陷数据。日志如表 12.2 所示, 它的使用说明见表 12.3。使用这种日志收集你所编写的每个程序的缺陷数据。尽可能详细地描述每个缺陷, 以便以后仍可理解它。完成每个程序后, 分析有关数据, 看一看程序哪些地方引入和排除了缺陷, 是哪种缺陷导致主要的问题。使用日志之前, 先阅读本章余下各节和表 12.3 中的使用说明。以下各节用一个缺陷记录日志的示例来说明如何完成日志, 如表 12.4:

1. 当开始开发一个程序时, 找几张缺陷记录日志表, 在第一页的表头登入所需要的内容; 登满第一张后, 按同样的方式使用第二张表格。

2. 当遇到第一个缺陷时, 在日志上记下它的编号, 直到修复了这个缺陷时, 再填写其它的数据。当学生 X 第一次编译程序 10 时, 编译器显示了 12 个以上的出错信息。尽管最初不知道是什么问题, 但知道至少有一个错误。这样他记下时间, 并在缺陷日志的第一行编号下登入 1, 这是程序 10 的第一个缺陷。这些编号有助于以后分析缺陷数据。在大程序中, 缺陷编号可以用于跟踪程序不正确的修复, 而且有助于进行缺陷预防。

3. 每个缺陷单独占一行。不要把几个相同的缺陷放在同一行上。

4. 记下发现缺陷的日期。如果一天发现了几个缺陷，从第二个起，日期项可以空白直到第二天的第一个输入再登。在表 12.4 中，学生 X 所有的错误都是在 10 月 28 日发现的，因此不必重复登入日期。空白的日期表示和前面的缺陷是在同一天发现的。

5. 在修复缺陷后，登入缺陷类型。有时可能拿不准登入哪种类型更合适，这时要发挥你的判断能力。不必在考虑是哪种缺陷类型更精确上浪费时间。不过，要尽量做到合理并前后一致。例如，在表 12.4 中，学生 X 发现的缺陷 1 是丢掉一个分号。当解决了这个问题，他在缺陷 1 的类型下登入数值 20。

6. 登入引入缺陷的过程阶段。有时可能说不清在哪个阶段引入这个缺陷，但对小程序不会有问題。发挥你的判断能力，但不必花费太多的时间。在这个示例中，学生 X 确信是在程序编码时丢掉分号，所以在引入项下登入“编码”一词。

7. 在排除缺陷后记录过程阶段。这一般是指发现缺陷的阶段。例如，如果是在开始编译时发现的，就在排除项下登入“编译”。此例中对于缺陷 1，学生 X 是在编译阶段发现和修复缺陷，所以在排除项下登入“编译”一词。

8. 对于缺陷修复时间，从开始意识到缺陷的存在时算起，直到修复完成且检查无误的估计时间。学生 X 开始修复缺陷 1 时记下时间，当解决问题并且确认修复完成时再看一下表，这样就能知道花了 1 分钟。一般地，修复编译缺陷需要花费大约 1 分钟左右的时间。对于测试阶段发现的缺陷修复需要的时间要长得多。可以用钟表或秒表来测量修复时间，但若修复时间很短，估计一下就可以了。

9. 修复缺陷项说明此缺陷是由于修复其它缺陷而引入的。这一项以后会很重要，但现在可以忽略它。

10. 在描述部分对这个缺陷进行简短的描述。说明要尽可能简洁但又描述清楚。例如只登入一个“；”来指明丢失分号。对于一个复杂的逻辑缺陷就需要写上几行，如果内容放不下，可以用下一个缺陷的说明部分。对于缺陷 1，学生 X 简单记下“遗漏；”，对于表 12.4 中的大部分缺陷，给出更详细的说明。因为说明只是由自己使用，只要能提示你记起问题即可，不必过细。

缺陷类型：	
10 文档	60 检查
20 语法	70 数据
30 联编打包	80 函数
40 赋值	90 系统
50 接口	100 环境

表 12.2

缺陷记录日志

学生 _____ 日期 _____
教师 _____ 程序号 _____

日期	编号	类型	引入阶段	排除阶段	修改时间	修复缺陷
	描述：					

续表

日期	编号	类型	引入阶段	排除阶段	修改时间	修复缺陷
	描述:					
	描述:					
	描述:					
	描述:					

表 12.3

缺陷记录日志使用说明

目 的	记录所发现和修复的每一个缺陷的详细数据。这些数据要用来填写项目计划总结表
一般性说明	记录所有在复查、编译和测试阶段发现的缺陷： 每个缺陷要分别记录，且信息应该完整。 如果一张表不够，请使用另一张表。
表头	如下输入： ● 你的名字 ● 今天的日期 ● 教师的名字 ● 程序号
日 期	登入发现缺陷的日期
编 号	每个缺陷给一个编号； 对于每个程序，使用一个序列号，如从 1 或 001 开始编号。
类 型	从表 12.1，（已把摘要列在缺陷记录日志的右上角）的缺陷类型单中选出这个缺陷的类型。 按照自己的判断来选择缺陷的类型。
引入阶段	登入缺陷引入的阶段； 根据自己的判断。
排除阶段	登入排除缺陷的阶段。 一般是发现和修复这个缺陷的阶段。
修复时间	估计或测量发现和修复这个缺陷所需要的时间。如果你愿意可以使用秒表来计算
修复缺陷	此时你可忽略此项的填写。 如果在修复另一个缺陷时引入了此缺陷，记录这不正当的修复缺陷的编号。 如果不能确定这个相关缺陷的编号，请在修复缺陷框中登入 X。
描 述	登入对当前缺陷的简单描述； 描述应该足够清楚，以便今后可以看出该缺陷的起因以及你为什么作出这种判断。

缺陷类型:	
10 文档	60 检查
20 语法	70 数据
30 联编打包	80 函数
40 赋值	90 系统
50 接口	100 环境

表 12.4

缺陷记录日志的示例

学生	学生 X	日期	10/28/96
教师	先生 Z	程序号	10

日期	编号	类型	引入阶段	排除阶段	修复时间	修复缺陷
10/28	1	20	编码	编译	1 min.	
	描述: 漏掉了分号“;”					
	2	20	编码	编译	1 min.	
	描述: 漏掉了分号“;”					
	3	40	设计	编译	1 min.	
	描述: 二元操作变量 Rhs 有类型错误, 必须将整型转换为浮点型					
	4	40	编码	编译	1 min.	
	描述: 二元操作变量 Rhs 有类型错误, 常量应为 0.0, 而不是 0					
	5	40	编码	编译	1 min.	
	描述: 二元操作变量 Rhs 有类型错误, 必须将整型转换为浮点型					
	6	40	设计	编译	7 min.	
	描述: 指数必须为整数。使用数学库计算 Sqrt。整数计算不正确					
	7	80	编码	测试	14 min.	
	描述: 答案不正确 (std.dev.)					
	8	80	编码	测试	28 min.	
	描述: 对负指数的循环没有终止, 忘记减小循环变量					

人们经常搞不清楚缺陷类型, 有人认为应该有一个特殊类型来说明那些误解或容易混淆的类型。例如, 如果不理解用户需求或对开发环境不熟悉, 就可能产生很多错误。这种情况很重要, 但它与缺陷的起因相关。当考虑缺陷类型时, 应该注意两个问题: 产品中是否有错误; 如果有, 则产品缺陷的类型是什么。所以, 为了预防缺陷了解错误的起因是必要的, 但缺陷类型只描述产品中哪里不正确。

12.8 统计缺陷个数

缺陷的定义似乎看起来很明白, 其实并不是这样。例如, 在编译阶段, 只需要对程序所

做的修改进行统计。也就是说，当在程序中漏掉一个分号时，编译器可能给出 10 条出错信息，但这里只有漏掉分号这一个缺陷。因此，不管修复的性质和编译器给出的出错信息的数目，对于程序的每一次修正，在缺陷记录日志中只记录一条缺陷。

同样，当在编码阶段发现设计问题时，把它记为一个设计缺陷。然而，在设计时，对如何实现却可能经常改变想法。如果你是在纠正需求或规格说明中的一个错误，就记录为一个需求或规格说明缺陷。如果想到一个更好的设计方法，这并不是一个缺陷。你也可能经常遇到这种情况，错误一出现就被发现和修复。这样的调整是创造性思维中的正常情况而不是缺陷。关键是记录那些当你完成初始设计或编码后仍然留在产品中的缺陷。

例如，在键入一行代码后立即发现并修复了一个拼错的参数名，这个拼写错误不是缺陷。可是，如果等到编码完成后，同样的拼写错误就是一个缺陷，并应该对它进行统计。因此，如果习惯在输入每一行代码后立即进行检查，这时发现的缺陷不必记录在内。

当完成一个产品或产品的一部分的一个阶段后，就应该开始统计这个阶段的缺陷数。例如设计阶段结束后，就应该开始记录在设计中发现的缺陷。可是，假设你在编写两个程序过程。编完第一个后，没有马上编译而是开始编写第二个。在第二个程序过程的编写过程中，你发现在第一个程序过程的代码中有一个写错的参数名。这就要记录为一个缺陷。因为，虽然仍在编码阶段，可是你已经完成了第一个过程的编写工作。

注意，本书中不要求你统计在设计或编码阶段发现的缺陷。最初，把精力集中在编译或测试阶段发现的缺陷是非常重要的。一旦习惯了收集缺陷数据，你就能更好地理解这些缺陷数据的用途。然后我可能希望对在设计和编码阶段可能出现的错误知道得更多一些。因为大多数的缺陷是在设计和编码阶段引入的，所以必须关注这些阶段，以寻找缺陷的起因和避免缺陷出现的方法。不过，现在我们还是从编译和测试阶段发现缺陷开始。

12.9 使用缺陷记录日志

为什么要累计缺陷呢？当你收集缺陷数据时，要记住为什么要这样做：

- **提高程序设计水平。**这些缺陷数据可以帮助你改进编写程序的方法。尽管人们很容易对缺陷辩解，但如果不能理解缺陷，就不能管理缺陷。因此，要管理好缺陷，就必须收集有关缺陷的准确数据。如果原谅缺陷，例如不累计编译器中的语法检查器检查出来的错误，那只会自欺欺人。如果你愿意这样，就别指望有所提高。
- **减少程序中缺陷的个数。**每个人都不可避免引入缺陷。但若使用谨慎、适合的方法，就可以减少引入的缺陷数。
- **节省时间。**错误可以引起更多的错误。缺陷在程序中的驻留时间越长，就需要更长的时间查找，而且更难以修复。需求问题会导致不正确的设计，设计问题会引起实现错误。实现错误就在程序中引入了缺陷。这就是为什么排除缺陷越早越好。
- **节约开支。**缺陷带来的费用是昂贵的。单元测试后，在以后的各个测试和维护阶段，其查找和修复缺陷的费用平均以十倍的速度增长。

● **负责地地完成工作。**缺陷是工程师引入的，因此这些工程师有责任查找和修复这些缺陷。

12.10 更新的PSP过程

更新的PSP过程脚本如表12.5所示。新增加的内容主要是缺陷数据的收集和记录。更新的项目计划总结表如表12.6所示，它的使用说明见表12.7。注意，表中不要求记录后置处理阶段引入或排除的缺陷。尽管一般不会在后置处理阶段发现或引入缺陷，但这种可能性还是有的。一个示例就是在统计新增加的和修复的源程序行数时发现缺陷。在后置处理阶段可能为了修复一个缺陷而出现了错误，从而引入新的缺陷。下面几段解释如何完成表12.8项目计划总结表中新增加部分。

表 12.5 **PSP 过程脚本**

过程目的	指导用户进行小程序的开发。
入口准则	问题描述； PSP 项目计划总结表； 以前开发程序的实际的规模和时间数据； 时间记录日志； 缺陷记录日志。
1 计划	获取对程序功能的描述； 估计整个程序的代码行数及其最大值和最小值； 确定开发效率 (Minutes/LOC)； 计算总开发时间及其最大值和最小值； 将计划数据登入项目计划总结表； 将计划阶段所花费的时间记入时间记录日志。
2 设计	设计程序； 按照指定的格式记录设计文档； 将设计阶段所花费的时间记入时间记录日志。
3 编码	实现设计； 使用标准的格式来书写程序代码； 将编码阶段所花费的时间记入时间记录日志。
4 编译	编译程序； 修复和记录所有发现的缺陷； 将编译阶段所花费的时间记入时间记录日志。
5 测试	测试整个程序； 修复和记录所有发现的缺陷； 将测试阶段所花费的时间记入时间记录日志。
6 后置处理	将实际的规模、时间和缺陷数据登入项目计划总结表； 将后置处理阶段所用的时间记入时间记录日志。

续表

过程目的	指导用户进行小程序的开发。
出口准则	经过详尽测试的程序； 完整的设计文档； 完整的源程序清单； 已经填写完成的项目计划总结表； 已经填写完成的时间 <i>和</i> 缺陷日志。

表 12.6

PSP 项目计划总结表

学生	_____	日期	_____
程序	_____	程序号#	_____
教师	_____	程序设计语言	_____

总 结	计 划	实 际	累 计	
Minutes/LOC	_____	_____	_____	
LOC/Hour	_____	_____	_____	
Defects/KLOC	_____	_____	_____	
过程效益				
A/FR				
程序规模 (LOC)				
新开发的与更改的	_____	_____	_____	
最大规模	_____	_____	_____	
最小规模	_____	_____	_____	
开发阶段时间 /min	计 划	实 际	累 计	累计百分比
计划	_____	_____	_____	_____
设计	_____	_____	_____	_____
编码	_____	_____	_____	_____
代码复查				
编译	_____	_____	_____	_____
测试	_____	_____	_____	_____
后置处理	_____	_____	_____	_____
总计	_____	_____	_____	_____
最大时间	_____	_____	_____	_____
最小时间	_____	_____	_____	_____
引入的缺陷	计 划	实 际	累 计	累计百分比
计划	_____	_____	_____	_____
设计	_____	_____	_____	_____

					续表
引入的缺陷	计划	实际	累计	累计百分比	Def./Hour
编码					
代码复查					
编译					
测试					
总计					
排除的缺陷	计划	实际	累计	累计百分比	Def./Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					

表 12.7

PSP 项目计划总结表使用说明

目 的	本表以一种方便而且易于存取的方式，记录估算的和实际的项目数据
表头	如下输入： ● 你的名字和今天的日期 ● 程序名和程序号 ● 教师的名字 ● 你将使用的编程语言
Minutes/LOC	在开发之前： ● 登入计划的每行代码的开发分钟数(Minutes/LOC)：使用在作业编号日志中最近开发项目的累计效率值或最近的项目计划总结表中的累计效率值作为参考。 在开发之后： ● 用全部的开发时间除以实际的程序规模，得到实际的和累计的 Minutes/LOC； ● 例如，如果该项目总共花了 196 分钟，代码行数 是 29，那么 Minutes/LOC 的值就是 $196/29 = 6.76$。
LOC/Hour	在开发之前： ● 用 60 除以计划的“Minutes/LOC”就得到了计划的“LOC/Hour”的值。 在开发之后： ● 用 60 除以实际的和累计的“Minutes/LOC”就得到了实际或和累计的“LOC/Hour”的值； 对于上面的示例，实际的 Minutes/LOC = 6.76，那么，实际的 LOC/Hour = $60/6.76 = 8.88$。

续表

目 的	本表以一种方便而且易于存取的方式, 记录估算的和实际的项目数据
程序规模(LOC)	在开发之前: - 在计划部分登入估算的全部新开发的与更改的代码行数及其最大值和最小值。 在开发之后: - 统计并登入实际新开发的与更改的代码行数; - 在累计一栏中, 把实际新开发的与更改的代码行数与上一个程序的新开发的与更改的代码行数相加, 得到累计值。
开发阶段时间 计划	用估算的新开发的与更改的代码行数, 乘以估算的开发效率“Minutes/LOC”, 得到项目的总开发时间; 使用估算的代码行的最大值乘以估算的开发效率得到开发时间的最大值; 使用估算的代码行的最小值乘以估算的开发效率得到开发时间的最小值; 从最近的上一个程序的项目计划总结表中, 找到各个阶段所花时间的累计百分比, 乘以总开发时间, 就得到各个阶段的开发时间。
实际	在作业完成之后, 根据时间记录日志中的数据登入各个阶段实际花费的时间
累计时间	对每个阶段, 计算并登入实际开发时间与最近的上一个程序的累计时间之和
累计时间百分比(%)	对于每个阶段, 用 100 乘以该阶段累计时间, 再除以总的累计时间, 得累计时间百分比
引入的缺陷 实际	在开发完成后, 统计并登入各个阶段实际引入的缺陷的数目
累计	对每个阶段, 计算并登入实际引入的缺陷个数与最近的上一个程序的累计缺陷引入个数之和
累计百分比(%)	对每个阶段, 用 100 乘以该阶段累计缺陷, 再除以总的累计缺陷, 得累计缺陷百分比; 即登入该阶段累计引入缺陷值占整个项目缺陷引入累计值的百分比
排除的缺陷 实际	在开发完成后, 统计并登入各个阶段实际排除的缺陷数
累 计	对每个阶段, 计算并登入实际排除的缺陷个数与最近的上一个程序的累计缺陷排除个数之和
累计百分比(%)	对每个阶段, 用 100 乘以该阶段累计缺陷, 再除以总的累计缺陷, 得累计缺陷百分比; 即登入该阶段累计排除缺陷个数占整个项目缺陷排除累计值的百分比

表 12.8

PSP 项目计划总结表的示例

学生	学生 X	日期	10/28/96
程序		程序号#	10
教师	先生 Z	程序设计语言	Ada
总 结	计 划	实 际	累 计
Minutes/LOC	6.76	6.12	6.50
LOC/Hour	8.88	9.80	9.23

续表

总 结	计 划	实 际	累 计		
<i>Defects/KLOC</i>					
<i>过程效益</i>					
<i>A/FR</i>					
程序规模 (LOC)					
新开发的与更改的	44	57	105		
最大规模	58				
最小规模	30				
开发时间 (Minutes)	计 划	实 际	累 计	累计百分比	
计划	13	18	33	4.8	
设计	11	43	55	8.1	
编码	130	162	308	45.2	
代码复查					
编译	44	21	70	10.2	
测试	82	73	165	24.2	
后置处理	17	32	51	7.5	
总计	297	349	682	100.0	
最大时间	392				
最小时间	203				
引入的缺陷	计 划	实 际	累 计	累计百分比	<i>Def./Hour</i>
计划					
设计		2	2	25.0	
编码		6	6	75.0	
代码复查					
编译					
测试					
总计		8	8	100.0	
排除的缺陷	计 划	实 际	累 计	累计百分比	<i>Def./Hour</i>
计划					
设计					
编码					
代码复查					
编译		6	6	75.0	
测试		2	2	25.0	
总计		8	8	100.0	

在后置处理阶段，复查缺陷日志中统计的每个阶段引入的缺陷数。从表 12.4 所示的缺陷记录日志中，学生 X 先统计在设计阶段引入的缺陷(第 3 号和第 6 号)共 2 个，所以在表 12.8 的设计行的实际这一列下登入 2。另外 6 个缺陷都是编码阶段引入的，所以在编码行登入 6。总计有 8 个引入缺陷。最初可能是在编码阶段引入大量缺陷，而在设计、编译、测试阶段引入很少的缺陷。对于更复杂的程序偶尔也可能在计划阶段引入错误。

然后，统计各个阶段排除的缺陷。学生统计出编译阶段排除的缺陷是 6 个，测试阶段是 2 个，所以在排除缺陷的相应行登上 6 和 2。总计也是 8。通过 PSP，你能在编译阶段就发现大部分的缺陷。测试一般需要更多的时间，因为在这一阶段更难发现和修复缺陷。

在记录了引入和排除的缺陷个数之后，使用与处理时间数据相同的累计方法完成累计和累计百分比（见第 11 章的 11.7 节）。这些数据直到第 15 章，开始估计引入和排除的缺陷个数时才会用到。

通过使用累计百分比，工程师估计他们引入和排除的缺陷数目准确得令人吃惊。人是有习惯的生物，习惯决定着我们的错误。如果我们不改变这些习惯，还会犯类似的错误。因此，除非你做出一些大的调整，如采用不同的过程，开发更复杂的应用程序，改变开发环境，否则你可能会在下一个程序中引入与上一个程序相同数量的缺陷。

用和以前同样的方法完成项目计划总结表的余下部分。但还有两点应当注意：第一，Minutes/LOC 行的累计列是用总计开发时间的累计值除以新开发和更改的代码行的累计值，即 $682/105=6.50$ Minutes/LOC。第二，累计的 LOC/Hour 是用 60 除以累计的 Minutes/LOC，即 $60/6.50=9.23$ LOC/Hour。注意，有了这些累计百分比的数据，就不必跟踪作业编号日志上的与程序开发有关的单元和比率。在日志中可以方便地应用项目信息，所以建议你使用作业编号日志来跟踪程序。如果你能像我一样有经验，就能迅速累加出大量数据。我已经跟踪过 200 个作业，并把有关项目数据保存在作业编号日志中。我发现，使用作业编号日志是记录历史数据和查找指定项目数据的好方法。

12.11 总 结

软件质量是要满足用户的要求并且要能可靠而稳定地完成用户的工作。这就要求你开发的软件完全没有或几乎没有缺陷。

软件缺陷是软件产品中的不正确的东西。缺陷是由人为的错误引起的。因为查找和修复缺陷的代价很大，所以最有效的方法是工程师能及时发现和修复引入的缺陷。

管理缺陷的第一步是了解它们。为此，程序员需要收集和分析这些数据，并决定怎样才能更好地预防、发现和修复这些缺陷。

本书用的缺陷类型标准是一个简单的标准，是为了帮助你找出最重要的缺陷类别。当你有一些缺陷数据后，你就可以扩充这些类别。使用缺陷记录日志收集缺陷数据。对于每个缺陷在该日志上记录一个条目。对于每个程序，在项目计划总结表中总结这些数据。

12.12 练习 12

使用缺陷记录日志记录你所编写的程序中发现的每个缺陷。标识出在那里发现了缺陷，每个缺陷作为一个单独的条目列出。完整地描述每个缺陷。对于每个程序，在项目计划总结表中总结缺陷数据。

另外，提交一份你以前没有提交过的时间记录日志、作业编号日志和周活动总结表。对每个新开发的程序，交一份完整的计划表和缺陷记录日志。

参考文献

[Chillarege] Chillarege, Ram, Inderpal S. Bhandari, Jarir K. Chaar, Michael J. Halliday, Diane S. Moebus, Bonnie K. Ray, and Man-Yuen Wong. "Orthogonal Defect Classification — A Concept for In-Process Measurements." *IEEE Transactions on Software Engineering*, vol. 18, no. 11, Nov. 1992, pp. 943~956

[Humphrey 89] Humphrey, W. S. *Managing the Software Process*. Reading, MA: Addison-Wesley, 1989

[Humphrey 95] Humphrey, W. S. *A Discipline for Software Engineering*. Reading, MA: Addison-Wesley, 1995

第 13 章 缺陷查找技术

有各种各样的方法用来发现缺陷。本章概要地总结了几种主要的方法，并详细说明了代码复查这一特定方法是怎样提高工作效率和产品质量的。本章的课后作业是对你要编写的下一个程序做代码复查。

13.1 个人对产品质量的承诺

尽管可以利用各种工具和方法，但决定程序质量的最重要的因素是软件工程师个人对产品质量的承诺。当一个工程师承诺对质量负责时，他会更仔细地完成工作并以自己生产的产品质量高为荣。PSP 将展示怎样使用有效的质量方法来生产合格产品。首先是了解在以前程序中引入的缺陷。然后，利用 PSP 方法，学会快速地找到和修复缺陷。本章和后面的几章将描述怎样做到这一点。

13.2 发现缺陷的步骤

尽管没有办法不引入缺陷，但是在开发过程中尽早地发现和修复缺陷还是可能的。学习 PSP 后，你就会知道尽早排除缺陷既节省时间又能开发出好的产品。譬如，如果在开始编码前能发现和修复设计的缺陷，就不会浪费时间去实现一个不正确的设计。同理，在编译和测试前发现编码的缺陷，就可节省在编译和测试阶段为查找和修复这些缺陷花费的时间。本章讲述怎样在开发过程中尽早发现缺陷，以及如何利用本章所提供的数据来评价这些排除缺陷方法的有效性。

有多种发现程序中缺陷的方法，基本上，这些方法都包括以下几个步骤：

1. 标识缺陷征兆。
2. 从征兆推断出缺陷的位置。
3. 确定程序中的错误。
4. 决定如何修复缺陷。

5. 修复缺陷。
6. 验证这个修复是否已经解决了这个问题。

13.3 发现和修复缺陷的方法

人们设计出各种工具和辅助手段来帮助软件工程师完成这些步骤。工程师最常用的工具是编译器。为了弄清编译器为什么能够和怎样帮助发现缺陷，重要的是搞清编译器的目的。编译器最基本的任务是生成目标代码。为此，编译器要扫描源代码看看能否生成目标码。若能，它不管源代码是否正确而生成目标码。

编译器如果遇到不能解释的字符，就不会产生目标码。例如，如果把一个没有声明的字符串 ABC 放到源程序中，编译器会标识此字符串为一个错误。编译器能标识出大部分语法缺陷，但是它不能辨别你的意图是什么。因此编译器经常为那些看起来很简单缺陷提供很多出错信息。另外，编译器只提供了缺陷的征兆，你必须自己对问题定位，并确定是什么问题。通常很快能做到这一点，但偶尔也需要较长的时间。

编译器不能检查出所有的拼写、标点符号或其它不符合语法的缺陷。原因是编译器经常可以在源程序有缺陷的情况下生成代码。大部分检查不出来的缺陷是设计错误，但也有一些是语法错误。编译器会漏掉语法错误似乎有些不可思议，但从几千个 C++ 缺陷数据说明，大约有 9.4% 的语法错误会通过编译器。就像拼写检查程序不能找到所有的拼写错误一样，编译器也不能查出所有的语法错误。

发现错误的第二种方法是测试。有许多种测试方法，但都要求测试人员提供测试数据和测试条件（有时称为测试用例或测试情景）。因此，测试的质量是由这些测试用例覆盖所有程序功能的程度决定的。测试人员运行这些测试用例来查看程序是否得出正确的结果。这也说明测试人员另外的一个职责：如果程序运行正确，要确定测试的期望结果。

测试可以用来验证程序几乎所有的功能。但它还有一些缺点。首先，像编译器一样，测试只能满足缺陷排除过程的第一个步骤。也就是说，你仍必须从缺陷征兆找出问题的根源，然后才能开始修复。另一个问题是每个测试只验证了一组程序条件。例如，一个求两个数（ x 和 y ）乘积的程序，如果用 $x=11$ 和 $y=18$ 进行测试，就可以知道这组值的工作情况；但还是不知道当输入负数、零、最大的正数、最大的负数或任何其它一对数时，其乘积会怎样。检查所有的可能情况需要大量的测试工作。因为即使很简单的程序也包括许多数据和可操作条件的可能组合，全面的测试会花费大量的时间。事实上，除了最简单的程序，任何程序的完全测试都是不可能的。

第三种发现缺陷的方法，也是一种很常见的方法，就是发行仍然含有缺陷的产品，然后等待用户发现和反馈缺陷信息。但事实证明，这是花费最大的策略。例如，IBM 每年花费大约 2.5 亿美元，用于修复 1.3 万个客户反馈缺陷和重新安装修复后的版本，每个缺陷大约花费 2 千美元。

最后，最有效的发现和修复缺陷的方法是个人复查源程序清单。虽然这种方法似乎很难

彻底清除程序中的缺陷，但事实证明，这是最快而且是最有效的方法。本章将解释其原因。

13.4 代码复查

代码复查是一种能快速找到缺陷的方法。进行代码复查就是研究源程序，并从中发现错误。代码复查最好是在源程序编码完成后并在编译和测试之前进行。由于大部分软件缺陷源于疏忽或愚蠢的失误，所以在刚刚完成设计或编码时是最容易发现缺陷的，因为这时程序员还可能记住他的意图，因而最有可能知道如何修复任何问题。

代码复查有很多种方法。最一般的方法是打出源程序清单，然后逐行地进行复查。也可以在计算机屏幕上复查代码，但工程师一般觉得看打印出来的程序清单似乎更方便，即使对很小的程序亦是如此。程序清单能使你的目光快速地在代码段之间切换、做笔记、标记出已经完成的部分。

尽管代码复查非常费时间，但它比测试更有效率。来自学生和工程师的数据显示，代码复查的效率是最初测试或单元测试效率的 3 ~ 5 倍。例如，对于一般的工程师，单元测试 1 小时可以找出 4 ~ 6 个缺陷，而代码复查每个小时可以找出 6 ~ 10 个缺陷。

代码复查更有效的原因是：在代码复查时看到的是问题本身而不是征兆。就是说，当从头到尾复查代码时，考虑的是程序应该做什么。因此，当看到某些地方好像不正确时，就可以看到可能的问题是什么，并能立即去验证代码。因为在编译和测试阶段，发现和修复缺陷需要经历从发现征兆到确定问题的过程，所以时间花费就大得多，而代码复查可以节省这些时间。

复查也有缺点。两个主要的缺点是：代码复查非常耗时，而且很难恰当地进行。复查也是一种技能，可以通过学习和实践来提高。即使对有经验的人员，也只能发现程序中平均 75% ~ 80% 的缺陷。对每 100 行源程序进行详细的复查至少需要 30 分钟的时间。如果复查得再快些，一般会漏掉更多的缺陷。本章以下部分讲述代码复查过程。后面几章讨论测量与技术，它们有助于改进做代码复查方法，并提供说明这些效果的数据。

13.5 为什么要尽早发现缺陷

有许多理由可以说明为什么要在编译和测试前复查程序。最重要的是：不能简单地拼凑出一个满是缺陷的程序，期望以后把它修正为一个合格的产品。一旦生产出一个有缺陷的程序，它将永远是有缺陷的。虽然你可以修复所有已知的问题，并且让它能够通过所有的测试，但它还是一个有许多补丁的有缺陷的程序。

举一个例子，假设你要买一辆新的小汽车。结束交易前，你参观两个制造商的生产线工厂。在一个工厂，你看到许多漂亮的汽车开下生产线并进行测试。虽然这些汽车看上去很好看，但经测试，每台车平均发现 10 个缺陷。当然这些缺陷将全部被修复，然后汽车发给经

销商。

在第二个工厂，汽车开下生产线后和前一个一样漂亮。可是在这儿，平均在每 10 辆车中只发现一个缺陷。即使第二个工厂的汽车价格贵一点，你也不会考虑这些区别而愿意买它。要知道，测试不能发现所有的问题，如果这个汽车生产过程产生出一个次品，不管经过多少次最终测试和检查，这辆车可能总是一个次品。

程序也是一样。如果工程师能宽容有缺陷的工作，他将生产出低质量的产品。“我很忙，以后再修复吧”这样的态度不可能生产出优质产品。为了生产出高质量的软件，开发过程中的每一个步骤都必须是高质量的。虽然这样严格的质量实践可能看起来花费很大，但实际上是节省时间的。

13.6 发现和修复缺陷的费用

在典型的软件项目中，产品被分成许多小的程序元素或模块。每个工程师开发一个或多个模块。在模块设计、实现、编译后，工程师做初始的测试或单元测试。在个人测试之后，多个模块组成一些子组件并进行集成测试。在经过各种级别的组件测试之后，这些组件集成成为产品进行产品测试。最后，将产品集成到系统中并进行系统测试。随着系统的规模和复杂程度不同，集成测试、组件测试、产品测试、系统测试的类型、持续时间、复杂度也不同。但几乎所有规模的软件产品，都要经过这个过程。

开发过程每前进一步，发现和修复缺陷的平均代价要增长 10 倍。尽管缺陷的修复时间变化很大，但平均时间总是遵循这样的规律，而与缺陷的类型无关。有些细小的语法错误，如分号的位置不正确或名字拼错等可能会通过编译，而且在测试阶段极难发现。在代码复查时，平均 1 ~ 2 分钟就可发现和修复一些缺陷。而在初始测试中，修复一个缺陷的平均时间是 10 ~ 20 分钟，甚至更多。实际上这是若干缺陷修复时间的平均值，其中，大部分缺陷的修复时间只花费 1~2 分钟，但少数缺陷要花费更多乃至几个小时的时间。

在集成测试、组件测试、系统测试时，发现缺陷的时间随着系统的规模和复杂性的不同而变化。一般来说，越是大型、复杂系统，发现和修复缺陷需要的时间就越多。譬如，在集成测试时，每个缺陷可花费 1 个小时或更多的时间，在系统测试时每个缺陷花费多达 10 ~ 40 个工程师小时。一旦产品发给客户，发现和修复缺陷的代价会更大，其值与产品的种类和客户的类型及数目有关。我自己发现和修复 C++ 缺陷的时间数据如图 13.1 所示。下面的几个示例说明，如果等到测试阶段再清除程序缺陷，它的花费是巨大的。

- 一个小型商业软件开发部门开发了一个有几个模块的程序。经过 PSP 训练的几位工程师在几周内完成了集成测试。然而，另一个组件是由一个未经 PSP 训练的小组开发的，集成测试花了几个月。发现和修复 36 个缺陷的测试时间是 300 小时。由于测试时间比计划长得多，交付给客户的时间延迟了两个月。

- 一个航天系统开发组在海军电子系统的系统测试阶段，发现和修复每个缺陷平均花费 40 工程师小时。

● 在数字设备公司，某系统发现和修复每个客户反馈缺陷最低花费时间是 88 个工程小时。

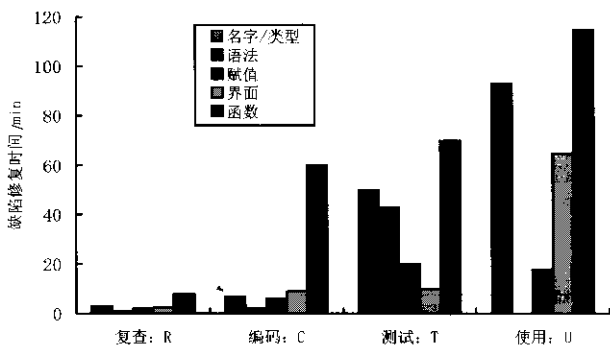


图 13.1 缺陷修复时间

除了花费之外，及早发现缺陷的另一个同等重要的原因是编译、调试和测试的效果非常有限。编译器是我们现有的最快的缺陷检测工具，但它只能查出大约 90% 的语法错误和很少的逻辑错误。单元测试一般是最有效的测试阶段，但它一般只能发现大约一半的缺陷。单元测试后，测试的有效性下降，系统测试一般仅能找到进入系统测试时产品中缺陷的大约 30% ~ 40%。

因此，如果想生产出高质量的产品，必须在开始时就生产出一个良好的程序，否则测试阶段要花费大量的时间。

13.7 利用代码复查发现缺陷

你也许很难相信跟踪和检查自己引入的缺陷可以改进工作，但是另外一些学生已经将他们在编译与测试阶段发现的缺陷数量减少了 5 ~ 10 倍。他们就是根据本章和以后几章所讲述的步骤做到这点的。代码复查方法的确很好，当你用过这种方法并看到工作改进的数据后，你就会把代码复查方法作为个体过程的一部分。

代码复查的第一步是了解自己引入的缺陷的种类。这是收集缺陷数据的主要原因。因为在下一个程序中引入的缺陷种类一般会 and 前面的基本类似。只要一直采用同样的软件开发方法，情况就会一直如此。但另一方面来说，当你有了技能和经验或改变了过程，缺陷的类型和数目也会随之变化。

由于你引入程序的缺陷的种类可能会与别人的不同，所以你的代码复查策略应该根据个人的缺陷情况而定。就像熟练的打字员很少犯打字错误，有经验的软件工程师很少犯编程错

误。通过不断的实践，自然能够提高。但是到了一定程度之后，改进就变得非常困难了。这时，就必须研究缺陷，这可以帮助你找到更好的发现和修复缺陷的方法。

代码复查的目标是在软件过程中尽可能早和尽可能多地发现缺陷。你也希望发现每个缺陷用的时间越少越好。表 13.1 是一个代码复查脚本。这个表是自说明性的，当进行代码复查时，特别要注意以下事项：

- 在第一次编译前进行代码复查。
- 在打印出的源程序清单上复查。
- 在缺陷记录日志上记录发现的每一个缺陷。
- 在代码复查时，根据以前在编译与测试阶段发现的缺陷类型进行检查。在第 14 章中，描述了一个有序的检查方法。

表 13.1

代码复查脚本

入口准则	在复查前，检查下列产品是否已经准备好： ● 需求陈述文档； ● 程序设计文档； ● 程序的源代码清单； ● 编码标准
1 复查规程	首先，完成源程序编码； 然后，在进行编译和测试之前，打印一份源程序清单； 下一步，进行代码复查； 进行代码复查时，仔细检查每一行源程序，以尽可能多地发现和修复缺陷
2 修复缺陷	修复所发现的每一个缺陷； 确保所做的修复正确无误； 将缺陷登入在缺陷记录日志
3 覆盖率复查	验证程序设计覆盖了需求文档中描述的每一个功能； 验证程序代码实现了所有设计
4 程序逻辑复查	验证程序设计在逻辑上是正确的； 验证程序代码正确地实现了设计中的逻辑
5 命名和类型检查	验证所有的名字和类型已经正确地声明和使用； 检查整型、长整型和浮点型是否正确声明
6 变量检查	确保每个变量已初始化； 检查上溢、下溢或越界问题
7 程序语法检查	验证程序代码符合编程语言的规格说明
出口准则	在复查结束时，应该有完整的、修复过的源程序清单； 填写完整的时间记录日志； 填写完整的缺陷记录日志

13.8 编译前的复查

有几个原因说明应在编译之前进行代码复查。基本原因是：

1. 不论在编译前还是在编译后，进行完整的代码复查的时间大约相同。
2. 先做复查将节省大量编译时间。若不做代码复查，工程师一般要花 12% ~ 15% 的开发时间进行编译。一旦使用代码复查方法后，编译时间可以缩短至 3% 或更少。
3. 工程师一旦先编译了自己的程序后，代码复查一般都很难彻底的进行。
4. 无论在代码复查之前或之后进行编译，对检查语法有效性的效果是一样的。
5. 经验证明，当编译阶段程序中有大量的缺陷时，一般在测试阶段也有许多缺陷。

应该在编译之前进行代码复查的基本观点是“照这种方法去做，你就会看到结果”。学了这门课后，如果你还认为应先编译后代码复查的话，可以按照每种方法编写几个程序，并对结果进行比较。然后，使用第 18 和 19 章描述的方法评估产品和过程的数据，看一看哪种方法更有效。重要的一点是不要仅仅根据直觉或其他人的做法去做；即收集自己的数据，根据这些数据做出合理的决定。

13.9 编译与测试缺陷的数据

在一个项目中，所有的工程师都经过 PSP 训练。他们开发一个有 6 个模块的新产品，每个模块有 600 到 2500 行代码。在开发过程中，他们在编码和编译后进行单元测试，接着是系统测试，最后把产品交付给客户，并且把客户反馈的缺陷记录下来。

这些模块在编译和测试阶段的缺陷数据显示在图 13.2、13.3 和 13.4 中。某个工程师坚决

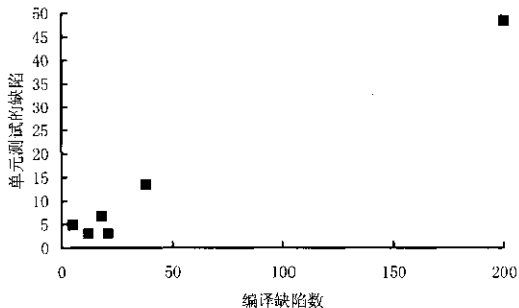


图 13.2 编译缺陷数与单元测试缺陷

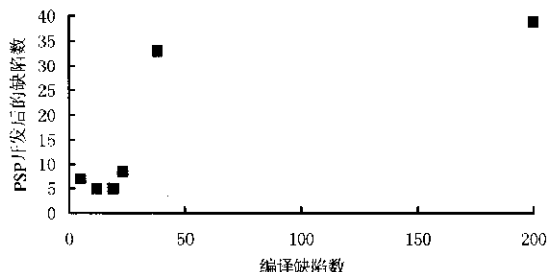


图 13.3 编译缺陷数与 PSP 开发后缺陷数

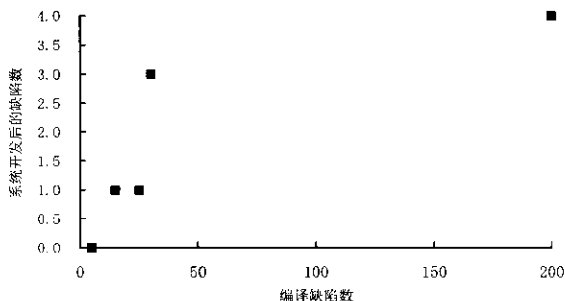


图 13.4 编译缺陷数与系统开发后缺陷数

反对代码复查方法，结果在编译时出现将近 200 个缺陷。如图 13.2 所示，他的单元测试中发现的缺陷比其它的模块多很多。更有趣的是这个模块在集成测试和系统测试中的缺陷也是最多的。虽然客户只对这些模块进行了简单的验收测试，但在这个模块中发现的缺陷还是最多的。最后，这个有缺陷的模块实际开发时间比计划长了两倍。其它所有模块都提前或按期交付，只有这个模块的交付时间延期五周。

细心工作是有回报的。当工程师觉得对自己开发的程序负有质量责任时，他就不会依赖于编译器或其它工具来发现缺陷。当你承诺要生产合格的产品时，你的承诺将体现在编译阶段发现的缺陷数、测试阶段发现的缺陷数和最终所完成的程序的质量上。

13.10 更新后的 PSP 项目计划总结表

现在的 PSP 过程包括了代码复查。这个修改后的过程脚本如表 13.2 中所示。项目计划总

结表也做过修改,如表 13.3、13.4 所示,而表 13.5 是一个项目计划总结表的示例。

这些表是自说明的,但还值得说明一点。在对这过程引进累计和累计百分比的计算时,记住引入代码复查会改变时间和缺陷的分布情况。当计划一个新的程序时,你就可以调整计算时间的估计以适合这些变化。

表 13.2 PSP 过程脚本

过程目的	指导用户进行小型程序的开发
入口准则	问题描述; PSP 项目计划总结表; 以前开发程序的实际的规模和时间数据; 时间记录日志; 缺陷记录日志
1 计划	获取对程序功能的描述; 估计整个程序的代码行数及其最大值和最小值; 确定开发效率 (Minutes/LOC); 计算开发时间及其最大值和最小值; 将计划数据填入项目计划总结表; 将计划阶段所花费的时间记入时间记录日志
2 设计	设计程序; 按照指定的格式记录设计文档; 将设计阶段所花费的时间记入时间记录日志
3 编码	实现设计; 使用标准的格式来书写程序代码; 将编码阶段所花费的时间记入时间记录日志
4 代码复查	复查所有的源程序代码; 遵照代码复查脚本; 修复并记录所发现的每一个缺陷; 将代码复查阶段所花费的时间记入时间记录日志
5 编译	编译程序; 修复并记录所有发现的缺陷; 将编译阶段所花费的时间记入时间记录日志
6 测试	测试整个程序; 修复并记录所有发现的缺陷; 将测试阶段所花费的时间记入时间记录日志
7 后置处理	将实际的规模、时间数据和缺陷数据填入项目计划总结表; 将后置处理阶段所用的时间记入时间记录日志
出口准则	经过详尽测试的程序; 完整的设计文档; 完整的源程序清单; 已经填写完成的项目计划总结表; 已经填写完成的时间日志和缺陷日志

表 13.3

PSP 项目计划总结表

学生					日期		
程序					程序号#		
教师					程序设计语言		

总 结	计 划	实 际	累 计		
Minutes/LOC					
LOC/Hour					
Defects/KLOC					
过程效益					
A/FR					
程序规模(LOC)					
新开发与更改的					
最大规模					
最小规模					
开发阶段时间/min	计 划	实 际	累 计	累计百分比	
计划					
设计					
编码					
代码复查					
编译					
测试					
后置处理					
总计					
最大时间					
最小时间					
引入的缺陷	计 划	实 际	累 计	累计百分比	Def./Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					
排除的缺陷	计 划	实 际	累 计	累计百分比	Def./Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					

表 13.4

PSP 项目计划总结表使用说明

目 的	本表以一种方便而且易于存取的方式，记录估算的和实际的项目数据
表头	如下输入： ● 你的名字和今天的日期 ● 程序名和程序号 ● 教师的名字 ● 你将使用的编程语言
Minutes/LOC	在开发之前： ● 登入该项目计划的 Minutes/LOC 值，使用最近开发项目的累计开发效率值。 在开发之后： ● 用全部的开发时间除以实际的程序规模，得到实际的和累计的 Minutes/LOC 值； ● 例如，如果该项目总共花费了 196 分钟，产生了 29 LOC，那么开发效率 Minutes/LOC 的值就是 $196/29=6.76$
LOC/Hour	在开发之前： ● 用 60 除以计划的“Minutes/LOC”就得到了计划的“LOC/Hour”的值 在开发之后： ● 用 60 除以实际的和累计的“Minutes/LOC”，就得到了实际的和累计的“LOC/Hour”的值； ● 对于上面的示例，实际的 Minutes/LOC = 6.76，那么，实际的 LOC/Hour = $60/6.76 = 8.88$
程序规模(LOC)	在开发之前： ● 在计划部分登入估算的全部新开发的与更改的代码行数、及其最大值和最小值。 在开发之后： ● 统计并填入实际新开发的与更改的代码行数； ● 在累计一栏中，把实际的新开发与更改的代码行数与上一个程序的新开发的与更改的代码行数相加，得到累计值
开发阶段时间 计划	用总的新开发的与更改的代码行数，乘以“Minutes/LOC”，得到项目的总开发时间； 使用代码行的最大值乘以“Minutes/LOC”，得到开发时间的最大值； 使用代码行的最小值乘以“Minutes/LOC”，得到开发时间的最小值； 从最近的前一个程序的项目计划总结表中，找到各个阶段所花费时间的累计百分比。 用这个累计百分比计算出各个阶段的计划时间。
实际	在作业完成之后，根据时间记录日志中的数据填入各个阶段实际花费的时间 从时间记录日志中得到这些数据
累计	对每个阶段，计算并登入实际开发时间与最近的前一个程序各阶段累计开发时间的和
累计百分比(%)	对于每个阶段，用 100 乘以该阶段的累计时间，再除以总的累计时间，得到累计时间百分比
引入的缺陷 实际	在开发完成后，统计并登入各个阶段实际引入的缺陷的数目
累计	对每个阶段，计算并登入实际引入的缺陷个数与最近的上一个程序累计引入的缺陷个数之和
累计百分比(%)	对每个阶段，用 100 乘以该阶段的累计缺陷，再除以总的累计缺陷，得到累计缺陷百分比；即登入该阶段累计引入缺陷值占整个项目缺陷引入累计值的百分比

续表

目 的	本表以一种方便而且易于存取的方式,记录估算的和实际的项目数据
排除的缺陷	
实际	在开发完成后,统计并登入各个阶段实际排除的缺陷数
累计	对每个阶段,计算并登入实际排除的缺陷个数与最近的上一个程序的累计缺陷排除个数之和
累计百分比(%)	对每个阶段,用 100 乘以该阶段的累计缺陷,再除以总的累计缺陷,得到累计缺陷百分比;登入该阶段累计排除缺陷个数占整个项目缺陷排除累计值的百分比

表 13.5

PSP 项目计划总结表的示例

学生	学生 X	日期	11/4/96
程序		程序号#	11
教师	先生 Z	程序设计语言	Ada

总 结	计 划	实 际	累 计	
Minutes/LOC	6.50	5.88	6.30	
LOC/Hour	9.23	10.20	9.52	
Defects/KLOC				
过程效益				
A/FR				
程序规模 (LOC)				
新开发与更改的	53	48	153	
最大规模	72			
最小规模	35			
开发阶段时间 /min	计 划	实 际	累 计	累计百分比
计划	17	15	48	5.0
设计	28	28	83	8.6
编码	156	132	440	45.6
代码复查	0	36	36	3.7
编译	35	7	77	8.0
测试	83	38	203	21.1
后置处理	26	26	77	8.0
总计	345	282	964	100.0
最大时间	468			
最小时间	228			

					续表
引入的缺陷	计划	实际	累 计	累计百分比	Def./Hour
计划					
设计		1	3	21.4	
编码		5	11	78.6	
代码复查					
编译					
测试					
总计		6	16	100.0	
排除的缺陷	计划	实际	累 计	累计百分比	Def./Hour
计划					
设计					
编码					
代码复查		3	3	21.4	
编译		2	8	57.2	
测试		1	3	21.4	
总计		6	14	100.0	

表 13.6 发现一个缺陷所用的时间/h

数据引用来源	检查阶段	测试阶段	使用阶段
Ackerman	1	2~10	
O'Neill	0.26		
Ragland		20	
Russell	1	2~4	33
Shooman	0.6	3.05	
vanGenuchten	0.25	8	
Weller	0.7	6	

13.11 其它种类的代码复查

在软件组织中，一种常用的方法是让几个工程师彼此复查程序。这叫做同行评审或同行检查。组织良好的同行检查一般会发现程序中 50%~70% 的缺陷。互查虽然需要许多时间，但是可以非常有效地发现缺陷。原因是工程师往往难于发现自己的设计错误。他们创作这个设计，他们知道程序应该完成什么。如果他们的概念有瑕疵、作了错误的设计或实现假定，

他们往往很难发现。检查者可以帮助他们克服这些问题。在检查时发现缺陷的时间数据显示在表 13.6 中。

对于非常小的课堂练习，一般没有必要进行同行检查。但对于大的项目或任何工业化程序，检查是应该做的。最佳的策略是，先做个人代码复查再进行编译。然后在任何测试前进行同行检查。本书不进一步讨论此类检查，在这方面有许多有用的参考资料[Fagan, Gilb, Humphrey 89]。

13.12 总 结

本章描述了在开发中怎样及为什么要尽早发现缺陷。程序质量中最重要的因素是工程师个人对质量的承诺。有了这样的承诺，工程师甚至在编译和测试程序前就能尽早地排除缺陷。

在 PSP 中主要的缺陷排除方法是个人代码复查。先打印出程序清单，然后逐行进行复查，尽可能地修复发现的每一个缺陷。最有效的方法是，找出以前的程序中在编译和测试阶段引起最大问题的那些缺陷类型。因为人们往往容易犯同样的错误，所以缺陷数据有助于在第一次编译和测试前找到全部或绝大部分程序缺陷。

经验表明，当工程师在第一次编译前仔细地复查了他们的代码时，就能减少编译时间，大约相当于 10% 的开发时间，并且可以节省更多的测试时间。虽然全面的代码复查要花费时间，但他们节省出来的时间比花费的时间多得多，并能生产出更好的产品。

13.13 练 习 13

对下一个程序，在编译和测试前先进行代码复查。按照表 13.1 代码复查说明进行复查。在缺陷记录日志上记录发现的所有缺陷，在已更新过的项目计划总结表中记录项目数据。

提交你前面没有提交的时间记录日志和周活动总结表。另外，对你开发的每个程序完整地填写项目计划总结表和缺陷记录日志。包括计划的和实际的开发时间和引入的和排除的缺陷。

参 考 文 献

[Ackerman] Ackerman, Frank A., Lynne S. Buchwald, and Frank H. Lewski. "Software Inspections: An Effective Verification Process." *IEEE Software*, May 1989, pp 31-36

[Fagan] Fagan, Michael. "Design and Code Inspections to Reduce Errors in Program Development." *IBM Systems Journal*, vol. 15, no.3, 1976

[Gilb 93] Gilb, Tom, and Dorothy Graham. *Software Inspection*. Edited by Susannah Finzi, Reading, MA: Addison-Wesley, 1993

[Humphrey 89] Humphrey, W.S. *Managing the Software Process*. Reading, MA: Addison-Wesley, 1989

[O'Neill] O'Neill, Don. Personal communication.

[Ragland] Ragland, Bryce. "Inspections Are Needed Now More Than Ever." *Journal of Defense Software Engineering*, 38. Software Technology Support Center, U.S. Department of Defense, Nov. 1992

[Russell] Russell, Glen W. "Experience with Inspection in Ultralarge-Scale Developments." *IEEE Software*, Jan. 1991, pp.25-31

[Shooman] Shooman, M.L., and M.I.Bolsky. "Types, Distribution, and Test and Correction Times for Programming Errors." *Proceedings of the 1975 Conference on Reliable Software*. Catalog no. 75 CHO 940-7CSR, IEEE, New York: pp.347

[vanGenuchten] vanGenuchten, Michael. Personal communication

[Weller] Weller, E.F., "Lessons Learned from Two Years of Inspection Data." *IEEE Software*, Sept. 1993, pp. 38-45

第 14 章 代码复查检查表

进行有效的代码复查的关键是要具有一个高效率的复查规程。本章描述了代码复查检查表，说明它们是如何帮助你快速而有效地发现程序中的缺陷，以及如何制定一个适合自己使用的检查表。在本章的练习中，针对自己经常引入的缺陷设计一个检查表，并在进行程序代码复查时使用。

14.1 检查表的用途

检查表包括一系列规程式的步骤，并要求你精确地遵循这些步骤去做。当人们需要严格地按照说明去做某件重要事情时，经常使用检查表。例如，民航飞行员在起飞前用检查表做飞行前的检查。即使他们在二小时前刚刚对这架飞机做过同样的检查，起飞前也仍然要再做一遍。美国空军基地的一项调查发现，造成每次事故的原由，都是由于没有严格按照飞行前检查表进行检查。另一个示例，美国国家宇航局在每次航天飞行前都使用完整而复杂的检查表进行倒计时检查。这一过程一般要进行几天，经过几百个步骤。由于这个过程太复杂，不得不使用计算机来监控倒计时的进展。

如果想发现和改正程序中的每一个缺陷，就必须遵照一个精确的规程。检查表可以帮助确保遵循这个规程。本章使用一种特殊类型的检查表，在为自已编写的程序进行代码复查时，用它来帮助查找缺陷。你将看到如何制定一个经过裁减的代码复查检查表，以准确地发现以前程序中曾引起大多数问题的缺陷。

检查表也是一个构思的来源。当按照检查表去做时，就知道如何进行代码复查。如果能够正确地使用这个表，还能知道在检查表的每个步骤发现了多少缺陷。这样就能测量出复查过程的效率并进一步改进检查表。建议把你的检查表和其他工程师的检查表进行比较，这也将有助于改进复查方法。

检查表包括了个人的经验。通过不断地使用和改进个人检查表，可以更好地发现在程序中的缺陷。检查表也可帮助你用较少的时间发现这些缺陷。

14.2 代码复查检查表的示例

表 14.1 是我设计的为我自己编写 C++ 程序进行代码复查的检查表。表 14.2 是为 Ada 语言设计的类似的检查表。这些检查表给出了在设计和使用自己的个人检查表时要考虑的几点建议。

第一步是确保编码实现了所设计的全部功能。在大程序中，容易忽略某些过程或操作的编码。这样的疏忽是常见的差错，它们偶尔还能通过此后的复查、编译和测试等步骤。这种缺陷一般很容易通过检查表来发现。

全面地检查 include(或 withs)语句、初始化、过程调用和名字也是有效的。这些是最容易出问题的地方，除非历史的缺陷数据说明你从未有过这样的缺陷，否则应该检查这些地方。

表 14.1

C++代码复查指南和检查表

程序名和程序号#:

目的	指导你进行有效的代码复查	#	#	#	#	累计	累计%
一般性说明	在完成每个复查步骤之后，将发现的某个类型的缺陷的个数记录在右面的栏目中。如果该步骤没有发现缺陷，就在右面的栏目中打个表示检查无误的交叉符号 (X)。在开始复查下一个程序单元之前，要按照检查表完成对程序、类、对象或方法的检查						
完整性	验证设计的所有功能都已经编码						
Includes	验证 Include 语句是完全的						
初始化	检查变量和参数的初始化： ● 在程序的开始； ● 在每个循环的开始； ● 在函数/过程的入口						
调用	检查函数调用的格式： ● 指针； ● 参数； ● ‘&’ 的使用						
名字	检查名字的拼写和使用： ● 是否前后一致？ ● 是否在说明的作用域之内？ ● 结构和类中变量的引用是否使用了 ‘.’						
字符串	检查所有的字符串： ● 是用指针来标识的； ● 是以 NULL 结尾的						

续表

目的	指导你进行有效的代码复查	#	#	#	累计	累计%
指针	检查所有的指针： ● 是初始化为 NULL； ● 只有在 New（新建）之后才 Delete（删除）； ● 在 New 并使用之后要删除					
输出格式	检查输出格式： ● 换行是否合适？ ● 间隔是否合适？					
{ } 对	保证检查 { } 是适当的而且是成对的					
逻辑操作符	验证 ==, =, 等逻辑操作符的使用是合适的。检查每个逻辑函数的 () 是合适的					
逐行检查	检查每一行代码： ● 指令的语法是否正确？ ● 标点是否正确符号是否正确？					
标准	保证所有代码符合编码标准					
文件的打开与关闭	验证所有的文件： ● 是合适地声明的； ● 是合适地打开的； ● 是合适地关闭的					
全面检查	对整个程序进行全面的检查以发现系统问题和非期望的问题					
总计						

表 14.2

ADA 代码复查指南与检查表

程序名和程序号#:

目的	指导有效的代码复查	#	#	#	累计	累计%
一般性说明	在完成每个复查步骤之后，将发现的某个类型的缺陷的个数记录在右面的栏目中。如果该步骤没有发现缺陷，就在右面的栏目中打个表示检查无误的交叉符号（X）。在开始复查下一个程序单元之前，要按照检查表完成对程序、类、对象或方法的检查					
完整性	验证设计的所有功能都已经编码					
Includes	验证 with 语句是完全的					
初始化	检查变量和参数的初始化： ● 在程序的开始； ● 在每个循环的开始； ● 在过程的入口。					

续表

目的	指导有效的代码复查	#	#	#	#	累计	累计%
调用	检查过程调用的格式： ● 指针； ● 参数。						
名字	检查名字的拼写和使用： ● 是否前后一致？ ● 是否在说明的作用域之内？ ● 结构和包中变量的引用是否使用了‘.’						
字符串	检查所有的字符串中的分组是合适的						
指针	检查所有的指针： ● 只有在 New（新建）之后才能 Delete（删除）。 ● 在 New 并使用之后要删除。						
输出格式	检查输出格式： ● 换行是否合适？ ● 间隔是否合适？						
（ ）对	保证（ ）是适当的而且是成对的						
逻辑操作符	验证所有逻辑操作符的使用是合适的。检查每个逻辑函数的（ ）是合适的						
逐行检查	检查每一行代码： ● 指令的语法是否正确？ ● 标点符号是否正确？						
标准	保证所有的代码符合编码标准						
文件的打开与关闭	验证所有的文件： ● 是合适地声明的； ● 是合适地打开的； ● 是合适地关闭的						
全面检查	对整个程序进行全面的检查以发现系统问题和非期望的问题						
总计							

另外，要考虑根据编码标准来复查代码，以确保代码中没有漏掉关键的注释，没有使用不正确的格式，或没有忽略重要的过程或产品信息。虽然编码标准看起来似乎并不重要，但它对以后的工作有很大帮助，如程序的修复、增添或重用。由于这些工作都是大多数软件组织的主要工作，所以应该养成遵守编码标准的好习惯（见 14.7 节）。

使用检查表要注意的问题是，通常只能找到那些已知可能的缺陷。虽然这是有用的，问题是，如果只是按照检查表去检查，就只能找到在检查表上列出的缺陷。但是，有些严重的问题经常是难以预料的。例如，程序间全局性的交互作用、未曾预料到的时序问题、复杂的

内存使用问题或不正常的操作条件。因此，一个很好的建议就是至少要对程序进行一次全面的审查，以查找那些未曾预料到的问题。这时，应该尽量从系统或用户的角度去检查程序。

14.3 使用代码复查检查表

使用代码复查检查表时，要逐个阅读每一项说明，精确地按照说明的每个动作去做。当完成每个动作后，在检查表的该项后面做标记。最后，复查整个检查表以确保每一项都检查到了。如果不是这样，返回去执行未做的部分，逐个做标记。然后按照检查表再检查一遍，以确保已经完成表中的每一项检查。使用检查表时，下面的实践应该是有帮助的。

1. 针对检查表中的每一项，从头至尾地对程序进行检查。例如表 14.1，首先复查全部程序以确保完全实现了所有的设计功能。在这期间，如果发现其它缺陷，就先修复它们。但要注意，这次检查的意图是根据设计通查程序。然后是复查检查表中的下一项，如此继续。

2. 当检查中发现了缺陷时，在右边第一个未用的 # 列记下一个小竖线标记。发现第二个缺陷，在同一格中再记下一个标记。这样，复查完成后，你可以返回来看一看每个复查步骤发现的缺陷数。

3. 完成每项检查后，如果没有发现缺陷，在右边第一个未用的 # 列记下标记 X。

4. 当要复查的程序有几个函数、对象或过程时，最好对它们分别进行复查。即先完全复查第一个过程，把标记 X 或这个过程的缺陷数记在右边的第一个 # 列下。对于第二个过程重复以上步骤把标记 X 或该过程的缺陷数记到第二个 # 列下。继续下去直到复查完所有的函数、对象或过程。

5. 如前面所说，一个好的主意是在最后重新对整个程序进行检查，找出那些非期望的、新的问题，或那些系统问题或用户问题。

按照代码复查检查表进行复查的一般过程见表 14.3，它是修改过的代码复查脚本。修改过的 PSP 过程脚本见表 14.4。这些脚本与上一章的脚本相比有几点改动，现在已经包括了检查表，并且要求在进行代码复查时这些检查表应该已经填好。PSP 项目计划总结表及其指南与前一章相比没有变化。

14.4 建立个人检查表

为了建立个人代码复查检查表，先检查缺陷数据并找出引起大部分问题的缺陷类型。刚开始时一般只有有限的缺陷数据，通过每个新程序可以得到更多的数据。为了达到更好的效果，检查表要根据自己的情况、所使用的程序语言、经常发现的或漏过的缺陷的类型来设计。在开始时，可以参考别人的检查表，但效果可能不如使用根据自己的特殊需要而定制的检查表来得有效。

表 14.3

代码复查脚本

入口准则	在复查前，检查下列产品是否已经准备好： ● 需求陈述文档； ● 程序设计文档； ● 程序的源代码清单； ● 编码标准； ● 代码复查检查表
一般性说明	使用代码复查检查表； 在复查时遵照代码复查检查表的使用说明； 在复查结束时，填写累计、累计百分比和总结栏目。
1 复查规程	首先，完成源程序编码； 然后，在进行编译和测试之前，打印一份源程序清单； 下一步，进行代码复查； 进行代码复查时，仔细检查每一行源程序，以尽可能多地发现和修复缺陷。
2 修复缺陷	修复所发现的每一个缺陷； 确保所做的修复正确无误； 将缺陷登录在缺陷记录日志。
3 覆盖率复查	验证程序设计覆盖了需求文档中描述的每一个功能； 验证程序代码实现了所有设计。
4 程序逻辑复查	验证程序设计的逻辑上是正确的； 验证程序代码正确地实现了设计中的逻辑。
5 命名和类型检查	验证所有的名字和类型已经正确地声明和使用； 检查整型、长整型和浮点型是否正确声明。
6 变量检查	确保每个变量已初始化； 检查上溢、下溢或越界问题。
7 程序语法检查	验证程序代码符合编程语言的规格说明。
8 程序检查	对整个程序进行全面的检查以发现系统问题和非期望的问题
出口准则	在复查结束时，应该有： 完整的、修复过的源程序清单； 填写完整的时间记录日志； 填写完整的缺陷记录日志。

表 14.4

PSP 过程脚本

过程目的	指导用户进行小型程序的开发
入口准则	问题描述； PSP 项目计划总结表； 代码复查检查表； 以前开发程序的实际的规模和时间数据； 时间记录日志； 缺陷记录日志。

续表

过程目的	指导用户进行小型程序的开发
1 计划	获取对程序功能的描述; 估计整个程序的代码行数及其最大值和最小值; 确定开发效率 (Minutes/LOC); 计算开发时间及其最大值和最小值; 将计划数据填入项目计划总结表; 将计划阶段所花费的时间记入时间记录日志。
2 设计	设计程序; 按照指定的格式记录设计文档; 将设计阶段所花费的时间记入时间记录日志。
3 编码	实现设计; 使用标准的格式来书写程序代码; 将编码阶段所花费的时间记入时间记录日志。
4 代码复查	复查所有的源程序代码; 遵照代码复查脚本和检查表; 修复并记录所发现的每一个缺陷; 将代码复查阶段所花费的时间记入时间记录日志。
5 编译	编译程序; 修复并记录所有发现的缺陷; 将编译阶段所花费的时间记入时间记录日志。
6 测试	测试整个程序; 修复并记录所有发现的缺陷; 将测试阶段所花费的时间记入时间记录日志。
7 后置处理	将实际的规模、时间数据和缺陷数据填入项目计划总结表; 复查缺陷数据并更新代码复查检查表; 将后置处理阶段所用的时间记入时间记录日志。
出口准则	经过详尽测试的程序; 完整的设计文档; 一个完成的代码复查检查表; 完整的源程序清单; 已经填写完成的项目计划总结表; 已经填写完成的时间日志和缺陷日志。

这里有一些提示可以帮助你设计出有用的个人检查表。

1. 根据在软件开发过程中每个阶段发现的缺陷类型和数目制作一个表。例如, 表 14.5 中学生 X 的数据, 其中在表的下半部分列出每个程序在各阶段发现的缺陷数。这样, 可以很容易地检查是否所有的缺陷都已统计。

表 14.5

学生 X 的缺陷数据分析

类型	引入的缺陷			排出的缺陷			复查中漏过的缺陷
	设计	编码	其它	复查	编译	测试	
10							
20		8		4	4		4
30							
40	2	3		1	4		4
50		2			1	1	2
60							
70							
80	2	3			1	4	5
90							
100							
总计	4	16		5	10	5	15
程序							
10	2	6			6	2	8
11	1	5		3	2	1	3
12	1	5		2	2	2	4

2. 按缺陷类型降序排列在编译和测试阶段发现的各种类型缺陷的数目。表 14.6 是这种排列方式的一个示例。

3. 对于有多数缺陷的那些少数缺陷类型，要检查缺陷记录日志，看看是什么问题引起了这些缺陷。在表 14.6 中可看到类型 80 是函数缺陷、类型 20 是语法缺陷、类型 40 是赋值缺陷。

4. 对于那些导致严重问题的缺陷，要确定在代码复查阶段采取什么样的步骤去发现他们。例如对类型 20 语法缺陷，学生 X 发现最常见的问题是丢掉分号和分号位置错。这样，他可以在检查表中增加一个检查项，对于源程序的每一行进行专门的分号检查。

5. 把这些条目放在代码复查检查表中以保证按照这些步骤实施。例如，学生 X 应该在检查表中增加一个分号项，就是说复查每一行源程序来验证分号使用是否正确。

6. 使用一个新的检查表后，用同样的方法重新检查缺陷数据。

7. 如果检查表在发现这些最重要类型的缺陷很有效，那么增加另一类型再继续使用它。

8. 如果检查表在发现某些类型的缺陷时无效，那么尽量修改检查表以便它能更好地找出这些缺陷，然后再试一次。这种情况下如果学生 X 发现自己经常把分号错录入成冒号，可以增加一个检查分号的部分以确保不再错把分号打成冒号。这样修改后的检查表见表 14.7。

9. 在制订或更新检查表时，把相似的检查放在一起且不要重复列出。如果某项单独的检查不能充分地发现问题，改用另外的检查项来完成同样的检查工作。替换掉不满意的检查项。在表 14.7 的示例中，学生 X 把分号检查放在其它标点符号中检查。

10. 开发完每个新程序后,用同样的方法简要检查一下缺陷数据和检查表,并标识出有用的更改和增加部分。

11. 建议想一想哪些步骤可以在以后预防这些缺陷。例如可以更新编码标准或在设计过程中增加一个步骤。

表 14.6 学生 X 的缺陷数据排序表

类型	引入的缺陷			排出的缺陷			复查中漏过的缺陷
	设计	编码	其他	复查	编译	测试	
80	2	3			1	4	5
20		8		4	4		4
40	2	3		1	4		4
50		2			1	1	2
60							
100							
30							
10							
70							
90							
总计	4	16		5	10	5	15

在表 14.5 和 14.6 中,学生 X 列出从开始收集缺陷数据以来他所引入和排除的所有缺陷。尽管这里只包括 20 个缺陷,但这是他收集的所有缺陷数据。在表 14.5 中,他首先从项目计划总结表中得到了程序各阶段缺陷总数,然后仔细检查了所有缺陷记录日志。在表 14.6 中,他把表 14.5 中最右列的缺陷数从大到小排序。在最上面的一项是代码复查中遗漏缺陷最多的类型,第二项次之,依次下去。这样排列方式叫做 Pareto 分布。这些分布排列是根据数据决定表项的先后排列次序。另外要注意的是,因为学生 X 没有对程序 10 做代码复查,所以他把编译与测试阶段发现的所有缺陷都记为代码复查漏过的缺陷。

14.5 改进检查表

应该养成定期复查缺陷数据和重新审核检查表的习惯。当步骤是有效时,保留它们。当某些步骤不理想时,找出怎样才能使这些步骤更有效并更新检查表。这样检查表就变成了个人经验的总结。它还可以帮助你坚持按照自己设计的步骤来查找和修复缺陷。

下面各段是改进个人检查表的一些建议。

当完成一个程序后,在检查表右边的累计这一列登入数据。累计项的值是从最近完成的

检查表中的缺陷数累加得到的。在累计列的每一行登入数据。表 14.7 是学生 X 更新过的检查表，它说明了它是如何做的。

首先通过统计累计例中值的总和，并将其登入检查表的最后一行(即总计)，完成累计百分比这一列的值。接下来用每行的累计值和总计的累计值得到累计百分比，把结果登入此行的累计百分比值这一列，见表 14.7。

表 14.7

学生 X 更新的 ADA 检查表

程序名和程序编号#:

目的	指导有效的代码复查	#	#	#	#	累计	累计%
一般性说明	在完成每个复查步骤之后，将发现的某个类型的缺陷的个数记录在右面的栏目中。如果该步骤没有发现缺陷，就在右面的栏目中打个表示检查无误的交叉符号(X)。 在开始复查下一个程序单元之前，要按照检查表完成对程序、类、对象或方法的检查						
完整性	验证设计的所有功能都已经编码	X					
Include	验证 with 语句是完全的	X					
初始化	检查变量和参数的初始化： ● 在程序的开始； ● 在每个循环的开始； ● 在过程的入口。	X					
函数调用	检查过程调用的格式： ● 指针； ● 参数。	X					
名字	检查名字的拼写和使用： ● 是否前后一致？ ● 是否在说明的作用域之内？ ● 结构和包中变量的引用是否使用了‘.’	1				2	40
字符串	检查所有的字符串中的分组是合适的： ● 是用指针来标识的； ● 是以 NULL 结尾的。	X					
指针	检查所有的指针： ● 是初始化为 NULL； ● 只有在 New (新建) 之后才 Delete (删除)； ● 在 New 并使用之后要删除。	X					
输出格式	检查输出格式： ● 换行是否合适？ ● 间隔是否合适？	X					
() 对	保证 () 是适当的而且是成对的	X					
逻辑操作符	验证所有逻辑操作符的使用是合适的。检查每个逻辑函数的 () 是合适的	X					

续表

目的	指导有效的代码复查	#	#	#	#	累计	累计%
逐行检查	检查每一行代码： ● 指令的语法是否正确？ ● 分号使用是否合适？ ● 检查分号是否打印成冒号？ ● 其它标点符号是否正确。	1				3	60
标准	保证所有的代码符合编码标准。	X					
文件的打开与关闭	验证所有的文件： ● 是合适地声明的； ● 是合适地打开的； ● 是合适地关闭的。	X					
全面检查	对整个程序进行全面的检查以发现系统问题和非期望的问题。	X					
总计		2				5	100

在每个程序的后置处理阶段，比较检查表和缺陷日志，找出检查表需要改进的地方并进行改进，以便更有效地查找缺陷。同时也要考虑删掉那些在最近 5 ~ 10 个程序中没有发现或漏过任何缺陷的复查步骤。例如，学生 X 复查了表 14.8 中记录的程序 12 的缺陷日志数据，以判断是否应该变更检查表来更好地找到遗漏的缺陷。正是这样的复查，他在表 14.7 中增加了分号检查这一项。

最好在收集 20 个以上的缺陷后再修改检查表。即使如此，也要在程序后置处理阶段复查缺陷数据。当编译或测试中几次看到同样的缺陷数据时，应考虑更新检查表以便处理这种特定问题。

缺陷类型：	
10 文档	60 检查
20 语法	70 数据
30 联编打包	80 函数
40 赋值	90 系统
50 接口	100 环境

表 14.8

学生 X 缺陷记录日志的实例

学生 学生 X 日期 11/11/96
教师 先生 Z 程序号# 12

日期	编号	类型	引入阶段	排除阶段	修改时间/min	相关缺陷
11/11	1	20	编码	编译	1	
	描述：漏掉了分号“;”					
	2	20	编码	复查	1	
	描述：变量拼写缺陷，把 X_axis 写成了 X_axes					

续表

日期	编号	类型	引入阶段	排除阶段	修改时间/min	相关缺陷
	3	20	编码	编译	1	
	描述: 分号: 写成了冒号;					
	4	50	编码	编译	1	
	描述: 过程 Normalize 的调用格式不正确					
	5	80	编码	测试	11	
	描述: 忘记初始化变量 X_axis 和 Y_axis					
	6	80	设计	测试	7	
	描述: 当 X_axis 为负数时, while 循环无法正确地执行					
	7					
	8					

定期更新检查表。随着时间的推移, 检查表自然地要变大。但是, 检查表的主要作用是帮助你把注意力集中在关键的方面。太大以后, 你将失去重点。所以要定期复查缺陷数据, 删除那些不能找到问题的表项, 这样做是很重要的。可以把删除的表项放在一个杂类中, 为复查其它的表项作参考。

从个人检查表的方法认识到, 每个工程师都有各自的特点, 某个工程师的实践经验对别人不一定适合。因而要设计出适合自己的检查表, 并定期地对它进行检查以保证检查表更有效。只要你在代码复查中还遗漏缺陷, 就要不断寻找改进检查表的方法。但是要记住, 进步通常是很缓慢的。最初, 你发现缺陷的能力随着每次复查都有所提高。此后, 提高将变得很困难。要坚持收集和分析缺陷数据, 并坚持思考如何才能预防缺陷的产生或怎样更好地找到缺陷。只要坚持不断地做下去, 就能在代码复查中不断进步, 也能不断地提高自己编写程序的质量。

14.6 编码标准

检查表很有效的一个原因是它提供了一个进行代码复查的标准。虽然最主要的代码复查标准是程序设计语言的语法规格说明, 但这些说明并没指明编码的格式和风格。因此, 你需要一个编码标准。

标准是官方接受的用于对照的基准。因此, 编码标准定义为被广泛接受的、能够作为工作样板的编码实践集。这个标准用于指导编写源程序代码。这样的标准一般定义了源程序的书写格式、什么语句分别写在单独的一行或语句怎样缩排。编码标准中一般要定义注释的书

写规范，包括必须的说明性注释。在程序清单的头部一般有关于工程师的名字、工作时间、程序名字、项目名字和版本号注释。一个 C++ 编码标准见表 14.9。

编码标准还有助于预防缺陷。例如，可以在编码标准中列出那些应该避免使用的方法，象 go-to 语句、过程多出口或使用递归算法。有些实践也很有用，如在循环入口前或在说明时初始化变量。不良的命名实践可能是差错的一个主要来源。使用那些与变量含义有关的字符作名字，每个名字应有明显的区别以免混淆。XY34B 与 XY35C 就是两个容易混淆和出错的变量。最好选用 AZIM34 和 UPDATE_AZIM 的名字来替代。

如果你的老师已经有编码标准，复制一份用就可以了。它可以帮助你编制出更加易读和易理解的程序。易读的代码有助于测试和调试程序，并且有助于其他人使用或修改你的程序。

表 14.9 C++ 编码标准

目 标	指导 C++ 程序的开发
程序头	所有程序都应从一个描述性的开头开始
程序头格式	<pre> /***** /* Program Assignment: 程序号 /* Name: 你的姓名 /* Date: 程序开发开始日期 /* Description: 对程序功能的一个简单描述 *****/ </pre>
	提供一份清单内容的概要
清单内容	
内容举例	<pre> /***** /* Listing Contents (清单内容): /* Reuse instruction (重用指导) /* Includes (包括) /* Class declarations (类说明): /* CData /* Aset /* Source code in c (用 C 编写的源代码) \classes\Cdata.cpp: /* CData /* CData() /* Empty() *****/ </pre>
重用指导	描述如何使用该程序的。提供说明格式、参数值和类型以及参数范围； 提供对非法值、溢出条件及其它潜在的有可能引起非正常操作的条件等方面的警告。
示例	<pre> /***** /* Reuse Instruction (重用指导) /* int Printline(char *line_of_character) /* 目的: 画线, 'Line_of_character', 位于同一打印行 /* 限制: 线长最多为 LINE_LENGTH /* Return: 0 if 打印机尚未准备好 /* else 1 *****/ </pre>

续表

目 标	指导 C++ 程序的开发
标识符	所有变量、函数名、常数以及其它标识符都使用带有描述性的名称。避免缩写形式或仅有一个字母的变量名
标识符示例	<pre>int number_of_students; /* 此标识符好 */ Float x4, j, ftave; /* 这些标识符不好 */</pre>
注释	编码，文档要充分，以帮助读者理解其功能； 注释既应说明编码的目的，也应说明编码的行为； 对变量声明进行注释，指出说明该变量的目的。
好的注释	<pre>If (record_count>limit) /* 是否所有的记录都处理了? */</pre>
不好的注释	<pre>If (record_count>limit) /*检查 record_count 是否超过限制 */</pre>
程序主体	程序主体前应有一段注释来描述由以下部分所做的处理
示例	<pre>/****** /*以下程序部分将检查数组“grades”的内容，并将计算班级成绩的平均值。 */ /******</pre>
空白区域	为了增加程序的可读性，写程序时应留出一定的空间； 每个程序结构之间至少有一空行将它们隔开。
缩进	不同级别的括号之间应该有缩进； 每一个起始括号和终止括号应单独占一行。
缩进示例	<pre>While (miss_distance > threshold) { success_code=move_robot (target_location); if (success_code==MOVE_FAILED) { printf(" The robot move has failed.\n"); } }</pre>
大小写	所有定义应大写； 所有其它标识符及保留字均应小写； 对用户的输出信息可以大、小写相混合，以便用户清晰明了。
大写示例	<pre>#define DEFAULT_NUMBER_OF_STUDENTS 15 int class_size=DEFAULT_NUMBER_OF_STUDENTS;</pre>

14.7 总 结

本章介绍了代码复查检查表以及如何设计和使用自己的某种检查表。检查表包括了一系

列应该精确遵循的步骤。使用根据白己的缺陷数据制定出来的检查表，就可更有效地进行代码复查。检查表不但能帮你找到更多的缺陷，而且还能帮你更快地发现缺陷。

在构造代码复查检查表时，要按照所使用的编程语言进行裁减，要根据缺陷数据进行设计，而且要随着技能和经验的提高，对原有的代码复查检查表作适当的调整。

在使用检查表时，按照步骤进行复查；在完成每一个程序或过程的复查后再开始下一个程序或过程的复查；完成每项检查之后做出标记。发现缺陷时，记下缺陷号及其检查步骤。当全部完成检查时计算出累计和累计百分比项。在完成每个程序后，复查数据和检查表，并从中发现如何对现有的代码复查检查表进行改进。

14.8 练 习 14

构造一个检查表以找出在编译和测试阶段引起大部分问题的缺陷类型。把最初的检查表限制在几个缺陷类型上，并在下一个程序中进行使用。提交完整的检查表、程序以及完整的项目计划总结表。另外，提交一份用来构造检查表的缺陷数据，并使用与表 14.6 相类似的格式分析缺陷数据。

提交以前没有提交过的时间记录日志和周活动总结表。另外，完成你开发程序的项目计划总结表、缺陷记录日志、代码复查检查表。包括计划的和实际的开发时间以及引入的和排除的实际的缺陷。

第 15 章 缺陷预测

本章讨论如何分析和使用缺陷数据来提高计划的准确性和产品的质量。并通过实例来说明一些有用的分析这些缺陷数据的方法。在练习中，将对本课程中开发的所有程序的缺陷数据写一个简短的报告。

使用缺陷数据的主要目的是确定怎样更好地预防引入缺陷，怎样才能更好地发现所引入的缺陷。随后的几章，将介绍一些测量方法来帮助控制工作的质量。这将有助于稳定地开发出高质量的程序。

15.1 缺陷率

有经验的工程师的缺陷引入率一般是每千行代码 50 ~ 250 个缺陷。在一个编制小型程序的练习中，38 位工程师的缺陷引入率的情况如图 15.1 所示。这是他们开始测量、跟踪、控制缺陷数日之前的情况。这张图反映了工程师们在未经过 PSP 训练时缺陷引入率的情况。

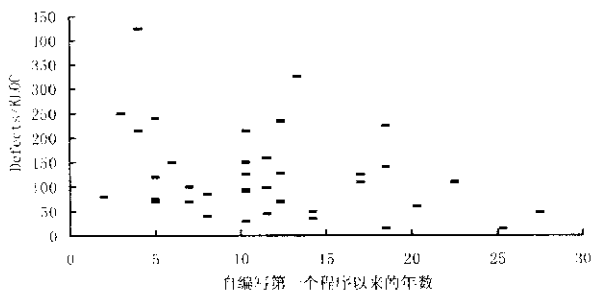


图 15.1 在 PSP 培训之前缺陷率与工作年限的关系

另外要注意，图 15.1 显示了缺陷密度和工程师写程序年数的函数关系。从图中可以看出，除

了几个经验特别少的工程师引入的缺陷数比其他人要多一些之外，其他工程师的缺陷率与写程序的年数的关系并不大。这说明，对于一些基本的质量实践，大部分工程师都可以从自己的经验中学到。只不过有些人能迅速地掌握这些方法，而另外一些人花的时间要长一些。有了这些基本的经验之后，缺陷数目的多少主要由每个人的编程规范所决定。

图 15.2 显示了上述 38 位工程师在学完 P S P 课程后的缺陷数据 [Humphrey 95]。此时，他们已经跟踪过 10 个小程序的缺陷，并且对这些缺陷的类型、分布和修复时间进行了两次系统的研究。他们知道自己通常引入的缺陷的类型和数目以及他们需要使用多长时间来修复缺陷。这有助于他们认识到规范化个体实践的重要性，并且激励他们在设计和编写程序时要更加仔细。最终，他们减少了缺陷的引入率。图 15.1 和图 15.2 中的数据说明，在减少引入缺陷的数目方面，个人的编程规范与对缺陷的跟踪和分析相结合，比多年的经验要有效的多。

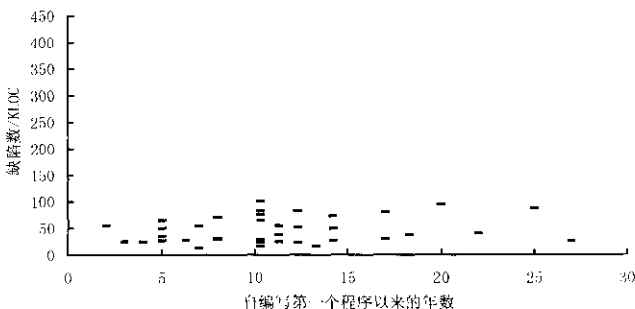


图 15.2 在 PSP 培训之后缺陷率与工作年限的关系

从图 15.1 和图 15.2 我们还可以看出，软件工程师在工作中一般会引入大量的缺陷。除个别情况外，平均的缺陷引入率在 100 个缺陷 / 千行代码左右或更多。即使是在工程师学过缺陷管理后，平均引入缺陷数也是 50 个 / 千行代码左右。也就是每 20 行有一个缺陷。这个数字看起来好像不多，但按照这个比率，1000 行代码有 50 个缺陷，10000 行代码有 500 个缺陷，1,000,000 代码就有 50,000 个缺陷。必须找到和修复所有这些缺陷，才能保证程序工作正常。在此，再次指出一个关键点，即引入缺陷是人类的正常现象，所有的软件工程师都会引入缺陷。因此所有的工程师都应该了解自己引入缺陷的类型和数目。

15.2 缺陷数据的使用

到目前为止，已经收集了一定数量的缺陷数据来帮助你了解自己引入的缺陷。然后，可以使用这些数据来设计一个人的代码复查检查表。本章将利用这些缺陷数据估计在新开发的程序中引入的缺陷数，同时还讲述怎样利用这些历史数据对程序开发的各个阶段将引入和排

除的缺陷数作出一个相当合理的预测。

准确地估计程序中缺陷的个数是很重要的。在开发过程中，总是可以再进行一轮测试或代码复查。但是，决定是否这样做的唯一的办法就是分析缺陷数据。通过把当前的项目数据和以前的数据相比较，就能大概知道正在开发的程序的质量情况。这样就能决定是否需要增加一些缺陷排除步骤。

工程师个人承诺要开发无缺陷的程序是最重要的，没有这种承诺这些测量方法和计划是没有太大帮助的。尽管生产这样高质量的产品很难，而且在开始时往往很难做到，但是经过不断的实践，在大多数情况下，你将能开发出无缺陷的程序。这一点很重要，因为有一天你可能要承担这样的项目，而无缺陷是这项目的基本要求。为此，我建议把每一个程序的开发都作为一次实际锻炼，为将来的这一天做好准备。

15.3 缺陷密度

在 PSP 中，基本的缺陷测量是以每千行代码的缺陷数(Defects/KLOC)来测量的。称为缺陷密度(Dd)，其测量单位是 Defects / KLOC。这里 K 代表 1000。可按照以下步骤来算一个程序的缺陷密度(Defects/KLOC)：

1. 累计开发过程中每个阶段发现的缺陷总数(D)。
2. 统计程序中新开发的和修改的代码行数(N)。
3. 计算每千行的缺陷数 $Dd=1000 \times D/N$ 。

例如，一个 96 行的源程序总共有 14 个缺陷，则缺陷密度是：

$$Dd = 1000 \times 14 / 96 = 145.83 \text{ defects / KLOC.}$$

在计算缺陷密度时，最重要的是要使用正确的规模测量。正如第 6 章所述，本书是用程序中新开发的和修改的代码行数作为计算程序的规模。也就是说，如果你使用了库函数或拷贝了某个程序的一部分，不要把它们代码计算在待开发程序的规模之内。应统计所有新编的代码行数以及对所引用的程序修改的行数。

15.4 缺陷率的预测

当开发一个新的程序时，可能会觉得很难估计你将引入多少缺陷。这理由是，这个数会因程序的不同而不同。缺陷数据不稳定是由以下几个原因造成的。首先是经验问题，个人的技能是在不断提高的。开始编程时，要面临着许多以前没有碰到过的问题。往往不能确定有些过程和函数是如何执行的，可能对语言的结构不清楚或者可能会遇到新的编译器或编程环境方面的问题。这些问题都会引起开发时间和缺陷率的波动。有了经验后，你将逐渐克服这些问题，犯的错误就减少了。这既减少缺陷的总数又减少缺陷数目的波动。缺陷的减少起初是由于经验的增加和对语言的熟练程度的提高。然而，经过这最初的提高后，就需要收集

和分析缺陷数据来进一步改进。

缺陷率波动的第二个原因是个体过程不稳定。也就是说，当开始学习写程序时你也同时开始学习使用新的过程和方法。你的过程将随着实际的经验不断地发展。这就会引起完成不同程序任务的时间和引入缺陷的数目的波动。最后，缺陷本身也是这种变化的原因。引入的缺陷越多，修复这些缺陷所花时间就越长。当工程师编写代码时，一般每小时引入 6 ~ 8 个缺陷，修复缺陷所花的时间也多，引入新的缺陷的机率也会增加。因为缺陷的修改时间变动幅度很大，所以，很难对一个引入很多缺陷的过程进行预测。

随着开发过程的改进，过程会逐步稳定下来。这种稳定将提高缺陷预测的准确性。如果在代码复查方面花了足够的时间，你的过程会迅速稳定下来。一旦你的过程相当稳定，缺陷也将容易预测。根据对最近的程序跟踪每千行引入和排除的缺陷数，就能相当准确地估计出在将来的程序中可能引入和排除的缺陷数。

15.5 缺陷估计

当为一个新的程序作计划时，先估计出程序可能有多少新的和修改的代码行，然后计算以前开发程序的每千行源代码的平均缺陷数。有了这些数据，就可以计算出在新的程序中的每千行代码的缺陷数。计算公式如下：

$$Dd_{plan} = 1000 \times (D_1 + \dots + D_j) / (N_1 + \dots + N_j)$$

例如，如表 15.1 所示，假设你有 5 个程序的数据，则计算 Dd_{plan} 的过程如下：

$$\begin{aligned} Dd_{plan} &= 1000 \times (6 + 11 + 7 + 9 + 5) / (37 + 62 + 49 + 53 + 28) \\ &= 1000 \times 38 / 229 = 165.94 \text{ defects / KLOC} \end{aligned}$$

假设新程序有同样的缺陷密度，可按照下面的公式计算预期的缺陷数：

$$D_{plan} = N_{plan} \times Dd_{plan} / 1000$$

用同样的示例，现在假设新程序规模的估计值为 56 行，则可计算出预期的缺陷数为：

$$\begin{aligned} D_{plan} &= 56 \times 165.94 / 1000 \\ &= 9.29 \text{ defects} \end{aligned}$$

使用这些数据你就能估计出计划要开发的程序有 56 行代码，会出现 9 个缺陷。

在项目计划总结表中的累计数据和缺陷数据是用于帮助进行计算的。

$$D_{plan} = N_{plan} \times D_{ToDate} / N_{ToDate}$$

使用表 15.1 的数据计算如下：

$$D_{plan} = 56 \times 38 / 229 = 9.29$$

这和前面的结果一致。

估算出新程序总的缺陷数目之后，就能计算出每个阶段预计引入和排除的缺陷数。用预计的缺陷总数乘以每个阶段的累计百分比值再除以 100 就得到每个阶段的缺陷数。这个数就是新的程序在这个阶段的预计缺陷数。这也就是要计算缺陷累计和缺陷累计百分比值的原因。它们为缺陷估计提供了必要的历史数据。15.7 节给出一个用更新的项目计划总结表计算

的示例。

表 15.1 缺陷数据的示例

程序编号	缺陷个数 (D)	LOC (N)
1	6	37
2	11	62
3	7	49
4	9	53
5	5	28
总 计	38	229

15.6 更新的项目计划总结表和示例

新的 PSP 过程描述表和项目计划总结表中加入了有关缺陷计划的内容, 如表 15.2 和表 15.3 所示。表 15.4 是新的项目计划总结表说明。在这些表中, 新增加的表项用*斜体*显示。其中在引入的和排除的缺陷数的两栏中分别增加了一行计划, 在总结一栏中增加了一行 Defects/KLOC (每千行源代码缺陷数)。

表 15.5 是学生 X 的程序 13 的项目计划总结表的示例。以下各段描述了该学生怎样利用他在前面编写的程序 12 的总结数据来完成表 15.6 的。在这些表中不同的值可用字母来标记数据的来源。首先在计划阶段:

表 15.2 PSP 过程脚本

过程目的	指导用户进行小型程序的开发
入口准则	问题描述; PSP 项目计划总结表; 一份代码复查检测表; 以前开发程序的实际的规模和时间数据; 时间记录日志; 缺陷记录日志。
1 计划	获取对程序功能的描述。 估计整个程序的代码行数及其可能的最大值和最小值。 确定开发效率 (Minutes/LOC)。 估算开发时间及其可能的最大和最小值。 <i>估计各个开发阶段将引入和排除的缺陷的个数。</i> 将计划数据登入项目计划总结表。 将计划阶段所花费的时间记入时间记录日志。

续表

过程目的	指导用户进行小型程序的开发
2 设计	进行系统设计。 按照指定的格式记录设计文档。 将设计阶段所花费的时间记入时间记录日志。
3 编码	编码实现程序设计。 使用标准的格式来书写程序代码。 将编码阶段所花费的时间记入时间记录日志。
4 代码复查	复查所有的源程序代码。 遵照代码复查脚本和代码复查检测表。 修复并记录发现的每一个缺陷。 将代码复查阶段所花费的时间记入时间记录日志。
5 编译	编译程序。 修复并记录所有发现的缺陷。 将编译阶段所花费的时间记入时间记录日志。
6 测试	测试整个程序。 修复并记录所有发现的缺陷。 将测试阶段所花费的时间记入时间记录日志。
7 后置处理	将实际的规模、时间数据和缺陷数据登入项目计划总结表。 总结缺陷数据并更新复查检测表。 将后置处理阶段所用的时间记入时间记录日志。
出口准则	经过详尽测试的程序； 设计文档； 已经填写完成的代码复查检测表； 源程序清单； 已经填写完成的项目计划总结表； 已经填写完成的时间日志和缺陷日志。

表 15.3

PSP 项目计划总结表

学生 _____ 日期 _____
 程序 _____ 程序号# _____
 教师 _____ 程序设计语言 _____

总 结	计 划	实 际	累 计
Minutes/LOC	_____	_____	_____
LOC/Hour	_____	_____	_____
Defects/KLOC	_____	_____	_____
过程效益	_____		
A/FR	_____		
程序规模 (LOC)	_____		

续表

总 结	计 划	实 际	累 计		
新开发与更改的					
最大规模					
最小规模					
开发阶段时间 /min	计 划	实 际	累 计	累计百分比	
计划					
设计					
编码					
代码复查					
编译					
测试					
后置处理					
总计					
最大时间					
最小时间					
引入的缺陷	计 划	实 际	累 计	累计百分比	Def/Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					
排除的缺陷	计 划	实 际	累 计	累计百分比	Def/Hour
计划					
设计					
编码					
代码复查					
编译					
测试					
总计					

表 15.4

PSP 项目计划总结表使用说明

目 的	本表以一种方便而且易于存取的方式，记录估算的和实际的项目数据
表头	如下输入： ● 你的名字和今天的日期 ● 程序名和程序号 ● 教师的名字 ● 你将使用的编程语言
Minutes/LOC	在开发之前： ● 登入该项目计划的 Minutes/LOC 值。使用最近开发项目的累计开发效率值作为参考。 在开发之后： ● 用全部的开发时间除以实际的程序规模，得到开发效率的值。 ● 例如，如果该项目总共花了 196 分钟，产生了 29LOC，那么开发效率 Minutes/LOC 的值就是 $196/29 = 6.76$。
LOC/Hour	在开发之前： ● 用 60 除以计划的“Minutes/LOC”就得到了计划的“LOC/Hour”的值。 在开发之后： ● 用 60 除以实际的“Minutes/LOC”就得到了实际的“LOC/Hour”的值。 ● 对于上面的示例，$Minutes/LOC = 6.76$，那么，$LOC/Hour = 60/6.76 = 8.88$。
缺陷密度 (Defects/KLOC)	在开发之前： ● 找到最近的前一个程序的缺陷密度的累计值； ● 把它作为当前项目的计划的缺陷密度的值。 在开发之后： ● 计算缺陷密度的实际值和到日前为止的累计值； ● 缺陷密度的实际值 = $1000 \times \text{实际的缺陷总数} / \text{实际新开发的和修改的代码行数}$； ● 用类似的方法计算缺陷密度的累计值； ● 例如，到日前为止发现的缺陷的总数为 17，新开发的和修改的代码的总行数为 153，那么缺陷密度的累计值 = $1000 \times 17/153 = 111.11$。
程序规模(LOC)	在开发之前： ● 在计划列登入估算的全部新开发的与更改的代码行数，及其最大值和最小值。 在开发之后： ● 统计并登入实际新开发的与更改的代码行数； ● 在累计一栏中，把实际新开发的与更改的代码行数与上一个程序的新开发的与更改的代码行数相加，得到累计值。
开发阶段时间 计 划	用新开发的与修改的代码行数，乘以开发效率“Minutes/LOC”，得到项目的总开发时间。使用代码行的最大值乘以开发效率得到开发时间的最大值。 使用代码行的最小值乘以开发效率得到开发时间的最小值。 从最近的程序的项目计划总结表中，找到各个阶段所花时间的累计百分比。 使用以前程序的累计百分比值，计算出各个阶段的计划时间。
实 际	在任务完成之后，根据时间记录日志中的数据登入各阶段实际花费的时间。 从时间记录日志中得到这些数据。
累 计	对每个阶段，登入实际时间与最近的上一个程序的对应阶段的累计时间之和
累计百分比(%)	对于每个阶段，用 100 乘以该阶段的累计时间，再除以总的累计时间，得到累计时间百分比。

续表

目 的	本表以一种方便而且易于存取的方式, 记录估算的和实际的项目数据
引入的缺陷	在开发之前, 估算程序中将引入的缺陷的总数。
计 划	计划的缺陷密度乘以计划新开发的和修改的代码行数, 然后除以 1000。 例如, 假设计划的缺陷密度为 75.9 defects/LOC, 计划新开发的和修改的代码行数为 75LOC, 那么计划引入的缺陷个数就是 $75.9 \times 75 / 1000 = 5.69$, 这时可以取值为 6。 在开发之前, 根据估算出来的缺陷总数和以前开发程序的缺陷引入的累计百分比, 估算各个阶段将引入的缺陷的个数。
实 际	在开发完成后, 统计并登入各个阶段实际引入的缺陷的数目
累 计	对每个阶段, 登入实际引入的缺陷个数与最近的上一个程序和累计引入的缺陷的总和
累计百分比(%)	对每个阶段, 用 100 乘以该阶段的累计缺陷, 再除以总的累计缺陷, 得到累计缺陷百分比
排除的缺陷	在“总计”一行中, 登入估算的总的缺陷个数。
计 划	根据最近开发的程序的缺陷排除的累计百分比计算各个阶段计划排除的缺陷数。
实 际	在开发完成后, 统计并登入各个阶段实际排除的缺陷数
累 计	对每个阶段, 计算并登入实际排除的缺陷数与最近程序的累计缺陷排除数之和
累计百分比(%)	对每个阶段, 用 100 乘以该阶段累计缺陷, 再除以总的累计缺陷, 得到累计缺陷百分比

1. 学生 X 估计该新程序的规模是 58 行, 最大值是 72 行, 最小值是 41 行(a)。
2. 他接下来从程序 12 的项目计划总结表找到每行的累计分钟数, 如表中所示为 5.92 min./LOC (b)。他用此数作为程序 13 计划的开发效率。
3. 用总的估计的代码行 58 来估计项目的时间 $58 \times 5.92 = 343.36$ 或 343 min.(c)。
4. 学生 X 计算最大和最小的开发时间, 分别用最大和最小规模(a)乘以 5.92(b)得到 426min. 和 243 min.(d)。
5. 为了计算各个阶段的时间, 他从程序 12 的总结表中取出累计百分比(e)乘以估计的总时间 343 分(c)再除 100。就得到每个阶段的时间(f)。为了方便, 他对时间进行了取整。
6. 为了计算总的引入和排除的缺陷数。学生 X 在项目计划总结表(见表 15.6)中找到程序 12 的每千行累计缺陷数是 94.79(g)。从累计代码行数(h)及累计缺陷数(i)可以得到结果, 计算过程如下。
7. 用每千行缺陷数 94.79(i)和计划代码行数 58(a), 他计算出估计的缺陷数是 $94.79 \times 58 / 1000 = 5.50$, 即大约 6 个缺陷(j)。
8. 基于程序 12 的引入和排除缺陷的累计百分比值(k), 他计算出各个阶段的预计引入和排除的缺陷数(l)。

这样就完成了程序 13 的缺陷预测。

表 15.5

PSP 项目计划总结表的示例

学生	学生 X	日期	11/8/96
程序		程序号#	13
教师	先生 Z	程序设计语言	Ada

总 结	计 划		实 际		累 计	
Minutes/LOC	b	5.92	p	4.87	x	5.73
LOC/Hour	10.14		q	12.32	y	10.49
Defects/KLOC	i	94.79	r	106.4	z	96.90
过程效益						
A/FR						

程序规模(LOC)

新开发的与更改的	a	58	o	47	w	258
最大规模	a	72				
最小规模	a	41				

开发阶段时间
/min

开发阶段时间 /min	计 划		实 际		累 计		累计百分比	
计划	f	18	m	22	s	88	u	6.0
设计	f	35	m	24	s	151	u	10.2
编码	f	149	m	93	s	637	u	43.1
代码复查	f	20	m	37	s	111	u	7.5
编译	f	24	m	4	s	92	u	6.2
测试	f	64	m	8	s	240	u	16.2
后置处理	f	33	m	41	s	160	u	10.8
总计	c	343	m	229	s	1479	u	100
最大时间	d	426						
最小时间	d	243						

引入的缺陷	计 划		实 际		累 计		累计百分比	Def./Hour
计划								
设计	l	1			t	4	u	16.0
编码	l	5	n	5	t	21	u	84.0
代码复查								
编译								
测试								

								续表
引入的缺陷	计划	实际	累计	累计百分比				Def./Hour
总计	j	6	n	5	t	25	u	100.0
排除的缺陷	计划	实际	累计	累计百分比				Def./Hour
计划								
设计								
编码								
代复查码	l	2	n	3	t	8	u	32.0
编译	l	3	n	2	t	12	u	48.0
测试	l	1			t	5	u	20.0
总计	j	6	n	5	t	25	u	100.0

表 15.6 学生 X 程序 12 所填写的项目计划总结表

学生	学生 X	日期	11/11/96
程序		程序号 #	12
教师	先生 Z	程序设计语言	Ada

总 结	计 划	实 际	累 计
Minutes/LOC	6.30	4.93	b 5.92
LOC/Hour	9.52	12.17	10.14
Defects/KLOC			g 94.79
过程效益			
A/FR			
程序规模 (LOC)			
新开发与更改的	51	58	h 211
最大规模	65		
最小规模	37		
开发阶段时间 /min	计 划	实 际	累 计
计划	16	18	s 66 e 5.3
设计	27	44	s 127 c 10.2
编码	146	104	s 544 e 43.5
代码复查	12	38	s 74 e 5.9
编译	26	11	s 88 e 7.0

续表

开发阶段时间 /min	计 划	实 际	累 计	累计百分比
测试	68	29	232	18.6
后置处理	26	42	119	9.5
总计	321	286	1250	100.0
最大时间	410			
最小时间	233			
引入的缺陷	计 划	实 际	累 计	累计百分比
计划				
设计		1	4	20.0
编码		5	16	80.0
代码复查				
编译				
测试				
总计		6	20	100.0
排除的缺陷	计 划	实 际	累 计	累计百分比
计划				
设计				
编码				
代复查码		2	5	25.0
编译		2	10	50.0
测试		2	5	25.0
总计		6	20	100.0

15.7 登入实际的数据

在程序开发完成后，学生X在表中登入实际数据，步骤如下：

1. 根据时间记录日志登入各个阶段的实际的时间值(m)。
2. 根据缺陷记录日志登入实际引入和排除的缺陷数(n)。
3. 他在最终程序中统计新开发的和修改的程序行是 47,把此数记录到实际列下(o)。
4. 有了这些数据，他用总计时间 229(m)除新开发的和修改的行数 47(o)得到每行代码的实际时间，即 $229/47=4.87(p)$ 。
5. 根据每行代码的实际时间(p)计算出每小时实际代码行数， $60/4.87=12.32(q)$ 。

6. 根据实际缺陷数(n)和实际代码行数(o)计算出每千行实际的缺陷数, $1000 \times 5/47=106.4(r)$ 。

7. 通过把各个阶段的时间与前一个项目的相应累计时间相加, 他得到各个阶段的累计时间(s)。

8. 他把前一个项目的各个阶段的实际缺陷数与累计缺陷数相加得到他引入和排除缺陷的累计值(t)。

9. 计算累计百分比(u)。

10. 用累计时间值 1479(s)除以新开发的和修改的行数值 258(w)得到累计的每行时间, $1479/258=5.73(x)$ 。

11. 用 5.73(x)这个值来计算累计每小时代码行数, $60/5.73=10.47(y)$ 。

12. 最后, 他用 1000 乘累计引入缺陷数(t)再除以累计新的和修改的代码行数(w)得到累计每千行缺陷数, $1000 \times 25/258=96.90(z)$ 。

要注意的是, 累计和累计百分比这两个数据记录的是总的时间和缺陷的分布情况。在有了更多的经验之后, 你可能会发现, 开发效率和缺陷数据的变化如此显著, 以至于早期程序的数据已经不再适合指导编制新的程序的计划。这时, 你可能希望不用那些早期程序的数据, 而重新计算累计数据和累计数据百分比。

15.8 总 结

本章讲述了如何利用缺陷的历史数据来预测新程序中各个阶段引入和排除的缺陷数。除了学会制订缺陷预测计划, 你还学会了如何制订更好的开发计划以及如何更好地控制你引入和排除的缺陷数。

制订缺陷预测计划时, 你需要以前程序的缺陷和程序规模的数据以及新程序的规模的估计值。为了计算预测的缺陷数, 假设新程序的每千行缺陷数与以前开发过的程序的每千行缺陷数平均值相同。还假设引入和排除缺陷的分布也相近。这样, 就可以用前一个程序的每千行缺陷数和缺陷数累计百分比来计算。

在已更新的项目计划总结表中引入了缺陷预测的内容。在每个项目的计划阶段给出计划值。在每个程序的后置处理阶段计算计划值、实际值和累计每千行代码缺陷数, 以便利用这些数据计划下一个项目。

15.9 练 习 15

对你写的下一个程序, 确定每千行缺陷数和估计每个阶段引入和排除的缺陷数。在项目计划总结表中记下这些数据, 如表 15.3。

提交你以前没有提交过的时间记录日志和周活动总结表。另外, 提交完成的每个程序的

项目计划总结表、缺陷记录日志和代码复查检查表。包括开发时间以及引入的和排除的缺陷的计划和实际值。

参 考 文 献

[Humphrey 95] Humphrey, W. S. *A Discipline for Software Engineering*. Reading, MA: Addison- Wesley, 1995.

第 16 章 缺陷排除的经济效益

本章讨论缺陷排除的经济效益。缺陷排除是一种经济学问题，因为它的决策要综合考虑到软件费用、时间进度和软件质量等三方面的因素。尽管应该总是优先考虑产品质量，但许多项目有进度和费用方面的限制。因此，开发过程中缺陷排除的相对费用、把缺陷留给客户的影响以及所导致的用户支持费用都是软件开发中的重要问题。本章的练习中，你将分析本学期到目前为止在你所开发的程序中引入和排除的缺陷率。

16.1 高质量工作的必要性

在过去的三十年中，计算机领域尽管在技术上取得了很大的进步，但软件工程师的工作却没有一点改变。软件工程师仍是把大程序分成几个小部分，然后他们自己再开发其中一个或几个部分。在这个过程中，他们要花 30% ~ 40% 或更多的时间让程序通过一些基本的测试。当程序可以运行后，程序员要测试并把它们逐级集成为较大型的系统。集成和测试的过程几乎全是为了发现和修复更多的缺陷。当产品最终交付后，产品质量往往很差，以致于工程师必须要再花费几个月的时间来修复客户反馈的缺陷。这是三十年前软件开发的普遍现象，但今天情况依然如此。

更好的软件开发方法确实是存在的。但基本的问题是，软件工程师在开始写程序时，往往匆忙地进行设计和编码，然后进行编译和测试。但是，保证工作质量的原则是在第一次就要开发出合格的产品。当你按照这个原则去做之后，就不会在编译和测试阶段需要去排除那么多的缺陷，你和你的组织也不需要花费那么多时间来查找和修复缺陷。

在 Motorola 工作的一些工程师的示例可以说明这个原则的重要性。他们的软件是用于运行生产高级寻呼机的工厂。无论产品或是产品过程改变时，工程师都必须对软件做出相应的改变。为了在每一个新发布的软件中查找缺陷，产品生产线只好停产来进行测试。即使经过测试，产品生产线也因为一些漏掉的软件缺陷而定期地被中断。当软件工程师学习 PSP 之后，在他们第一次对软件进行更新时，在测试阶段只发现了一个缺陷，而且在随后产品生产的几个月中没有发现任何缺陷。这不仅节省测试时间，而且也减少停产的情况。从而工厂能够生

产出更多的产品，公司能够得到更多回报。

16.2 缺陷排除问题

因为缺陷难于查找和修复，所以排除缺陷的花费很大。当构造大型复杂软件系统时，这个问题就更严重。软件系统的规模和复杂度大约每十年增长十倍。一个早期的激光打印机的支持程序需要 20,000 行代码，而它的最新版本需要 100 万行以上的代码。再有，十年前，汽车里没有软件，可今天最新的汽车装有成千上万行的软件。由于软件的规模和复杂度将继续增加，缺陷排除也将变得更昂贵、更耗时。

为了了解和控制缺陷带来的费用，很有必要测量缺陷排除的效果。有一种测量方法是计算每小时排除缺陷的个数；另一种是计算缺陷排除效益，即测量通过某一排除方法所发现的缺陷的百分比。因此，如果一个产品在开始测试时有 100 个缺陷，在测试阶段发现了 45 个缺陷，那么测试阶段的缺陷排除效益就是 45%。当知道每种缺陷排除方法的排除效益和排除率之后，就知道如何更好地发现和修复缺陷。本章将讨论这些缺陷排除效益和缺陷排除率的测量问题，下面的几章将讨论如何使用这些测量。

16.3 缺陷排除时间

对于几十行到几百行的程序，可能只引入很少的几个缺陷。尽管发现和修复这些缺陷会带来一点麻烦，但通常不会花费很长的时间。但是，对于较大型的程序，就需要很多时间来发现和修复缺陷。例如，对 Microsoft 的 NT 系统进行系统测试时，250 个工程师花了一年的时间，共发现和修复 30,000 个缺陷[Zachary]，平均每 16 个小时修复一个缺陷。

在大型系统中，如果工程师不能排除所有在开发程序中引入的缺陷，那么就必须由其他人来查找和修复这些缺陷。缺陷越多，修复越迟，修改工作的费用就越大。对于大型软件产品，这种费用的增长是惊人的。在第 13 章曾提到，在代码复查阶段工程师发现和修复一个缺陷只用几分钟；在单元测试时，费用增加了几倍，但修复一个缺陷仍然平均只有 15 分钟到半小时。可是对于那些由于工程师漏掉的缺陷，必须等到以后的测试或客户使用时才可能发现，这样，一般每个缺陷需要若干个小时来查找和修复。

16.4 缺陷引入和排除的经验

在设计阶段，学生和工程师一般每小时引入 1~3 个缺陷；在编码阶段，一般每小时引入 5~8 个缺陷。在测试阶段，每小时排除 2~4 个缺陷，而在代码复查阶段，每小时可发现 6~12 个缺陷。图 16.1 给出了一个班 14 个工程师的缺陷引入率，图 16.2 给出了他们的缺陷

排除率。在做程序 1 时, 学生们一般不做代码复查或跟踪缺陷排除效益及缺陷排除率。但从程序 1 到程序 10, 工程师逐渐学会如何计算缺陷排除效益和缺陷排除率, 他们对 PSP 的缺陷排除方法已经相当熟练。

图 16.1 和图 16.2 中的数据有几点很有趣。首先, 图 16.1 和图 16.2 中的比率是 14 个人的平均值。因此他们看起来相当有规律。可是单个工程师的比率变化会大一些。个人的比率根据程序不同会有很大变化。但是, 只要坚持使用 PSP 来跟踪自己的工作, 那么最终必然看到进步。

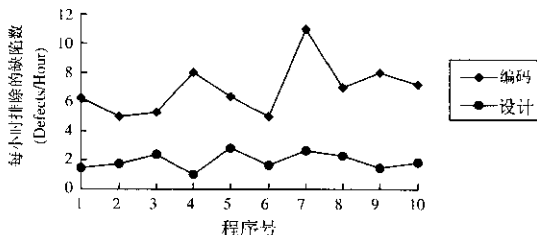


图 16.1 一个班级的平均缺陷引入率

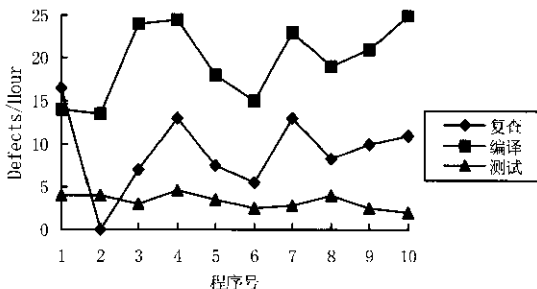


图 16.2 一个班级的平均缺陷排除率

在测试阶段, 每小时排除缺陷数一般在 2~4 个之间。对于没有经验的学生和非常有经验的工程师, 这个值差不多一样。

代码复查是一种技能。经过实践, 大多数工程师能达到每小时排除 8~10 个缺陷。关键是要使用第 13 章和 14 章中所描述的方法。

缺陷管理有一点像沿一个下降的自动扶梯向上走。在设计和编码阶段引入缺陷。当发现和排除这些缺陷时, 又需要花费更多的时间进行重新设计和重新编码。在这段时间也会引入更多的缺陷。为了赢得这场比赛, 就需要减少缺陷引入率, 增加缺陷排除率。为了做到这一点, 必须计算和跟踪缺陷的引入率和排除率。

为了计算个人的缺陷引入率和缺陷排除率, 需要使用 PSP 方法收集必要的数

16.5 节省缺陷排除时间

了解缺陷排除率对大型项目是很重要的，对每个软件工程师也都有帮助的。当开发一个中等规模的程序时，假设你的数据和其他工程师的数据差不多，在每千行代码中大约引入 100 个缺陷，在编译时每千行代码大约发现 50 个缺陷，在单元测试中每千行代码大约发现另外 40 个缺陷。

如表 16.1 所示，这就意味着 500 行代码中有 50 个缺陷。在编译时将发现其中的 25 个缺陷，运气好的话，在测试时再发现 20 个。如果不做代码复查，一般要花 2 个小时去发现和修复这 25 个编译缺陷。单元测试大约需要花 10 个小时去发现和修复这 20 个预期的缺陷。缺陷排除总共需要花费 12 个小时。

学习了 PSP 后，每千行代码应该只引入 50 个缺陷（对规模为 500LOC 的程序来说，则引入 25 个缺陷）。在第一次编译之前的代码复查将发现其中的 60% ~ 70%。现在编译仅需要大约 30 分钟的时间。在单元测试阶段只发现了 4 或 5 个缺陷。然而，大约需要花费了 2 个小时。假设你花 2.5 小时进行代码复查，总共的缺陷排除时间大约是 5 小时左右，这就节省了 7 个小时。

表 16.1 缺陷引入和排除的示例

	500 LOC		10,000 LOC	
	未经 PSP 训练	经过 PSP 训练	未经 PSP 训练	经过 PSP 训练
缺陷数				
缺陷总数	50	25	1000	500
代码复查阶段发现的缺陷数	0	15	0	300
残留缺陷数	50	10	1000	200
编译阶段发现的缺陷个数	25	5	500	100
单元测试阶段发现的缺陷数	20	4	400	80
残留缺陷数	5	1	100	20
时间 (Hour)				
代码复查时间	0	2.5	0	50
编译时间	2	0.5	40	10
单元测试时间	10	2	200	40
个人缺陷排除时间	12	5	240	100
集成和系统测试时间			1000	200
缺陷排除时间合计	12	5	1240	300

虽然对于规模只有几天的作业，为了节省7小时而付出这么大的努力，似乎不太值得。但要考虑到这些数字按比例放大将会怎么样。如果开发的不是500行程序而是10,000行程序，开始你将引入1000个缺陷。但经过PSP训练后将只出现500个缺陷。如果花50小时进行代码复查，将把编译时间从40小时减少到10小时，测试时间从200小时减少到40小时。这样就节省了140个小时，也就是几周的工作时间。

但是，更重要的费用是花费过程中那些没有发现的缺陷的费用。在那个500行代码的示例中只有几个漏掉的缺陷。对于那个大型程序将约有100个漏掉的缺陷。而发现这些缺陷一般需要1000个小时。在经过PSP训练后的过程中，将只留下20个缺陷，节省测试时间800个小时。

看一下在实践中这意味着什么，假设你和另外4位工程师要开发一个50,000行代码的软件产品。你们计划每个人分别开发一个10,000行代码的组件，然后集成为一个系统进行测试。根据一般的工程师的数据，你和你的组员每千行代码中将引入100个缺陷。这就意味着大约有5000个缺陷需要发现和修复。按照典型的比率，在编译阶段大约会发现2500个缺陷，在测试阶段再发现2000个缺陷，大约还有500个缺陷要等到集成和系统测试时才可能发现。假设你的产品比Microsoft的NT系统简单，你大约以每10小时一个缺陷的速度发现这些缺陷，则排除这些缺陷大约需要这5位工程师花5000小时。如果整个组不做其它事，每周工作40小时，这就需要花费6个月的时间。可是，如果每个人首先自己复查了程序，然后同组之间进行相互检查，这将至少节省5个月的测试时间。对于一个为期两年的项目，在测试时间上的这样的差别将对项目是按期交付还是严重推迟有重大影响。

16.6 在PSP项目计划总结表中每小时缺陷数的计算

表16.2显示如何在已更新的项目计划总结表中计算累计缺陷排除率。对每个阶段，从这个阶段的累计引入缺陷数和累计花费分钟数开始，然后计算累计每小时引入缺陷数，其计算公式为： $60 \times (\text{该阶段累计引入缺陷数}) / (\text{该阶段累计花费分钟数})$ ；同样可计算累计每小时排除缺陷数，其计算公式为： $60 \times (\text{该阶段累计排除缺陷数}) / (\text{该阶段累计花费分钟数})$ 。在表16.2的示例中，这些计算结果在缺陷引入和缺陷排除两栏的最右列中。计算过程如下。

对于各阶段累计每小时引入缺陷数：

在设计阶段累计每小时引入缺陷数

$$= 60 \times (\text{设计阶段累计引入缺陷数}) / (\text{设计阶段累计花费分钟数})$$

$$= 60 \times 5 / 195$$

$$= 1.54 (\text{defects/hour}).$$

在编码阶段累计每小时引入缺陷数

$$= 60 \times (\text{编码阶段累计引入缺陷数}) / (\text{编码阶段累计花费分钟数})$$

$$= 60 \times 25 / 792$$

$$= 1.89 (\text{defects/hour}).$$

用同样方法可以计算所有其它阶段的缺陷引入率,但上面这两个是最重要的。对于各阶段累计每小时缺陷排除率:

在代码复查阶段累计每小时排除缺陷数

$$=60 \times (\text{代码复查阶段累计排除缺陷数}) / (\text{代码复查阶段累计花费分钟数})$$

$$=60 \times 12/145$$

$$=4.97 \text{ (defects/hour)}.$$

在编译阶段累计每小时排除缺陷数

$$=60 \times (\text{编译阶段累计排除缺陷数}) / (\text{编译阶段累计花费分钟数})$$

$$=60 \times 13/100$$

$$=7.80 \text{ (defects/hour)}.$$

在测试阶段累计每小时排除缺陷数

$$=60 \times (\text{测试阶段的累计排除缺陷数}) / (\text{测试阶段累计花费分钟数})$$

$$=60 \times 6/279$$

$$=1.29 \text{ (defects/hour)}.$$

上面只计算了一些累计值。如果需要,可以分别计算计划的和实际的每小时缺陷数。注意,这些值随着你的过程的变化而变化。另外,因为较早的程序没有记录缺陷数据,所以缺陷数比实际值要低一点。对于更新的项目计划总结表的说明在表 16.3。

表 16.2

PSP 项目计划总结表

学生	学生 X		日期	11/28/96
程序			程序号#	14
教师	先生 Z		程序设计语言	Ada
总结	计划	实际	累计	
Minutes/LOC	5.73	4.65	5.48	
LOC/Hour	10.47	12.90	10.95	
Defects/KLOC	96.90	77.9	92.53	
过程效益	33.3	80.0	40.0	
A/FR				
程序规模(LOC)				
新开发与更改的	67	77	335	
最大规模	85			
最小规模	49			
开发阶段时间/min	计划	实际	累计	累计百分比
计划	23	32	120	6.5
设计	39	44	195	10.6

续表

开发阶段时间/min	计 划	实 际	累 计	累计百分比	
编码	166	155	792	43.1	
代码复查	29	34	145	7.9	
编译	24	8	100	5.5	
测试	62	39	279	15.2	
后置处理	41	46	206	11.2	
总计	384	358	1837	100.0	
最大时间	487				
最小时间	281				
引入的缺陷	计 划	实 际	累 计	累计百分比	Def./Hour
计划					
设计	1	1	5	16.1	1.54
编码	5	4	25	80.7	1.89
代码复查					
编译		1	1	3.2	
测试					
总计	6	6	31	100.0	
排除的缺陷	计 划	实 际	累 计	累计百分比	Def./Hour
计划					
设计					
编码					
代码复查	2	4	12	38.7	4.97
编译	3	1	13	41.9	7.80
测试	1	1	6	19.4	1.29
总计	6	5	31	100.0	

表 16.3

PSP 项目计划总结表使用说明

目 的	本表以一种方便而且易于存取的方式，记录估算的和实际的项目数据
表头	如下输入： ● 你的名字和今天的日期 ● 程序名和程序号 ● 教师的名字 ● 你将使用的编程语言

目 的	本表以一种方便而且易于存取的方式, 记录估算的和实际的项目数据
Minutes/LOC	在开发之前: ● 登入该项目计划的 Minutes/LOC 值。使用最近开发项目的累计开发效率值作为参考。 在开发之后: ● 用全部的开发时间除以实际的程序规模, 得到开发效率的值。 ● 例如, 如果该项目总共花了 196 分钟, 产生了 29LOC, 那么开发效率 Minutes/LOC 的值就是 $196/29 = 6.76$。
LOC/Hour	在开发之前: ● 用 60 除以计划的 “Minutes/LOC” 就得到了计划的 “LOC/Hour” 的值。 在开发之后: ● 用 60 除以实际的 “Minutes/LOC” 就得到了实际的 “LOC/Hour” 的值。 ● 对于上面的示例, Minutes/LOC = 6.76, 那么, LOC/Hour = $60/6.76 = 8.88$。
缺陷密度 (Defects/KLOC)	在开发之前: ● 找到最近的前一个程序的缺陷密度的累计值; ● 把它作为当前项目的计划的缺陷密度的值。 在开发之后: ● 计算缺陷密度的实际值和到目前为止的累计值; ● 缺陷密度的实际值 = $1000 \times \text{实际的缺陷总数} / \text{实际新开发的和修改的代码行数}$; ● 用类似的方法计算缺陷密度的累计值; ● 例如, 到目前为止发现的缺陷的总数为 17, 新开发的和修改的代码的总行数为 153, 那么缺陷密度的累计值 = $1000 \times 17/153 = 111.11$。
过程效益	计算过程效益的计划值、实际值和累计值。 过程效益 = $100 \times (\text{编译前排除的缺陷数}) / (\text{编译前引入的缺陷的个数})$。如果在编译前引入了 5 个缺陷且发现和修复了 4 个, 那么过程效益 $\text{yield} = 100 \times 4/5 = 80.0\%$。
程序规模(LOC)	在开发之前: ● 在计划列登入估算的全部新开发的与更改的代码行数, 及其最大值和最小值。 在开发之后: ● 统计并登入实际新开发的与更改的代码行数; ● 在累计一栏中, 把实际新开发的与更改的代码行数与上一个程序的新开发的与更改的代码行数相加, 得到累计值。
开发阶段时间 计 划	用新开发的与修改的代码行数, 乘以开发效率 “Minutes/LOC”, 得到项目的总开发时间。 使用代码行的最大值乘以开发效率得到开发时间的最大值。 使用代码行的最小值乘以开发效率得到开发时间的最小值。 从最近的程序的项目计划总结表中, 找到各个阶段所花时间的累计百分比。 使用以前程序的累计百分比值, 计算出各个阶段的计划时间。
实 际	在任务完成之后, 根据时间记录日志中的数据登入各阶段实际花费的时间。 从时间记录日志中得到这些数据。
累 计	对每个阶段, 登入实际时间与最近的上一个程序的对应阶段的累计时间之和。
累计百分比(%)	对于每个阶段, 用 100 乘以该阶段的累计时间, 再除以总的累计时间, 得到累计时间百分比。

续表

目 的	本表以一种方便而且易于存取的方式，记录估算的和实际的项目数据
引入的缺陷	在开发之前，估算程序中将引入的缺陷的总数。
计 划	计划的缺陷密度乘以计划新开发的和修改的代码行数，然后除以 1000。 例如，假设计划的缺陷密度为 75.9 defects/LOC，计划新开发的和修改的代码行数为 75LOC，那么计划引入的缺陷个数就是 $75.9 \times 75 / 1000 = 5.69$ ，这时可以取值为 6。 在开发之前，根据估算出来的缺陷总数和以前开发程序的缺陷引入的累计百分比，估算各个阶段将引入的缺陷的个数。
实 际	在开发完成后，统计并登入各个阶段实际引入的缺陷的数目
累 计	对每个阶段，登入实际引入的缺陷个数与最近的上一个程序和累计引入的缺陷的总和
累计百分比(%)	对每个阶段，用 100 乘以该阶段的累计缺陷，再除以总的累计缺陷，得到累计缺陷百分比
缺陷引入率 (defects/Hour)	计算设计和编码阶段的缺陷引入率。 例如，对于设计阶段，用设计阶段引入的缺陷的累计值乘以 60 再除以设计阶段花费的时间累计值 $= 60 \times 5 / 195 = 1.54$ (defects/Hour)
排除的缺陷	在“总计”一行中，登入估算的总的缺陷个数。
计 划	根据最近开发的程序的缺陷排除累计百分比计算各个阶段计划排除的缺陷数。
实 际	在开发完成后，统计并登入各个阶段实际排除的缺陷数
累 计	对每个阶段，计算并登入实际排除的缺陷数与最近程序的累计缺陷排除数之和
累计百分比(%)	对每个阶段，用 100 乘以该阶段累计缺陷，再除以总的累计缺陷，得到累计缺陷百分比
缺陷排除率 (defects /Hour)	计算代码复查、编译和测试阶段的缺陷排除率。 例如，对于测试阶段，用测试阶段排除的缺陷累计值乘以 60 再除以测试阶段花费的时间累计值 $= 60 \times 6 / 279 = 1.29$ (defects/Hour)

16.7 缺陷排除效益的计算

当在某一个阶段排除缺陷时，你可能想知道自己在这个阶段发现了多少缺陷，还漏掉多少缺陷。虽然这在开发阶段是无法确切知道的，但过程的历史数据可以给你一个很好的参考。其方法就是计算和跟踪缺陷排除效益。在 PSP 中，我们定义缺陷排除效益是在第一次编译和测试前发现缺陷的百分比。

对计划的、实际的和累计的过程效益的计算结果显示在项目计划总结表中上部的总结一栏中，参见表 16.2。例如，计划阶段的过程效益值的计算如下：

编译前计划的引入缺陷数是 $1+5=6$ 。

编译前计划的排除缺陷数是 2。

这样，计划的过程效益值的计算如下：

$$\begin{aligned}\text{过程效益}_{\text{plan}} &= 100 \times (\text{编译前计划的排除缺陷数}) / (\text{编译前计划的引入缺陷数}) \\ &= 100 \times 2 / (1+5) \\ &= 100 \times 2/6 \\ &= 33.3\%.\end{aligned}$$

实际的过程效益值的计算如下：

$$\begin{aligned}\text{过程效益}_{\text{Actual}} &= 100 \times (\text{编译前实际的排除缺陷数}) / (\text{编译前实际的引入缺陷数}) \\ &= 100 \times 4 / (1+4) \\ &= 100 \times 4/5 \\ &= 80.0\%.\end{aligned}$$

对于累计值：

$$\begin{aligned}\text{过程效益}_{\text{ToDate}} &= 100 \times (\text{编译前累计的排除缺陷数}) / (\text{编译前累计的引入缺陷数}) \\ &= 100 \times 12 / (5+25) \\ &= 100 \times 12/30 \\ &= 40.0\%.\end{aligned}$$

注意，在表 16.2 中，在编译阶段引入一个缺陷。由于过程效益的计算只关心编译前的缺陷排除率，所以只考虑在编译前引入的和排除的缺陷。在编译或测试阶段引入的任何缺陷都不在代码复查阶段的程序中，因此不包括在缺陷排除效益的计算中。

16.8 提高缺陷排除率

通过代码复查可以很快地提高缺陷排除率。然而，在最初的提高之后，进一步的提高将变的很困难。由于软件工程师面临的挑战每年都在增加，每个人都需要不断进步。下面是一些怎样持续改善个人的缺陷排除率的建议：

- 首先把精力集中到缺陷排除效益上。记住目标是排除所有的缺陷。因此，第一个目标是必须使缺陷排除效益稳定地达到 70% 或更高。
 - 在第一次编译之前做代码复查。如第 13 章指出的那样，尽可能高地达到缺陷排除效益，并采用编译器来检查代码复查的质量。
 - 一旦能够稳定地达到很高的缺陷排除效益，用第 13 和 14 章描述的方法来提高代码复查速度。跟踪检查表找出在哪里发现和漏掉缺陷，并定期进行调整。如果有些步骤不能发现缺陷或漏掉很多缺陷，应该考虑把它们删除。同时，当经常漏掉某些缺陷时，应该考虑在检查表中增加一个步骤来发现此类缺陷。
 - 记住“愚蠢”一词的含义就是重复做同一件事并期望有不同的结果 [Brown]。如果不对检查表作出调整，你将一直会漏掉同类型的缺陷。
- 继续收集缺陷数据和计算缺陷排除效益、缺陷引入率和缺陷排除率。然后跟踪这些数据，并试验各种方法，看看是什么因素帮助你提高的。

本书的一个目标就是让你意识到缺陷所带来的费用和后果,这样生产高质量产品就会变成个人的承诺。事实证明,这也是节省时间的方法。正如在表 16.3 中看到的,学生通过跟踪和管理缺陷排除效益和缺陷排除率,减少了花在编译和测试阶段的时间。在这个课程开始,这些学生在编写程序 1 和程序 2 时,花在编译和测试阶段的时间大约占开发时间的 30%。到课程结束时,通过不断使用 PSP,在编译和测试阶段大约平均仅花费开发时间的 10%。

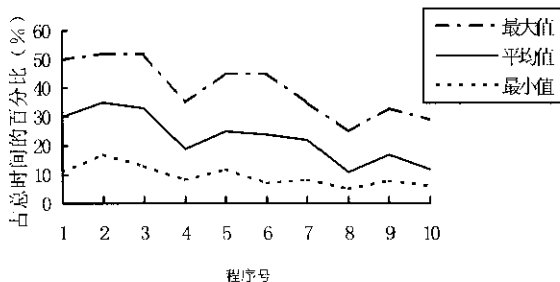


图 16.3 一个班级的编译和测试时间

16.9 减少缺陷引入率

因为在过程的每个阶段都可能引入缺陷,所以缺陷引入率很难降低。因此需要考虑软件开发中的每项任务。下面给出几种降低软件缺陷引入率的方法。

1. 记录所有的缺陷数据。只有知道了这些数据,才能更仔细地工作,也才能减少引入的缺陷数。

2. 产生更好的软件设计。通过产生具有更完整和文档化的设计,可以从两个方面提高程序质量。第一,这将有效地防止那些由于不清楚或不完整的设计所引起的缺陷。第二,更完整的设计可以节省编码时间。而且由于设计阶段的缺陷引入率的降低,也将减少编码阶段的缺陷。有关讨论见第 17 章。

3. 使用更好的方法。由于每个阶段都可能引入缺陷,因此对需求分析、规格说明、设计、测试用例和源程序代码等方面的改进都将有助于减少缺陷。在许多软件工程课程中,讲授了这些活动的有用方法。把这些方法用在 PSP 中并测量自己的工作结果,就能看到这些方法对改进工作非常有效。

4. 使用更好的工具。如果一个工具可以节省时间,它很可能也会减少引入缺陷的数目。现在每年都有新的软件工具面市,可以利用 PSP 数据来测量和评价它们。然后,就能知道哪个软件对自己有帮助,且帮助有多大。

16.10 总 结

缺陷是引起软件问题的主要原因。修复缺陷需要花钱,也会延迟项目进度。由于缺陷修复的时间难以预测,因此它会降低计划的准确性。因此,缺陷管理可以被看成是一个经济学问题。本章讲述缺陷的经济后果,并介绍几种测量方法来帮助管理自己开发的程序中的缺陷。

即使是有经验的工程师也会引入许多缺陷。在编译后的各个阶段查找缺陷的代价,随着每个阶段的进行要递增 10 倍。此外,典型的缺陷排除率数据说明,测试阶段的缺陷排除效率不高。在单元测试中,工程师一般每小时发现和 2 ~ 4 个缺陷。在代码复查中,每小时发现和修复 6 ~ 10 个缺陷。编译器能有效地发现程序中的语法缺陷,但它不能发现程序中所有的语法缺陷。在此后的集成测试、系统测试和产品使用中,发现和修复每个在个人过程中漏掉的缺陷平均需要花费几个小时。

为了改进缺陷查找和修复的有效性,引入了两种新的测量方法:测量每个阶段的缺陷引入和排除的效率,单位为每小时缺陷数(Defects/Hour)。在 PSP 中,过程效益测量在第一次编译前缺陷排除的百分比。为了知道在哪里改进和如何改进你的过程,需要计算缺陷引入率、缺陷排除率和过程效益。

16.11 练 习 16

对本学期你编写的每个程序,找出设计和编码阶段的累计缺陷引入率,并找出代码复查、编码、编译和测试阶段的缺陷排除率。另外,计算这些程序的实际的过程效益。以图表的方式显示这些数据,简要描述这些数据的含义,并把这些图表和描述交上来。

提交你以前没有提交过的时间记录日志和周活动总结表。另外,完成每个程序的更新过的项目计划总结表、缺陷记录日志和代码复查检查表,包括开发时间、缺陷引入、缺陷排除和缺陷排除效益的计划值、实际值和累计值。

参 考 文 献

[Brown] Brown, Rita Mae. Private communication.

[Humphrey 89] Humphrey, W.S. *Managing the Software Process*. Reading, MA: Addison-Wesley, 1989.

[Zachary] Zachary, G. Pascal. *Showstopper!* New York: The Free Press, 1994.

第 17 章 设计缺陷

本章讨论设计缺陷、设计过程和设计表示；并讨论减少引入设计缺陷数目的方法。管理设计缺陷的主要问题是了解设计缺陷是什么时候引入的以及怎样引入的。在课后作业中，要分析自己程序中的设计缺陷，确定它们是什么时候引入的和怎样引入的。

17.1 设计缺陷的本质

在测试阶段发现的大部分缺陷很多可能是在编码阶段引入的，这说明主要的缺陷问题是单纯的编码错误。事实上，从参加我所主讲的 PSP 课程的 14 位同学记录的数据发现，他们在编码阶段引入的缺陷是设计阶段引入的缺陷的一倍半。由于编译器已经排除了大部分语法错误，所以这个结果似乎有点让人吃惊。可是如表 17.1 所示，当学生学习了 PSP 后，这个比率有了变化。在测试阶段发现的缺陷中，在课程开始时，学生在编码阶段引入的缺陷数是设计阶段的一倍半。到课程结束时，他们在编码阶段引入的缺陷比设计阶段平均大约只多 14%。他们在设计和编码两个阶段的缺陷数都减少了，但设计阶段的改进要小一点。注意，表 17.1 中的数值对应的是缺陷引入的阶段。然而，如果看一下缺陷类型，我们将得到不同的信息。在 PSP 中，缺陷类型是按照问题的复杂度来排列的，从最简单的类型 10 和类型 20 到最复杂的类型 90 和类型 100。因此我们可以说，类型 10 ~ 40 是简单缺陷或编码类型的缺陷，类型 50 ~ 100 是复杂缺陷或设计类型的缺陷。

表 17.1 在设计和编码阶段引入在测试阶段发现的缺陷

	设计阶段缺陷引入率 (Defects/KLOC)	编码阶段缺陷引入率 (Defects/KLOC)	编码缺陷与设计缺陷之比
练习 1	8.66	12.99	1.50
练习 10	5.05	5.77	1.14
递减率(%)	41.67%	55.56%	

如果按照类型对上述缺陷进行分类,可以得到表 17.2。从表中可以看出,测试阶段发现的缺陷大部分是设计类型的缺陷,但它们多数是在编码阶段引入的。尽管这些工程师显著地减少了引入的缺陷数,但在编码阶段他们还是引入了许多设计类型的错误。与在第 16 章看到的那样,同一组工程师在编码阶段每小时引入 5~8 个缺陷,而在设计阶段每小时只引入 1~3 个缺陷。这就说明一个潜在的有效的减少设计缺陷的方法是:不要等到编码阶段才去完成设计工作。本章将讨论这一问题以及如何解决这一问题。

表 17.2 测试阶段发现的缺陷按引入阶段分类表

	设计类型 Defects/KLOC	编码类型 Defects/KLOC	设计类型缺陷 的百分比
练习 1			
设计阶段	7.22	1.44	83.33%
编码阶段	9.38	3.61	72.22%
总计	16.59	5.05	76.67%
练习 10			
设计阶段	3.61	1.44	71.43%
编码阶段	3.61	2.16	62.50%
总计	7.22	3.61	66.67%
递减率			
设计阶段	50.00%	0%	
编码阶段	61.54%	40.00%	
总计	56.52%	28.57%	

17.2 识别设计缺陷

某些人认为编译器发现的所有缺陷都是编码类型的缺陷,这就意味着没有被编译器发现的缺陷都是设计缺陷。很遗憾,事实并不是这样。即使是最好的编译器也不能发现所有的语法缺陷,有些设计缺陷也同样会引起语法错误。

为什么编译器不能发现所有的语法缺陷呢?因为有些带有语法错误的代码符合编译器的有效语法。就是说,一些随机的录入错误偶尔会产生符合语法的语句。这样的缺陷通常占有语法缺陷的 10%。例如,当想输入 $x==y$ 却键入了 $x=y$, C++ 编译器将无法发现这一错误。这样,原本希望在逻辑上判断 x 是否等于 y ,实际上却把 y 的值,置入 x 。同样的问题在 Ada 和 Pascal 中也会遇到,当想输入 $x=y$ 却键入了 $x:=y$ 。这些错误一般认为不是设计错误,但却

导致程序执行结果与程序逻辑不符。

因此，没有简单、客观的方法来定义什么是设计缺陷。在这里我们有两个选择：

- 将设计阶段引入的所有缺陷定义为设计缺陷。
- 将那些与程序功能、逻辑、性能和时序有关的缺陷定义为设计缺陷。

在本书中，我们使用第二个定义。

17.3 什么是设计

在考虑设计缺陷时，重要的是给设计下个定义。但是不容易给出这个定义，因为任何与程序结构和实现有关的内容都涉及到设计：这些内容包括程序流程、语言构造的结构和组成以及主源代码的标点符号等。这些都属于设计范畴，但它们的详细程度不同。

问题在于，设计本身是一个从哪个视角观察的问题。可以很详细地进行设计，也可以在很高的层次上进行设计。在开发程序时，几乎每一步都在做某种设计。在实现一个选择语句或循环结构的细节时，进行的是详细设计。当定义一个数据结构或建立一个模块接口时，进行的是更高层次的设计。

因此，设计和编码的界线不是截然可分的。软件工程师一般喜欢按照工作内容的详细程度来划分各种活动，这有助于他们按照正确的顺序把精力集中在应该处理的事情上。例如，如果工程师在实现之前能够全面考虑设计，就会减少犯错误的机会，工作也会更加有效。这样做并不意味着他们采用自顶向下的方法来完成所有的工作，而是说明一般应该先从高层概念开始，然后再深入钻研细节。

我们用“抽象”一词来描述高层的概念方法。因此当说到像求平方根这样的功能抽象时，是在说一个计算平方根的程序过程。在这个层次上，并不考虑计算平方根的各种细节。遵循这种抽象策略，工程师可以先完成全部的高层设计，再深入到每个抽象或功能的实现细节。

例如，通过定义四个功能抽象或过程（输入、文件、计算和输出），就可以开始设计一个简单的程序，然后使用这些抽象就能设计出程序的主例程。另外还必须定义程序主要的变量和文件，这样就能用功能抽象方法来完成程序的高层设计。当然，下一步就是设计每一个抽象功能。对于大型程序，可能要进行几个层次的抽象，甚至可以把以前开发的功能抽象建成一个库，为以后进行程序设计时使用。

17.4 设计过程

在进行设计时，有经验的工程师经常在各个设计层次之间动态地进行切换[Curtis]。原因是他们要处理很多抽象功能。首先要理解这个抽象是如何工作的，然后才能在高层设计中放心地使用。如果他们以前用过这些功能，就可以先不考虑细节。如果以前从未用过这样的功能抽象，通常要停下来先进行详细设计。甚至会编写一个原型并且进行测试，然后才有足够

的信心继续进行高层设计。有经验的设计人员都会这样做，因为他们知道，看起来简单的抽象可能包含许多难以捉摸的复杂问题。因此，当他们使用一个不知道如何建立起来的抽象时，常常会感到这个作业比预想的要困难得多。实际上，有时当系统设计建立在一个看似简单但却包含有定义笨拙的抽象上时，最终会发现整个设计方案是不可行的。

当处理已知或已定义的功能时，抽象的概念非常有用。但是，当用这种概念方法来推迟困难的设计工作，就容易出现困难。因此，有经验的工程师不愿在高层设计中使用任何抽象，除非他们以前用过类似的功能或者知道用这样的功能来实现是可行的。

因此，很难对设计和编码之间的区别给出一个明确解释；也就是说，很难定义什么是完整的设计规格说明。因此在 PSP 中，重要的是要区分设计过程和特定的设计阶段。设计阶段是指生产设计制品的阶段，设计过程是描述为生产这个设计产品或制品而进行的活动。从某种意义上讲，我们把设计定义为在设计阶段所做的工作。我们同样也可以把“思想”定义为人们按照自己的想法去做的工作。然而，根据人们生产的产品来定义活动经常是有用的。本章后面的章节将讨论怎样表达设计制品。

17.5 设计缺陷的起因

设计缺陷是由几种原因引起的。首先是设计错误。在对问题深思熟虑之后却作出了错误的设计决策。这可能是因为误解了某个数学函数，或设计了一个在错误的条件下执行的循环，或忽略了某一个系统条件；也可能因为你缺乏信息、劳累或者没有接受合适的训练。不论什么原因，都是在神智清楚的情况下，作出了一个错误的设计决策。

第二个原因是，知道应该做什么样的设计，但犯了一个简单的错误。例如，你知道在一定的条件下循环应该结束，但却忘记把所有情况都包括进去。这可能是由于疏忽或者是由于思考简单。知道要做什么，也知道怎么做，但自己犯了愚蠢的错误。在仓促或过度疲劳的情况下，经常会出现这种错误。

第三个引起设计缺陷的原因是误解了要做的内容。要么是误解了高层设计，要么是不理解需求。哪一种情况都会导致设计错误。实际上，也许所做的设计正确地实现了你想要的功能，但所建造的功能并不是系统所需要的。

另一种情况与第三种情况非常类似。不但理解局部设计和系统设计，甚至还理解需求，只是不明白系统的语境。我称这种情况为“文字解释”问题。例如，当你问某人“spilled the beans”的意义时，你得到的答案可能会因人而异。懂得美国俚语的人解释为“不慎泄密”，而其他国家学英语的人则解释为“打翻的食物”。

在软件中也存在同样的问题，因为设计决策经常涉及到许多影响系统用户的细节。如果设计者不熟悉用户的使用语境，他们可能会按照字面的意思去解释需求而不能理解其运作的含义。尽管这样的系统一般也能满足所有陈述的需求，但往往使用起来很不方便，有时甚至无法使用。

常常令人吃惊的是，设计错误经常来自过分自信。最常见的例子是在一个大程序中进行

一行或两行修复的问题。因为这种改变很小，工程师往往认为很简单，而没有花时间去真正理解问题或修复的含义。这些错误可以是上述分类中的任何一种，一般归入第三类或第四类，即由于错误的理解或不熟悉语境所造成。

17.6 设计缺陷的影响

在 IBM，一个软件开发经理曾经向我汇报一个典型的设计问题。她偶然听到两个工程师辩论应该如何实现一个特定的程序功能。他们对客户希望什么功能知道的都不精确，他们的见解也不一样。但他们解决问题的方式不是找出客户准确的需求，而是采用某种第三种的折中解释。当工程师作出这种信息不足的设计决策时，经常会产生不方便的设计，甚至可能是不可操作的设计。

所有这些原因都存在，而且都会导致总的设计缺陷数目的增加。到目前为止，大部分缺陷是由单纯的疏忽大意、愚蠢和理解错误引起的。这些缺陷比其它缺陷多的原因是：软件工程师只有在面对挑战性的设计问题时才会格外小心。他们知道此时容易犯错误，因此仔细地进行检查。由于软件工程师犯了许多简单错误，而这些错误又经常很难发现，所以这些简单缺陷是引起大部分问题的原因。许多这样的缺陷可能通过整个开发和测试过程，最后影响到系统用户。这些细微的缺陷导致许多用户在使用软件系统时出现问题。

事实上，许多这类缺陷也是可以避免的。工程师知道应该做什么，设计也是合适的，只是表达方式不好。因此，当进入编码阶段时，实现人员看不懂想要完成什么。这时，许多程序员通常不是停下来去找设计人员，而是飞快地实现设计。由于他们在实现层工作，经常不能完全理解所有的设计含义，因此他们更容易出现错误。虽然设计人员知道要做什么，但实现人员并不知道。

当实现自己作的设计时，也一样会出现误解。在作设计时，一般做到这样一种程度，即此时设计的其它部分好像是不言而喻的。如果也要计划进行编码，似乎没有必要对这部分设计编写文档。遗憾的是，到了以后实现时，你可能记不起曾经认为不言而喻的设计，因而不得不在实现期间重新进行设计。因为这时已经忘记了设计的语境，必须重建有关的设计概念和条件。由于在这些重建过程中容易出错，就可能犯设计错误。错误的起因不是由于拙劣的实现，而是拙劣的设计表达。

17.7 设计表达

完整清晰的设计表达有助于减少引入缺陷的数目。这就意味着当构思出设计方案后，还需要把它表达出来，这样不管哪个实现者都能清楚地理解你的设计。对 10,000 行以上的程序来说，设计表达至关重要，即使是对 50 ~ 100 行代码的程序模块，也要注意设计表达问题。

通过完整地表达你所创建的设计，可以节省开发时间。因为在一般情况下，根据清晰完整的设计来编码比根据模糊不清的设计来编码要快很多。编码时间少意味着引入的编码缺陷也少，一个好的设计更能生产出较高质量的产品。

有三种常用的表达设计的方法：图形、伪码或数学方法。下面讨论这些表达方法。这是一个很大的主题，包括许多内容，这里只对设计模块一级的程序给出一些概念。同时也为将来使用 PSP 考虑设计时提供语境。这里的目标不是讲授设计表达方法，而是让你确信进行设计和清晰地表达设计的重要性。这样，在其它课程学习或工作中面临各种表达方法时，你会知道它们为什么很重要，并考虑如何使用它们。

图形设计表达方法

图形一般比公式和文字更容易让人理解。在描述一个复杂的设计时，一张图可以帮助人们更好地理解其设计。因为设计的大部分问题涉及理解，所以通常应该使用图形表达来帮助人们理解设计。

最常用的图形表达方式是流程图。在流程图中，用方块表示程序功能，用方块之间的连线表示逻辑的程序流。尽管画这种流程图有许多方法，我们只描述基本的流程图符号，如图 17.1 所示。

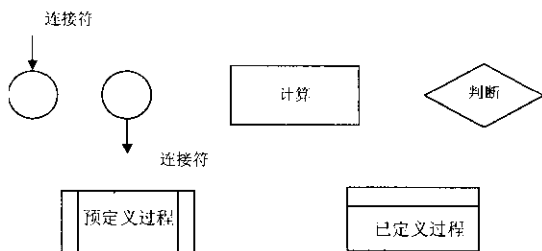


图 17.1 流程图的基本符号

使用这些符号描述流程图的一个简单示例见图 17.2。在流程图中，上边的连接符说明 8Ax 是 x 的输入，下边的连接符显示输出 y 到 3Ay 和 5By。连接符指向设计文档的其它页。这种方法表示：变量 x 来自设计的第 8 页的 Ax 点，同理变量 y 去往设计的第 3 页的 Ay 点和第 5 页的 By 点。

图 17.2 中间的框表示一个判断功能，可以用 if-then-else 结构或 case 结构来实现。使用哪种结构由实现者来选择。计算框 $y=f(x)$ 描述一个计算。

图 17.2 中，左边和右边的两个框表示已经定义的函数。左边的这个 Error-Routine 有两条竖线，表示这是一个预定义功能。预定义功能是指已经实现、在这里被重用的功能。右边的这个 Null-Correction 框有一条横线，表示一个已定义的功能。这是一个新开发的功能抽象，对它进行实现后在程序中使用。

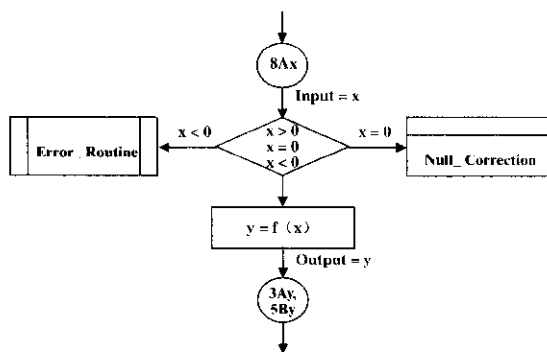


图 17.2 一个逻辑图的示例

图形表达方法易于理解，但通常不够精确或篇幅过长。这不是方法本身固有的问题，而只是一个不精确和不完整表达的示例。使用完整的书面设计材料，并使用图形来辅助说明程序的逻辑，可以解决精确表达设计的问题。然而要记住，放入流程图的信息越多，流程图就越乱，也就越难理解。解决这一问题的最好方法是使用多层流程图，每一层流程图都包括对更低一层流程图的抽象。

因为图形表示有很多优点，所以应该利用图形表示来说明你所生产的每一个程序的设计。实际上，通过画流程图，经常可以发现一些没有考虑到的关系。无论怎样画出流程图，它确实让你在以后的实现、测试、修复或扩充程序时节省时间和减少混淆。

伪码设计表达方法

伪码是另外一种表达程序逻辑的方法。这种方法使用一种与编程语言类似的语言，采用简洁的方法来表示复杂的表达式，以便易于理解。这种方法的目的是表达程序的逻辑，但忽略程序设计语言的许多语法细节。一个伪码设计描述的示例如下：

```
Algorithm (Function f(x) calculation)
```

```

If (x<0)
  Then ( Error_Routine )
elseif ( x=0 )
  then(Null Correction )
else
  y=f(x)
end
  
```

你可以增加语句来描述函数 $f(x)$ 是怎样计算的，以对本示例进行扩展。在实现时，根据

需要也可以增加类型定义、说明或其它细节。

伪码设计描述是人类语言与程序结构的混合。对人们熟知的功能不用复杂的逻辑而用简单的语句来表示。同样，用传统的条件语句表示程序流程，用写好的表达式来描述逻辑。目前尚无通用的伪码符号标准，但采用与高级程序设计语言相类似的语言结构是一个好主意。这样做的好处是，在实现时对这种伪码看起来比较熟悉，并且给出了一个实现框架。

伪码不涉及程序设计语言所要求的细节，因此通常比生产一个完整的源程序要容易、要快，而且可以根据自己的需要编写任意详细程度的设计文档。这就使得在作第一次设计时就能比较容易把握所研究的问题。也就是说，通过伪代码，可以在对设计理解最透彻时用最有效的方法把它记录下来。

伪码的主要优点是可以根据要求把伪码精确到所需要的任意详细的程度，而且伪码还易于理解。伪码的主要缺点是一个伪码设计可能太详细以至于难以理解。这就是为什么对于任何设计（最简单的设计除外），最好的表达方式是既有流程图又有伪码。

使用伪码的一个潜在问题是，在用伪码编写的程序中，忽略了许多用来完整定义源程序中各种含义的标点符号和结构。因此，一些常见的逻辑结构很容易出错。一个好的主意是用缩进或标点符号把 else 语句与其对应的 if 语句连接在一起，或显示循环或选择条件的范围。变量和参数的范围也容易引起混淆，在那些不明显的地方应该小心。

使用伪码的一个常见问题是关于设计的详细程度。许多工程师写一行伪码大约相当于最终源程序的 3~5 行。可是我也看到有些工程师的每一行伪码仅对应一行源代码。虽然这种伪码表达也称为设计，但实在是一种格式不严格的实现。由于简单的拷贝也会引入缺陷，因此设计到如此详细的程度，可能比直接用某一种程序设计语言编码时引入的缺陷要多。

由于伪码的灵活性，设计可以详细到任何程度。无法精确说明用伪码设计必须详细到什么程度。虽然伪码表达非常有用，但我们仍然留下了一开始就提出的问题：如何精确地定义设计所需要的详细程度。实际上，伪码的主要优点是允许工程师根据情况决定设计的详细程度。

当使用伪码时，PSP 数据有助于你决定合适的详细程度。例如，当复查缺陷数据时，如果发现在编码阶段引入了设计类型缺陷，应考虑使用更精确的设计。相反，如果没有设计表达问题，可以尝试使用粗一些的伪码设计，看看是否加快设计工作并且在实现时有没有引起设计缺陷。

其它表达方法

各种数学方法都可以用来精确定义软件系统。这些方法的优点是表达精确，缺点是不易学习，至少对那些没有经过严格数学训练的工程师是这样。然而形式化方法能很好地帮助工程师生产出缺陷较少的程序。

设计表示是一个重要的问题，在将来无疑会有许多想法和方法出现。当你从各种方法中选择使用哪种方法时，要考虑以下几点：

- 设计是一个思考过程。
- 丰富的设计符号有助于精确地思考和表达一个复杂的设计。
- 但是，丰富的设计符号难以学习。
- 当使用不熟悉的设计符号时，可能不习惯用这些符号进行思考。

- 这时，必须用熟悉的符号通盘考虑设计，然后再将其转换成不熟悉的符号。
- 这个转换过程会影响创造力、延缓设计工作和引起错误。

当应该试用新的方法和符号时，要对这些方法和符号的使用尽量熟练，然后再给出评价。还要记住，设计符号是一种交流媒体。如果实现人员对所使用的方法和符号不很习惯，就可能出现上面提到的许多问题。

17.8 总 结

许多设计缺陷表现在编码阶段。一种减少设计缺陷的方法是分析缺陷类型。在 PSP 中，把设计缺陷定义为从类型 50 ~ 100 的缺陷。如果在编码阶段引入了许多这种类型的缺陷，应该考虑在设计上花更多的时间。最有效的方法是在设计阶段而不是编码阶段产生逻辑设计和功能设计。

精确的设计表达可以节省实现时间，减少设计缺陷；拙劣的设计表达会引起缺陷。图形表达易于理解但可能不精确。伪码表达可以详细到任意程度，可以使用与采用的程序设计语言相类似的语言。在选择设计表达方法时，要对表达方法熟悉后再给出评价。使用自己的 PSP 数据进行评价并作出决定。

17.9 练 习 17

本作业要根据你记录的缺陷数据复查所有发现的程序缺陷。使用表 17.1 和表 17.2 给出的方法，说明哪些缺陷是不精确设计表达的结果，并给出防止缺陷的步骤。在课后作业中，根据问题类型列出缺陷数目，给出可以防止缺陷的设计表示，并给出这些改进总共可以节省多少改正时间。

为下一个程序设计伪码和流程图。把这些设计转化成程序，并简单写一段关于各种设计方法对引入设计缺陷数目的影响，给出得到这些结论的数据。注意缺陷分析和设计作业是选做的，除非你的指导老师有要求，否则可以不做。

提交以前没有交过的时间记录日志和周活动总结表。还要完成每个程序的 PSP 项目计划总结表、缺陷记录日志和代码复查检查表，这些表中要包括开发时间、缺陷引入、缺陷排除和缺陷排除效益的计划值、实际值和累计值。

参 考 书 目

[Curtis] Curtis, Bill, Herb Krasner, and Neil Iscoe. "A Field Study of the Software Design Process for Large Systems." *Communications of the ACM*, November 1988, vol. 31, no. 11.

[Gries] Gries, David. *The Science of Programming*. New York: Springer-Verlag, 1981.

第 18 章 产 品 质 量

本章讨论个人工作规范如何对产品质量产生影响，还将说明在编译和测试阶段发现的缺陷数目和残留在最终产品中的缺陷数目之间的关系。然后描述采取哪些步骤可以提高程序的质量。在本章的作业中，要对本学期所开发的程序中编译缺陷数目和测试缺陷数目的关系进行考察。

18.1 质 量 第 一

正如我们所看到的那样，即使对小程序或中等规模的程序，测试的代价也很大。我们也看到，如果能尽早发现缺陷，就能节省时间。在编译和测试之前对代码进行复查，可以减少测试阶段所发现的缺陷数，从而可以节省测试时间。我们还看到，经过代码复查后，编译和测试所节省的时间与代码复查所花时间大致相同。代码复查的另外一个优点是，缺陷发现得越早，生产出的产品质量越高。本章将解释这种情况的原因。

许多软件工程师认为在编译和测试前不必考虑缺陷问题，因此他们匆匆走过设计和编码阶段的许多步骤，以便开始测试。但在使用 PSP 时，工程师从一开始就把注意力集中于生产出清晰的、无缺陷的程序。采用这种策略的原因是：一旦工程师生产出一个草率的产品，没有任何工具能把它整理好。为了证明这一点，一种方法是了解编译阶段、测试阶段和用户使用阶段发现的缺陷数据之间的关系；另一种方法是收集关于自己工作情况的数据，并且看看这种策略产生的效果。本章将介绍这两种方法。

18.2 测 试

产品越复杂，测试花费的时间就越长，费用就越大，测试发现缺陷的效率也就越低。这就是说，对于大型复杂的程序，发现和修复每个缺陷的花费很大，能发现的缺陷的百分比在下降。

出现这种情况的原因有很多。第一，缺陷会掩盖或加重其它缺陷。也就是说，当一个程

序有许多缺陷时，由于缺陷相互作用，使得发现和修复缺陷的过程更加复杂。这使得一些缺陷很难查找和修复。一个缺陷可能掩盖其它缺陷的征兆，使得这些被掩盖的缺陷难以发现，增加了它们逃过检查的可能性。

另一个原因是，即使是对中等规模的程序，由于程序的逻辑路径数目巨大，因而很难测试；即使是小程序，要对所有可能的路径组合进行测试一般也是不实际的。当程序变得越来越大、越来越复杂时，要对程序进行适度的全面测试也变得越来越困难。例如，对于分析字符串这样的程序，如果只有 60 个可能的字符，那么一个由 10 个字符组成的字符串将会有 60^{10} 种可能组合。这个数是 6 后面跟 17 个零那么大。如果每秒能测试 100 万种组合，要测试所有的可能组合需要 2 万年。因此，即使是简单的程序，也不可能测试所有可能的组合。

在这个示例中，许多字符组合在逻辑上是一样的，但系统测试人员不知道应该跳过哪一个、测试哪一个。当然，设计人员可以说明哪些组合可以忽略。这样的策略带来的问题是设计人员可能会犯错误。如果他们不会出错，程序也就不必测试了。程序之所以需要测试，很大程度上是因为它们的表现和我们的期望不一样。对于复杂的系统，如果只测试认为重要的条件，一般会漏过许多缺陷。

遵照规范化的方法，仔细复查和测试每个小程序模块，这比让任何测试小组在你的程序中发现缺陷的效果要好。因为你知道程序的逻辑，知道程序要做什么，也知道有哪些异常情况和偶然条件，并且能迅速看出某种情况是否能正常工作。如果你最终能肯定程序模块中已经没有缺陷，将大大减少以后清除程序缺陷的时间和花费。尽管不能避免一些偶然的错误，但至少可以保证自己的产品基本上是无缺陷的。进一步说，你是唯一能做到这一点的人。如果你自己不能生产出无缺陷程序，其他任何人都无法帮你做到这一点。

18.3 过滤器概念

缺陷排除就像一个过滤器，每一次复查、编译和测试都排除产品中的一部分缺陷。因此，放入过滤器上的缺陷越多，能发现的缺陷就越多。例如，如果被测程序有 25 个缺陷，你将会发现一部分缺陷，假如 12 个。同样，在另一个程序中有 50 个缺陷，假设会发现同样比例的缺陷，则大约是 24 个。存在的缺陷越多，则发现更多缺陷的可能性也就越大。

但是相反，每个缺陷排除过程都会漏过一部分缺陷。因此，进入过滤器的缺陷越多，漏过的缺陷也就越多。漏过过滤器的缺陷数与进入过滤器的缺陷数是成比例的。这种比例的大小随着缺陷排除过程的质量和类型的不同而有很大的差异，但这种比例关系是一直存在的。因此在进入测试、编译或代码复查阶段的入口处的缺陷越多，在结束时遗留在产品中的缺陷也就越多。

当然，下一个问题是过滤器的质量。如果能设计出缺陷排除效益接近 100% 的过滤器，则缺陷排除问题也就解决了。遗憾的是，在编译和测试阶段的缺陷排除效益却无法让人放心。如表 18.1 所示，代码复查和代码互查的缺陷排除效益最高，而编译、单元测试和其它形式的测试效果则较差[Humphrey 89]。这些数字是基于有限数据的统计结果，不一定适合你的特殊

情况，但这是我们所拥有的全部数据。当然，最好的答案是根据自己的缺陷排除方法收集缺陷排除效益数据并得出自己的结论。在表 18.1 中，结果有趣地表明，缺陷排除效益最高的方法是工程师自己的代码复查。第二个有效的方法是几个工程师相互检查别人的设计和编码。两种最有效的方法都是手工方法，不包括任何形式的自动工具。这两种方法比其它方法好的原因是，人脑是最强大的缺陷检测仪器，它比任何已知的软件工具都好。

表 18.1 缺陷排除效益

方 法	缺陷排除效益的近似值 (%)
代码复查	70~80
代码互查	50~70
编译	50
单元测试	40~50
集成测试	45
需求测试	45
算法测试	8

从这些数据可以得出结论，为了生产高质量的产品，在开始测试时产品必须几乎没有有什么缺陷。进入测试阶段几乎没有缺陷就需要在编译阶段结束时也几乎没有缺陷。而在编译结束时几乎没有缺陷就必须在编译开始前排除缺陷。当然，为了生产出高质量的产品，应该对缺陷排除的各个阶段都进行测量、分析和改进。

18.4 仔细工作的好处

把测试看作过滤器的另一种说法是：“一旦成了次品，就永远是一个次品”。在本质上，当工程师生产出很差的软件产品时，进入编译和测试阶段的缺陷会很多，遗留在各阶段的缺陷也会很多。但是，当工程师仔细地工作时，其好处在整个过程中都会表现出来。这就意味着，例如，当编译阶段有许多缺陷时，在测试阶段也可能会有许多缺陷。测试阶段的缺陷多，遗留在最终程序中的缺陷也就多。

图 18.1、图 18.2 和图 18.3 是三个学生编译阶段和测试阶段所发现的缺陷数，这些数据说明，在编译阶段发现的缺陷数和在测试阶段发现的缺陷数有一定的关系。这种关系是统计学上的弱关系，通常编译时缺陷数高，则指明在测试阶段的缺陷数也高。

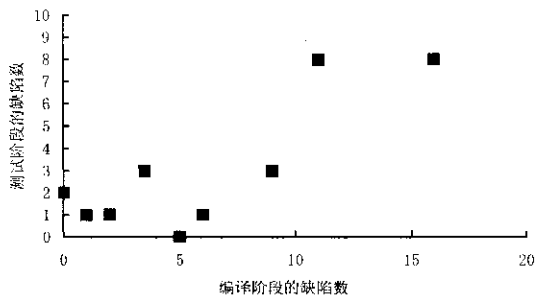


图 18.1 学生 1 的编译和测试阶段的缺陷

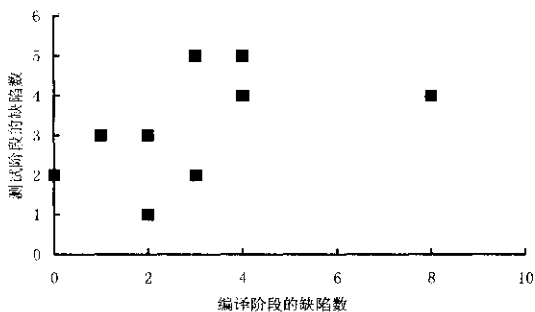


图 18.2 学生 14 的编译和测试阶段的缺陷

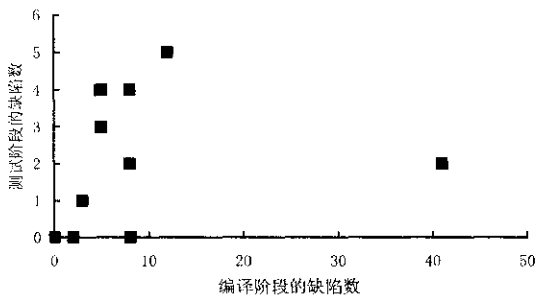


图 18.3 学生 20 的编译和测试阶段的缺陷

18.5 缺陷排除效益的计算

在第 16 章介绍的过程效益的测量方法,关心的是在第一次编译前缺陷排除的百分比。实际上,过程效益可以应用于缺陷排除的各个阶段。因此,各个阶段的缺陷排除效益的计算方法如下:

阶段缺陷排除效益 = $100 \times (\text{本阶段排除的缺陷数}) / (\text{进入本阶段时产品中的缺陷数})$

要准确计算缺陷排除效益需要拥有此后各个阶段的缺陷数据。例如,如果在代码复查中发现 5 个缺陷、编译阶段发现 3 个、测试阶段又发现 2 个,则代码复查发现总共 10 个缺陷中的 5 个。这时,代码复查的缺陷排除效益就是 50%。尽管程序中可能还有许多缺陷,但这个数据就是此时的代码复查缺陷排除效益。

如图 18.4 所示,在代码复查结束时发现了 5 个缺陷。由于你只发现了这些,所以认为程序中只有这些缺陷。因此,此时你认为代码复查缺陷排除效益是 100%。如果此后又发现了新的缺陷,那么漏过这些缺陷的所有阶段的缺陷排除效益的值都要下降。

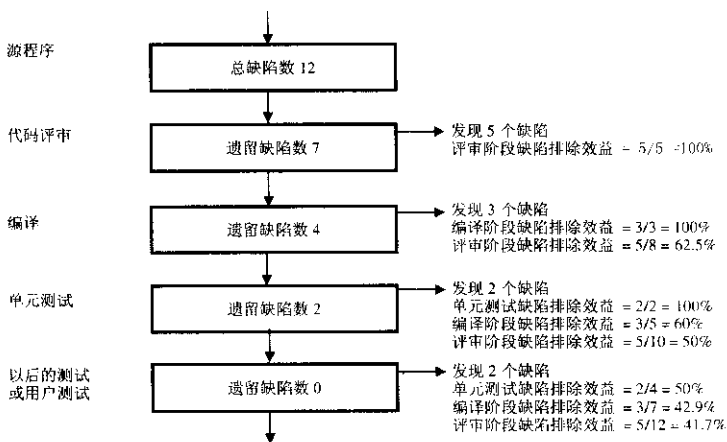


图 18.4 缺陷排除效益的计算

例如,如果在编译阶段又发现了 3 个缺陷,则代码复查中只发现了 8 个缺陷中的 5 个。因此,代码复查缺陷排除效益是 62.5%而不是 100%。显然,此时编译阶段的缺陷排除效益是 100%。

如果在单元测试中再次发现 2 个缺陷,显然代码复查只发现 10 个缺陷中的 5 个。因此代码复查缺陷排除效益只有 50%。此时,编译器发现 3 个缺陷,漏过 2 个缺陷,因此缺陷排除效益是 60%。由于假设单元测试发现了剩下的所有缺陷,它的缺陷排除效益显然是 100%。

假设此后的程序测试中又发现了 2 个缺陷。那么代码复查就只发现产品的 12 个缺陷中的 5 个。那时的代码复查缺陷排除效益是 41.7%。同样地, 编译器发现 7 个缺陷中的 3 个缺陷, 此时缺陷排除效益是 42.9%。但是, 到现在你还不能肯定最终的缺陷排除效益的数值。一旦产品交付最终用户, 还会发现许多缺陷, 因此还应进一步减少各个阶段的缺陷排除效益。

缺陷排除效益的测量方法可以用在过程的每个阶段。例如用在单元测试中, 它指的是单元测试中排除的缺陷数占进入单元测试时缺陷总数的百分比。如第 16 章所描述的那样, 过程效益指的是第一次编译前排除缺陷的百分比。因此, 过程效益的计算如下:

$$\text{过程效益} = 100 \times (\text{编译前排除的缺陷数}) / (\text{编译前引入的缺陷数})$$

18.6 最终的缺陷排除效益的估计

当完成一个阶段时, 还不能完全肯定此时的缺陷排除效益, 但缺陷排除效益测量方法有助于评价和改进过程。例如在代码复查时发现 17 个缺陷, 编译时发现 2 个, 测试时发现 1 个, 则就有理由相信代码复查的缺陷排除效益是 85%。可能以后还会发现个别缺陷, 但你已经找到大部分缺陷。可是, 如果在代码复查阶段发现 17 个缺陷, 编译时发现 15 个, 测试时又发现 8 个, 那么这时的代码复查缺陷排除效益是 42.5%; 将来还很可能要下降。因为在编译和测试阶段发现了 23 个缺陷, 因此可以肯定还会有若干个缺陷残留在产品中。

要想确定有多少缺陷残留在最终产品中, 唯一的方法是跟踪产品整个使用生命期间发现的缺陷数。但即使经过若干年的使用也不能发现所有的缺陷, 所以永远无法知道确切数值。可是如果在缺陷排除的各个阶段发现的缺陷数迅速下降, 则所求得的缺陷排除效益值可能相当准确。在收集缺陷数据之后, 就可以求出每个阶段的缺陷排除效益的测量值, 然后就能估算可能残留在产品中的缺陷数目。

一个有用的拇指规则是, 假设残留的缺陷数和最后一个阶段排除的缺陷数相等。这等于假设最后阶段的缺陷排除效益是 50%。根据我的经验数据, 这个值对认真进行了同行互查和代码复查的有点低, 对编译阶段大约正好, 对大多数测试阶段有点偏高。

再看一下前面的例子, 代码复查发现 17 个缺陷, 编译时发现 2 个, 测试时发现 1 个, 此时代码复查的缺陷排除效益是 $17 / (17 + 2 + 1) = 85\%$ 。按照拇指规则, 还应该发现一个缺陷。因此最终的缺陷排除效益是: $17 / (17 + 2 + 1 + 1) = 80.95\%$ 。在另外一个例子中, 代码复查时发现 17 个缺陷, 编译时发现 15 个, 测试时发现 8 个, 此时代码复查的缺陷排除效益是 $17 / (17 + 15 + 8) = 42.5\%$ 。根据拇指规则还会发现 8 个缺陷, 因此, 估计的最终代码复查的缺陷排除效益为 $17 / (17 + 15 + 8 + 8) = 35.4\%$ 。

根据每个阶段实际的缺陷排除效益的历史数据, 可以对最终缺陷排除效益作出更准确的估计。可以根据最后一个阶段的缺陷排除效益值计算出可能漏过的缺陷值, 来进行上述这种准确的估计。有了足够的缺陷排除效益数据, 甚至可以计算出可能漏掉的缺陷数的统计范围。尽管无法知道实际的缺陷排除效益的数值, 但这些数据有助于进行更准确的估计。

18.7 100%过程效益的好处

代码复查的目标是使过程效益稳定地达到 100%。如果能够做到这一点,在代码复查阶段将发现所有的缺陷,而不会将缺陷留到编译和测试阶段。这不仅节省编译和测试时间,而且对项目的费用和进度都有很大好处。对于无差错的程序模块,程序开发组无须花费很多时间来测试程序。这将节省测试费用,减少测试时间,加快开发进度,而且可以生产出更好的产品。

要使过程效益达到 100%并不容易,需要花费时间,需要总结出好的实践,并且需要收集和分析大量的数据。然而高效益的软件开发是一种可以学习和提高的技能。可以把它比做演奏乐器,当开始学习演奏时,可能会弹错很多音符。通过实践,逐渐增加弹奏正确的音符,减少弹错的音符。经过大量的实践后,最终将学会不出现任何错误的演奏。这时乐手开始成为真正的艺术家。

生产高质量软件的作业也与演奏乐器非常相似,直到能生产出含有极少缺陷或者无缺陷的代码,你才是一个有造诣的软件工程师。只要还把大部分时间花在查找和修复缺陷上,你就不够一个专业人员的水平。然而,使用规范化的质量控制方法,就能相当稳定地生产出无缺陷的小程序,然后就可以把精力集中到生产更具有挑战性的高质量的大程序上。尽管掌握这些方法需要一定的时间和持续不断的努力,但你会看到你在逐步地进步。然而你的个人目标必须是稳定地达到 100%的过程效益。记住,除非能够始终保持生产出高质量的小程序,否则你将永远不会生产出高质量的大程序。做到这一点需要个人的训练和大量的实践。

18.8 缺陷排除效益的经验

尽管要达到较高的缺陷排除效益是很困难的,但这个目标是可以达到的。图 18.5~图 18.7

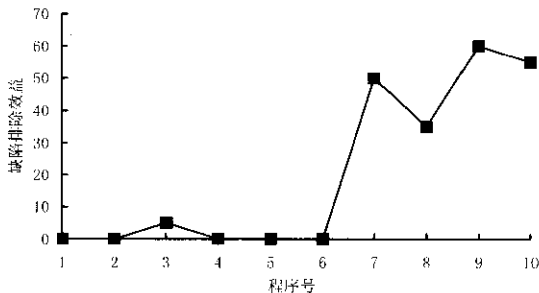


图 18.5 学生 1 的过程效益数据

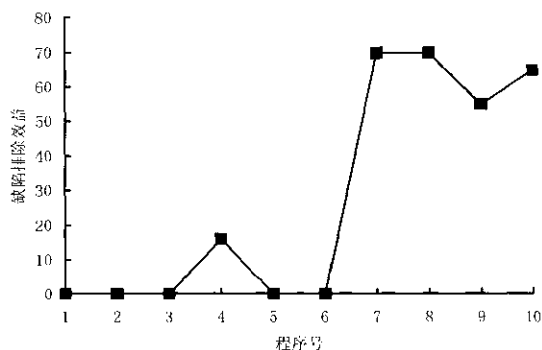


图 18.6 学生 14 的过程效益数据

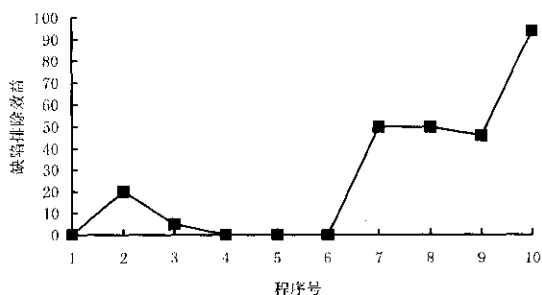


图 18.7 学生 20 的过程效益数据

给出了三个工程师的过程效益数据。这些工程师都学了一个学期的 PSP 课程，在课程学习中共写了 10 个小程序。他们从程序 1 开始，在以后的每个程序中逐步加入测量和方法。到程序 7 时，要求进行代码复查。如图中所示，所有工程师在这里都显著提高了缺陷排除效益。

可是要注意，这里没有标准的改进模式。有些工程师将稳定地提高缺陷排除效益，而另一些工程师的提高过程有相当的波动。有些学生可以自然地达到 80% 或更高的缺陷排除效益，而其他一些学生则要经过相当的努力才能达到 50% 以上的缺陷排除效益。我在 PSP 开发中，经过学习期望稳定地达到 80% 的缺陷排除效益。可是，这是我在改善缺陷排除效益方面经过三年的努力、通过编写 62 个 PSP 程序才达到的结果。

每个人有不同的技能。有的人很自然地生产出高质量的代码，有的人能很容易地构想出复杂的设计，有的人是天生的测试员，而有的人则善于设计易用的用户界面。所有的工程师都应能稳定地达到 70% 以上的缺陷排除效益，但有的人可能比其他人花的时间要长一些。

18.9 原型方法

在稳定地达到 70% 以上的过程效益之后, 进一步的提高将变得非常困难。虽然你可能会偶尔漏掉一、两个语法缺陷, 但大部分漏过的缺陷与设计有关。你可能会遇到新的系统、新的支持问题或者需要使用不熟悉的程序功能。对于大型程序, 接口、性能、存储管理以及其它许多问题都将变得更加重要。

人们每次从事新的工作时, 都很可能犯错误。因此, 为了消除所有的缺陷, 应该试着写一些小的原型程序。在使用每一个不熟悉的函数和过程时, 首先进行实验, 然后再在程序中使用它们。在测试一些新的想法、以前没有用过的结构或构造方法时, 也应该建立简单的原型。这样可以防止出现那些通常在代码复查时容易漏过的缺陷。

你还会看到有些缺陷类型比其它类型难于发现或防止。当设计一些方法以发现那些最经常漏掉的缺陷时, 很多方法都涉及到应用、系统支持和开发环境问题。每个引起混淆的地方都是缺陷的来源。关键是认识到真正了解的事与自认为理解的事的区别。对于真正知道的事情偶尔也会犯错误, 但更多的时候, 犯错误是由于你的假设不完全正确。首先要学会认识这些设想; 然后, 在把设想建造成产品以前, 建立一个原型来对它们进行测试。

18.10 总 结

随着程序的增大, 发现和修复缺陷的费用也会增大。缺陷排除过程的效果也会变差。开发高质量的大程序的策略是首先要排除组成大程序的模块中的所有的缺陷。

缺陷排除是一个过滤过程: 把每个缺陷排除阶段看成是一个过滤器。放入过滤器的缺陷越多, 将要发现的缺陷就越多。同样, 放入过滤器的缺陷越多, 将要漏掉的缺陷也就越多。大多数测试阶段的缺陷排除效益少于 50%。因此, 要在测试结束时得到一个高质量的产品, 测试前的产品就必须是高质量的。

在过程中的每个阶段, 仔细工作都是有回报的, 而且也节约时间。如果生产了一个很差的程序, 在编译阶段以及以后的各个测试阶段都将发现很多缺陷。

效益可以测量缺陷排除的质量。过程效益是指在第一次编译前排除产品中缺陷的百分比。效益也可以测量每个缺陷排除阶段排除产品中缺陷的百分比。目标应该是达到 100% 的过程效益。需要注意的是, 只有当能稳定地生产高质量的小程序, 才能生产高质量的大程序。要达到这个目的, 需要坚持质量原则和个体规范, 也需要大量的实践。

为了使程序达到高质量, 应该建立小的原型来测试每一种设想的情况, 然后才能在产品中使用。要学会认识自以为知道和真正了解之间的区别。每作一次假设, 就可能引入一个缺陷。

18.11 练习 18

在作业中，给出本学期你写的所有程序中编译和测试阶段发现的缺陷数之间的关系，并像图 18.1、18.2 和 18.3 那样用图表示。一旦发现了程序中所有的残留缺陷，计算每个程序的过程效益，估计可能的最终的过程效益。注意缺陷分析是选做作业，如果指导老师不要求则可以不做的。

提交你以前没有提交的时间记录日志和周活动总结表。另外，完成每个程序的 PSP 项目计划总结表、缺陷记录日志和代码复查检查表，这些表中要包括开发时间、缺陷引入、缺陷排除和缺陷排除效益计划的值、实际的值和累计值。

参 考 资 料

[Humphrey 89] Humphrey, W.S. *Managing the Software Process*. Reading, MA: Addison-Wesley, 1989.

第 19 章 过程质量

本章描述过程质量和用于评价软件工作质量的一些测量方法，同时说明如何计算和跟踪这些测量方法。在本章的练习中，将选用其中的一种方法对本学期所开发的程序进行测量，并用图来说明这个测量是如何变化的。

19.1 过程测量

程序的质量取决于过程的质量，而过程的质量又取决于人们的工作方式。因此，为了开发出更好的程序，需要改变工作方式。为了能够有效地做到这一点，需要知道自己当前的过程是好还是不好。要知道这一点，就需要测量过程的质量。

最基本的过程测量包括所生产产品的产量、产品的质量以及完成工作需要的时间和资源。从这些方面可以确定当前过程的性能以及将来如何改进过程以生产更好的产品。本书 3 ~ 11 章介绍了各种费用和规模的测量方法。从 12 章开始，跟踪了缺陷并介绍了一些缺陷排除的测量方法和每小时排除的缺陷。16 ~ 18 章介绍了过程效益和其它在发现和修复缺陷方面测量过程有效性的测量方法。现在，我们来讨论过程质量的测量。

需要注意的是，本书只从缺陷引入和排除两个方面来讨论质量问题。当学完本书所描述的 PSP 方法并经过实践后，就要考虑如何在其它一些重要的领域改进所使用的过程。例如，真正地了解需求的问题。解决对需求误解问题的一些过程步骤包括开发一个早期原型，并用它来与指导老师或客户共同复查需求。其它一些重要的领域包括可用性、性能测试、兼容标准或接口复查，等等。对其中的每一个方面，都可以设计出各种步骤来提高产品的质量。质量是一个很大的主题，有许多潜在的、可以利用的过程活动可以用来保证向客户交付高质量的产品。

19.2 缺陷排除中的矛盾

你可能已经注意到这样一种现象，就是当产品质量提高时缺陷排除率会下降。例如以图

16.2 为例, 经过 10 个程序后, 测试阶段的平均缺陷排除率从每小时大约 4 个降到每小时大约 2 个。在同一期间, 学生在测试阶段发现的缺陷数大约降低了十倍: 每千行源码中缺陷的数量从 40 个左右减少到 4 个左右。

考虑一下极端的情况, 就会感到这种下降似乎是很自然的。当缺陷很多时, 一般很快就会发现许多缺陷。尽管个别缺陷可能花费时间较长, 但平均时间相对较短。另一种极端的情况是, 在一个非常大的程序中只有一个缺陷, 所有花在代码复查和测试上的时间都是为了找到这个缺陷。虽然所需要的时间与缺陷的特点以及代码复查和测试的次序有关, 但都可能需要花费很长的时间。

因此, 当缺陷很少时, 无论使用什么方法都需要花费很长时间去找到这些缺陷。这对代码复查和测试明显是这样的, 但对编译情况却不同。尽管编译器的缺陷排除率有一点下降的趋势, 但编译器的自动特性使它对缺陷密度不敏感。代码复查和测试的下降趋势对少量数据可能不明显, 但每种方法的缺陷率一般都会随着缺陷的减少而下降。

因此, 当程序质量日益提高并成功地通过集成测试和系统测试等几个阶段后, 对程序中残留的缺陷的查找和修复将变得日益艰难。因此建议, 如果发现的缺陷越多, 就要更加注意去全部发现和修复这些缺陷。

19.3 缺陷排除策略

有人可能认为, 大型程序中的缺陷大多是在系统中而不是在模块中。但实际情况并不是这样。除了个别情况外, 大型系统中的缺陷都存在于组成系统的模块之中。这些缺陷可以分为两类: 仅涉及一个模块的缺陷; 涉及几个模块之间的交互作用的缺陷。

当达到一个较高的 PSP 缺陷排除效益时, 几乎可以排除所有的第一类缺陷。可是, 第二类缺陷的排除就非常困难。原因是大型复杂系统通常包括许多相互作用的模块, 即使是设计人员本人, 也很难设想到所有的相互作用关系。解决这个问题的好方法就是遵循以下的策略:

- 尽自己的努力开发高质量的程序模块。
- 全面检查所有的模块接口和交互作用(注意: 检查是指一组工程师复查一个产品。在本书 13.11 中对检查进行了简要的讨论)。
- 检查需求, 以确保所有重要的能力都被正确地理解、设计和实现。
- 根据需求检查系统和程序的设计, 以确保正确地实现了所有的关键需求。
- 代码检查后进行彻底的单元测试。
- 进行全面的集成测试。
- 进行全面的系统测试。

除第一步外, 其它的步骤都不属于个体过程的范围。但是, 你应该努力做好第一步。如果模块的质量不高, 其它步骤的效果也不会好。因此, 开发过程必须集中精力去查找和修复人们留在模块中的缺陷。如果不这样做, 则将意味着有可能无法完成其它步骤要做的工作。

这时，系统测试和集成测试的主要精力都花在寻找和修复残留在模块中的缺陷，而不是放在确保系统正确地完成用户想要的功能。

但是，如果最初生产的模块是高质量的，就可以按照前面提到的策略去做，这样就会有机会生产出高质量的系统。集成测试应该集中精力进行模块间接口的测试，而不是在查找和修复模块内部的缺陷上浪费时间。系统测试则是针对一些重要的系统问题（如性能、可用性、失效恢复和安全等）进行测试。除非工程师能稳定地达到很高的个人缺陷排除效益，否则很难进行这样的测试。

19.4 质量的成本

无论花费多少时间寻找缺陷，也无法肯定是否已经找到了全部缺陷。如果目标只是生产无缺陷程序，可以无限次地进行代码复查和系统测试。当经过一段时间没有发现缺陷之后，你可能会停下来。但可能仍没有找到全部缺陷。然而，一个软件工程师需要全面地权衡所花费的时间和所生产的产品质量等问题。质量成本(COQ)提供一种处理这些问题的方法。质量成本包括三个主要元素：过失成本、质检成本和预防成本。

过失成本包括修复产品中缺陷的所有费用。当修复一个缺陷时，就增加过失成本。类似地，当运行调试器来查找有缺陷的语句时，也会增加过失成本。任何与修复缺陷有关的工作都记入过失成本，这甚至包括重新设计、重新编译或者重新测试。

质检成本包括评估产品质量以确定在产品中是否仍然留有缺陷的所有工作，但不包括修复缺陷花费的时间。这包括对无缺陷产品的代码复查时间、编译时间和测试时间。因此，质检成本不包括修复缺陷的费用。质检成本是确保产品无缺陷的保障费用。

预防成本是由修改过程以避免引入缺陷所带来的费用。例如，包括对理解缺陷所做的分析以及改进需求、设计或编码过程的过程开发工作。重新设计和测试一个新的过程所花费的时间也属于预防成本。

另一种重要的预防缺陷的活动是为测试或实现想法而建立原型。当使用不熟悉的库函数时，总倾向于直接使用它。然而如果直接使用，经常会导致设计错误和程序缺陷。写一个小的原型程序来检查函数的运行情况的费用属于预防成本。

19.5 质量成本的计算

PSP 用一种简化的方法测量质量成本。尽管编译时间中通常包括一些无缺陷的编译时间，但在 PSP 中，把所有的编译时间都统计为过失成本。同样，尽管测试时间一般也会包含一些没有发现缺陷的测试时间，PSP 也把所有的测试时间统计为过失成本。最后，所有的复查时间累计为质检成本。复查时间一般包括一些修复缺陷的时间，但 PSP 把所有的复查时间统计为质检成本。可以用 19.8 节描述的方法精确计算质量成本值，但这种方法工作量大，而且不

会增加测量的有效性。因此，我建议你们使用这种简化的 PSP 定义。

质量成本（COQ）是按照总开发时间的百分比来计算的。在 PSP 中，质检成本和过失成本计算过程如下：

- 质检质量成本是所有复查时间的总和占总开发时间的百分比。
- 过失质量成本是所有编译和测试时间的总和占总开发时间的百分比。

例如，假设有显示在表 19.1 中的项目计划总结表中的过程数据，就可以按照下面的步骤计算质检质量成本：

- 实际总开发时间 = 262 min
- 实际代码复查时间 = 29 min
- 质检质量成本 = $100 \times 29 / 262 = 11.07\%$

对于过失成本则有：

- 实际编译时间 = 5 min
- 实际测试时间 = 10 min
- 过失质量成本 = $100 \times (5 + 10) / 262 = 100 \times 15 / 262 = 5.73\%$

表 19.1 PSP 项目计划总结表

学生	学生 X		日期	12/9/96	
程序			程序号#	15	
教师	先生 Z		程序设计语言	Ada	
总 结	计 划		实 际		累 计
Minutes/LOC	5.48		4.60		5.35
LOC/Hour	10.95		13.04		11.21
Defects/KLOC	92.53		52.6		86.7
过程效益	40.0		100.0%		45.5
A/FR	0.375		1.93		0.44
程序规模（LOC）					
新开发与更改的	49		57		392
最大规模	62				
最小规模	36				
开发阶段时间/min					
	计 划		实 际		累 计
计划	17		20		140
设计	29		38		233
编码	116		119		911
代码复查	21		29		174
编译	15		5		105
					5.0

					续表
开发阶段时间/min	计 划	实 际	累 计	累计百分比	
测试	41	10	289	13.8	
后置处理	30	41	247	11.7	
总计	269	262	2099	100.0	
最大时间	340				
最小时间	197				
引入的缺陷	计 划	实 际	累 计	累计百分比	Def./H our
计划					
设计	1		5	14.7	1.29
编码	4	3	28	82.4	1.84
代码复查					
编译			1	2.9	
测试					
总计	5	3	34	100.0	
排除的缺陷	计 划	实 际	累 计	累计百分比	Def./ Hour
计划					
设计					
编码					
代码复查	2	5	15	44.1	5.17
编译	2		13	38.2	7.43
测试	1		6	17.1	1.25
总计	5	5	34	100.0	

表 19.2

PSP 项目计划总结表使用说明

目 的	以一种方便而且易于存取的方式记录估算和实际的项目数据。
表头	如下输入： ● 你的名字和今天的日期 ● 程序名字和程序号 ● 教师的名字 你将使用的编程语言
Minutes/LOC	在开发之前： ● 登入计划的 Minutes/LOC 值，使用最近开发项目的累计开发效率值。 在开发之后： ● 用全部的开发时间除以实际的代码行数，得到实际的和累计的 Minutes/LOC 值。 例如，如果项目总共花了 196 分钟，产生了 29 LOC，那么开发效率 Minutes/LOC 的值就是 $196/29 = 6.76$ 。

续表

目 的	以一种方便而且易于存取的方式记录估算和实际的项目数据。
LOC/Hour	在开发之前: ● 用 60 除以计划的“Minutes/LOC”就得到了计划的“LOC/Hour”的值。 在开发之后: ● 用 60 除以实际的和累计的“Minutes/LOC”, 就得到了实际的和累计的“LOC/Hour”的值; 对于上面的示例, 实际的 Minutes/LOC = 6.76, 那么, 实际的 LOC/Hour = $60/6.76 = 8.88$。
缺陷密度 (Defects/KLOC)	在开发之前: ● 找到最近的前一个程序的缺陷密度的累计值; ● 使用此值作为当前项目的计划的缺陷密度的值。 在开发之后: ● 计算缺陷密度的实际值和到目前为止的累计值; ● 缺陷密度的实际值 = $1000 \times \text{实际的缺陷总数} / \text{实际的新开发和修改的代码行数}$; ● 用类似的方法计算缺陷密度的累计值; ● 例如, 到目前为止发现的缺陷的总数为 17, 新开发的和修改的代码的总行数为 153, 那么缺陷密度的累计值 = $1000 \times 17/153 = 111.11$。
过程效益	计算过程效益的计划值、实际值和累计值。 过程效益 = $100 \times (\text{编译前排除的缺陷的个数}) / (\text{编译前引入的缺陷的个数})$。如果在编译前引入了 5 个缺陷而发现了 4 个, 那么过程效益 = $100 \times 4/5 = 80.0\%$。
A/FR	计算计划的、实际的和累计的 A/FR 的值。 例如, A/FR 的实际值 = $\text{代码复查的实际的时间} / (\text{实际的编译时间} + \text{实际的测试时间})$。如果复查时间是 29 分钟, 编译时间是 5 分钟, 测试时间是 10 分钟, 那么 $A/FR = 29/(5+10) = 1.93$。
程序规模(LOC)	在开发之前: ● 在计划列登入估算的全部新开发与更改的代码行数, 最大规模, 最小值。 在开发之后: ● 统计并登入实际的新开发与更改的代码行数; 在累计一栏中, 登入实际的新开发与更改的代码行数加上上一个程序的新开发与更改的代码行数的累计值。
开发阶段时间 计划	用估算的新开发与修改的代码行数, 乘以估算的开发效率“Minutes/LOC”, 得到项目的总开发时间; 使用估算的代码行的最大值乘以估算的开发效率得到开发时间的最大值; 使用估算的代码行的最小值乘以估算的开发效率得到开发时间的最小值; 从最近的程序的项目计划总结表中, 找到各个阶段所花时间的累计百分比; 使用以前程序的累计百分比值, 计算出各个阶段的计划时间。
实 际	在任务完成之后, 根据时间记录日志中的数据登入各阶段实际花费的时间。 从时间日志中得到这些数据。
累 计	对每个阶段, 登入实际时间与最近的前一个程序的对应阶段的累计时间的和
累计百分比(%)	对于每个阶段, 计算 100 乘以该阶段累计时间, 然后除以总的累计时间, 并登入
引入的缺陷 计划	在开发之前, 估算程序中将引入的缺陷的总数; 计划的缺陷密度乘以计划的新开发和修改的代码行数, 然后除以 1000; 例如, 假设计划的缺陷密度为 75.9, 计划的新开发和修改的代码行数为 75, 那么计划引入的缺陷个数就是 $75.9 \times 75/1000 = 5.69$, 这时可以取值为 6; 在开发之前, 根据估算出来的缺陷总数和以前开发程序的缺陷引入的累计百分比, 估算各个阶段将引入的缺陷的个数。

续表

目 的	以一种方便而且易于存取的方式记录估算和实际的项目数据。
实 际	在开发完成后, 统计并登入各个阶段实际引入的缺陷的数目
累 计	对每个阶段, 登入实际缺陷和累计缺陷的总和
累计百分比(%)	对每个阶段, 计算 100 乘以该阶段累计缺陷, 然后除以总的累计缺陷, 并登入
缺陷引入率 (Defects/hour)	计算设计和编码阶段的缺陷引入率。 例如, 对于设计阶段, 用设计阶段引入的缺陷的累计值乘以 60 再除以设计时间的累计值= $60 \times 5/195=1.54$ 缺陷数/hour。
排除的缺陷	在“总计”一行中, 登入估算的总的缺陷个数。
计 划	根据缺陷排除的累计百分比的历史数据计算各个阶段计划排除的缺陷数。
实 际	在开发完成后, 统计并登入各个阶段实际排除的缺陷的数目
累 计	对每个阶段, 登入实际缺陷和最近程序的累计缺陷的总和
累计百分比(%)	对每个阶段, 计算 100 乘以该阶段累计缺陷, 然后除以总的累计缺陷, 并登入
缺陷排除率 (Defects/Hour)	计算代码复查、编译和测试阶段的缺陷排除率。 例如, 对于测试阶段, 用测试阶段排除的缺陷的累计值乘以 60 再除以测试时间的累计值= $60 \times 6/279=1.29$

19.6 质检/过失比

在 PSP 中, 一种有用的测量方法是质检/过失比, 即质检成本与过失成本之比, 简写为 A/FR。因此, A/FR 的值等于质检成本除以过失成本。按照 19.5 节示例中的数据, 质检成本是 11.07%, 过失成本是 5.73%, 那么 $A/FR=11.07/5.73=1.93$ 。

一个更简单的计算质检/过失比 (A/FR) 的方法是用复查时间除以编译和测试时间。使用这种方法, $A/FR=29/(5+10)=1.93$, 这个结果与上面的结果一样。这些计算方法在项目计划总结表使用说明中也有说明, 见表 19.2。

A/FR 用来测量在第一次编译前花在查找缺陷上的时间的相对值。学生的数据说明, A/FR 的值能很好地指示测试中发现缺陷的可能性。图 19.1 画出了 14 个接受 PSP 培训的学生的 A/FR 值与他们在测试中发现的每千行代码中的缺陷数的关系。这些数据来源于 140 个程序。

当 A/FR 小于 1 时, 程序测试时一般会发现很多错误。虽然并不绝对是这样, 但在这个范围内的大多数程序的测试缺陷数/KLOC 都相当高。另外, 从图 19.1 中可以明显看出, 对 A/FR 大于 2 的 PSP 过程, 无论如何每千行源码都只有很少的几个缺陷。这说明, 对 A/FR 值大于 2 的过程比 A/FR 小于 1 的过程生产无缺陷产品的可能性更大。因此, 应该努力使质检/过失比 A/FR 的值大于 2。

来自某个公司的数据说明了用 A/FR 来判断产品质量的效果。图 19.2 显示了六个产品组

件在单元测试阶段发现缺陷数与 A/FR 值的情况。图 19.3 显示了同样的组件在集成测试、系统测试和客户使用时发现的缺陷数与 A/FR 值的情况。注意，一个 A/FR 值较低的组件在单元测试中发现的缺陷数较少，但此后发现的缺陷很多。这很可能是由于单元测试进行的不够全面。图 19.4 画出编译时发现的缺陷数与编译后发现的缺陷数的情况。显然在编译时发现的缺陷越多，则在此后的测试和使用时发现缺陷也就越多。这个信息清楚地说明：如果想在测试阶段减少缺陷，就必须在编译前清除程序中的缺陷。有最高编译缺陷数目的组件在此后的每个过程阶段也有最多的缺陷。就是在产品交付用户使用后，它还是有最多的缺陷。

为了说明在开发过程的早期排除缺陷的效果，图 19.5 显示了测试小时数与 A/FR 值的关系。这里 A/FR 最低的两个组件在测试阶段分别花了 140~245 小时，而其它组件只花了 10~40 小时。图 19.6 显示每千行源码的测试小时数。对一个低质量的过程，每千行源码测试花了 80~100 小时；而对高质量的过程，只花了 20 小时左右。当开发 100,000 行以上的产品时，这种每千行 60~80 小时的差距将相当显著。一个低质量的个体软件过程 PSP 将多花 6000~8000 个测试小时，或者说浪费程序员 3~4 年的时间。随着几百万行源码的产品越来越常见，这将成为按时交付成功的产品和彻底失败的产品之间的区别。

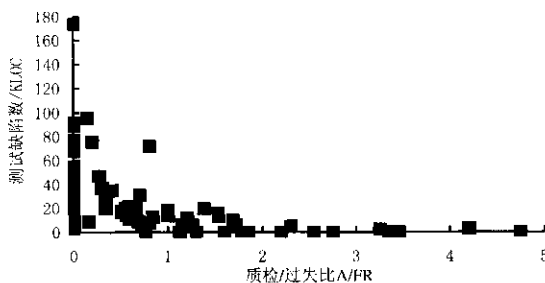


图 19.1 测试缺陷与 A/FR 的关系

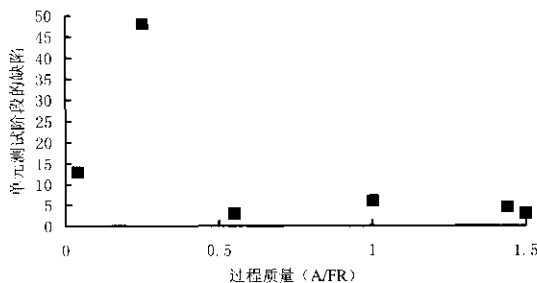


图 19.2 单元测试缺陷与过程质量的关系

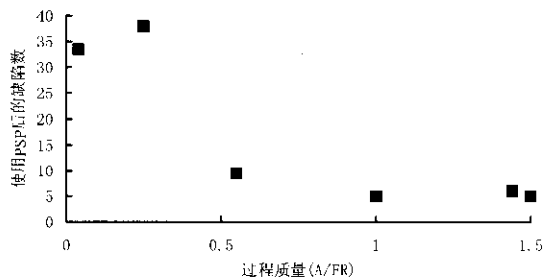


图 19.3 使用 PSP 开发之后发现的缺陷与过程质量的关系

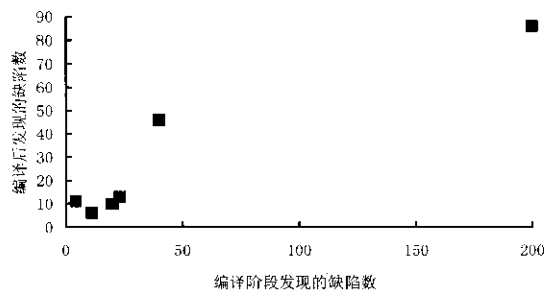


图 19.4 编译缺陷与编译后发现的缺陷的关系

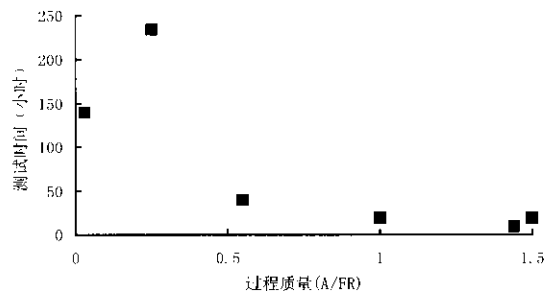


图 19.5 测试时间与过程质量的关系

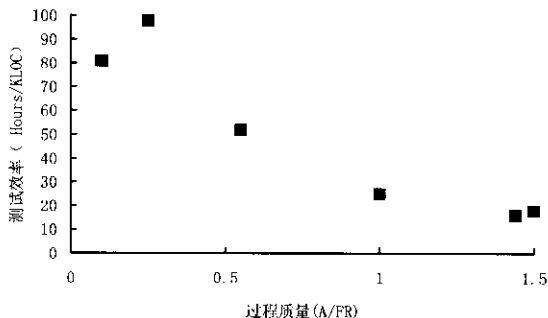


图 19.6 测试效率 (Hours/KLOC) 与过程质量的关系

使用质检/过失比

为了使质检/过失比 A/FR 的值达到 2.0 以上，应该复查以前程序的编译和测试时间，并且准备在下一个程序的代码复查中花费至少是这个时间两倍的时间。当然，可以仅仅通过增加代码复查时间来提高 A/FR 的值。但除非能发现缺陷，否则很难提高程序的质量。因此，能否高效率地利用复查时间来查找缺陷就显得很重要。

只要过程效益还没有达到 80% ~ 100% 的范围，就要继续增加 A/FR 的值。但是仅仅多花时间是远远不够的。应该复查最近开发的程序的缺陷数据，设计能查找经常漏掉的那些缺陷的方法。如果不知道如何做到这一点，请阅读第 14 章的内容。

确定了如何发现那些经常漏掉的缺陷之后，在代码复查检查表中插入适当步骤。最后，按照这些复查步骤进行代码复查。如果做到所有这些工作，就需要更多的代码复查时间，也会找到更多的缺陷和增加 A/FR 的值。这样做几乎将肯定减少在编译和测试阶段发现的缺陷数，也将节省大量的测试时间，减少过失成本，生产出高质量的产品。

19.7 改进复查的效率

直到在编译和测试阶段前能稳定地发现所有的缺陷，都不要考虑代码复查发现缺陷的效率 (defect/hour)。而要继续检查缺陷数据，看一看那些复查步骤应该发现可能在编译和测试阶段出现的缺陷；继续更新个人代码复查检查表以便找到这些缺陷。

一旦代码复查阶段发现了大部分缺陷，就应考虑改进代码复查发现缺陷的效率。为了做到这一点，首选要标识出每个既不能发现又没有漏掉缺陷的代码复查步骤。然后，重新考察原先包含这些步骤的理由。如果这些不再是问题，则跳过这些步骤；另一方面，如果认为这些检查仍然重要，把它们中相关的几个步骤组合起来，以便能做得更快。对于每一个程序，继续监控缺陷数据，恢复任何可以发现以后漏掉的那些缺陷的步骤。但是，当步骤不再有效时，要毫不犹豫地将其删除。

19.8 质量成本的精确计算

对 PSP, 简单的质量成本计算是足够了。但是, 当开发大型的项目时, 就需要使用更精确的质量成本测量方法。

为了计算过失成本和质检成本, 必须把代码复查时间、编译时间和测试时间分别分成错误修复和检验两个部分。例如, 我们把没有发现缺陷的编译时间标记为编译 Appraisal 或 C_A , 把有缺陷修复的编译时间记为编译 Failure 或 C_F 。因此总的编译时间是 $C_F + C_A = C$ 。同理, 总的代码复查时间是 $R_F + R_A = R$, 而总的测试时间是 $T_F + T_A = T$ 。

使用这些参数, 就能精确地计算出质检成本和过失成本。计算过程如下:

质检质量成本 = $100 \times (R_A + C_A + T_A) / (\text{总开发时间})$

过失质量成本 = $100 \times (R_F + C_F + T_F) / (\text{总开发时间})$

缺陷类型	
10 文档	60 检查
20 语法	70 数据
30 联编打包	80 函数
40 赋值	90 系统
50 接口	100 环境

表 19.3

缺陷记录日志的示例

学生 学生 X 日期 12/9/96
 教师 先生 Z 程序号# 15

日期	编号	类型	引入阶段	排除阶段	修复时间	相关缺陷
12/9	1	40	编码	代码复查	2	
	描述: 漏掉对 Set_X 的说明					
	2	80	设计	代码复查	8	
	描述: while 循环错误, 忘记步进, 仅考虑了奇数					
	3	20	编码	代码复查	1	
	描述: 漏掉“:”					

下面的示例使用了表 19.1 项目计划总结表和表 19.3 缺陷记录日志中的数据。首先按照如下步骤计算 R_A , C_A , T_A , R_F , C_F 和 T_F 。

● 为了计算 R_A , 首先要根据缺陷记录日志中的数据计算出 R_F , 也就是代码复查阶段用于排除缺陷的时间的总和: $R_F = 2 + 8 + 1 = 11$ 。

- 下一步计算 $R_A: R_A = R - R_P = 29 - 11 = 18$ 。

- 由于在编译阶段没有发现缺陷，因此所有的编译时间均为质量检验时间。因此 $C_A = 5$ 而 $C_P = 0$ 。

- 由于在测试阶段没有发现缺陷，因此所有的测试时间均是质量检验时间。因此 $T_A = 10$ 而 $T_P = 0$ 。

有了这些数值，现在就可以来计算质检质量成本和过失质量成本。计算过程如下：

$$\begin{aligned}\text{质检质量成本} &= 100 \times (R_A + C_A + T_A) / (\text{总开发时间}) \\ &= 100 \times (18 + 5 + 10) / 262 = 100 \times 33 / 262 = 12.60\%\end{aligned}$$

$$\begin{aligned}\text{过失质量成本} &= 100 \times (R_P + C_P + T_P) / (\text{总开发时间}) \\ &= 100 \times (11 + 0 + 0) / 262 = 100 \times 11 / 262 = 4.20\%\end{aligned}$$

这些值与前面算出来的那些值有点不同。 A/FR 的值也明显地增加到 3.0 而不是 1.93。由于这些更加准确的质检/过失比 (A/FR) 和质量成本 (COQ) 的值与缺陷修复时间密切相关，因此除非使用计时表测定缺陷的修复时间，否则不必使用这种方法。此外， A/FR 的值为 2.0 时仍然可能有许多缺陷，所以应将 A/FR 的目标值设置得更高一些。

19.9 总 结

质量成本 (COQ) 用三个部分 (过失成本、质检成本和预防成本) 来测量软件过程的质量。PSP 使用简化的质量成本计算方法。质检成本用代码复查时间除以总开发时间再乘 100；这个计算忽略了代码复查阶段的缺陷修复时间。过失成本用编译和测试时间之和除以总开发时间乘以 100；这个计算忽略了无缺陷的编译和测试时间。

质检/过失比 (A/FR) 测量过程质量。它是用质检质量成本除以过失质量成本。 A/FR 的值指出了第一次编译前花在查找和修复缺陷上的时间的相对值。 A/FR 的值也指出了在测试阶段发现缺陷的可能性。 A/FR 的值小于 1，一般说明在测试阶段将会发现许多缺陷；当此值大于 2 时，一般说明在测试阶段将会发现很少缺陷。当在测试阶段发现很少缺陷时，一般表明程序中残留的缺陷也很少。

虽然简化的质量成本计算适合于目前的开发情况，但当开发大型项目时，应该使用更加精确的质量成本计算方法。

19.10 练 习 19

在本章的练习中，计算本学期你写的所有程序的 A/FR 的值。另外找出这些程序在测试阶段的每千行源码中的缺陷数 (Defects/KLOC)，像图 19.1 那样显示 A/FR 的值和测试阶段的每千行源码的缺陷数 (Defects/KLOC) 之间的关系。

提交你以前没有提交过的时间记录日志和周活动总结表。完成每个程序的 PSP 项目计划

总结表、缺陷记录日志和代码复查检查表，这些表中应包括开发时间、缺陷引入、缺陷排除、过程效益和 A/FR 的计划值、实际值和累计值。

第 20 章 个人对质量的承诺

高质量的工作对你的雇主、客户和你自己都很重要。本章讨论为什么应该把树立质量观念作为个人的最高优先级。

20.1 质量的重要性

作为软件工程师，你所生产的程序的质量对你的雇主和客户都至关重要。软件应用在许多有迫切需要的领域，软件缺陷可能引起经济损失，甚至造成人身伤害。当你编写的程序用在金融工作或文字处理时，这些缺陷可能只是令人烦恼，也可能会为之付出巨大的代价，但不可能出现致死或致残的情况。但是，当你的软件是飞机飞行、汽车驾驶、空中交通管理、控制工厂运行或控制电厂的系统的一个部分时，软件系统中的缺陷可能造成无法计算甚至可能是灾难性的后果。

已经有因软件缺陷而致人死亡的先例[Leveson]。到目前为止，虽然这种灾难并不多见，但是这个数据以后肯定会有所增加。尽管有许多问题，但由于软件不会出现疲劳或损坏。因此，软件非常适合于包括安全在内的一些重要领域中应用。例如，今天的核电厂在最初建造时，控制仪器全都是电气机械设备，这是当时最可靠的技术。可是在许多年后，那些早期的设备已经逐渐损坏并不得不被替换下来。今天，最可靠的控制系统都是计算机化的系统。虽然第一代核控制相对简单，使用了很少的软件，然而在未来更复杂的控制中将有大量的程序代码。这就意味着对于那些建造和管理核电厂的人来说软件质量将日益重要；对那些住在核电厂附近的人来说软件质量也将同样重要。

每个领域的软件应用都遵循着同样的趋势。计算机化的控制系统是如此多种多样、经济和可靠，以致于软件几乎应用在生活的各个方面。因此，重要的是所有的软件工程师都要认识到，他们的工作对千百万人的健康、安全和幸福将有至关重要的影响。

20.2 低质软件的危险正在增长

大型程序中一小部分的任何缺陷都可能导致严重的问题，一个大型系统中某个环节上的

一个简单错误可能具有潜在的灾难性。这听起来似乎不太可能，但实际上这是经常出现问题的根源。因为软件系统不像机械系统那样有机械保护。通过设计合适的机械安全系统，工程师可以消除各种事故或故障。常见的情况是冲压操作员穿的安全服，冲压机与安全服机械地连在一起，这样当机器冲压时，操作员就不可能把手放在里面了。

然而对软件系统来说，安全系统通常也是由软件完成的。只有当软件按照系统规则和约定运行时，软件保护才起作用。这就意味着当软件缺陷打破约定运行时，将会出现不可预测的后果。虽然系统安全应该在系统级别上实现，并应同时实施软件和硬件的保护措施，但这并不总是可行的，因此，为了拥有真正的安全系统，必须努力生产出无缺陷的软件。

大型多用户实时系统的设计可能会特别复杂。这些复杂的设计问题吸引了设计者大量的注意力。人们通常会对困难的设计问题进行认真的研究、仔细的复查和测试，而对那些简单的设计问题则容易忽视。结果表明，大型软件系统中常见问题的起因经常是一些简单的疏忽大意。这就是那种典型的软件工程师的个人错误。尽管大部分这类错误在集成测试和系统测试时会发现，但有一部分则会通过所有的测试而进入实际应用。

问题是软件工程师经常把简单的和容易的两个问题弄混。他们认为那些经常出现的简单错误可以很容易找到。但他们吃惊地发现，这些细微的错误，如少写一个标点、写错一个参数名、设置不正确的条件或不正确地结束循环，经常能够逃过测试，并在实际应用中引起严重问题。正是这类简单错误几乎引起了所有各种各样的问题，使得软件供应商不得不花费几百万美元来查找和修复。

大程序的质量依赖于组成它们的小程序的质量。因此，为了生产出高质量的大程序，每个软件工程师在开发一个或多个模块时必须保证工作的高质量。这意味着所有工程师必须对软件质量作出个人承诺。当他们有了承诺，就会非常细心地跟踪和管理自己的缺陷，尽可能地让缺陷留到集成测试、系统测试或客户使用阶段。

当工程师对质量有个人承诺时，他们会为自己的成就而骄傲。他们知道自己生产了多少代码以及别人在他们的产品中发现多少缺陷。有些工程师曾经告诉我有多长时间没有人在他们的产品中发现缺陷。当软件工程师对质量没有个人承诺时，他们就没有意识到质量的重要性。他们会告诉你测试会发现大部分缺陷，残留的缺陷也将会很容易被发现和修复。当工程师这样想的时候，他们不可能生产出高质量的产品。

20.3 制订个人质量承诺

许多软件工程师把软件缺陷叫作 bugs，把它们当作是从别处爬来的什么东西，而没有把它看作是自己的责任。这种观点是错误的。缺陷是工程师引入的，并且当一个程序中有一个缺陷时，它就是一个有缺陷的程序！编写这个程序的工程师，生产了一个有缺陷的产品。当工程师对质量作出承诺时，他们就会在乎缺陷。当他们对此关心时，他们在工作中就会更加精心。当他们更加精心时，他们就会生产出更好的产品。

因此，生产高质量软件的第一步是要明确质量的重要性。第二步是要建立生产无缺陷产

品的目标。为了赢得各种机会达到这个目标，必须测量程序的质量，并采取相应的措施改进质量。

每个人都会偶尔引入缺陷。管理我们易犯错误的毛病是一种挑战，也是一个永无止境的奋斗。不能仅仅学会一些质量保证的诀窍就无忧了。未来的软件产品会变化多端且日益复杂，你面对的问题也会不断地改变。问题不仅是你能否生产出无缺陷的产品，而且是你能否持续地、充分地关注这一点。生产出无缺陷产品不仅使你和你的组织的效率更高，而且也会让你在工作中感受到更多满足。

20.4 个人的目标

人生要追求什么？这个问题很大，很多人难于作出回答。当你考虑这个问题的答案时，有几点值得注意。

从工作中得到满足的一个途径是拥有权利或地位。人们可以通过成为老板或重要部门的主管来得到这些。权利和地位也可以是间接的，如挣许多钱、在重要公司工作或开着一辆别致的汽车。这些都是成功者的象征。

当然，拥有地位并没有错，但这是暂时的。你可以在一段时间之内拥有一份重要的工作，但在或长或短的一段时间后，你可能会失去它。失去地位可能出现危机。有些人在第一次丢掉重要工作时就变得颓废。人们往往容易把工作的重要性 and 个人的重要性相混淆。

我认识这样一些经理，他们因降级而一蹶不振。他们已经建立了一个认为自己是重要人物的自我印象。只要他们拥有重要的工作，每个人都会把他们看成是重要人物。可是一旦他们失去这些工作，就和其他人一样，没有人在意他们说些什么，他们不再有特殊待遇，失去了单独的办公室，不再有秘书。如此严重的打击，使得有些人精神崩溃、心脏病发作或出现家庭危机。他们追求的是地位，但现在却失去了它。

20.5 成就的回报

需要决定的是，你追求的是什么。想想未来，当你退休时，将对什么样的人生感到满足？我认为，你做的工作比你得到的回报要有价值的多。譬如，如果你计划从事工程工作，你可能有建筑师的本能。你可能会建立一些系统或构件，你可能会总结出一些建造方法或建造过程。或者你可能爱好科学、建立理论或在基础知识方面进行研究。

但不管做什么，质量都是关键。没有人会对很差的工作感到满足。而且，即使没有其他人发现，你自己会知道自己的工作做得不好。这会失去你在工作中的骄傲，也限制了你人生的满足感。你不能诚实地对自己说，你真的相信质量，这只是一次失误。总是可以找到很多借口，你甚至可以找到有利的回答而让其他人感到满意，但是你永远不会让自己满意。

当完成高质量的工作时，你会为此而感到骄傲。即使没有其他人知道，但你知道自己完

成了一流的工作并为此作出了最大的努力而满足。出乎意料的是，高质量的工作迟早会为人们所知的；它可能要用相当长的时间，但或迟或早总会被人承认的。不管你是否知道，你都会因为你的高质量工作而得到信誉。

因此，要问自己这样的问题：“我希望为自己所做之事而骄傲吗？”很多人都会回答“是”。但如果真的想做到这一点，就需要建立个人标准，并为达到这一标准而努力奋斗。在达到这些标准时，再提高标准并继续努力。挑战自己去完成更优良的工作，你将会为自己的成就而感到惊讶。

参 考 书 目

[Leveson] Leveson, Nancy G. *Safeware, System Safety and Computers*. Reading, MA: Addison-Wesley, 1995.